Virtual University	CS304-OOP (Solved Macq's) LECTURE FROM (23 to 45)	VU LMS HELP vulmshelp.com
Junaidfazal08@gmail.com Bc190202640@vu.edu.pk	FOR MORE VISIT VULMSHELP.COME	**JUNAID MALIK** **0304-1659294**

AL-JUNAID TECH INSTITUTE



For CS619 Project Feel Free To Contact With Me

Ph# 0304-1659294
Email: junaidfazal08@gmail.com

AL-JUNAID INSTITUTE GROUP

1. Function func1of class1 is_____ in class2.
   - ► Public
   - ► Protected
   - ► Private
   - ► None of the given
2. The default inheritance mode is,
   a) Public inheritance
   b) Protected inheritance
   c) Private inheritance
   d) None of the given
3. A template provides a convenient way to make a family of
   a) Variables and data members
   b) Functions and classes
   c) Classes and exceptions
   d) Programs and algorithms
4. Then int member i of base class is accessible in class,
   a) derived 1 only
   b) derived 2 only
   c) both derived 1 derived 2
   d) none of the given options
5. In case of public inheritance, protected members of base class will be_____in derived class?
   a) Private
   b) Public
   c) Protected
   d) hidden
6. By default assignment operator (=) performs,
   a) Shallow copy
   b) Deep copy
   c) Both of these
   d) None of these
7. In protected inheritance the public members of base class become _____in derived class.
   a) Public
   b) Private
   c) Protected
   d) None of the given
8. We can access public members of the class from outsides the class using_____operator with its object pointer
   a) ->

b) .
c) &
d) @

9. Compiler can automatically generates the following constructor/s for a class,
   a) Default constructor
   b) Copy constructor
   c) Both of these
   d) None of these
10. Friend's functions of class are _____members of that class.
   a) Public
   b) Private
   c) Protected
   d) None of the given
11. Methodologies to the development of reusable software relate to_____.
   a) Structure programming
   b) Procedural programming
   c) Generic programming
   d) None of the given
12. Function overriding is done in context of,
   a) Single class
   b) Single derived class
   c) Single base class
   d) Derived and base classes
13. Which of the following is not type of inheritance in c++?
   a) Public
   b) Protected
   c) Restricted
   d) Private
14. If there is a pointer ,p, to objects of a base class and it contains the address of an object of a derived class and both classes contain a virtual member function, ding(),then the statement p->ding(); will cause the version of ding in the_____class to be executed.
   a) Base
   b) Derived
   c) Virtual
   d) Implemented
15. Target class of a____function call is determined t run time.
   a) Instance
   b) Virtual
   c) Operator
   d) None of the given
16. In c++ dynamic binding and polymorphism will be achieved when the member function will be_____

AL-JUNAID TECH INSTITUTE
www.yulmshelp.com

a) Private
b) Public
c) Virtual
d) Inline

17. To make a pure virtual, we need to give____after() of this function.
a) =0;
b) =1;
c) Null;
d) None of the given

18. If we have not given any constructor for the class,compiler generates which of the following constructors?
a) Emplicit Default Constructorb) Implicit Default Constructor
c) Emplicit Parameterized Constructor
d) Implicit Parameterized Constructor

19. In c++ by default access of classes is____.
a) Private
b) Protected
c) Public
d) None of the given

20. In public inheritance the public members of base class become_____in derived class
a) Private
b) Protected
c) Public
d) None of the given

21. A class hierarchy
a) Shows the same relationships as a organization chart
b) Describe "has a "relationship.
c) Describe "is a kind of" relationship.
d) Shows the same relationships as a family tree.

22. In private Inheritance the public members of the base class become_____in derived class
a) Public
b) private
c) Protected
d) None of the given option

23. In the line No.2, which constructor of student class will be
a) Default constructor of student class
b) Copy constructor of student class
c) Both Default and Copy constructor of student class
d) No constructor will be called

24. Which of the following is Not casting operator in c++ standard?
a) Static-cast

AL-JUNAID TECH INSTITUTE 03215659294 www.vulmshelp.com

b) Var-cast
c) Dynamic-cast
d) Reinterpret-cast

25. Which of the following type of inheritance is used to model "Implemented in terms of" relationship?
a) Public
b) Private
c) Protected
d) Confidential

26. When we want to implement one class in terms of another class then we use,a) Public inheritance
b) Protected inheritance
c) Private inheritance
d) None of these options

27. Public inheritance represents;
a) "IS A "relationship
b) "Has A" relationship
c) "IS Special kind of" relationship
d) None of these option

28. Suppose we have defined derived class copy constructor but have not defined base class copy constructor then compiler will,
a) Use base class default constructor
b) Generate base class copy constructor itself
c) Use base class assignment operator
d) Nine of these option is correct

29. Child class can call constructor of its,a) Direct base class
b) Indirect base class
c) Both direct and indirect base classes
d) None of these

30. We can have_____type of member functions in a class.
a) Public
b) Private
c) Protected
d) All of these options

31. Compiler generated copy constructor performs_____
a) Shallow copy
b) Deep copy
c) Both Shallow and Deep copy
d) None of these options

32. In c++ generic programming is done using_____
a) Procedures

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

b) Packages
c) Templates
d) None of the given

33. Target class of a____function call is determined at run time.
a) Instance
b) Virtual
c) Operator
d) None of the given

34. Suppose person is a user defined class.In statement "Person*pPtr", static type of pPtr is ----.
a) pPtr
b) pointer
c) person
d) None of the given

35. Which statement is true for concrete class?
a) It cannot be inherit from an abstract class
b) It can be instantiated
c) It cannot be instantiated
d) None of given

36. In c++, compiler can generate which of the following operators 'code,
a) ==
b) =
c) &
d) &&

37. In specialization we can,
a) Replace child class with its base class
b) Replace base class with its child class
c) Replace both child and base classesinterchangeably
d) None of the given option

38. In private inheritance derived class pointer can be assigned to base class pointer in,
a) Main function
b) In derived class member and friend class function
c) In base class member and friend functions
d) None of the given option

39. When we access private members of the class from outside the class using_____operator with its object pointer
a. ->
b. .
c. &
d. None of above

40. Which of the following access specifier ensure that base class member is accessible in derived class and Not outside of this class
a. Public

AL JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

b. Private
c. Protected
d. All of above

41. Static casting is
    a. C++ way of calling base class function from derived class
    b. C way of calling base class function from derived class
    c. Both of these
    d. None of these
42. Suppose both derive and base classes have compilers generated copy constructors then derived class copy construct is
    a. Compiler generated default constructor
    b. Compiler generated copy constructor
    c. Both of these options are correct
    d. None of these operators are correct
43. In case of protected inheritance, public members of the base class will be_____in derived class
    a. Private
    b. Public
    c. Protected
    d. Hidden
44. Methodologies to the development of reusable software related to_____
    a. Structured programming
    b. Procedural programming
    c. Generic programming
    d. None of the given
45. A child class can call constructor of the parent call through
    a. Its constructor initialization list
    b. Its constructor body
    c. Both from its constructor initialization list or body
    d. Cannot call the constructor of its parent class
46. In case of dynamic memory allocation in or class we should use
    a. Default assignment operator
    b. User defined assignment operator
    c. Default copy constructor
    d. None of these
47. Consider the code below.

```
Class class1 {
Protected:
Int I;
};
Class class2: private calls 1{
};
```

    a) Public

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

b) Protected
c) Private
d) None of the given options

48. Consider the following two lines of code written for class student
    1. Student sobj1;
    2. Student sobj2(sobj1)

In line 2 which constructor of student class will be called?

a) Default constructor of student class
b) Copy constructor of student class
c) Both default and copy constructor of student class
d) No constructor will be called

49. Which of the following is /are casting operator(s) inc++?
    a. Static_cast
    b. Const_cast
    c. Dynamic_cast
    d. All of the given options

50. In case of private inheritance, private members of base class will be_____in derived class?
    a. Private
    b. Public
    c. Protected
    d. Hidden

51. Consider the code below

```
class class1{
Private:
Void func1();
};
Class class2 : public class1{
};
```

Function func1 of class 1 is_____in class2.
    a. Public
    b. Protected
    c. Private
    d. None of the given options

52. We can call base class assignment operator in derived class user defined assignment operator
    a. Implicitly
    b. Explicitly
    c. Using both of these options
    d. Using none of these options

53. A function call is resolved at run time in

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

a. Non virtual members functionb. Virtual member function

c. Both none-virtual and virtual member function

d. None of given

54. If the user does not specify the type if inheritance, then the default type of inheritance is _____

a. Public inheritance b. Private inheritance

c. Protected inheritance

d. None of given

55. Consider the code belowclass class1 {

public:

Void func1();

};

Class class2: protected class1{

};

Function func1 of class 1 is_____in class2.

a. Public

b. Protected

c. Private

d. None of the given options

56. Which of the following causes run time binding?

- Declaring object of abstract class
- Declaring pointer of abstract class
- Declaring overridden methods as non-virtual (Page 226)
- None of the given

57. Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?

- Templates (Page 256)
- Overloading
- Data hiding
- Encapsulation

58. A copy constructor is invoked when

- a function do not returns by value.
- an argument is passed by value. (Page 78)
- a function returns by reference.
- an argument is passed by reference.

59. Like template functions, a class template may not handle all the types successfully.

- True (Page 258)

ALHUNAID TECH INSTITUTE 0304-1659294 www.ulmshelp.com

► False

60. A class template may inherit from another class template.

► True (Page 288)
► False

61. By default the vector data items are initialized to _____
► 0 google
► 0.0
► 1
► null

62. In Private---------- only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object

► specialization
► inheritance (Page 216)
► abstraction
► composition

63. Which of the following is/are advantage[s] of generic programming?
► Reusability
► Writability
► Maintainability
► All of given (Page 256)

64. Template functions use __ _____than ordinary functions.

► Greater Memory
► Lesser Memory
► Equal Memory
► None of the given options

65. Non Template Friend functions of a class are friends of_____instance/s of that class.

► All google
► One specific
► All instances of one date type
► None of the given options

66. A pointer to a base class can point to objects of a derived class.

► True google
► False

67. A template argument is preceded by the keyword_____.

AJ JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

► vector
► class
► template
► type*

68. Which one of the following terms must relate to polymorphism?

► Static allocation
► Static typing
► Dynamic binding (Page 239)
► Dynamic allocation

69. Multiple inheritance can be of type

► Public
► Private
► Protected
► All of the given

70. Assume a class Derv that is privately derived from class Base. An object of class Derv located in main() can access

► public members of Derv
► protected members of Derv.
► private members of Derv.
► protected members of Base.

71. A function call is resolved at run-time in_____

► non-virtual member function.
► virtual member function. (Page 239)
► Both non-virtual member and virtual member function.
► None of given

72. Two important STL associative containers are_____and_____.

► set,map
► sequence,mapping
► setmet,multipule
► sit,mat

73. An abstract class is useful when,

► We do not derive any class from it.
► There are multiple paths from one derived class to another.
► We do not want to instantiate its object
► You want to defer the declaration of the class.

74. Suppose you create an uninitialized vector as follows:

AL-JUNAID TECH INSTITUTE
0304-1639294
www.vulmshelp.com

vector<int> evec;
After adding the statment,
evec.push_back(21);
what will happen?

► The following statement will add an element to the start (the back) of evec and will initialize it with the value 21.

► The following statement will add an element to the center of evec and will reinitialize it with the value
21.

► The following statement will delete an element to the end (the back) of evec and will reinitialize it with the value 21.

► The following statement will add an element to the end (the back) of evec and initialize it with the value 21.

75. Default constructor is such constructor which either has no ---------or if it has some parameters these have -----
- values

► Parameter, temporary
► Null, Parameter
► Parameter, default (Page 75)
► non of the given

76. Classes like TwoDimensionalShape and ThreeDimensionalShape would normally be concrete, while classes like Sphere and Cube would normally be abstract.

► True
► False

77. In order to define a class template, the first line of definition must be:

► template <typename T> (Page 257)
► typename <template T>
► Template Class <ClassName>
► Class <Template T>

78. A function template can not be overloaded by another function template.

► True
► False

0304-1659294
www.vulmshelp.com

79. Identify the correct way of declaring an object of user defined template class A for char type members?

- A< char > obj;
- <char>A obj;
- A obj<char>;
- Obj <char> A;

80. The user must define the operation of the copy constructor.

- True
- False

81. The find() algorithm

- finds matching sequences of elements in two containers.
- finds a container that matches a specified container.
- takes iterators as its first two arguments. (Object-Oriented Programming in C++)
- takes container elements as its first two arguments.

82. Compiler performs_____type checking to diagnose type errors,

- Static (Page 261)
- Dynamic
- Bound
- Unbound

83. Vectors contain contiguous elements stored as a[an]____.

- variable
- array (Page 306)
- function
- datatype

84. In a de-queue, (chose the best option)

- data can be quickly inserted or deleted at any arbitrary location.
- data can be inserted or deleted at any arbitrary location, but the process is relatively slow.
- data can not be quickly inserted or deleted at either end.
- data can be inserted or deleted at either end, but the process is relatively slow.

85. Algorithms can only be implemented using STL containers.

- True
- False(Object-Oriented Programming in C++)

ALJUNAID TECH INSTITUTE
0304-1659294
www.aljunaid.com

86. What is a class?

- A class is a section of computer memory containing objects.
- A class is a section of the hard disk reserved for object oriented programs
- A class is the part of an object that contains the variables.
- A class is a description of a kind of object. Click here for detail

87. Inheritance is a way to

- organize data.
- pass arguments to objects of classes.
- add features to existing classes without rewriting them. (Page 27)
- improve data-hiding and encapsulation.

88. We can use "this" pointer in the constructor in the body and even in the initialization list of any class if we are careful,

- True
- False

89._____and_____methods may not be declared abstract.

- private,staticClick here for detail
- private,public
- static,public
- none of given

90----------members are somewhere between public and private members. They are used in inheritance

- protected
- public
- private
- global (Page 187)

91. Which of these are examples of error handling techniques?

- Abnormal Termination
- Graceful Termination

AJ IJAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

- Return the illegal
- all of the given (Page 329)

92---------- follow try block to catch the object thrown

- catch block (Page 333)
  - throw block
  - main block
  - non of the given

93. Graphical representation of the classes and objects is called object model it shows -------

- Class Name only
- Class Name and attributes
- Relationships of the objects and classes
- all of the given

94. Destructor can be overloaded

- True

- False(Page 92)

95. Which of the following is the best approach to implement generic algorithms with minimum number of coding lines?

- Templates (Page 256)
- Overloading
- Overriding
- Friend function/class

ALJINAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

96. Each try block can have_____no. of catch blocks.

- ► 1
- ► 2
- ► 3
- ► As many as necessary.

97. class DocElement

```
{
public:
      virtual void Print() { cout << "Generic element"; }
};
class Heading : public DocElement
{
public:
      void Print() { cout << "Heading element"; }
};
class Paragraph : public DocElement
{
public:
      void Print() { cout << "Paragraph element"; }
};
void main()
{
      DocElement * p = new Paragraph();
```

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

```
    p->Print();
}
```

98. When you run this program, it will print out a single line to the console output.

What will be in that line?

Select one correct answer from the following list:

► Generic element ►
Heading element
► Paragraph element
► Nothing will be printed.

99. Suppose we have two derived classes from a single class, can we write a method with same name in both these derived classes? Choose the best option.

► No
► Only if the two classes have the same name (Page 204)
► Only if the main program does not declare both kinds
► Yes

100. When a virtual function is called by referencing a specific object by name and using the dot member selection operator (e.g., squareObject.draw()), the reference is resolved at compile time.

► True
► False

101. Considering the resolution order in which Considering the resolution order in which compiler search for functions in a program; the first priority is given to; the first priority is given to,

► general template
► partial specialization
► complete specialization
► ordinary function (Page 287)

102. One purpose of an iterator in the STL is to connect algorithms and containers.

► True
► False

103. In_____, a base class can be replaced by its derived class,

► Sub-typing (Page 31)
► Super-typing
► Multiple-typing
► Restricted-typing

ALJUNAID TECH INSTITUTE
0304-1659291
www.vulmshelp.com

104. one this pointer does not point to current object of any class,

- True
- False

105. Which of the following operator(s) take(s) one or no argument if overloaded?

- ++(Page 162)
- -
- +
- All of the given

106. Which of the following operators can not be overloaded?

- Scope resolution operator (::) (Page 141)
- Insertion operator (<<)
- Extraction operator (>>)
- The relation operator (>)

107. Which of these are examples of error handling techniques ?

- Abnormal Termination
- Graceful Termination
- Return the illegal
- all of the given (Page 329)

108. _____"is a" relationship

- Inheritance (Page 25)

ALHUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

► Polymarphism
► abstraction
► encapsulation

109. Virtual functions allow you to

► create an array of type pointer-to-base class that can hold pointers to derived classes.
► create functions that can never be accessed.
► group objects of different classes so they can all be accessed by the same function code.
► use the same function call to execute member functions of objects from different classes

110. The copy() algorithm returns an iterator to

► the last element copied from.
► the last element copied to.
► the element one past the last element copied from.
► the element one past the last element copied to.

111. If you define a vector v with the default constructor, and define another vector w with a one-argument constructor to a size of 11, and insert 3 elements into each of these vectors with push_back(), then the size() member function will return_____for v and_____for w.

► 11 for v and 3 for w.
► 0 for v and 0 for w.
► 0 for v and 3 for w.
► 3 for v and 11 for w.

112. Which is not the Advantage of inheritance?

► providing class growth through natural selection. (Object-Oriented Programming in C++)
► facilitating class libraries.
► avoiding the rewriting of code.
► providing a useful conceptual framework.

113. Which type of inheritance is being represented by the following statement, class X : public A, public B { ..... };

► Single inheritance
► Multiple inheritance (Page 41)

0304-1659294 www.vulmshelp.com

- Double inheritance
- None of the given options

114. When we write a class template the first line must be:

- template < class *class_name*>
- template < class *data_type*>
- template < class T > (Page 257)

115.Function templates should be used where code and behavior must be identical.

- True (Page 262)
- False

116.The specialization pattern <T*> after the name says that this specialization is to be used for every,

- data type
- meta type
- virtual type
- pointer type (Page 286)

117.A range is often supplied to an algorithm by two _____values.

- italic
- iteration (Object-Oriented Programming in C++)
- iterator
- None of given

118.Which of the following is an integral part of an object?

- State
- Behavior
- Unique identity
- All of the given (Page 12)

119.Consider the following statement

Cupboard has books
What is the relationship between Cupboard and books?

- Composition
- Aggregation
- Inheritance
- None of the given options

120. Which sentence clearly defines an object?

- one instance of a class. (Page 23)

ALHUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

- another word for a class.
- a class with static methods.
- a method that accesses class attributes.

121._____, which means if A declares B as its friend it does NOT mean that A can access private
data of B. It only means that B can access all data of A.

- Friendship is one way only
- Friendship is two way only
- NO Friendship between classes
- Any kind of friendship

122. The statement objA=objB; will cause a compiler error if the objects are of different classes.

- True
- False

123. Consider the call given below of an overloaded operator "+", *Rational_number_1 + Rational_number_2*

Where Rational_number_1 and Rational_number_2 are the two objects of Rational_number class (a user defined class). Identify which of the above two objects will be passed as an argument to the overloaded operator function?

- Rational_number_1
- Rational_number_2
- Both Rational_number_1 & Rational_number_2
- any of the two objects, randomly

124. If a class D has been derived using protected inheritance from class B (If B is a protected base and D is derived class) then public and protected members of B ---------accessed by member functions and friends of class D and classes derived from D

- can be
- cannot be
- does restirct to be
- not given

A-JOINED TECH INSTITUTE www.vulmshelp.com

The type that is used to declare a reference or pointer is called its_____

► abstract type
► reference type
► ► static type
► defaulttype

125. How the information hidden within an object can be accessed?
► Through its interface
► Through its private data members
► Through its private member functions
► Through both public and private members

126. The sub-object's life is not dependant on the life of master class in_____.
► Separation
► Composition

► None of the given

127. Encapsulation means
Select correct option:
► Extending the behaviour of class in another class
► Data and behaviour are tightly coupled within an entity (Page 16)
► One entity takes all the attributes and operations of the other
► Taking out the common features and put those in a separate class

128. An STL container can not be used to,

► hold objects of class employee.
► store elements in a way that makes them quickly accessible.
► compile c++ programs. (Object-Oriented Programming in C++)
► organize the way objects are stored in memory

129. Which of the following may not be an integral part of an object?
► State
► Behavior
► Protected data members (Page 12)
► All of given

130. Public methods of base class can----------be accessed in its derived class
► directly (Page 179) rep
► inderectly
► simultaniously
► non of the given

ARJUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

131. What is true about function templates?

► The compiler generates only one copy of the function template
► The compiler generates a copy of function respective to each type of data (Page 256)
► The compiler can only generate copy for the int type data
► None of the given.

132. When the base class and the derived class have a member function with the same name, you must be more specific which function you want to call (using _____).

► scope resolution operator
► dot operator
► null operator
► Operator overloading

133. A template provides a convenient way to make a family of

► variables and data members
► functions and classes(Object-Oriented Programming in C++)
► classes and exceptions
► programs and algorithms

134. A non-virtual member function is defined in a base class and overridden in a derived class; if that function is called through a base-class pointer to a derived class object, the derived-class version is used.

► True
► False

135. If there is a pointer p to objects of a base class, and it contains the address of an object of a derived class, and both classes contain a nonvirtual member function, ding(), then the statement p->ding(); will cause the version of ding() in the_____class to be executed.

► Base (Object-Oriented Programming in C++)
► Derived
► Abstract
► virtual

136. In case of multiple inheritance a derived class inherits,

► Only the public member functions of its base classes
► Only the public data members of its base classes
► Both public data members and mem er functions of all its base classes
► Data members and member functions of any two base classes

ALJUNAID TECH INSTITUTE
0304-1639294
www.vulmshelp.com

137. Which of the following is incorrect line regarding function template?

- ► template<class T>
- ► template <typename U>
- ► Class<template T> (Page 257)
- ► template < class T, class U>

138. Consider a class named Vehicle, which of the following can be the instance of class Vehicle?

i. Car
ii. Computer
iii. Desk
iv. Ahmed
v. Bicycle
vi. Truck

- 1,4,5
- 2,5,6
- 1,2,3,6
- 1, 5, 6 Conceptual

139. Consider the code below,

```
class Fred {
public:
Fred();
...
};
int main()
{
Fred a[10];
Fred* p = new Fred[10];
...
}
```

Select the best option,

► Fred a[10]; calls the default constructor 09 times Fred* p = new Fred[10]; calls the default constructor 10 times

► Produce an error

ALJUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

► Fred a[10]; calls the default constructor 11 times Fred* p = new Fred[10]; calls the default constructor 11 times

► Fred a[10]; calls the default constructor 10 times Fred* p = new Fred[10]; calls the default constructor 10 times

140. When a variable is define as static in a class then all object of this class,

► Have different copies of this variable
► Have same copy of this variable (Page 110)
► Can not access this variable
► None of given

141. Which of the following operators always takes no argument if overloaded?
► /
► -
► +
► ++ (Page 162)

142. It is sometimes useful to specify a class from which no objects will ever be created.
► True(Object-Oriented Programming in C++)
► False

143. When a subclass specifies an alternative definition for an attribute or method of its superclass, it is_____the definition in the superclass.

► overload
► overriding (Page 34)
► copy riding
► none of given

144. The mechanism of selecting function at run time according to the nature of calling object is called,
► late binding
► static binding
► virtual binding
► None of the given options (Page 227)

145. Dynamic binding means that target function for a call is selected at run time

Which one of the following functions returns the total number of elements in a vector.
► length();
► size(); (Page 318)
► ele();
► veclen();

AL JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

146. The main function of scope resolution operator (::) is,
- ► To define an object
- ► To define a data member
- ► To link the definition of an identifier to its declaration Google
- ► To make a class private

147. When is a constructor called?
- ► Each time the constructor identifier is used in a program statement
- ► During the instantiation of a new object (Object-Oriented Programming in C++)
- ► During the construction of a new class
- ► At the beginning of any program execution

148. Associativity can be changed in operator overloading.

- ► True
- ► False (Page 141)

149. A normal C++ operator that acts in special ways on newly defined data types is said to be

- ► glorified.
- ► encapsulated.
- ► classified.
- ► overloaded.

150. Which operator can not be overloaded?

- ► The relation operator (>=)
- ► Assignment operator (=)
- ► Script operator ([])
- ► Conditional operator (? :) (Page 141)

151. Suppose obj1 and obj2 are two objects of a user defined class A. An + operator is overloaded to add obj1 and obj2 using the function call obj1+obj2.
Identify the correct function prototype against the given call?
- ► A operator + (A &obj);
- ► int + operator(); (Page 143)
- ► int operator (plus) ();
- ► A operator(A &obj3);

ALJUNAID TECH INSTITUTE
0304 1659294
www.vulmshelp.com

152. Suppose that the Test class does not have an overloaded assignment operator. What happens when an assignment a=b; is given for two Test objects a and b?

- The automatic assignment operator is used
- **The copy constructor is used**
- Compiler error
- Run-time error

153. A static member function can be called, even when a class is not_____.

- Declared
- Define
- **Instantiated**
- Called

154. Identify which of the following overloaded operator function's declaration is appropriate for the given call?

Rational_number_1 + 2.325

Where Rational_number_1 is an object of user defined class Rational_number.

- Rational_number operator+(Rational_number & obj);
- Rational_number operator+(double& obj);
- Rational_number operator+(Rational_number &obj, double& num);
- operator+(double& obj);

155. What problem(s) may occur when we copy objects without using deep copy constructor?

- Dangling pointer
- Memory Leakage
- **All of the given**
- System crash

156._____provide the facility to access the data member.

- **accesser function**
- private function
- inline function
- None of the given

157. Constant objects cannot change their state,

- **True**
- False

`

158. The_____relationship indicates that an object contains other objects.

- None of given
- **'has-a'**
- 'is-a'
- 'be-

159. Which one of the following features of OOP is used to derive a class from another?

- Encapsulation
- Polymorphism
- Data hiding
- **Inheritance**

160._____**satisfy the condition of polymorphism**

- Carbon
- Diamond
- Coal
- **all of the given**

161.**A generalization-specialization relation between classes are implemented using**

- data hiding
- friend classes
- encapsulation
- **inheritance**

162.**The >= operator can be overloaded.**

- **True**
- False

163.**In order to free the memory occupied by the object, we use ----------**

- Constructor
- **Destructor**
- Shallow Copy
- Deep Copy

`

164. **Which of the following is not an example of multiple inheritances?**

- ► Mermaid
- ► **Woman**
- ► None of the given
- ► Amphibious Vehicle

165. **Static variable can be initialized more than once.**

- ► True
- ► **False**

166. **A generic class showing all the common attributes and a behavior of other classes represents a very important feature in oop called --------**

- ► **Inheritance**
- ► Encapsulation
- ► Polymarphism
- ► Abstraction

167. The parameters given in temple definition other than those used for mentioningtemplates types are called ____

a. Type Parameters
b. Non Type parameters
c. Default Type Parameters
d. None of the given option

168. Which of the following is the correct way to define a template class X?

e. Class < typename T>class X{}
f. Template class C{}
g. Template < typename T> classX{}
h. Typename<class T > class X{}

169. In resolution order of function template, compiler searches for_____in the end

a. Complete specialization
b. Partial specialization
c. Ordinary function
d. Generic template

170. __may inherit from a complete specialization

a. Partial specialization
b. Complete specialization
c. Ordinary class
d. All of given options

171. Which of the following may inherit form the ordinary class?

a. Class template

www.vulmshelp.com

b. Partial specialization
c. Complete specialization
d. All of given option

172. Which of the following is a sequence container?
a. Map
b. Vector
c. Set
d. multiset

173. Each_____of a template class by default becomes function template
a. Date member
b. Member function
c. Type parameter
d. Object

174. When we specialize a function template, it is called _____
a. Function overloading
b. Function overriding
c. Function template overloading
d. Function template overriding

175. Which of the following is NOT an associative container?
a. Set
b. Multiset
c. Map
d. Group

176A A_____is a pointer which is declared outside the _____
a. Container, cursor
b. Cursor, container
c. Container, class
d. This, cursor

177. Which of the following are collectively referred to as the first class containers?
a. Associative containers and containers adapters
b. Sequence containers and associative containers
c. Sequence containers and container adapters
d. None of the given options

178. _____class is a single class that provides functionality to operate on different type of data
a. Friend
b. Template
c. Ordinary
d. None of the give options

179. Which of the following can be passed as type argument to template?
a. Primitive type
b. User defined types

`

c. Both primitive type and user defined types
d. None of the given options

180. Which of the following is known as Dereference operator in C++?
a. +b.
*
c. &
d. ::

181. We can change behavior of template using_____
a. Template parameters
b. Function parameters
c. Class Templates
d. none of the given options

182. Which of the following represents partial specialization?
a. Template < int, char >
b. Template < class T, Type t>
c. Template<class T, Class U, int>
d. Template< typename T, class W>

183. When we declare template function as friend of any class, then which of the followingspecialization(s) of template function also become friend of the class granting friendship?
a. Explicit
b. Implicit
c. Partial
d. All of the given options

184. In statement "template <class T, class U, int I = 5>" then non-type parameters is
a. Class T
b. Class U
c. int I
d. All of the given option

185. Which of the following is the correct syntax for passing two type argument to atemplate?
a. template < type T, type U>
b. template < typename T, typename U>
c. template <type T,U>
d. template Typename < T,U >

186. Which of the following represents complete specialization?a. Template < int, char >
b. Template < class T, float>
c. Template<class T, Class U , int>
d. Template< class , class W>

187. Which of the following is/are the key component of standard template library?
a. Containers
b. Iterators

`

c. Algorithm
d. All of the given option

188. Which of the following statement is true about partial specialization?
a. Function templates cannot have partial specialization
b. Class template cannot have partial specialization
c. Both class templates and function template can have partial specialization.
d. None of the given

189. Which of the following is correct code portion to initialize static data member "value"of int type for class X?
a. Template <class T> X::value =0;
b. Template<class T>int X::value =0;
c. Template<class T> int X < T >::value = 0;
d. Template<class T> X < T >::value =0;

190. When we want to have exactly identical operations on different data type, _____are used
a. Function Template
b. Function Overloading
c. Function Overriding
d. None of the given option

191. In order to define a class template the first line of definition must be:a. Template <typename T>
b. typename < template T>
c. Template class <ClassName>
d. Class <Template T>

192. In case of template specialization, if compiler cannot find required completespecialization then it searches for some _
a. General template
b. Complete template
c. Partial specialization
d. None of the given

193. Which of the following is the correct code to instantiate the object of given templatevector class of int type?

```
Template <class T>
class vector
{
};
```

a. Vector obj <int>;
b. Vector obj <> int;
c. Vector <int>obj;
c. Vector int obj;

194. Which of the following types of container is NOT provided by standard TemplateLibrary?
a. Sequence containers

`

b. Structured containers
c. Associative containers
d. Container adapters

195. A class template _____
a. Facilitates rescue of class
b. Dose not facilitates reuse of class
c. Does not support generic methods
d. Does not support static members

196. Suppose a template class "test" has a static data members, how many copies of thisstatic data member will be created when template class is instantiated by writing following line of C++ code?

```
Test <char>x,y,z
```

a. Zero
b. One Correct
c. Two

197. STL stands for:
a. Stack Template Library
b. Standard Template Library
c. Structured Template Library
d. Secure Template Library

198. Which of the following is not a sequence container?
a. Vector
b. Dequec. Map
d. List

199. Suppose a template class "test" has a static data members, how many copies of thisstatic data member will be created when template class is instantiated by writing following line of C++ code?

```
Test <int>x; Test <char>y;
```

a. Zero
b. Onec. Two
d. Three

200. Suppose we have a template class "Test" which have a static data member "data"which is initialized with 0. What will be output when following code is written inmain() function for given class?

```
Test<int> a, b;a.data = 5;
b.data = 7; cout<<a.data;
```

a. 0
b. 1c. 5
d. 7 Correct

201. In resolution order, highest priority is given to_____in template specialization.
a. General template
b. Complete specialization
c. Partial specialization
d. None of the given options

`

ALHAMD TECH INSTITUTE
0304-1659294
www.vulmshelp.com

202. Which of the following is a constrained version of some first-class container?

a. Sequence containers
b. Associative containers
c. Container Adapter
d. None of the given options

Question No: 1 (Marks: 1) - Please choose one
Which of the following causes run time binding?
► Declaring object of abstract class
► Declaring pointer of abstract class
► Declaring overridden methods as non- virtual (Page 226)
► None of the given
Question No: 2 (Marks: 1) - Please choose one Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?

► Templates (Page 256)
► Overloading
► Data hiding
► Encapsulation
Question No: 3 (Marks: 1) - Please choose one A copy constructor is invoked when

► a function do not returns by value.
► an argument is passed by value. (Page 78)
► a function returns by reference.
► an argument is passed by reference.
Question No: 4 (Marks: 1) - Please choose one Like template functions, a class template may not handle all the types successfully

. ► True (Page 258)
► False 2
Question No: 6 (Marks: 1) - Please choose one A class template may inherit from another class template
.
► True (Page 288)
► False
Question No: 7 (Marks: 1) - Please choose one By default the vector data items are initialized to ____

► 0 Click here for detail
► 0.0
► 1
► null

`

Question No: 8 (Marks: 1) - Please choose one In Private ----------only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object

► specialization
► inheritance (Page 216)
► abstraction
► composition Question

No: 9 (Marks: 1) - Please choose one Which of the following is/are advantage[s] of generic programming?

► Reusability
► Writability
► Maintainability
► All of given (Page 256)

Question No: 10 (Marks: 1) - Please choose one Template functions use_____than ordinary functions

. ► Greater Memory
► Lesser Memory
► Equal Memory
► None of the given options

Question No: 11 (Marks: 1) - Please choose one Non Template Friend functions of a class are friends of_____instance/s of that class.
► All Click here for detail
► One specific
► All instances of one date type
► None of the given options 3
Question No: 12 (Marks: 1) - Please choose one A copy constructor is invoked when
► a function do not returns by value.
► an argument is passed by value. (Page 78)
► a function returns by reference.
► an argument is passed by reference
. Question No: 13 (Marks: 1) - Please choose one A pointer to a base class can point to objects of a derived class.
► True Click here for detail
► False
Question No: 14 (Marks: 1) - Please choose one A template argument is preceded by the keyword _____.
► vector
► class Click here for Detail
► template
► type*

`

Question No: 15 (Marks: 1) - Please choose one Which one of the following terms must relate to polymorphism?

► Static allocation
► Static typing
► Dynamic binding (Page 239)
► Dynamic allocation

Question No: 16 (Marks: 1) - Please choose one Multiple inheritance can be of type

► Public
► Private
► Protected
► All of the given Click here for detail

Question No: 17 (Marks: 1) - Please choose one Assume a class Derv that is privately derived from class Base. An object of class Derv located in main() can access

► public members of Derv. (Object-Oriented Programming in C++)
► protected members of Derv.
► private members of Derv.
► protected members of Base. 4

Question No: 18 (Marks: 1) - Please choose one A copy constructor is invoked when

► a function do not returns by value.
► an argument is passed by value. (Page 78) (rep)
► a function returns by reference.
► an argument is passed by reference.

Question No: 19 (Marks: 1) - Please choose one A function call is resolved at run-time in_____

► non-virtual member function.
► virtual member function. (Page 239)
► Both non-virtual member and virtual member function.
► None of given

Question No: 20 (Marks: 1) - Please choose one Two important STL associative containers are _____and_____.

► set,map (Object-Oriented Programming in C++)
► sequence,mapping
► setmet,multipule
► sit,mat

Question No: 21 (Marks: 1) - Please choose one An abstract class is useful when,

► We do not derive any class from it.
► There are multiple paths from one derived class to another.
► We do not want to instantiate its object. (Object-Oriented Programming in C++)
► You want to defer the declaration of the class.

Question No: 22 (Marks: 1) - Please choose one Which of the following is/are advantage[s] of generic programming?

► Reusability
► Writability
► Maintainability
► All of given (Page 256) rep

Question No: 23 (Marks: 1) - Please choose one By default the vector data items are initialized to ____

`

► 0 Click here for Detail rep
► 0.0
► 1
► null 5
Question No: 24 (Marks: 1) - Please choose one Suppose you create an uninitialized vector as follows: vector evec; After adding the statment, evec.push_back(21); what will happen?
► The following statement will add an element to the start (the back) of evec and will initialize it with the value 21.
► The following statement will add an element to the center of evec and will reinitialize it with the value 21.
► The following statement will delete an element to the end (the back) of evec and will reinitialize it with the value 21.
► The following statement will add an element to the end (the back) of evec and initialize it with the value 21. Click here for detail
Question No: 25 (Marks: 1) - Please choose one Default constructor is such constructor which either has no ---------or if it has some parameters these have---------- values
► Parameter, temporary
► Null, Parameter
► Parameter, default (Page 75)
► non of the given
Question No: 26 (Marks: 1) - Please choose one Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?
► Templates (Page 256) rep
► Overloading
► Data hiding
► Encapsulation
Question No: 27 (Marks: 1) Describe the way to declare a template class as a friend of any class.
Question No: 28 (Marks: 1) Classes like TwoDimensionalShape and ThreeDimensionalShape would normally be concrete, while classes like Sphere and Cube would normally be abstract.
► True Click here for detail
► False 6
Question No: 29 (Marks: 1) In order to define a class template, the first line of definition must be:
► template (Page 257)
► typename
► Template Class
► Class
Question No: 30 (Marks: 1) In case of multiple inheritance a derived class inherits,
► Only the public member functions of its base classes
► Only the public data members of its base classes
► Both public data members and member functions of all its base classes Click here for detail
► Data members and member functions of any two base classes
Question No: 31 (Marks: 1) In Private---------- only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object
► specialization
► inheritance (Page 216) rep

`

► abstraction
► composition
Question No: 32 (Marks: 1) Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?
► Templates (Page 256) rep
► Overloading
► Data hiding
► Encapsulation
FINALTERM EXAMINATION Spring 2010 CS304- Object Oriented Programming
Question No: 1 (Marks: 1) - Please choose one A template argument is preceded by the keyword _____.
► vector
► class (Object-Oriented Programming in C++)
► template
► type*
Question No: 2 (Marks: 1) - Please choose one Which of the following causes run time binding?
► Declaring object of abstract class
► Declaring pointer of abstract class
► Declaring overridden methods as non-virtual (Page 226)
► None of the given
Question No: 3 (Marks: 1) - Please choose one A function template can not be overloaded by another function template.
► True (Object-Oriented Programming in C++)
► False
Question No: 4 (Marks: 1) - Please choose one Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?
► Templates (Page 256) rep
► Overloading
► Data hiding
► Encapsulation Question No: 5 (Marks: 1) - Please choose one Identify the correct way of declaring an object of user defined template class A for char type members?
► A< char > obj; (Object-Oriented Programming in C++)
► A obj;
► A obj;
► Obj A;
Question No: 6 (Marks: 1) - Please choose one The user must define the operation of the copy constructor.
► True Click here for detail
► False
Question No: 7 (Marks: 1) - Please choose one Template functions use_____than ordinary functions.
► Greater Memory
► Lesser Memory
► Equal Memory
► None of the given options

`

Question No: 8 (Marks: 1) - Please choose one The find() algorithm
► finds matching sequences of elements in two containers.
► finds a container that matches a specified container.
► takes iterators as its first two arguments. (Object-Oriented Programming in C++)
► takes container elements as its first two arguments.
Question No: 9 (Marks: 1) - Please choose one Compiler performs _____type checking to diagnose type errors,
► Static (Page 261)
► Dynamic
► Bound
► Unbound
Question No: 10 (Marks: 1) - Please choose one Which of the following is/are advantage[s] of generic programming?
► Reusability
► Writability
► Maintainability
► All of given (Page 256) rep
Question No: 11 (Marks: 1) - Please choose one Vectors contain contiguous elements stored as a[an]___.
► variable
► array (Page 306)
► function
► datatype
Question No: 12 (Marks: 1) - Please choose one Suppose you create an uninitialized vector as follows: vector evec; After adding the statment, evec.push_back(21); what will happen?
► The following statement will add an element to the start (the back) of evec and will initialize it with the value 21.
► The following statement will add an element to the center of evec and will reinitialize it with the value 21.
► The following statement will delete an element to the end (the back) of evec and will reinitialize it with the value 21.
► The following statement will add an element to the end (the back) of evec and initialize it with the value 21. Click here for detail
Question No: 13 (Marks: 1) - Please choose one In a de-queue, (chose the best option)
► data can be quickly inserted or deleted at any arbitrary location.
► data can be inserted or deleted at any arbitrary location, but the process is relatively slow. (Object-Oriented Programming in C++)
► data can not be quickly inserted or deleted at either end.
► data can be inserted or deleted at either end, but the process is relatively slow.
Question No: 14 (Marks: 1) - Please choose one Algorithms can only be implemented using STL containers.
► True
► False (Object-Oriented Programming in C++)
Question No: 15 (Marks: 1) - Please choose one What is a class?
► A class is a section of computer memory containing objects.
► A class is a section of the hard disk reserved for object oriented programs
► A class is the part of an object that contains the variables.

`

► A class is a description of a kind of object. Click here for detail
Question No: 16 (Marks: 1) - Please choose one Inheritance is a way to
► organize data.
► pass arguments to objects of classes.
► add features to existing classes without rewriting them. (Page 27)
► improve data-hiding and encapsulation.
Question No: 17 (Marks: 1) - Please choose one We can use "this" pointer in the constructor in the body and even in the initialization list of any class if we are careful,
► True Click here for detail
► False
Question No: 18 (Marks: 1) - Please choose one_____and_____methods may not be declared abstract.
► private,static Click here for detail
► private,public
► static,public
► none of given
Question No: 19 (Marks: 1) - Please choose one Default constructor is such constructor which either has no ---------or if it has some parameters these have--------- values
► Parameter, temporary
► Null, Parameter
► Parameter, default (Page 75) rep
► non of the given
Question No: 20 (Marks: 1) - Please choose one Public methods of base class can----------be accessed in its derived class
► directly (Page 179)
► inderectly
► simultaniously
► non of the given
Question No: 21 (Marks: 1) - Please choose one The type that is used to declare a reference or pointer is called its ---------
► default type
► static type (Page 185)
► abstract type
► reference type
Question No: 22 (Marks: 1) - Please choose one ----------members are somewhere between public and private members. They are used in inheritance
► protected (Page 187)
► public
► private
► global
Question No: 23 (Marks: 1) - Please choose one Which of these are examples of error handling techniques?
► Abnormal Termination
► Graceful Termination
► Return the illegal
► all of the given (Page 329)

`

Question No: 24 (Marks: 1) - Please choose one ---------- follow try block to catch the object thrown
► catch block (Page 333)
► throw block
► main block
► non of the given
Question No: 25 (Marks: 1) - Please choose one Graphical representation of the classes and objects is called object model it shows -------
► Class Name only
► Class Name and attributes
► Relationships of the objects and classes
► all of the given Click here for detail
Question No: 26 (Marks: 1) - Please choose one Destructor can be overloaded
► True
► False (Page 92)
FINALTERM EXAMINATION Spring 2010 CS304- Object Oriented Programming (Session - 3) 12
Question No: 1 (Marks: 1) - Please choose one Which one of the following terms must relate to polymorphism?
► Static allocation
► Static typing
► Dynamic binding (Page 239)
► Dynamic allocation
Question No: 2 (Marks: 1) - Please choose one Which of the following causes run time binding?
► Declaring object of abstract class
► Declaring pointer of abstract class
► Declaring overridden methods as non-virtual (Page 226)
► None of the given
Question No: 3 (Marks: 1) - Please choose one Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?
► Templates (Page 256) rep
► Overloading
► Data hiding
► Encapsulation
Question No: 4 (Marks: 1) - Please choose one Which of the following is the best approach to implement generic algorithms with minimum number of coding lines?
► Templates (Page 256)
► Overloading
► Overriding
► Friend function/class
Question No: 5 (Marks: 1) - Please choose one Like template functions, a class template may not handle all the types successfully.
► True (Page 258) rep
► False
Question No: 6 (Marks: 1) - Please choose one A class template may inherit from another class template.
► True (Page 288) rep

`

► False 13

Question No: 7 (Marks: 1) - Please choose one Assume a class Derv that is privately derived from class Base. An object of class Derv located in main() can access

► public members of Derv. (Object-Oriented Programming in C++) rep

► protected members of Derv.

► private members of Derv.

► protected members of Base

. Question No: 8 (Marks: 1) - Please choose one A copy constructor is invoked when

► a function do not returns by value.

► an argument is passed by value. (Page 78) rep

► a function returns by reference.

► an argument is passed by reference.

Question No: 9 (Marks: 1) - Please choose one Each try block can have _____no. of catch blocks.

► 1

► 2

► 3

► As many as necessary. Click here for detail A single try block can have multiple catch blocks but only one finally block.

Question No: 10 (Marks: 1) - Please choose one class DocElement { public: virtual void Print() { cout << "Generic element"; } }; class Heading : public DocElement { public: void Print() { cout << "Heading element"; } }; class Paragraph : public DocElement { public: void Print() { cout << "Paragraph element"; } }; void main() { DocElement * p = new Paragraph(); 14 p->Print(); } When you run this program, it will print out a single line to the console output. What will be in that line? Select one correct answer from the following list:

► Generic element

► Heading element

► Paragraph element

► Nothing will be printed.

Question No: 11 (Marks: 1) - Please choose one Suppose we have two derived classes from a single class, can we write a method with same name in both these derived classes? Choose the best option.

► No

► Only if the two classes have the same name (Page 204)

► Only if the main program does not declare both kinds

► Yes

Question No: 12 (Marks: 1) - Please choose one When a virtual function is called by referencing a specific object by name and using the dot member selection operator (e.g., squareObject.draw()), the reference is resolved at compile time.

► True

► False (Object-Oriented Programming in C++) (the function invocation is resolved at compile time)

Question No: 13 (Marks: 1) - Please choose one Considering the resolution order in which Considering the resolution order in which compiler search for functions in a program; the first priority is given to; the first priority is given to,

► general template

► partial specialization

► complete specialization

`

► ordinary function (Page 287)
Question No: 14 (Marks: 1) - Please choose one Vectors contain contiguous elements stored as a[an]___.
► variable
► array (Page 306) rep
► function
► datatype
Question No: 15 (Marks: 1) - Please choose one By default the vector data items are initialized to ____
► 0 Click here for detail rep
► 0.0
► 1
► null
Question No: 16 (Marks: 1) - Please choose one One purpose of an iterator in the STL is to connect algorithms and containers.
► True (Object-Oriented Programming in C++)
► False
Question No: 17 (Marks: 1) - Please choose one Algorithms can only be implemented using STL containers.
► True
► False (Object-Oriented Programming in C++) rep
Question No: 18 (Marks: 1) - Please choose one In_____, a base class can be replaced by its derived class,
► Sub-typing (Page 31)
► Super-typing
► Multiple-typing
► Restricted-typing
Question No: 19 (Marks: 1) - Please choose one this pointer does not point to current object of any class,
► True
► False Click here for detail The this pointer is a hidden pointer inside every class member function that points to the class object the member function is working with.
Question No: 20 (Marks: 1) - Please choose one Which of the following operator(s) take(s) one or no argument if overloaded?
► ++ (Page 162)
► -
► +
► All of the given
16 Question No: 21 (Marks: 1) - Please choose one Which of the following operators can not be overloaded?
► Scope resolution operator (::) (Page 141)
► Insertion operator (<<)
► Extraction operator (>>)
► The relation operator (>)
Question No: 22 (Marks: 1) - Please choose one The type that is used to declare a reference or pointer is called its ---------
► default type

`

► static type (Page 185) rep
► abstract type
► reference type
Question No: 23 (Marks: 1) - Please choose one ---------- members are somewhere between public and private members. They are used in inheritance
► protected (Page 187) rep
► public
► private
► global
Question No: 24 (Marks: 1) - Please choose one Which of these are examples of error handling techniques ?
► Abnormal Termination
► Graceful Termination
► Return the illegal
► all of the given (Page 329)
Question No: 25 (Marks: 1) - Please choose one_____"is a" relationship

► Inheritance (Page 25)
► Polymarphism
► abstraction ►
encapsulation
Question No: 26 (Marks: 1) - Please choose one Graphical representation of the classes and objects is called object model it shows -------
► Class Name only
► Class Name and attributes
► Relationships of the objects and classes
► all of the given Click here for detail rep
17 FINALTERM EXAMINATION Spring 2010 CS304- Object Oriented Programming (Session - 4)
Question No: 1 (Marks: 1) - Please choose one Classes like TwoDimensionalShape and ThreeDimensionalShape would normally be concrete, while classes like Sphere and Cube would normally be abstract.
► True Click here for detail rep
► False
Question No: 2 (Marks: 1) - Please choose one Virtual functions allow you to
► create an array of type pointer-to-base class that can hold pointers to derived classes.
► create functions that can never be accessed.
► group objects of different classes so they can all be accessed by the same function code.
► use the same function call to execute member functions of objects from different classes (Object-Oriented Programming in C++)
Question No: 3 (Marks: 1) - Please choose one .
► True Click here for detail rep
► False
Question No: 4 (Marks: 1) - Please choose one A copy constructor is invoked when
► a function do not returns by value.
► an argument is passed by value. (Page 78) rep
► a function returns by reference.
► an argument is passed by reference.

`

Question No: 5 (Marks: 1) - Please choose one Each try block can have _____no. of catch blocks.

► 1
► 2
► 3
► As many as necessary. Click here for detail rep

18 Question No: 6 (Marks: 1) - Please choose one Non Template Friend functions of a class are friends of_____instance/s of

► All Click here for detail rep
► One specific
► All instances of one date type
► None of the given options

Question No: 7 (Marks: 1) - Please choose one Template functions use_____than ordinary functions.

► Greater Memory
► Lesser Memory
► Equal Memory
► None of the given options

Question No: 8 (Marks: 1) - Please choose one The find() algorithm

► finds matching sequences of elements in two containers.
► finds a container that matches a specified container.
► takes iterators as its first two arguments. (Object-Oriented Programming in C++)
► takes container elements as its first two arguments.

Question No: 9 (Marks: 1) - Please choose one The copy() algorithm returns an iterator to

► the last element copied from.
► the last element copied to.
► the element one past the last element copied from.
► the element one past the last element copied to. (Object-Oriented Programming in C++)

Question No: 10 (Marks: 1) - Please choose one If you define a vector v with the default constructor, and define another vector w with a one-argument constructor to a size of 11, and insert 3 elements into each of these vectors with push_back(), then the size() member function will return _____ for v and_____for w.

► 11 for v and 3 for w.
► 0 for v and 0 for w.
► 0 for v and 3 for w.
► 3 for v and 11 for w. (Object-Oriented Programming in C++)

19 Question No: 11 (Marks: 1) - Please choose one Which is not the Advantage of inheritance?

► providing class growth through natural selection. (Object-Oriented Programming in C++)
► facilitating class libraries.
► avoiding the rewriting of code.
► providing a useful conceptual framework.

Question No: 12 (Marks: 1) - Please choose one class DocElement { public: virtual void Print() { cout << "Generic element"; } }; class Heading : public DocElement { public: void Print() { cout << "Heading element"; } }; class Paragraph : public DocElement { public: void Print() { cout << "Paragraph element"; } }; void main() { DocElement * p = new Paragraph(); p->Print(); } When you run this program, it will print out a single line to the console output. What will be in that line? Select one correct answer from the following list:

`

► Generic element
► Heading element
► Paragraph element
► Nothing will be printed. 20
Question No: 13 (Marks: 1) - Please choose one Which type of inheritance is being represented by the following statement, class X : public A, public B { ……};
► Single inheritance
► Multiple inheritance (Page 41)
► Double inheritance
► None of the given options
Question No: 14 (Marks: 1) - Please choose one When we write a class template the first line must be:
► template < class class_name>
► template < class data_type>
► template < class T > (Page 257) Here T can be replaced with any name but it is preferable.
► class class-name() class template
Question No: 15 (Marks: 1) - Please choose one Function templates should be used where code and behavior must be identical.
► True (Page 262)
► False
Question No: 16 (Marks: 1) - Please choose one Which of the following is/are advantage[s] of generic programming?
► Reusability
► Writability
► Maintainability
► All of given (Page 256) rep
Question No: 17 (Marks: 1) - Please choose one The specialization pattern after the name says that this specialization is to be used for every,
► data type
► meta type
► virtual type
► pointer type (Page 286)
Question No: 18 (Marks: 1) - Please choose one A range is often supplied to an algorithm by two _____values.
► italic
► iteration (Object-Oriented Programming in C++)
► iterator
► None of given 21
Question No: 19 (Marks: 1) - Please choose one Which of the following is an integral part of an object?
► State
► Behavior
► Unique identity
► All of the given (Page 12)
Question No: 20 (Marks: 1) - Please choose one Consider the following statement Cupboard has books What is the relationship between Cupboard and books?
► Composition

`

► Aggregation
► Inheritance
► None of the given options
Question No: 21 (Marks: 1) - Please choose one Which sentence clearly defines an object?
► one instance of a class. (Page 23)
► another word for a class.
► a class with static methods.
► a method that accesses class attributes.
Question No: 22 (Marks: 1) - Please choose one_____, which means if A declares B as its friend it does NOT mean that A can access private data of B. It only means that B can access all data of A.
► Friendship is one way only Click here for detail
► Friendship is two way only
► NO Friendship between classes
► Any kind of friendship
Question No: 23 (Marks: 1) - Please choose one The statement objA=objB; will cause a compiler error if the objects are of different classes.
► True
► False (Object-Oriented Programming in C++) 22
Question No: 24 (Marks: 1) - Please choose one Consider the call given below of an overloaded operator "+", Rational_number_1 + Rational_number_2 Where Rational_number_1 and Rational_number_2 are the two objects of Rational_number class (a user defined class). Identify which of the above two objects will be passed as an argument to the overloaded operator function?
► Rational_number_1
► Rational_number_2
► Both Rational_number_1 & Rational_number_2
► any of the two objects, randomly
Question No: 25 (Marks: 1) - Please choose one If a class D has been derived using protected inheritance from class B (If B is a protected base and D is derived class) then public and protected members of B--------- accessed by member functions and friends of class D and classes derived from D
► can be Click here for detail
► cannot be
► does restirct to be
► not given
Question No: 26 (Marks: 1) - Please choose one In Private ----------only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object
► specialization
► inheritance (Page 216) rep
► abstraction
► composition
FINAL TERM EXAMINATION SPRING 2010
Question No: 1 (Marks: 1) - Please choose one Classes like TwoDimensionalShape and ThreeDimensionalShape would normally be concrete, while classes like Sphere and Cube would normally be abstract.
► True Click here for detail rep

`

► False 23
Question No: 2 (Marks: 1) - Please choose one Each try block can have _____no. of catch blocks.
► 1
► 2
► 3
► As many as necessary. Click here detail
Question No: 3 (Marks: 1) - Please choose one Function templates should be used where code and behavior must be identical.
► True (Page 262)
► False
Question No: 4 (Marks: 1) - Please choose one Consider the following statement Cupboard has books What is the relationship between Cupboard and books?
► Composition
► Aggregation
► Inheritance
► None of the given options
Question No: 5 (Marks: 1) - Please choose one Identify the correct way of declaring an object of user defined template class A for char type members?
► A< char > obj; (Object-Oriented Programming in C++)
► A obj;
► A obj;
► Obj A;
Question No: 6 (Marks: 1) - Please choose one The user must define the operation of the copy constructor.
► True Click here for detail
► False
Question No: 7 (Marks: 1) - Please choose one Default constructor is such constructor which either has no ---------or if it has some parameters these have--------- values
► Parameter, temporary
► Null, Parameter
► Parameter, default (Page 75) rep
► non of the given
Question No: 8 (Marks: 1) - Please choose one The type that is used to declare a reference or pointer is called its ---------
► default type
► static type (Page 185)
► abstract type
► reference type
24 Question No: 9 (Marks: 1) - Please choose one How the information hidden within an object can be accessed?
► Through its interface
► Through its private data members
► Through its private member functions
► Through both public and private members
Question No: 10 (Marks: 1) - Please choose one The sub-object's life is not dependant on the life of master class in_____.

`

► Separation
► Composition
► Aggregation (Page 134)
► None of the given
Question No: 11 (Marks: 1) - Please choose one Encapsulation means Select correct option:
►Extending the behaviour of class in another class
►Data and behaviour are tightly coupled within an entity (Page 16)
►One entity takes all the attributes and operations of the other
►Taking out the common features and put those in a separate class
Question No: 12 (Marks: 1) - Please choose one Algorithms can only be implemented using STL containers.
► True
► False (Object-Oriented Programming in C++) rep
Question No: 13 (Marks: 1) - Please choose one When we write a class template the first line must be:
► template < class class_name>
► template < class data_type>
► template < class T > (Page 257) rep Here T can be replaced with any name but it is preferable.
► class class-name() class template
Question No: 14 (Marks: 1) - Please choose one An STL container can not be used to,
► hold objects of class employee.
► store elements in a way that makes them quickly accessible.
► compile c++ programs. (Object-Oriented Programming in C++)
► organize the way objects are stored in memory 25
Question No: 15 (Marks: 1) - Please choose one_____, which means if A declares B as its friend it does NOT mean that A can access private data of B. It only means that B can access all data of A.
► Friendship is one way only Click here for detail
► Friendship is two way only
► NO Friendship between classes
► Any kind of friendship
Question No: 16 (Marks: 1) - Please choose one Which of the following may not be an integral part of an object?
► State
► Behavior
► Protected data members (Page 12)
► All of given
Question No: 17 (Marks: 1) - Please choose one Public methods of base class can----------be accessed in its derived class
► directly (Page 179) rep
► inderectly
► simultaneously
► non of the given
Question No: 18 (Marks: 1) - Please choose one If a class D has been derived using protected inheritance from class B (If B is a protected base and D is derived class) then public and protected members of B--------- accessed by member functions and friends of class D and classes derived from D

`

► can be Click here for detail
► cannot be
► does restirct to be
► not given
Question No: 19 (Marks: 1) - Please choose one What is true about function templates?
► The compiler generates only one copy of the function template
► The compiler generates a copy of function respective to each type of data (Page 256)
► The compiler can only generate copy for the int type data
► None of the given.
Question No: 20 (Marks: 1) - Please choose one Which of the following is an integral part of an object?
► State
► Behavior
► Unique identity
► All of the given (Page 12) rep 26
Question No: 21 (Marks: 1) - Please choose one When the base class and the derived class have a member function with the same name, you must be more specific which function you want to call (using _____).
► scope resolution operator
► dot operator
► null operator
► Operator overloading Click here for detail
FINALTERM EXAMINATION Fall 2009 CS304- Object Oriented Programming (Session - 4)
Question No: 1 (Marks: 1) - Please choose one A template provides a convenient way to make a family of
► variables and data members
► functions and classes (Object-Oriented Programming in C++)
► classes and exceptions
► programs and algorithms
Question No: 2 (Marks: 1) - Please choose one Which one of the following terms must relate to polymorphism?
► Static allocation
► Static typing
► Dynamic binding (Page 239) rep
► Dynamic allocation
Question No: 3 (Marks: 1) - Please choose one What is true about function templates?
► The compiler generates only one copy of the function template
► The compiler generates a copy of function respective to each type of data (Page 256) rep
► The compiler can only generate copy for the int type data
► None of the given.
Question No: 4 (Marks: 1) - Please choose one Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?
► Templates (Page 256) rep
► Overloading
► Data hiding
► Encapsulation 27

`

Question No: 5 (Marks: 1) - Please choose one template <> class Vector { } This is an example of partial specialization.
► True (Page 287)
► False
Question No: 6 (Marks: 1) - Please choose one Classes like TwoDimensionalShape and ThreeDimensionalShape would normally be concrete, while classes like Sphere and Cube would normally be abstract.
► True Click here for detail rep
► False
Question No: 7 (Marks: 1) - Please choose one A non-virtual member function is defined in a base class and overridden in a derived class; if that function is called through a base-class pointer to a derived class object, the derived-class version is used.
► True
► False
Question No: 8 (Marks: 1) - Please choose one Assume a class Derv that is privately derived from class Base. An object of class Derv located in main() can access
► public members of Derv. (Object-Oriented Programming in C++) rep
► protected members of Derv.
► private members of Derv.
► protected members of Base.
Question No: 9 (Marks: 1) - Please choose one In order to define a class template, the first line of definition must be:
► template (Page 257) rep
► typename
► Template Class
► Class 28
Question No: 10 (Marks: 1) - Please choose one If there is a pointer p to objects of a base class, and it contains the address of an object of a derived class, and both classes contain a nonvirtual member function, ding(), then the statement p->ding(); will cause the version of ding() in the _____ class to be executed.
► Base (Object-Oriented Programming in C++)
► Derived
► Abstract
► virtual
Question No: 11 (Marks: 1) - Please choose one When the base class and the derived class have a member function with the same name, you must be more specific which function you want to call (using _____).
► scope resolution operator
► dot operator
► null operator
► Operator overloading Click here for detail
Question No: 12 (Marks: 1) - Please choose one Non Template Friend functions of a class are friends of_____instance/s of that class.
► All Click here for detail
► One specific
► All instances of one date type
► None of the given options

`

Question No: 13 (Marks: 1) - Please choose one The find() algorithm
► finds matching sequences of elements in two containers.
► finds a container that matches a specified container.
► takes iterators as its first two arguments. (Object-Oriented Programming in C++) rep
► takes container elements as its first two arguments.
Question No: 14 (Marks: 1) - Please choose one If you define a vector v with the default constructor, and define another vector w with a one-argument constructor to a size of 11, and insert 3 elements into each of these vectors with push_back(), then the size() member function will return _____ for v and_____for w.
► 11 for v and 3 for w.
► 0 for v and 0 for w.
► 0 for v and 3 for w.
► 3 for v and 11 for w. (Object-Oriented Programming in C++) rep 29
Question No: 15 (Marks: 1) - Please choose one Which of the following may not be an integral part of an object?
► State
► Behavior
► Protected data members (Page 12)
► All of given
Question No: 16 (Marks: 1) - Please choose one Which is not the Advantage of inheritance?
► providing class growth through natural selection. (Object-Oriented Programming in C++) rep
► facilitating class libraries.
► avoiding the rewriting of code
► providing a useful conceptual framework.
Question No: 17 (Marks: 1) - Please choose one 1class DocElement { public: virtual void Print() { cout << "Generic element"; } }; 2class Heading : public DocElement { public: void Print() { cout << "Heading element"; } }; 3class Paragraph : public DocElement { public: void Print() { cout << "Paragraph element"; } }; void main() { DocElement * p = new Paragraph(); p->Print(); } When you run this program, it will print out a single line to the console output. 30 What will be in that line? Select one correct answer from the following list:
► Generic element
► Heading element
► Paragraph element
► Nothing will be printed.
Question No: 18 (Marks: 1) - Please choose one When a virtual function is called by referencing a specific object by name and using the dot member selection operator (e.g., squareObject.draw()), the reference is resolved at compile time
. ► True
► False (Object-Oriented Programming in C++) (the function invocation is resolved at compile time)
Question No: 19 (Marks: 1) - Please choose one In case of multiple inheritance a derived class inherits,
► Only the public member functions of its base classes
► Only the public data members of its base classes
► Both public data members and member functions of all its base classes Click here for detail
► Data members and member functions of any two base classes
Question No: 20 (Marks: 1) - Please choose one When we write a class template the first line must be:

`

► template < class class_name>
► template < class data_type>
► template < class T > (Page 257) rep Here T can be replaced with any name but it is preferable.
► class class-name() class template
Question No: 21 (Marks: 1) - Please choose one Which of the following is incorrect line regarding function template?
► template
► template
► Class (Page 257)
► template < class T, class U> 31
Question No: 22 (Marks: 1) - Please choose one An STL container can not be used to,
► hold objects of class employee.
► store elements in a way that makes them quickly accessible.
► compile c++ programs. (Object-Oriented Programming in C++) rep
► organize the way objects are stored in memory
Question No: 23 (Marks: 1) - Please choose one Algorithms can only be implemented using STL containers.
► True
► False (Object-Oriented Programming in C++) rep
Question No: 24 (Marks: 1) - Please choose one Consider a class named Vehicle, which of the following can be the instance of class Vehicle? 1. Car 2. Computer 3. Desk 4. Ahmed 5. Bicycle 6. Truck
► 1, 4, 5
► 2, 5, 6
► 1, 2, 3, 6
► 1, 5, 6 (correct)
Question No: 25 (Marks: 1) - Please choose one Consider the code below, class Fred { public: Fred(); ... }; int main() { Fred a[10]; Fred* p = new Fred[10]; ... } Select the best option,
► Fred a[10]; calls the default constructor 09 times Fred* p = new Fred[10]; calls the default constructor 10 times
► Produce an error 32
► Fred a[10]; calls the default constructor 11 times Fred* p = new Fred[10]; calls the default constructor 11 times
► Fred a[10]; calls the default constructor 10 times Fred* p = new Fred[10]; calls the default constructor 10 times Click here for detail
Question No: 26 (Marks: 1) - Please choose one When a variable is define as static in a class then all object of this class,
► Have different copies of this variable
► Have same copy of this variable (Page 110)
► Can not access this variable
► None of given
Question No: 27 (Marks: 1) - Please choose one The life of sub object is dependant on the life of master class in_____.
► Separation
► Composition (Page 53)
► Aggregation
► None of the given

`

Question No: 28 (Marks: 1) - Please choose one_____, which means if A declares B as its friend it does NOT mean that A can access private data of B. It only means that B can access all data of A.

► Friendship is one way only Click here for detail
► Friendship is two way only
► NO Friendship between classes
► Any kind of friendship

Question No: 29 (Marks: 1) - Please choose one Which of the following operators always takes no argument if overloaded?

► /
► -
► +
► ++ (Page 162)

Question No: 30 (Marks: 1) - Please choose one In Private ----------only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object

► specialization
► inheritance (Page 216) rep
► abstraction
► composition

33 FINALTERM EXAMINATION Fall 2009 CS304- Object Oriented Programming (Session - 1)

Question No: 1 (Marks: 1) - Please choose one Which one of the following terms must relate to polymorphism?

► Static allocation
► Static typing
► Dynamic binding (Page 239) rep
► Dynamic allocation

Question No: 2 (Marks: 1) - Please choose one Multiple inheritance can be of type

► Public
► Private
► Protected
► All of the given Click here for detail

Question No: 3 (Marks: 1) - Please choose one When a subclass specifies an alternative definition for an attribute or method of its superclass, it is_____the definition in the superclass.

► overload
► overriding (Page 34)
► copy riding
► none of given

Question No: 4 (Marks: 1) - Please choose one Like template functions, a class template may not handle all the types successfully.

► True (Page 258) rep
► False

Question No: 5 (Marks: 1) - Please choose one It is sometimes useful to specify a class from which no objects will ever be created.

► True (Object-Oriented Programming in C++)
► False

`

Question No: 6 (Marks: 1) - Please choose one Assume a class Derv that is privately derived from class Base. An object of class Derv located in main() can access

► public members of Derv. (Object-Oriented Programming in C++) rep

► protected members of Derv.

► private members of Derv.

► protected members of Base. 34

Question No: 7 (Marks: 1) - Please choose one A pointer to a base class can point to objects of a derived class. \

► True Click here for detail

► False

Question No: 8 (Marks: 1) - Please choose one A copy constructor is invoked when

► a function do not returns by value.

► an argument is passed by value. (Page 78) rep

► a function returns by reference.

► an argument is passed by reference.

Question No: 9 (Marks: 1) - Please choose one A function call is resolved at run-time in_____

► non-virtual member function.

► virtual member function. (Page 239) rep

► Both non-virtual member and virtual member function.

► None of given

Question No: 10 (Marks: 1) - Please choose one When the base class and the derived class have a member function with the same name, you must be more specific which function you want to call (using _____).

► scope resolution operator

► dot operator

► null operator

► Operator overloading Click here for detail

Question No: 11 (Marks: 1) - Please choose one Each try block can have_____no. of catch blocks.

► 1

► 2

► 3

► As many as necessary. Click here for detail rep

Question No: 12 (Marks: 1) - Please choose one Two important STL associative containers are _____and_____.

► set,map (Object-Oriented Programming in C++) rep

► sequence,mapping

► setmet,multipule

► sit,mat 35

Question No: 13 (Marks: 1) - Please choose one The mechanism of selecting function at run time according to the nature of calling object is called,

► late binding

► static binding

► virtual binding

► None of the given options (Page 227) Dynamic binding means that target function for a call is selected at run time

`

Question No: 14 (Marks: 1) - Please choose one An abstract class is useful when
► We do not derive any class from it.
► There are multiple paths from one derived class to another.
► We do not want to instantiate its object. (Object-Oriented Programming in C++) rep
► You want to defer the declaration of the class.
Question No: 15 (Marks: 1) - Please choose one Which of the following is incorrect line regarding function template?
► template
► template
► Class (Page 257) rep
► template < class T, class U>
Question No: 16 (Marks: 1) - Please choose one Which of the following is/are advantage[s] of generic programming?
► Reusability
► Writability
► Maintainability
► All of given (Page 256) rep
Question No: 17 (Marks: 1) - Please choose one By default the vector data items are initialized to ____
► 0 Click here for detail rep
► 0.0
► 1
► null
Question No: 18 (Marks: 1) - Please choose one Which one of the following functions returns the total number of elements in a vector.
► length();
► size(); (Page 318)
► ele();
► veclen();
36 Question No: 19 (Marks: 1) - Please choose one Suppose you create an uninitialized vector as follows: vector evec; After adding the statment, evec.push_back(21); what will happen?
► The following statement will add an element to the start (the back) of evec and will initialize it with the value 21.
► The following statement will add an element to the center of evec and will reinitialize it with the value 21.
► The following statement will delete an element to the end (the back) of evec and will reinitialize it with the value 21.
► The following statement will add an element to the end (the back) of evec and initialize it with the value 21. Click here for detail
Question No: 20 (Marks: 1) - Please choose one An STL container can not be used to,
► hold objects of class employee.
► store elements in a way that makes them quickly accessible.
► compile c++ programs. (Object-Oriented Programming in C++) rep
► organize the way objects are stored in memory
Question No: 21 (Marks: 1) - Please choose one Algorithms can only be implemented using STL containers.
► True

`

► False (Object-Oriented Programming in C++) rep
Question No: 22 (Marks: 1) - Please choose one The main function of scope resolution operator (::) is,
► To define an object
► To define a data member
► To link the definition of an identifier to its declaration Click here for detail
► To make a class private
Question No: 23 (Marks: 1) - Please choose one When is a constructor called?
► Each time the constructor identifier is used in a program statement
► During the instantiation of a new object (Object-Oriented Programming in C++)
► During the construction of a new class
► At the beginning of any program execution 37
Question No: 24 (Marks: 1) - Please choose one Consider the code below, class Fred { public: Fred(); ... }; int main() { Fred a[10]; Fred* p = new Fred[10]; ... } Select the best option,
► Fred a[10]; calls the default constructor 09 times Fred* p = new Fred[10];
► Produce an error
► Fred a[10]; calls the default constructor 11 times Fred* p = new Fred[10]; calls the default constructor 11 times
► Fred a[10]; calls the default constructor 10 times Fred* p = new Fred[10]; calls the default constructor 10 times Click here for detail
Question No: 25 (Marks: 1) - Please choose one Associativity can be changed in operator overloading.
► True
► False (Page 141)
Question No: 26 (Marks: 1) - Please choose one A normal C++ operator that acts in special ways on newly defined data types is said to be
► glorified.
► encapsulated.
► classified.
► overloaded. (Object-Oriented Programming in C++)
Question No: 27 (Marks: 1) - Please choose one Which operator can not be overloaded?
► The relation operator (>=)
► Assignment operator (=)
► Script operator ([])
► Conditional operator (? :) (Page 141) 38
Question No: 28 (Marks: 1) - Please choose one Suppose obj1 and obj2 are two objects of a user defined class A. An + operator is overloaded to add obj1 and obj2 using the function call obj1+obj2. Identify the correct function prototype against the given call?
► A operator + (A &obj);
► int + operator(); (Page 143)
► int operator (plus) ();
► A operator(A &obj3);
Question No: 29 (Marks: 1) - Please choose one Default constructor is such constructor which either has no ---------or if it has some parameters these have---------- values
► Parameter, temporary
► Null, Parameter
► Parameter, default (Page 75) rep

`

► non of the given

Question No: 30 (Marks: 1) - Please choose one Public methods of base class can----------be accessed in its derived class

► directly (Page 179) rep

► inderectly

► simultaneously

► non of the given

0304-1659294
www.vulmshelp.com


AL-JUNAID TECH INSTITUTE
AJ
0304-1659294
www.vulmshelp.com


AL-JUNAID TECH INSTITUTE
AJ
0304-1659294
www.vulmshelp.com

FINALTERM EXAMINATION
14 Feb, 2011
CS304- Object Oriented Programming (Session - 3)

Question No: 1 (Marks: 1) - Please choose one
Which of the following causes run time binding?

- ► Declaring object of abstract class
- ► Declaring pointer of abstract class
- ► **Declaring overridden methods as non-virtual (Page 226)**
- ► None of the given

Question No: 2 (Marks: 1) - Please choose one
Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?

- ► **Templates (Page 256)**
- ► Overloading
- ► Data hiding
- ► Encapsulation

Question No: 3 (Marks: 1) - Please choose one
A copy constructor is invoked when

- ► a function do not returns by value.
- ► **an argument is passed by value. (Page 78)**
- ► a function returns by reference.
- ► an argument is passed by reference.

Question No: 4 (Marks: 1) - Please choose one
Like template functions, a class template may not handle all the types successfully.

- ► **True (Page 258)**
- ► False

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 6 (Marks: 1) - Please choose one
A class template may inherit from another class template.

► **True (Page 288)**
► False

Question No: 7 (Marks: 1) - Please choose one
By default the vector data items are initialized to ____

► **0** **Click here for detail**
► 0.0
► 1
► null

Question No: 8 (Marks: 1) - Please choose one
In Private ---------- only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object

► specialization
► **inheritance (Page 216)**
► abstraction
► composition

Question No: 9 (Marks: 1) - Please choose one
Which of the following is/are advantage[s] of generic programming?

► Reusability
► Writability
► Maintainability
► **All of given (Page 256)**

Question No: 10 (Marks: 1) - Please choose one
Template functions use _____ than ordinary functions.

► Greater Memory
► **Lesser Memory**
► Equal Memory
► None of the given options

Question No: 11 (Marks: 1) - Please choose one
Non Template Friend functions of a class are friends of _____instance/s of that class.

► **All** **Click here for detail**
► One specific
► All instances of one date type
► None of the given options

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 12 (Marks: 1) - Please choose one
A copy constructor is invoked when
- ► a function do not returns by value.
- ► **an argument is passed by value. (Page 78)**
- ► a function returns by reference.
- ► an argument is passed by reference.

Question No: 13 (Marks: 1) - Please choose one
A pointer to a base class can point to objects of a derived class.

- ► **True** **Click here for detail**
- ► False

Question No: 14 (Marks: 1) - Please choose one
A template argument is preceded by the keyword _____.
- ► vector
- ► **class** **Click here for Detail**
- ► template
- ► type*

Question No: 15 (Marks: 1) - Please choose one
Which one of the following terms must relate to **polymorphism?**
- ► Static allocation
- ► Static typing
- ► **Dynamic binding (Page 239)**
- ► Dynamic allocation

Question No: 16 (Marks: 1) - Please choose one
Multiple inheritance can be of type

- ► Public
- ► Private
- ► Protected
- ► **All of the given** **Click here for detail**

Question No: 17 (Marks: 1) - Please choose one
Assume a class Derv that is privately derived from class Base. An object of class Derv located in main() can access
- ► **public members of Derv. (Object-Oriented Programming in C++)**
- ► protected members of Derv.
- ► private members of Derv.
- ► protected members of Base.

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 18 (Marks: 1) - Please choose one
A copy constructor is invoked when
- ► a function do not returns by value.
- ► **an argument is passed by value. (Page 78) (rep)**
- ► a function returns by reference.
- ► an argument is passed by reference.

Question No: 19 (Marks: 1) - Please choose one
A function call is resolved at run-time in_____
- ► non-virtual member function.
- ► **virtual member function. (Page 239)**
- ► Both non-virtual member and virtual member function.
- ► None of given

Question No: 20 (Marks: 1) - Please choose one
Two important STL associative containers are _____ and _____.

- ► **set,map (Object-Oriented Programming in C++)**
- ► sequence,mapping
- ► setmet,multipule
- ► sit,mat

Question No: 21 (Marks: 1) - Please choose one
An abstract class is useful when,

- ► We do not derive any class from it.
- ► There are multiple paths from one derived class to another.
- ► **We do not want to instantiate its object. (Object-Oriented Programming in C++)**
- ► You want to defer the declaration of the class.

Question No: 22 (Marks: 1) - Please choose one
Which of the following is/are advantage[s] of generic programming?

- ► Reusability
- ► Writability
- ► Maintainability
- ► **All of given (Page 256) rep**

Question No: 23 (Marks: 1) - Please choose one
By default the vector data items are initialized to ____
- ► **0** **Click here for Detail** **rep**
- ► 0.0
- ► 1
- ► null

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 24 (Marks: 1) - Please choose one
Suppose you create an uninitialized vector as follows:

vector<int> evec;
After adding the statment,
evec.push_back(21);
what will happen?

► The following statement will add an element to the start (the back) of evec and will initialize it with the value 21.

► The following statement will add an element to the center of evec and will reinitialize it with the value 21.

► The following statement will delete an element to the end (the back) of evec and will reinitialize it with the value 21.

► **The following statement will add an element to the end (the back) of evec and initialize it with the value 21.** **Click here for detail**

Question No: 25 (Marks: 1) - Please choose one
Default constructor is such constructor which either has no ---------or if it has some parameters these have ---------- values

► Parameter, temporary
► Null, Parameter
► **Parameter, default (Page 75)**
► non of the given

Question No: 26 (Marks: 1) - Please choose one
Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?

► **Templates (Page 256) rep**
► Overloading
► Data hiding
► Encapsulation

Question No: 27 (Marks: 1)
Describe the way to declare a template class as a friend of any class.

Question No: 28 (Marks: 1)
Classes like TwoDimensionalShape and ThreeDimensionalShape would normally be concrete, while classes like Sphere and Cube would normally be abstract.

► **True** **Click here for detail**
► False

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 29 (Marks: 1)
In order to define a class template, the first line of definition must be:

► **template <typename T> (Page 257)**
► typename <template T>
► Template Class <ClassName>
► Class <Template T>

Question No: 30 (Marks: 1)
In case of multiple inheritance a derived class inherits,

► Only the public member functions of its base classes
► Only the public data members of its base classes
► **Both public data members and member functions of all its base classes**
Click here for detail
► Data members and member functions of any two base classes

Question No: 31 (Marks: 1)
In Private ---------- only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object

► specialization
► **inheritance (Page 216) rep**
► abstraction
► composition

Question No: 32 (Marks: 1)
Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?

► **Templates (Page 256) rep**
► Overloading
► Data hiding
► Encapsulation

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

FINALTERM EXAMINATION
Spring 2010
CS304- Object Oriented Programming

Question No: 1 (Marks: 1) - Please choose one
A template argument is preceded by the keyword _____.

- ► vector
- ► **class (Object-Oriented Programming in C++)**
- ► template
- ► type*

Question No: 2 (Marks: 1) - Please choose one
Which of the following causes run time binding?

- ► Declaring object of abstract class
- ► Declaring pointer of abstract class
- ► **Declaring overridden methods as non-virtual (Page 226)**
- ► None of the given

Question No: 3 (Marks: 1) - Please choose one
A function template can not be overloaded by another function template.

- ► **True (Object-Oriented Programming in C++)**
- ► False

Question No: 4 (Marks: 1) - Please choose one
Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?

- ► **Templates (Page 256) rep**
- ► Overloading
- ► Data hiding
- ► Encapsulation

Question No: 5 (Marks: 1) - Please choose one
Identify the correct way of declaring an object of user defined template class **A** for char type members?

- ► **A< char > obj; (Object-Oriented Programming in C++)**
- ► <char>A obj;
- ► A obj<char>;
- ► Obj <char> A;

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 6 (Marks: 1) - Please choose one
The user must define the operation of the copy constructor.

► **True** **Click here for detail**
► False

Question No: 7 (Marks: 1) - Please choose one
Template functions use _____ than ordinary functions.

► Greater Memory
► **Lesser Memory**
► Equal Memory
► None of the given options

Question No: 8 (Marks: 1) - Please choose one
The find() algorithm

► finds matching sequences of elements in two containers.
► finds a container that matches a specified container.
► **takes iterators as its first two arguments. (Object-Oriented Programming in C++)**
► takes container elements as its first two arguments.

Question No: 9 (Marks: 1) - Please choose one
Compiler performs _____ type checking to diagnose type errors,

► **Static (Page 261)**
► Dynamic
► Bound
► Unbound

Question No: 10 (Marks: 1) - Please choose one
Which of the following is/are advantage[s] of generic programming?

► Reusability
► Writability
► Maintainability
► **All of given (Page 256) rep**

Question No: 11 (Marks: 1) - Please choose one
Vectors contain contiguous elements stored as a[an] ___.

► variable
► **array (Page 306)**
► function
► datatype

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 12 (Marks: 1) - Please choose one
Suppose you create an uninitialized vector as follows:

vector<int> evec;
After adding the statment,
evec.push_back(21);
what will happen?

► The following statement will add an element to the start (the back) of evec and will initialize it with the value 21.
► The following statement will add an element to the center of evec and will reinitialize it with the value 21.
► The following statement will delete an element to the end (the back) of evec and will reinitialize it with the value 21.
► The following statement will add an element to the end (the back) of evec and initialize it with the value 21. **Click here for detail**

Question No: 13 (Marks: 1) - Please choose one
In a de-queue, (chose the best option)

► data can be quickly inserted or deleted at any arbitrary location.
► data can be inserted or deleted at any arbitrary location, but the process is relatively slow. (Object-Oriented Programming in C++)
► data can not be quickly inserted or deleted at either end.
► data can be inserted or deleted at either end, but the process is relatively slow.

Question No: 14 (Marks: 1) - Please choose one
Algorithms can only be implemented using STL containers.

► True
► False (Object-Oriented Programming in C++)

Question No: 15 (Marks: 1) - Please choose one
What is a class?

► A class is a section of computer memory containing objects.
► A class is a section of the hard disk reserved for object oriented programs
► A class is the part of an object that contains the variables.
► A class is a description of a kind of object. **Click here for detail**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 16 (Marks: 1) - Please choose one
Inheritance is a way to

- ► organize data.
- ► pass arguments to objects of classes.
- ► **add features to existing classes without rewriting them. (Page 27)**
- ► improve data-hiding and encapsulation.

Question No: 17 (Marks: 1) - Please choose one
We can use **"this"** pointer in the constructor in the body and even in the initialization list of any class if we are careful,

- ► **True** **Click here for detail**
- ► False

Question No: 18 (Marks: 1) - Please choose one
_____ and _____ methods may not be declared abstract.

- ► **private,static** **Click here for detail**
- ► private,public
- ► static,public
- ► none of given

Question No: 19 (Marks: 1) - Please choose one
Default constructor is such constructor which either has no ---------or if it has some parameters these have -------- values

- ► Parameter, temporary
- ► Null, Parameter
- ► **Parameter, default (Page 75) rep**
- ► non of the given

Question No: 20 (Marks: 1) - Please choose one
Public methods of base class can --------- be accessed in its derived class

- ► **directly (Page 179)**
- ► inderectly
- ► simultaniously
- ► non of the given

Question No: 21 (Marks: 1) - Please choose one
The type that is used to declare a reference or pointer is called its ---------

- ► default type
- ► **static type (Page 185)**
- ► abstract type
- ► reference type

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 22 (Marks: 1) - Please choose one
---------- members are somewhere between public and private members. They are used in inheritance

► **protected (Page 187)**
► public
► private
► global

Question No: 23 (Marks: 1) - Please choose one
Which of these are examples of error handling techniques?

► Abnormal Termination
► Graceful Termination
► Return the illegal
► **all of the given (Page 329)**

Question No: 24 (Marks: 1) - Please choose one
---------- follow try block to catch the object thrown

► **catch block (Page 333)**
► throw block
► main block
► non of the given

Question No: 25 (Marks: 1) - Please choose one
Graphical representation of the classes and objects is called object model it shows -------

► Class Name only
► Class Name and attributes
► Relationships of the objects and classes
► **all of the given Click here for detail**

Question No: 26 (Marks: 1) - Please choose one
Destructor can be overloaded

► True
► **False (Page 92)**

FINALTERM EXAMINATION
Spring 2010
CS304- Object Oriented Programming (Session - 3)

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 1 (Marks: 1) - Please choose one
Which one of the following terms must relate to **polymorphism?**

- ► Static allocation
- ► Static typing
- ► **Dynamic binding (Page 239)**
- ► Dynamic allocation

Question No: 2 (Marks: 1) - Please choose one
Which of the following causes run time binding?

- ► Declaring object of abstract class
- ► Declaring pointer of abstract class
- ► **Declaring overridden methods as non-virtual (Page 226)**
- ► None of the given

Question No: 3 (Marks: 1) - Please choose one
Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?

- ► **Templates (Page 256) rep**
- ► Overloading
- ► Data hiding
- ► Encapsulation

Question No: 4 (Marks: 1) - Please choose one
Which of the following is the best approach to implement generic algorithms with minimum number of coding lines?

- ► **Templates (Page 256)**
- ► Overloading
- ► Overriding
- ► Friend function/class

Question No: 5 (Marks: 1) - Please choose one
Like template functions, a class template may not handle all the types successfully.

- ► **True (Page 258) rep**
- ► False

Question No: 6 (Marks: 1) - Please choose one
A class template may inherit from another class template.

- ► **True (Page 288) rep**
- ► False

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 7 (Marks: 1) - Please choose one
Assume a class Derv that is privately derived from class Base. An object of class Derv located in main() can access

- ► **public members of Derv. (Object-Oriented Programming in C++) rep**
- ► protected members of Derv.
- ► private members of Derv.
- ► protected members of Base.

Question No: 8 (Marks: 1) - Please choose one
A copy constructor is invoked when

- ► a function do not returns by value.
- ► **an argument is passed by value. (Page 78) rep**
- ► a function returns by reference.
- ► an argument is passed by reference.

Question No: 9 (Marks: 1) - Please choose one
Each try block can have _____ no. of catch blocks.

- ► 1
- ► 2
- ► 3
- ► **As many as necessary.** **Click here for detail**

A single `try` block can have multiple `catch` blocks but only one `finally` block.

Question No: 10 (Marks: 1) - Please choose one

```
class DocElement
{
public:
    virtual void Print() { cout << "Generic element"; }
};
class Heading : public DocElement
{
public:
    void Print() { cout << "Heading element"; }
};
class Paragraph : public DocElement
{
public:
    void Print() { cout << "Paragraph element"; }
};
void main()
{
    DocElement * p = new Paragraph();
```


Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

```
    p->Print();
}
```
When you run this program, it will print out a single line to the console output.

What will be in that line?

Select one correct answer from the following list:

- ► Generic element
- ► Heading element
- ► Paragraph element
- ► **Nothing will be printed.**

Question No: 11 (Marks: 1) - Please choose one
Suppose we have two derived classes from a single class, can we write a method with same name in both these derived classes? Choose the best option.

- ► No
- ► **Only if the two classes have the same name (Page 204)**
- ► Only if the main program does not declare both kinds
- ► Yes

Question No: 12 (Marks: 1) - Please choose one
When a virtual function is called by referencing a specific object by name and using the dot member selection operator (e.g., squareObject.draw()), the reference is resolved at compile time.

- ► True
- ► **False (Object-Oriented Programming in C++)**

(the function invocation is resolved at compile time)

Question No: 13 (Marks: 1) - Please choose one
Considering the resolution order in which Considering the resolution order in which compiler search for functions in a program; the first priority is given to; the first priority is given to,

- ► general template
- ► partial specialization
- ► complete specialization
- ► **ordinary function (Page 287)**

Question No: 14 (Marks: 1) - Please choose one
Vectors contain contiguous elements stored as a[an] ___.

- ► variable
- ► **array (Page 306) rep**
- ► function
- ► datatype

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 15 (Marks: 1) - Please choose one
By default the vector data items are initialized to ____

► **0** **Click here for detail** rep
► 0.0
► 1
► null

Question No: 16 (Marks: 1) - Please choose one
One purpose of an iterator in the STL is to connect algorithms and containers.

► **True** **(Object-Oriented Programming in C++)**
► False

Question No: 17 (Marks: 1) - Please choose one
Algorithms can only be implemented using STL containers.

► True
► **False** **(Object-Oriented Programming in C++)** **rep**

Question No: 18 (Marks: 1) - Please choose one
In _____, a base class can be replaced by its derived class,

► **Sub-typing** **(Page 31)**
► Super-typing
► Multiple-typing
► Restricted-typing

Question No: 19 (Marks: 1) - Please choose one
this pointer does not point to current object of any class,
► True
► **False** **Click here for detail**
The **this pointer** is a hidden pointer inside every class member function that points to the class object the member function is working with.

Question No: 20 (Marks: 1) - Please choose one
Which of the following operator(s) take(s) one or no argument if overloaded?

► ++ **(Page 162)**
► -
► +
► All of the given

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 21 (Marks: 1) - Please choose one
Which of the following operators can not be overloaded?

- ► **Scope resolution operator (::) (Page 141)**
- ► Insertion operator (<<)
- ► Extraction operator (>>)
- ► The relation operator (>)

Question No: 22 (Marks: 1) - Please choose one
The type that is used to declare a reference or pointer is called its ---------

- ► default type
- ► **static type (Page 185) rep**
- ► abstract type
- ► reference type

Question No: 23 (Marks: 1) - Please choose one
---------- members are somewhere between public and private members. They are used in inheritance

- ► **protected (Page 187) rep**
- ► public
- ► private
- ► global

Question No: 24 (Marks: 1) - Please choose one
Which of these are examples of error handling techniques ?

- ► Abnormal Termination
- ► Graceful Termination
- ► Return the illegal
- ► **all of the given (Page 329)**

Question No: 25 (Marks: 1) - Please choose one
_____ "is a" relationship

- ► **Inheritance (Page 25)**
- ► Polymarphism
- ► abstraction
- ► encapsulation

Question No: 26 (Marks: 1) - Please choose one
Graphical representation of the classes and objects is called object model it shows -------

- ► Class Name only
- ► Class Name and attributes
- ► Relationships of the objects and classes
- ► **all of the given Click here for detail rep**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

FINALTERM EXAMINATION
Spring 2010
CS304- Object Oriented Programming (Session - 4)

Question No: 1 (Marks: 1) - Please choose one
Classes like TwoDimensionalShape and ThreeDimensionalShape would normally be concrete, while classes like Sphere and Cube would normally be abstract.

► **True** **Click here for detail** **rep**
► False

Question No: 2 (Marks: 1) - Please choose one
Virtual functions allow you to

► create an array of type pointer-to-base class that can hold pointers to derived classes.
► create functions that can never be accessed.
► group objects of different classes so they can all be accessed by the same function code.
► **use the same function call to execute member functions of objects from different classes (Object-Oriented Programming in C++)**

Question No: 3 (Marks: 1) - Please choose one
.

► **True** **Click here for detail** **rep**
► False

Question No: 4 (Marks: 1) - Please choose one
A copy constructor is invoked when

► a function do not returns by value.
► **an argument is passed by value.** **(Page 78) rep**
► a function returns by reference.
► an argument is passed by reference.

Question No: 5 (Marks: 1) - Please choose one
Each try block can have _____ no. of catch blocks.

► 1
► 2
► 3
► **As many as necessary.** **Click here for detail** **rep**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 6 (Marks: 1) - Please choose one
Non Template Friend functions of a class are friends of _____instance/s of

- **All** **Click here for detail** **rep**
- One specific
- All instances of one date type
- None of the given options

Question No: 7 (Marks: 1) - Please choose one
Template functions use _____ than ordinary functions.

- Greater Memory
- **Lesser Memory**
- Equal Memory
- None of the given options

Question No: 8 (Marks: 1) - Please choose one
The find() algorithm

- finds matching sequences of elements in two containers.
- finds a container that matches a specified container.
- **takes iterators as its first two arguments. (Object-Oriented Programming in C++)**
- takes container elements as its first two arguments.

Question No: 9 (Marks: 1) - Please choose one
The copy() algorithm returns an iterator to

- the last element copied from.
- the last element copied to.
- the element one past the last element copied from.
- **the element one past the last element copied to. (Object-Oriented Programming in C++)**

Question No: 10 (Marks: 1) - Please choose one
If you define a vector v with the default constructor, and define another vector w with a one-argument constructor to a size of 11, and insert 3 elements into each of these vectors with push_back(), then the size() member function will return _____ for v and _____ for w.

- 11 for v and 3 for w.
- 0 for v and 0 for w.
- 0 for v and 3 for w.
- **3 for v and 11 for w. (Object-Oriented Programming in C++)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 11 (Marks: 1) - Please choose one
Which is not the Advantage of inheritance?

► **providing class growth through natural selection. (Object-Oriented Programming in C++)**
► facilitating class libraries.
► avoiding the rewriting of code.
► providing a useful conceptual framework.

Question No: 12 (Marks: 1) - Please choose one

```
class DocElement
{
public:
    virtual void Print() { cout << "Generic element"; }
};
class Heading : public DocElement
{
public:
    void Print() { cout << "Heading element"; }
};
class Paragraph : public DocElement
{
public:
    void Print() { cout << "Paragraph element"; }
};
void main()
{
    DocElement * p = new Paragraph();

    p->Print();
}
```

When you run this program, it will print out a single line to the console output.

What will be in that line?

Select one correct answer from the following list:

► Generic element
► Heading element
► Paragraph element
► **Nothing will be printed.**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 13 (Marks: 1) - Please choose one
Which type of inheritance is being represented by the following statement,
class X : public A, public B { ... ... };

- ► Single inheritance
- ► **Multiple inheritance (Page 41)**
- ► Double inheritance
- ► None of the given options

Question No: 14 (Marks: 1) - Please choose one
When we write a class template the first line must be:

- ► template < class *class_name*>
- ► template < class *data_type*>
- ► **template < class T > (Page 257)**

Here T can be replaced with any name but it is preferable.

- ► class class-name()

class template<*class_name*>

Question No: 15 (Marks: 1) - Please choose one
Function templates should be used where code and behavior must be identical.

- ► **True (Page 262)**
- ► False

Question No: 16 (Marks: 1) - Please choose one
Which of the following is/are advantage[s] of generic programming?

- ► Reusability
- ► Writability
- ► Maintainability
- ► **All of given (Page 256) rep**

Question No: 17 (Marks: 1) - Please choose one
The specialization pattern <T*> after the name says that this specialization is to be used for every,

- ► data type
- ► meta type
- ► virtual type
- ► **pointer type (Page 286)**

Question No: 18 (Marks: 1) - Please choose one
A range is often supplied to an algorithm by two _____ values.

- ► italic
- ► **iteration (Object-Oriented Programming in C++)**
- ► iterator
- ► None of given

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 19 (Marks: 1) - Please choose one
Which of the following is an integral part of an object?

- ► State
- ► Behavior
- ► Unique identity
- ► **All of the given (Page 12)**

Question No: 20 (Marks: 1) - Please choose one
Consider the following statement

Cupboard has books
What is the relationship between Cupboard and books?

- ► Composition
- ► **Aggregation**
- ► Inheritance
- ► None of the given options

Question No: 21 (Marks: 1) - Please choose one
Which sentence clearly defines an object?

- ► **one instance of a class. (Page 23)**
- ► another word for a class.
- ► a class with static methods.
- ► a method that accesses class attributes.

Question No: 22 (Marks: 1) - Please choose one
_____, which means if A declares B as its friend it does NOT mean that A can access private data of B. It only means that B can access all data of A.

- ► **Friendship is one way only** **Click here for detail**
- ► Friendship is two way only
- ► NO Friendship between classes
- ► Any kind of friendship

Question No: 23 (Marks: 1) - Please choose one
The statement objA=objB; will cause a compiler error if the objects are of different classes.

- ► True
- ► **False (Object-Oriented Programming in C++)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 24 (Marks: 1) - Please choose one
Consider the call given below of an overloaded operator "+",
Rational_number_1 + Rational_number_2
Where Rational_number_1 and Rational_number_2 are the two objects of Rational_number class (a user defined class). Identify which of the above two objects will be passed as an argument to the overloaded operator function?

- ► Rational_number_1
- ► Rational_number_2
- ► Both Rational_number_1 & Rational_number_2
- ► **any of the two objects, randomly**

Question No: 25 (Marks: 1) - Please choose one
If a class D has been derived using protected inheritance from class B (If B is a protected base and D is derived class) then public and protected members of B -------- accessed by member functions and friends of class D and classes derived from D

- ► **can be** **Click here for detail**
- ► cannot be
- ► does restirct to be
- ► not given

Question No: 26 (Marks: 1) - Please choose one
In Private ---------- only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object

- ► specialization
- ► **inheritance** **(Page 216) rep**
- ► abstraction
- ► composition

FINAL TERM EXAMINATION
SPRING 2010

Question No: 1 (Marks: 1) - Please choose one
Classes like TwoDimensionalShape and ThreeDimensionalShape would normally be concrete, while classes like Sphere and Cube would normally be abstract.

- ► **True** **Click here for detail** **rep**
- ► **False**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 2 (Marks: 1) - Please choose one
Each try block can have _____ no. of catch blocks.
► 1
► 2
► 3
► **As many as necessary.** **Click here detail**

Question No: 3 (Marks: 1) - Please choose one
Function templates should be used where code and behavior must be identical.
► **True** **(Page 262)**
► False

Question No: 4 (Marks: 1) - Please choose one
Consider the following statement
Cupboard has books
What is the relationship between Cupboard and books?
► Composition
► **Aggregation**
► Inheritance
► None of the given options

Question No: 5 (Marks: 1) - Please choose one
Identify the correct way of declaring an object of user defined template class **A** for char type members?
► **A< char > obj;** **(Object-Oriented Programming in C++)**
► <char>A obj;
► A obj<char>;
► Obj <char> A;

Question No: 6 (Marks: 1) - Please choose one
The user must define the operation of the copy constructor.
► **True** **Click here for detail**
► False

Question No: 7 (Marks: 1) - Please choose one
Default constructor is such constructor which either has no ---------or if it has some parameters these have -------- values
► Parameter, temporary
► Null, Parameter
► **Parameter, default** **(Page 75) rep**
► non of the given

Question No: 8 (Marks: 1) - Please choose one
The type that is used to declare a reference or pointer is called its ---------
► default type
► **static type** **(Page 185)**
► abstract type
► reference type

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 9 (Marks: 1) - Please choose one
How the information hidden within an object can be accessed?
► Through its interface
► Through its private data members
► **Through its private member functions**
► Through both public and private members

Question No: 10 (Marks: 1) - Please choose one
The sub-object's life is not dependant on the life of master class in _____.
► Separation
► Composition
► **Aggregation (Page 134)**
► None of the given

Question No: 11 (Marks: 1) - Please choose one
Encapsulation means
Select correct option:
►Extending the behaviour of class in another class
►**Data and behaviour are tightly coupled within an entity (Page 16)**
►One entity takes all the attributes and operations of the other
►Taking out the common features and put those in a separate class

Question No: 12 (Marks: 1) - Please choose one
Algorithms can only be implemented using STL containers.

► True
► **False (Object-Oriented Programming in C++) rep**

Question No: 13 (Marks: 1) - Please choose one
When we write a class template the first line must be:

► template < class *class_name*>
► template < class *data_type*>
► **template < class T > (Page 257) rep**
Here T can be replaced with any name but it is preferable.
► class class-name()
class template<*class_name*>

Question No: 14 (Marks: 1) - Please choose one
An STL container can not be used to,

► **hold objects of class employee.**
► store elements in a way that makes them quickly accessible.
► **compile c++ programs. (Object-Oriented Programming in C++)**
► organize the way objects are stored in memory

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 15 (Marks: 1) - Please choose one
_____, which means if A declares B as its friend it does NOT mean that A can access private data of B. It only means that B can access all data of A.
► **Friendship is one way only** **Click here for detail**
► Friendship is two way only
► NO Friendship between classes
► Any kind of friendship

Question No: 16 (Marks: 1) - Please choose one
Which of the following may not be an integral part of an object?
► State
► Behavior
► **Protected data members (Page 12)**
► All of given

Question No: 17 (Marks: 1) - Please choose one
Public methods of base class can --------- be accessed in its derived class
► **directly (Page 179) rep**
► inderectly
► simultaniously
► non of the given

Question No: 18 (Marks: 1) - Please choose one
If a class D has been derived using protected inheritance from class B (If B is a protected base and D is derived class) then public and protected members of B -------- accessed by member functions and friends of class D and classes derived from D
► **can be** **Click here for detail**
► cannot be
► does restirct to be
► not given

Question No: 19 (Marks: 1) - Please choose one
What is true about function templates?
► The compiler generates only one copy of the function template
► **The compiler generates a copy of function respective to each type of data (Page 256)**
► The compiler can only generate copy for the int type data
► None of the given.

Question No: 20 (Marks: 1) - Please choose one
Which of the following is an integral part of an object?
► State
► Behavior
► Unique identity
► **All of the given (Page 12) rep**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 21 (Marks: 1) - Please choose one
When the base class and the derived class have a member function with the same name, you must be more specific which function you want to call (using _____).
► scope resolution operator
► dot operator
► null operator
► **Operator overloading** **Click here for detail**

FINALTERM EXAMINATION
Fall 2009
CS304- Object Oriented Programming (Session - 4)

Question No: 1 (Marks: 1) - Please choose one
A template provides a convenient way to make a family of
► variables and data members
► **functions and classes** **(Object-Oriented Programming in C++)**
► classes and exceptions
► programs and algorithms

Question No: 2 (Marks: 1) - Please choose one
Which one of the following terms must relate to **polymorphism?**

► Static allocation
► Static typing
► **Dynamic binding** **(Page 239) rep**
► Dynamic allocation

Question No: 3 (Marks: 1) - Please choose one
What is true about function templates?

► The compiler generates only one copy of the function template
► **The compiler generates a copy of function respective to each type of data (Page 256) rep**
► The compiler can only generate copy for the int type data
► None of the given.

Question No: 4 (Marks: 1) - Please choose one
Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?
► **Templates** **(Page 256) rep**
► Overloading
► Data hiding
► Encapsulation

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 5 (Marks: 1) - Please choose one
template <>
class Vector<char*> { }

This is an example of partial specialization.

- **True (Page 287)**
- False

Question No: 6 (Marks: 1) - Please choose one
Classes like TwoDimensionalShape and ThreeDimensionalShape would normally be concrete, while classes like Sphere and Cube would normally be abstract.

- **True Click here for detail rep**
- False

Question No: 7 (Marks: 1) - Please choose one
A non-virtual member function is defined in a base class and overridden in a derived class; if that function is called through a base-class pointer to a derived class object, the derived-class version is used.

- True
- False

Question No: 8 (Marks: 1) - Please choose one
Assume a class Derv that is privately derived from class Base. An object of class Derv located in main() can access

- **public members of Derv. (Object-Oriented Programming in C++) rep**
- protected members of Derv.
- private members of Derv.
- protected members of Base.

Question No: 9 (Marks: 1) - Please choose one
In order to define a class template, the first line of definition must be:

- **template <typename T> (Page 257) rep**
- typename <template T>
- Template Class <ClassName>
- Class <Template T>

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 10 (Marks: 1) - Please choose one
If there is a pointer p to objects of a base class, and it contains the address of an object of a derived class, and both classes contain a nonvirtual member function, ding(), then the statement p->ding(); will cause the version of ding() in the _____ class to be executed.

- **Base (Object-Oriented Programming in C++)**
- Derived
- Abstract
- virtual

Question No: 11 (Marks: 1) - Please choose one
When the base class and the derived class have a member function with the same name, you must be more specific which function you want to call (using _____).

- scope resolution operator
- dot operator
- null operator
- **Operator overloading** **Click here for detail**

Question No: 12 (Marks: 1) - Please choose one
Non Template Friend functions of a class are friends of _____instance/s of that class.

- **All** **Click here for detail**
- One specific
- All instances of one date type
- None of the given options

Question No: 13 (Marks: 1) - Please choose one
The find() algorithm

- finds matching sequences of elements in two containers.
- finds a container that matches a specified container.
- **takes iterators as its first two arguments. (Object-Oriented Programming in C++) rep**
- takes container elements as its first two arguments.

Question No: 14 (Marks: 1) - Please choose one
If you define a vector v with the default constructor, and define another vector w with a one-argument constructor to a size of 11, and insert 3 elements into each of these vectors with push_back(), then the size() member function will return _____ for v and _____ for w.

- 11 for v and 3 for w.
- 0 for v and 0 for w.
- 0 for v and 3 for w.
- **3 for v and 11 for w. (Object-Oriented Programming in C++) rep**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 15 (Marks: 1) - Please choose one
Which of the following may not be an integral part of an object?

- ► State
- ► Behavior
- ► **Protected data members (Page 12)**
- ► All of given

Question No: 16 (Marks: 1) - Please choose one
Which is not the Advantage of inheritance?

- ► **providing class growth through natural selection. (Object-Oriented Programming in C++) rep**
- ► facilitating class libraries.
- ► avoiding the rewriting of code.
- ► providing a useful conceptual framework.

Question No: 17 (Marks: 1) - Please choose one

```
1class DocElement
{
public:
    virtual void Print() { cout << "Generic element"; }
};

2class Heading : public DocElement
{
public:
    void Print() { cout << "Heading element"; }
};

3class Paragraph : public DocElement
{
public:
    void Print() { cout << "Paragraph element"; }
};

void main()
{
    DocElement * p = new Paragraph();

    p->Print();
}
```

When you run this program, it will print out a single line to the console output.

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

What will be in that line?

Select one correct answer from the following list:

- ► Generic element
- ► Heading element
- ► Paragraph element
- ► **Nothing will be printed.**

Question No: 18 (Marks: 1) - Please choose one
When a virtual function is called by referencing a specific object by name and using the dot member selection operator (e.g., squareObject.draw()), the reference is resolved at compile time.

- ► True
- ► **False (Object-Oriented Programming in C++)**

(the function invocation is resolved at compile time)

Question No: 19 (Marks: 1) - Please choose one
In case of multiple inheritance a derived class inherits,

- ► Only the public member functions of its base classes
- ► Only the public data members of its base classes
- ► **Both public data members and member functions of all its base classes**

Click here for detail

- ► Data members and member functions of any two base classes

Question No: 20 (Marks: 1) - Please choose one
When we write a class template the first line must be:

- ► template < class *class_name*>
- ► template < class *data_type*>
- ► **template < class T > (Page 257) rep**

Here T can be replaced with any name but it is preferable.

- ► class class-name()

class template<*class_name*>

Question No: 21 (Marks: 1) - Please choose one
Which of the following is incorrect line regarding function template?

- ► template<class T>
- ► template <typename U>
- ► **Class<template T> (Page 257)**
- ► template < class T, class U>

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 22 (Marks: 1) - Please choose one
An STL container can not be used to,

- ► hold objects of class employee.
- ► store elements in a way that makes them quickly accessible.
- ► **compile c++ programs. (Object-Oriented Programming in C++) rep**
- ► organize the way objects are stored in memory

Question No: 23 (Marks: 1) - Please choose one
Algorithms can only be implemented using STL containers.

- ► True
- ► **False (Object-Oriented Programming in C++) rep**

Question No: 24 (Marks: 1) - Please choose one
Consider a class named Vehicle, which of the following can be the instance of class Vehicle?

1. Car
2. Computer
3. Desk
4. Ahmed
5. Bicycle
6. Truck

- ► 1, 4, 5
- ► 2, 5, 6
- ► 1, 2, 3, 6
- ► **1, 5, 6 (correct)**

Question No: 25 (Marks: 1) - Please choose one
Consider the code below,

```
class Fred {
public:
Fred();
...
};
int main()
{
Fred a[10];
Fred* p = new Fred[10];
...
}
```

Select the best option,

- ► Fred a[10]; calls the default constructor 09 times

Fred* p = new Fred[10]; calls the default constructor 10 times

- ► Produce an error

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

► Fred a[10]; calls the default constructor 11 times

Fred* p = new Fred[10]; calls the default constructor 11 times

► **Fred a[10]; calls the default constructor 10 times**

Fred* p = new Fred[10]; calls the default constructor 10 times **Click here for detail**

Question No: 26 (Marks: 1) - Please choose one

When a variable is define as **static** in a class then all object of this class,

► Have different copies of this variable

► **Have same copy of this variable (Page 110)**

► Can not access this variable

► None of given

Question No: 27 (Marks: 1) - Please choose one

The life of sub object is dependant on the life of master class in _____.

► Separation

► **Composition (Page 53)**

► Aggregation

► None of the given

Question No: 28 (Marks: 1) - Please choose one

_____, which means if A declares B as its friend it does NOT mean that A can access private data of B. It only means that B can access all data of A.

► **Friendship is one way only** **Click here for detail**

► Friendship is two way only

► NO Friendship between classes

► Any kind of friendship

Question No: 29 (Marks: 1) - Please choose one

Which of the following operators always takes no argument if overloaded?

► /

► -

► +

► **++ (Page 162)**

Question No: 30 (Marks: 1) - Please choose one

In Private ---------- only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object

► specialization

► **inheritance (Page 216) rep**

► abstraction

► composition

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

FINALTERM EXAMINATION
Fall 2009
CS304- Object Oriented Programming (Session - 1)

Question No: 1 (Marks: 1) - Please choose one
Which one of the following terms must relate to **polymorphism?**
- ► Static allocation
- ► Static typing
- ► **Dynamic binding (Page 239) rep**
- ► Dynamic allocation

Question No: 2 (Marks: 1) - Please choose one
Multiple inheritance can be of type
- ► Public
- ► Private
- ► Protected
- ► **All of the given** **Click here for detail**

Question No: 3 (Marks: 1) - Please choose one
When a subclass specifies an alternative definition for an attribute or method of its superclass, it is _____ the definition in the superclass.
- ► overload
- ► **overriding (Page 34)**
- ► copy riding
- ► none of given

Question No: 4 (Marks: 1) - Please choose one
Like template functions, a class template may not handle all the types successfully.
- ► **True (Page 258) rep**
- ► False

Question No: 5 (Marks: 1) - Please choose one
It is sometimes useful to specify a class from which no objects will ever be created.
- ► **True (Object-Oriented Programming in C++)**
- ► False

Question No: 6 (Marks: 1) - Please choose one
Assume a class Derv that is privately derived from class Base. An object of class Derv located in main() can access
- ► **public members of Derv. (Object-Oriented Programming in C++) rep**
- ► protected members of Derv.
- ► private members of Derv.
- ► protected members of Base.

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 7 (Marks: 1) - Please choose one
A pointer to a base class can point to objects of a derived class.

- **True** **Click here for detail**
- False

Question No: 8 (Marks: 1) - Please choose one
A copy constructor is invoked when

- a function do not returns by value.
- **an argument is passed by value. (Page 78) rep**
- a function returns by reference.
- an argument is passed by reference.

Question No: 9 (Marks: 1) - Please choose one
A function call is resolved at run-time in_____

- non-virtual member function.
- **virtual member function. (Page 239) rep**
- Both non-virtual member and virtual member function.
- None of given

Question No: 10 (Marks: 1) - Please choose one
When the base class and the derived class have a member function with the same name, you must be more specific which function you want to call (using _____).

- scope resolution operator
- dot operator
- null operator
- **Operator overloading** Click here for detail

Question No: 11 (Marks: 1) - Please choose one
Each try block can have _____ no. of catch blocks.

- 1
- 2
- 3
- **As many as necessary. Click here for detail rep**

Question No: 12 (Marks: 1) - Please choose one
Two important STL associative containers are _____ and _____.

- **set,map (Object-Oriented Programming in C++) rep**
- sequence,mapping
- setmet,multipule
- sit,mat

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 13 (Marks: 1) - Please choose one

The mechanism of selecting function at run time according to the nature of calling object is called,

- ► late binding
- ► static binding
- ► virtual binding
- ► **None of the given options (Page 227)**

Dynamic binding means that target function for a call is selected at run time

Question No: 14 (Marks: 1) - Please choose one

An abstract class is useful when

- ► We do not derive any class from it.
- ► There are multiple paths from one derived class to another.
- ► **We do not want to instantiate its object. (Object-Oriented Programming in C++) rep**
- ► You want to defer the declaration of the class.

Question No: 15 (Marks: 1) - Please choose one

Which of the following is incorrect line regarding function template?

- ► template<class T>
- ► template <typename U>
- ► **Class<template T> (Page 257) rep**
- ► template < class T, class U>

Question No: 16 (Marks: 1) - Please choose one

Which of the following is/are advantage[s] of generic programming?

- ► Reusability
- ► Writability
- ► Maintainability
- ► **All of given (Page 256) rep**

Question No: 17 (Marks: 1) - Please choose one

By default the vector data items are initialized to ____

- ► **0** **Click here for detail** **rep**
- ► 0.0
- ► 1
- ► null

Question No: 18 (Marks: 1) - Please choose one

Which one of the following functions returns the total number of elements in a vector.

- ► length();
- ► **size(); (Page 318)**
- ► ele();
- ► veclen();

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 19 (Marks: 1) - Please choose one
Suppose you create an uninitialized vector as follows:

vector<int> evec;
After adding the statment,
evec.push_back(21);
what will happen?

► The following statement will add an element to the start (the back) of evec and will initialize it with the value 21.

► The following statement will add an element to the center of evec and will reinitialize it with the value 21.

► The following statement will delete an element to the end (the back) of evec and will reinitialize it with the value 21.

► **The following statement will add an element to the end (the back) of evec and initialize it with the value 21.** **Click here for detail**

Question No: 20 (Marks: 1) - Please choose one
An STL container can not be used to,

► hold objects of class employee.

► store elements in a way that makes them quickly accessible.

► **compile c++ programs. (Object-Oriented Programming in C++) rep**

► organize the way objects are stored in memory

Question No: 21 (Marks: 1) - Please choose one
Algorithms can only be implemented using STL containers.

► True

► **False (Object-Oriented Programming in C++) rep**

Question No: 22 (Marks: 1) - Please choose one
The main function of scope resolution operator (**::**) is,

► To define an object

► To define a data member

► **To link the definition of an identifier to its declaration** Click here for detail

► To make a class private

Question No: 23 (Marks: 1) - Please choose one
When is a constructor called?

► Each time the constructor identifier is used in a program statement

► **During the instantiation of a new object (Object-Oriented Programming in C++)**

► During the construction of a new class

► At the beginning of any program execution

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 24 (Marks: 1) - Please choose one

Consider the code below,

```
class Fred {
public:
Fred();
...
};
int main()
{
Fred a[10];
Fred* p = new Fred[10];
...
}
```

Select the best option,

- ► Fred a[10]; calls the default constructor 09 times

Fred* p = new Fred[10];

- ► Produce an error
- ► Fred a[10]; calls the default constructor 11 times

Fred* p = new Fred[10]; calls the default constructor 11 times

- ► **Fred a[10]; calls the default constructor 10 times**

Fred* p = new Fred[10]; calls the default constructor 10 times **Click here for detail**

Question No: 25 (Marks: 1) - Please choose one

Associativity can be changed in operator overloading.

- ► True
- ► **False (Page 141)**

Question No: 26 (Marks: 1) - Please choose one

A normal C++ operator that acts in special ways on newly defined data types is said to be

- ► glorified.
- ► encapsulated.
- ► classified.
- ► **overloaded. (Object-Oriented Programming in C++)**

Question No: 27 (Marks: 1) - Please choose one

Which operator can not be overloaded?

- ► The relation operator (>=)
- ► Assignment operator (=)
- ► Script operator ([])
- ► **Conditional operator (? :) (Page 141)**

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Question No: 28 (Marks: 1) - Please choose one

Suppose obj1 and obj2 are two objects of a user defined class A. An + operator is overloaded to add obj1 and obj2 using the function call obj1+obj2.

Identify the correct function prototype against the given call?

- A operator + (A &obj);
- **int + operator(); (Page 143)**
- int operator (plus) ();
- A operator(A &obj3);

Question No: 29 (Marks: 1) - Please choose one

Default constructor is such constructor which either has no ---------or if it has some parameters these have ---------- values

- Parameter, temporary
- Null, Parameter
- **Parameter, default (Page 75) rep**
- non of the given

Question No: 30 (Marks: 1) - Please choose one

Public methods of base class can --------- be accessed in its derived class

- **directly (Page 179) rep**
- inderectly
- simultaniously
- non of the given

Muhammad Moaaz Siddiq – MCS (2nd)
mc100401285@Gmail.com
Campus:- Institute of E-Learning & Modern Studies (IEMS) Samundari

Lecture No.23

Lecture Contents:

- **Protected Access Specifier in Inheritance**
- **Implicit and explicit use of IS A relationship**

Previous lecture discussion:

Definition of Some Terms:

Default constructor: Default constructor is such constructor which either has no parameter or if it has some parameters these have default values. The benefit of default constructor is that it can be used to create class object without passing any argument.

Implicit Default constructor:
Compiler generates implicit default constructor for any class in case we have not given any constructor for the class.

Explicit Default constructor:
If user has given constructor for any class without any arguments or with all arguments with default parameters then it is also default constructor according to definition but it is explicit default constructor.

Now if a base class has only **non-default constructor** (implicit or explicit), then when we will create object of any class derived from this base class compiler will not be able to call base class constructor as base class has no default constructor (constructor that can be called without giving any parameters) so compiler will generate error.
We can avoid this error by calling base class non-default constructor in derived class constructor initializer list.

Accessing base class member functions in derived class:

Public methods of base class can directly be accessed in its derived class (***derived class interface consists of its own member functions plus member functions of its base class***).

However there are some class members functions that are written just as helper functions for other class member functions and they need not to be called directly using class object for example,

1. Suppose some function in our class wants to get some input from user in the form of integers only for this we can write another function that checks whether the entered string by user consists of integers only.

2. Suppose our class has implement encryption of data, and it encodes and decodes data using some functions these functions will also to helper functions and should not be accessible to outside word.

3. Similarly take the example of our Date class we studied in lecture no.13 has a function to check whether an year is leap year or not and according to result it allows or disallows the days of February to be 29. This bool **IsLeapYear(int)** will used by other members functions of Date class like void setDay(int) and void addDay(int), this functions is also helper function.

These helper functions are made private as there is no need to access them using class object directly as shown below,

```
class Date{
        int day, month, year;
        static Date defaultDate;
public:
        void SetDay(int aDay);
        int GetDay() const;
        void AddDay(int x);
        …
         static void SetDefaultDate(int aDay,int aMonth, int aYear);
        …
private:

        bool IsLeapYear(int );
};

int main(){
        Date aDate;
        aDate.IsLeapYear( year );
        return 0;
}
```

Error:
bool IsLeapYear(int) is private method of Date class so it is not accessible here.

Making such functions private works fine as long as we don't derive any child class from this class, but when we derive some class for specialization these functions will not be accessible in the derived class as private members functions are accessible only in those class to which they belong but these functions are needed in derived classes as well example is given below,

Example:
Suppose we specialize our Date class by adding a child class SpecialDate to handle only working days in the an year, we will need to use **IsLeapYear** function in this child class as well but **IsLeapYear** function is private member function of base Date class (in accordance with the principles of data hiding and encapsulation), so it will not be accessible in child class as shown below,

```
class SpecialDate: public Date{
        …
public:
        void AddSpecialYear(int i){
                …
                if(day == 29 && month == 2
                &&      !IsLeapyear(year+i)){       //ERROR!
                        …
```

bool IsLeapYear(int) is private method of Date class so it is not accessible in its derived SpecialDate class.

```
            }
        }
};
```

Solution:

Modify Access Specifier

One solution of this problem is that we make "IsLeapYear" function public from private as shown below,

```
class Date{
public:
        ...
        bool IsLeapYear(int );
};
void SpecialDate :: AddSpecialYear (int i) {
        ...
        if(day == 29 && month == 2
                &&      !IsLeapyear(year+i)){
                ...
        }
}
```

But the problem with making this function public is that now this functions will be accessible to everyone using Date or SpecialDate class object but we don't want this, we want that our base or derived class can access this function only. the solution of this problem is protected access specifier.

"*protected*" access specifier:

c++ provides us with ***protected access specifier*** for these sorts of situations; protected access specifier ensures that function in base class is accessible in derived class of this base class and NOT outside of this class. So we can say that scope of protected access specifier is somewhere in between private and public access specifiers it is similar to private in the sense that it doesn't allow the external world to access protected member and it is similar to public access specifier in the sense that it ensures that protected member is accessible in derived classes of the protected member class.

Protected members of a class can not be accessed outside the class but only in the derived class of that class. Protected members of base class become protected member of derived class.[10]

```
class Date{
        ...
```

[10] This is the case in public and protected inheritance whereas in private inheritance protected members become private members of derived class public, protected and private inheritance has been discussed in detail in lecture no.26.

```
protected:
                bool IsLeapYear(int );
};

int main(){
        Date aDate;
        aDate.IsLeapYear(); //Error
        return 0;
}
void SpecialDate :: AddSpecialYear (int i) {
        ...
        if(day == 29 && month == 2
                &&      !IsLeapyear(year+i)){
                ...
        }
}
```

Now it is ok to call function **bool IsLeapYear** in derived class *SpecialDate*.

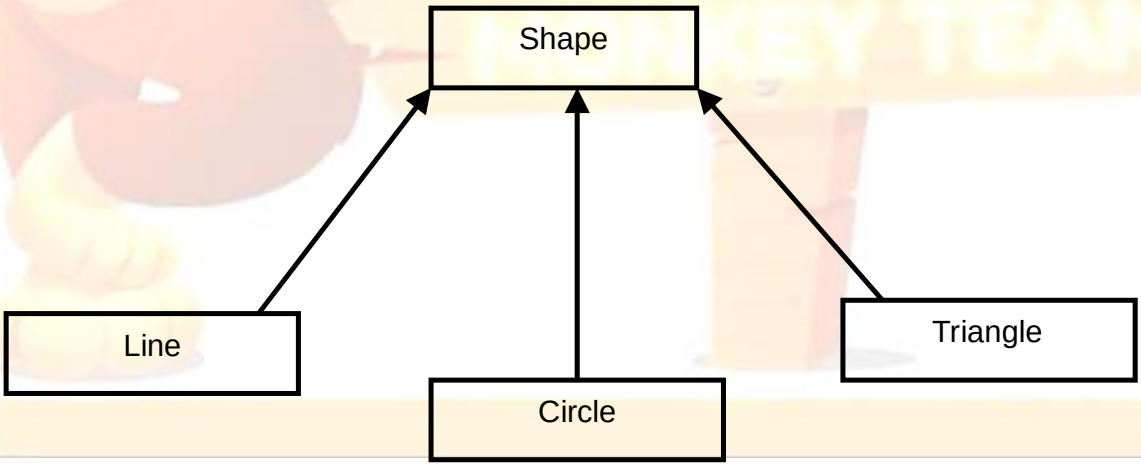
Disadvantages of protected Members:

Breaks encapsulation

The protected member is part of base class's implementation as well as derived class's implementation. So we can say that protected members breaks the principle of Encapsulation to some extent which says "A class data members and functions should be encapsulated in the class itself"

"IS A" Relationship

We have seen previously that Public inheritance models the "IS A" relationship for example see the diagram below,



Here,

- Line IS A Shape

© Virtual University of Pakistan

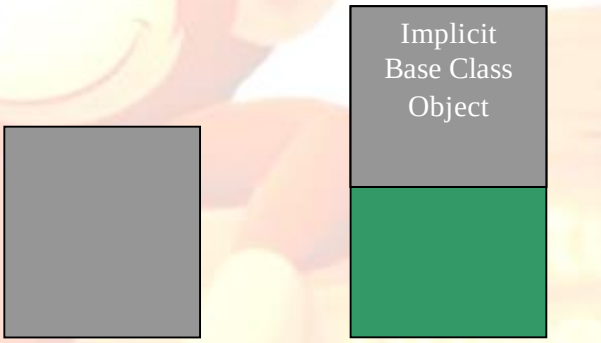
- Circle IS A Shape
- Triangle IS A Shape

Generally we can say that,

"Derived Object IS A kind of Base Object"

It means that derived class object is special kind of base object with extra properties (attributes and behaviour) so derived object has its own properties as well as properties of its base object this is the reason why base class constructor is also called while creating derived class object.

We can use derived class object where base class object is required as **derived class object has implicit base class object** also but the reverse of this statement is not true, we can not use base class object where derived class object is required because if we create base class it will only have base part not its derived class part.



Base Class Object Derived Class Object

Example

See the code below in it first we are using derived class object sobj to call methods *GetName* and *GetRollNo* belonging to classes Person and Student respectively, this is in accordance to the principle of inheritance that says child class object can access public and protected members of base class.

```
class Person {
        char * name;
public: ...
        const char * GetName();
};
class Student: public Person{
        int rollNo;
public: ...
```

```
        int GetRollNo();
};

int main()
{
        Student sobj;
        cout << sobj.GetName();
        cout << sobj.GetRollNo();
        return 0;
}
```

Now as we discussed previously that inheritance represents IS A relationship meaning relationship like,

Derived class "IS A" Base class

Hence Derived Class Object can be used where its Base Class object is required but Base Class Object can not be used where Derived Class Object is required example given below explain this point,

```
int main(){
        Person * pPtr = 0; // base class pointer
        Student s; // derived class object
        pPtr = &s;
/* assigning derived class object address to base class pointer derived class pointer is converted to base class pointer */
        cout << pPtr->GetName(); // calling base class methods using base class reference
        return 0;
}
```

So parent class object reference can hold the reference of derived class object as shown below,

```
pPtr = &s;
```

But parent class reference will have access only to the interface of parent class, if we try to access derived class functions from this parent class reference error will be generated as shown below,

```
int main(){
        Person * pPtr = 0;
        Student s;
        pPtr = &s;
        cout << pPtr->GetRollNo();          //Error
        return 0;
}
```

Compiler use the concept of static typing to decide the access to member functions as described below,

© Virtual University of Pakistan

Static Type

The type that is used to declare a reference or pointer is called its static type

1. In Person * pPtr = 0;
   The static type of **pPtr** is Person * (Person pointer).
2. Student s;
   The static type of **s** is Student.

Member Access

As the static type of pPtr is Person so following call is erroneous

pPtr->GetRollNo();

As pPtr static type is Person pointer and in Person class there is no GetRollNo() function so compiler will generate an error.

In c++ we can also use references (called as reference identifiers or reference variables or reference constants) to any type instead of pointers, for example see the example below where the reference of base object is being initialized with derived class object,

Example (Explicit use of IS A relationship)

```
int main(){

Person p;
Student s;
Person & refp = s;

/*Here refp is becoming reference (alias) of Student object but as its own static type is Person so it can access member functions of Student class only. */

cout << refp.GetName();
cout << refp.GetRollNo(); //Error
return 0;
}
```

Example (Implicit use of IS A relationship)

```
  Play(const Person& p){
        cout << p.GetName()
                << " is playing";
}
void Study(const Student& s){
        cout << s.GetRollNo()
                << " is Studying";
}

int main(){
Person p;
```

```
Student s;
Play(p); /* parameter of function Play is being initialized with argument p */
Play(s);
/* parameter of function Play is being is initialized with argument s as student IS
A kind of person*/
return 0;
}
```



Lecture No.24

Lecture Overview:

Protected Access Specifier
IS-A relationship
Copy constructor and inheritance

Protected Members:

Protected members are somewhere between public and private members. They are used in inheritance.
From outside the class no once can access protected access specifier however any publicly derived class can access the protected access specifiers and they behave as protected members of derived class.
In a single standalone class protected access specifier has no significance but when we derive any class from this class protected access specifier becomes significant.

IS-A relationship:

Inheritance is used when two classes have IS A kind of relationship between two classes.
In c++ public inheritance is used for IS A relationship.
We can use publicly derived class pointer for its base class pointer because derived class is a kind of base class or in other words derived class is base class with some extra properties. In case of "Student IS-A Person relationship" student has all properties of person for example student walks, eats, drinks and so on as person does but student has some extra properties like student studies is some study program.

(We can assign derived class pointer to its base class pointer because for compiler it is explicit manifestation of IS-A relationship.)

Static type of an identifier:

Static type of an identifier is type used to declare it.
Static type of an identifier is the type used for declaration of that identifier for example,

```
Student * student;
Person * person;
```

Here *static type* of pointer *student* is Student and *person* is Person.

Access to members of an identifier is governed by static type of that identifier, compiler uses static type for an identifier to allow the identifier to use different members (functions and variables).

Copy Constructor:[11]

Copy constructor is a member function of a class that is used to create an object of that class ***by copying values from an already created object.***

Copy Constructor in case of Inheritance:

Consider the code given below having two classes Person and Student in this code Person is base and Student is derived class. Person has one attribute ***name*** to store name of the person and Student has one attribute ***major*** to store study program of the student,

```
#include <iostream>
using namespace std;


/*Listing 24.1 */
/*Program to demonstrate Copy Constructor in case of base and derived classes. */

/*    Base Class Person    */
class Person{
        char * name;
public:
        Person(char * = NULL);
        const char * GetName() const;
        ~Person();
};

Person::Person(char * nm): name(NULL){

  if (nm != NULL)
     {
        name = new char[strlen(nm)+1];
        strcpy(name,nm);
     }

  }
const char * Person::GetName() const {
  if (name != NULL )
  return name;
  else
```

[11] See Lecture No.9 for more details about copy constructor.

© Virtual University of Pakistan

```
   return "NULL";

   }

Person::~Person(){
delete []name;
   }

/*                  Child Class Student                                    */
class Student: public Person{
        char* major;
public:
        Student(char * , char * );
        void Print() const;
        ~Student();
};

Student::Student(char *_name, char *_maj) : Person(_name), major(NULL)
{
   if (_maj != NULL) {
        major = new char [strlen(_maj)+1];
        strcpy(major,_maj);
   }
}
void Student::Print() const{
    cout << "Name: "<< GetName() << endl;
    cout << "Major: " << major << endl;

}

Student::~Student(){
        delete []major;
}

int main(){

        Student sobj1("Ali","Computer Science");
        Student sobj2 = sobj1;
          //Student sobj2(sobj1); Similar to above statement to call copy constructor
   sobj2.Print();
        system("PAUSE");
        return 0;

}
```

Code Description:

The code has two classes Person and Student this code is also manifestation of IS-A kind of relationship (public inheritance in c++).

Person class has an attribute name to store name of any person as all persons have specific names.
Student class has an attribute major to store the major of the student.

Person has the following in its public interface:

```
Constructor                 // one parameter name with default value NULL
GetName()                   // getter function
Destructor()                // used due to dynamic memory allocation
```

Student has the following in its public interface:

```
Constructor         // two parameter constructor with default values NULL
Print()             // showing student attributes
Destructor          // used to free memory dynamically allocated
```

The constructor's code of student class is as shown below,

```
Student::Student(char *_name, char *_maj) : Person(_name), major(NULL)
{
   if (_maj != NULL) {
         major = new char [strlen(_maj)+1];
         strcpy(major,_maj);
   }
}
```

The constructor of student class is taking two char *'s as inputs, one is name of student and second is major subject of that student.
In initialization list we are initializing major with null value and we are also calling base class constructor of person class as we discussed today base class part of an objects is created first and then its derived class part is created.
Then there is code of student constructor in which are checking maj for NULL if it is not (some value has passed for maj subject) we are creating memory equal to passed value and assigning it to char * major.

The print function code of student class is given below,

```
void Student::Print() const {
    cout << "Name: "<< GetName() << endl;
    cout << "Major: " << major << endl;

}
```

In print method we are doing two things first we are printing student name using person GetName method and then we are showing student major.

The main function is given below,

```
int main(){

        Student sobj1("Ali", "Computer Science");
        Student sobj2 = sobj1;
            //Student sobj2(sobj1);      Similar to above statement to call copy
constructor
       sobj2.Print();
        system("PAUSE");
        return 0;

}
```

In main we are creating one object of Student class with name ***sobj1*** with values "Ali" and "Study Program" for name and major,

Student sobj1("Ali","Computer Science");

Then we are assigning ***sobj1*** to a new student object ***sobj2*** with the line given below,

Student sobj2 = sobj1;

This line will invoke copy constructor of Student class as we are creating an object of student class in terms of another object that already exist for student class.

This line is exactly same as commented line,

//Student sobj2(sobj1);

Here one object **sobj1** values have been assigned to newly created object using **default compiler generated copy constructo**r, this copy constructor is created by compiler in the same way as compiler generates default constructor in case user has not defined default constructor for that class.

Then we are calling print method of student class to ensure that we values have been assigned to new object as well,

sobj2.Print();

The output of this code will be as follows:

C++ Out Put:

```
Name: Ali
Major: Computer Science
```

As we know compiler may generate copy constructor for any class, if needed. In this case when we are assigning derived class object **sobj1** to another newly created

object **sobj2**, derived class object Copy constructor will be invoked which in turn will call the copy constructor of the Base class as base class anonymous object is created first and then derived part is created.

Shallow Copy

Compiler by default uses shallow copy so compiler generated **default copy constructor** will also use shallow copy which simply copies all values of already existing object to newly created object as shown in the diagram below,

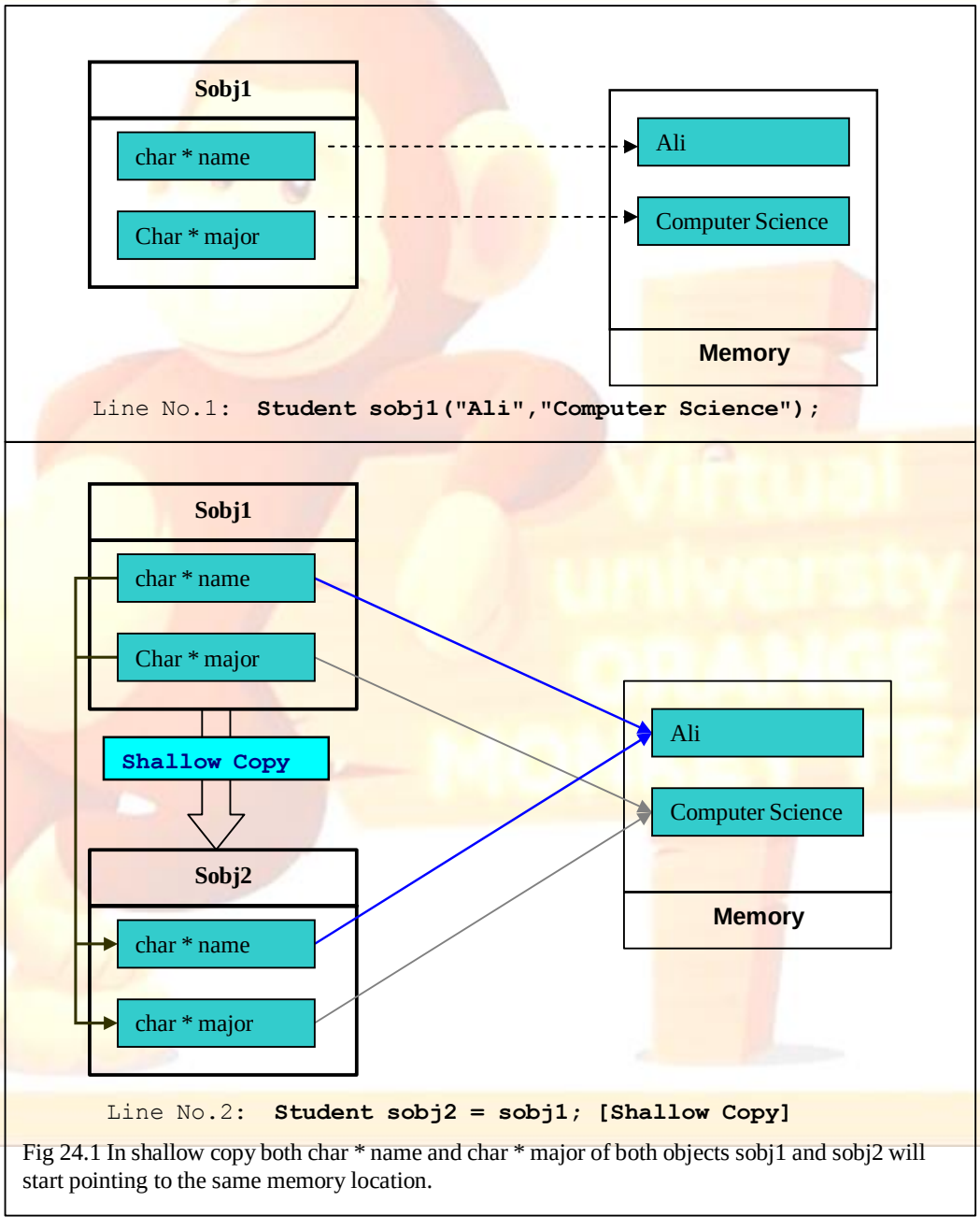


Fig 24.1 In shallow copy both char * name and char * major of both objects sobj1 and sobj2 will start pointing to the same memory location.

Compiler generates copy constructor for derived class, calls the copy constructor of the base class and then performs the shallow copy of the derived class's data members.

The problem with shallow copy remains same as was in the case of copy constructor for a single class that it doesn't work correctly in case of dynamic memory because in that case we are only storing pointer of the dynamic memory in class when we will do copy only the pointer value will be copied and pointers of both objects of student class will start pointing to same memory location as shown in the diagram now suppose obj1 is destroyed it frees it's memory allocated using new operator using its destructor the result will be that obj2 pointer will also become invalid. The solution is Deep Copy in deep copy we will write copy constructor code by our self to ensure that when one object is copied from other object new dynamic memory is allocated for it as well so that it doesn't rely on previous object memory.

Deep Copy:

Let we first write code for deep copy in our base class Person.
For this we will write our own copy constructor as we wrote our own default constructor as shown below,

```
Person::Person(const Person & rhs): name(NULL){
   /* Code for deep copy*/
   if (rhs.name != NULL)
      {
         name = new char[strlen(rhs.name)+1];
         strcpy(name,rhs.name);
      }

   }


int main(){

         Student sobj1("Ali","Computer Science");
         Student sobj2 = sobj1;
            //Student sobj2(sobj1); Similar to above statement to call copy constructor
   sobj2.Print();
         system("PAUSE");
         return 0;

}
```

When we will run this code the output will again be same,

C++ Out Put:

```
Name: Ali
Major: Computer Science
```

In this case as we have written our self written copy constructor of base class so Compiler will generate copy constructor for only derived class, will call our written copy constructor of the base class so it will perform the **Shallow copy** for derived class's data members and **Deep copy** for base class as our base class has our own written copy constructor with **Deep copy** code.

Now let us write our own copy constructor for derived class as well as shown below,

```
Student::Student(const Student & rhs) : major(NULL)
{
   if (rhs.major != NULL) {
         major = new char [strlen(rhs.major)+1];
         strcpy(major,rhs.major);
   }
}

int main(){

         Student sobj1("Ali","Computer Science");
         Student sobj2 = sobj1;
            //Student sobj2(sobj1); Similar to above statement to call copy constructor
   sobj2.Print();
         system("PAUSE");
         return 0;

}
```

But now when we executed this code the output is as follows:

C++ Out Put:

```
Name: NULL
Major: Computer Science
```

Name of sobj2 was not copied from sobj1 or we can say that only copy constructor of derived class executed but base class copy constructor did not execute.
The reason for this error is that we have not explicitly called copy constructor of base class from derived class. And compiler called base class default constructor by itself to complete the object creation process.

We can check whether base class default constructor was called or not by adding a cout statement in our base class default constructor as shown below,

24.1.Modified Default Constructor

```
Person::Person(char * nm): name(NULL){

   if (nm != NULL)
      {
         name = new char[strlen(nm)+1];
         strcpy(name,nm);
      }
   else {
         cout<<"In Person default constructor..\n";
      }


   }

int main(){

         Student sobj1("Ali","Computer Science");
         Student sobj2 = sobj1;
            //Student sobj2(sobj1); Similar to above statement to call copy constructor
         sobj2.Print();
         system("PAUSE");
         return 0;

}
```

Now when we executed this code the output was,

C++ Out Put:

```
In Person default constructor..
Name: NULL
Major: Computer Science
```

Now we modify our derived class copy constructor code such that it explicitly call copy constructor of base class from its initialization list,

Example

```
Person::Person(const Person & rhs): name(NULL){

   if (rhs.name != NULL)
      {
         name = new char[strlen(rhs.name)+1];
```

```
        strcpy(name,rhs.name);
    }

}

Student::Student(const Student & rhs) : Person(rhs), major(NULL)
{
  if (rhs.major != NULL) {
        major = new char [strlen(rhs.major)+1];
        strcpy(major,rhs.major);
  }
}
```

Now our main function shown previously will give following output

C++ Out Put:

```
Name: Ali
Major: Computer Science
```

The output again is correct now so this is our final code with user defined copy constructors for both base and derived class and derived class copy constructor *calling* base class copy constructor.

In table below we have shown the steps number wise involved in creation of an object using copy constructor when we executed following lines in main,

```
int main(){

        Student sobj1("Ali","Computer Science");
        Student sobj2 = sobj1;
// Here copy constructor of student class will be called
        return 0;

}
```

3	Person::Person(const Person &rhs) :	
4		name(NULL) {
5	//code for deep copy	
	}	
1	Student::Student(const Student & rhs) :	
6		major(NULL),
2	Person(rhs){	
7	//code for deep copy	
	}	

Table 24.1

Assignment Operator

Compiler also generates code for *assignment operator* for a class, if needed. In the case of inheritance when we assign one derive class object to another derive class object compiler uses assignment operator. Derived class copy assignment operator is invoked *which in turn calls* the assignment operator of the base class.
There are lot of similarities between copy constructor and assignment operator these are,
In case our class involves dynamic memory allocation we had to write assignment operator code by our self as we had to write the user defined code for copy constructor.

Similarly derived class assignment operator has to call base class assignment operator to complete the assignment process as derived class also contains implicit base class part in case of compiler generated assignment operator compiler does it by itself but if case we programmer has to call assignment operator of base class explicitly in case of user defined assignment operator.

The example given below explain this concept in detail,

Example

```
class Person{
public:
        Person & operator =
                                (const Person & rhs){
                cout << "Person Assignment";
                // Code for deep copy assignment
        }
};
Example
class Student: Public Person{
public:
        Student & operator = (const Student & rhs){
                cout<< "Student Assignment";
                // Code for deep copy assignment
        }
};
int main()
{
        Student sobj1, sboj2("Ali", "CS");
        sobj1 = sobj2;
        return 0;
}
```

Output

Student Assignment

The assignment operator of base class is not called

Calling Base Class member functions:
We can not call base class assignment operator from derived class using initialization list.

There are two ways of calling base class function from derived class one is calling base class function explicitly (mentioning base class name) and the other is calling base class function implicitly (without mentioning base class name).

Explicit Way:

In the code below we are calling base class **const char *GetName()** method explicitly,

```
const char * Person::GetName() const {

   …
   …

}


void Student::Print() const{

    //cout << "Name: "<< GetName() << endl;
    cout << "Name: "<< Person::GetName() << endl;

/*explicit call to base class GetName method (if we even do not write base class name even then call will work as derived class can call its base class public methods*/

    cout << "Major: " << major << endl;

}
```

Implicit Way:

We can call the same method implicitly as well as shown below,

```
const char * Person::GetName() const {

   …
   …

}
void Student::Print() const {
```


© Virtual University of Pakistan

```
/* Implicit call to base class GetName method using this pointer*/
cout <<"Name: "<<static_cast<const Person &>(*this).GetName()<<endl;
cout << "Major: " << major << endl;

}
```

Assignment Operator

In the same way we can also call assignment operator in base class in two ways also,
Calling assignment operator of base class explicitly
Calling assignment operator of base class implicitly

Explicitly Calling operator =

```
Student & Student ::operator =(Student & rhs){


   Person::operator = (rhs);  /*Explicit call to base class*/
   /*Student object rhs will be converted to Person object in function call
   above as derived class IS A is a kind of Base class*/

   /*Implicit calls to base class part*/
   //static_cast<Person &>(*this) = rhs;
   //Person(*this) = rhs;
   //(Person)*this = rhs;
   /*All the above three statements have the same meaning.*/

   if (major != NULL)
   delete [] major; // deleting previous allocated memory for major

   if(rhs.major != NULL){
   major = new char[strlen(rhs.major)+1];
   strcpy(major,rhs.major);
   }

}
```

Implicitly Calling operator =

```
Student & Student ::operator =(Student & rhs){


   //Person::operator = (rhs); /*Explicit call to base class*/
   /*Student object rhs will be converted to Person object in function call
   above as derived class IS A is a kind of Base class*/

   /*Implicit calls to base class part*/
   static_cast<Person &>(*this) = rhs; /* c++ way of type casting */
   //Person(*this) = rhs;  /* c way of type casting */
```

```
    //(Person)*this = rhs;   /* c way of type casting */
    /*All the above three statements have the same meaning.*/

    if (major != NULL)
    delete [] major; // deleting previous allocated memory for major

    if(rhs.major != NULL){
    major = new char[strlen(rhs.major)+1];
    strcpy(major,rhs.major);
    }

}
```

Appendix:

Type conversion:

C++ is strongly typed language mean we can not use one type instead of other type we need to convert one type in other type before using it.
This conversion from one type to another type is called **casting**.

Casting:

Casting can be done in two ways ,

Implicit casting conversion :

Conversions that compiler can perform automatically using build in casting operations without casting request from programmer.

Explicit casting conversion:

Conversion in which programmer requests the compiler to convert one type to another.
There are four types of casting operators that programmer can use to convert one type ot another as given below,

The C++ draft standard includes the following four casting operators,

1. static_cast
2. const_cast
3. dynamic_cast
4. reinterpret_cast

In perspective of inheritance casting can be of two kinds,

a. Up casting (casting from derived to base class)
b. Down casting (casting from base to derived class)

© Virtual University of Pakistan

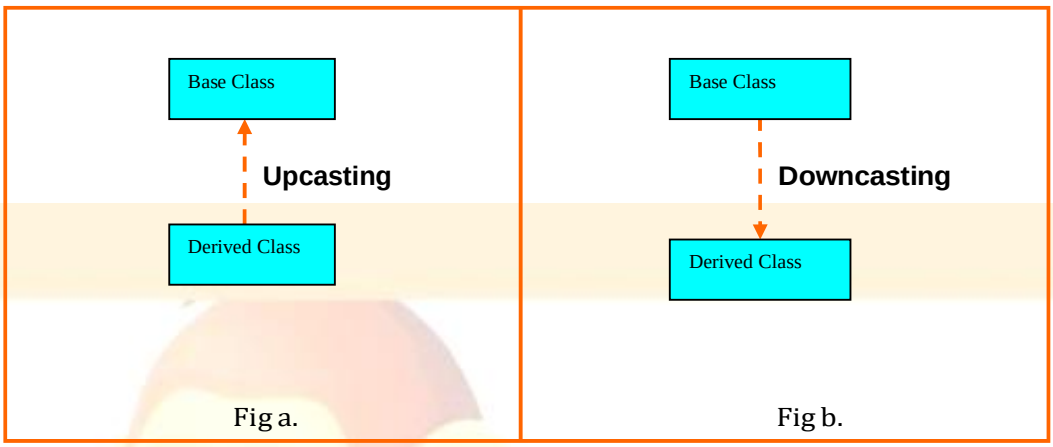


Fig a. Fig b.

Downcasting:

Example: (Base to Derive class)

Dynamic cast: (polymorphic behavior)

```
#include <iostream>
#include <stdlib.h>

using namespace std;

class Base{
public:
virtual void function() { cout<<"I am in Base"<<endl; }
};
class Derived : public Base {
public:
void function() { cout<<"I am in Derived"<<endl; }
};
int main()
{
Base * base = new Derived();
base->function();
Derived * derived = dynamic_cast<Derived*>(base);
derived->function();
system("PAUSE");
return 0;
}
```

Reinterpret cast:

```
#include <iostream>
#include <stdlib.h>

using namespace std;
```

```
class Base{
public:
void function() { cout<<"I am in Base"<<endl; }
};
class Derived : public Base {
public:
void function() { cout<<"I am in Derived"<<endl; }
};
int main()
{
Base * base = new Derived();
base->function();
Derived * derived = reinterpret_cast<Derived*>(base);
derived->function();
system("PAUSE");
return 0;
}
```

Note:

1. char * is c++ built in data type that can be used exactly like char arrays in fact char arrays are also treated as char * in c++ so we used char * where we needed to use char array.
2. In video lecture at duration 24 min there should only be one memory location to store values, Ali newly created object pointer will be NULL for name and will not point to any memory location.

References:

http://www.acm.org/crossroads/xrds3-1/ovp3-1.html (Casting Tutorial see this link for more details about casting)

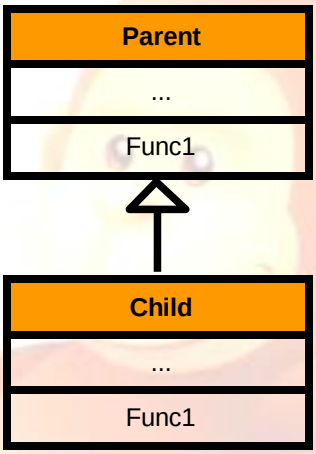
Open Source Editor Note Pad++ has been used in this document for code highlighting.

© Virtual University of Pakistan

Lecture No.25

Overriding Member Functions of Base Class in Derived Class (Function Overriding)

Derived class can override the member functions of its base class. To override a function the derived class simply provides a function with the same signature[12] (prototype) as that of its base class



Overriding

Overloading vs. Overriding

Function Overloading is done within the scope of one class whereas Function Overriding is done in scope of parent and child classes (inheritance).

[12] same name, parameters and return type

```
class Parent {
public:
    void Func1();
    void Func1(int);
};

class Child: public
    Parent {
public:
    void Func1();
};
```

Function Overriding:
Two functions in two base derived classes

```
class Parent {
public:
    void Func1();
    void Func1(int);
};

class Child: public
    Parent {
public:
    void Func1();
};
```

Function Overloading:
Two functions in same class with same name but different parameters and return type.

```
class Parent {
public:
    void Func1();
    void Func1(int);
};

class Child: public
    Parent {
public:
    void Func1();
};
```

Overriding within the scope of single class is error due to duplicate declaration (two member functions with same prototype)

```
class Parent {
public:
        void    Func1();
        void    Func1();        //Error
};
```

Overriding Member Functions of Base Class

Derive class can override member function of base class such that the working of function is totally changed.

Example

```
class Person{
public:
        void Walk();
};
class ParalyzedPerson: public Person{
public:
        void Walk();
};
```


© Virtual University of Pakistan

Overriding Member Functions of Base Class

Derive class can override member function of base class such that the working of function is similar to former implementation.

Example

```
class Person{
        char *name;
public:
        Person(char *=NULL);
        const char *GetName() const;
        void Print(){
                cout << "Name: " << name
                        << endl;
        }
};
class Student : public Person{
        char * major;
public:
        Student(char * aName, char* aMajor);
        void Print(){
                cout    <<"Name: "<< GetName()<<endl
                        << "Major:" << major<< endl;
        }
};
int main(){
        Student a("Ahmad", "Computer Science");
        a.Print();
        return 0;
}
```

Output:

```
Name: Ahmed
Major: Computer Science
```

Overriding Member Functions of Base Class

Derive class can call base class member function from its overridden member function to perform the base class part and then can perform its own tasks for example student class can call base class person method Print to show name of student and then can show study program of student by itself, this approach is in accordance with the Object Oriented Programming principles which says that each class should perform its tasks by itself, this is shown in example below,

Example

```
class Student : public Person{
        char * major;
public:
        Student(char * aName, char* m);
```

```
        void Print(){
                Print();      //Calling Print of Person to print student name
                cout<<"Major:" << major <<endl; // Displaying study program of
student
        }
};
int main(){
        Student a("Ahmad", "Computer Science");
        a.Print();
        return 0;
}
```

But there will be no output on the screen after executing this code due to minor mistake it is due to the reason that our both classes Student and Person have methods with name Print and we are calling Print method of Person from Student Print method compiler will call to Print of Student class again and again recursively as we are calling Print method form Student class.

In this case it is necessary to mention that we are calling base class Print method as shown below,

```
class Student : public Person{
        char * major;
public:
        Student(char * aName, char* m);

        void Print(){
                Person::Print();
                cout<<"Major:" << major <<endl;
        }
...
};
```

Example

```
int main(){
        Student a("Ahmad", "Computer Science");
        a.Print();
        return 0;
}
```

Output

```
Name: Ahmed
Major: Computer Science
```

Overriding Member Functions of Base Class

© Virtual University of Pakistan

As we see previously when we use pointers to call overridden methods they are called according to static type of pointer as shown below,

Example

```
int main(){
        Student a("Ahmad", "Computer Science");
        Student *sPtr = &a;
        sPtr->Print(); // static type of sPtr is Student * so Student Print method will be called

        Person *pPtr = sPtr; // static type of pPtr is Person * so Person Print method will be called
        pPtr->Print();
        return 0;
}
```

Output:

```
Name: Ahmed
Major: Computer Science

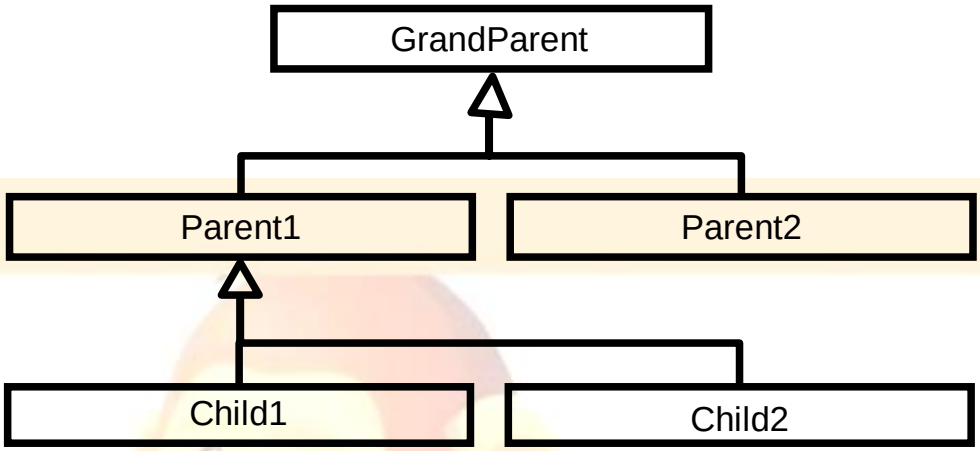
Name: Ahmed
```

This may be undesirable and to avoid this situation avoid using simple method overriding in this way its modified form (virtual functions will be covered in coming lectures).

Hierarchy of Inheritance

We represent the classes involved in inheritance relation in tree like hierarchy

Example



Direct Base Class

A direct base class is explicitly listed in a derived class's header with a colon (:)

```
class Child1:public Parent1 {        // Here Parent1 is Direct Base Class of Child1
…
};
```

Indirect Base Class

An indirect base class is not explicitly listed in a derived class's header with a colon (:)

It is inherited from two or more levels up the hierarchy of inheritance

```
class GrandParent {
…
};
class Parent1: public GrandParent {
…
};
class Child1:public Parent1 {        // Here GrandParent is InDirect Base Class of Child1
…
};
```


© Virtual University of Pakistan

Lecture No.26

Base Initialization

We saw in the previous lectures, that in the case of copy constructor we have to call base class constructor from the initialization list of child class because implicit base class object is created first and then derived class object is created. Now we see this concept in detail in the perspective of class hierarchy,

- The child can only call constructor of its direct base class to perform its initialization using its constructor *initialization list.*
- The child cannot call the constructor of any of its indirect base classes to perform their initialization using its constructor *initialization list*

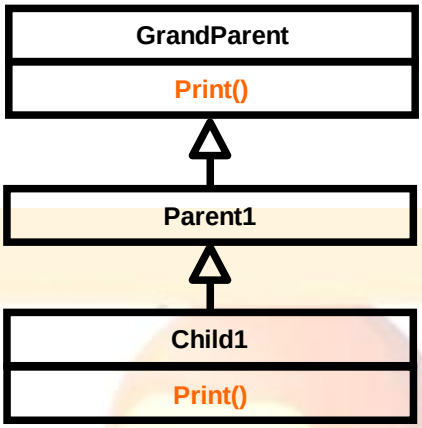
It is explained in example code given below,

Example

```
class GrandParent{
        int gpData;
public:
        GrandParent() : gpData(0){...}
        GrandParent(int i) : gpData(i){...}
        void Print() const;
};
class Parent1: public GrandParent{
        int pData;
public:
        Parent1() : GrandParent(), pData(0) {...}
};
class Child1 : public Parent1 {
public:
        Child1() : Parent1()    {...}
        Child1(int i) : GrandParent (i) //Error: Child1 can not call its indirect base class GrandParent Constructor from its constructor initialization list.
        {...}
        void Print() const;
};
```

Overriding in case of class hierarchy:

In class hierarchy Child class can override the function of any of its Parent class (direct or indirect) as shown in the diagram below (we see this concept in detail in types of inheritance given below),



Example

```
void GrandParent::Print() {
        cout  << "GrandParent::Print"
                << endl;
}

void Child1::Print() {
        cout << "Child1::Print" << endl;
}
int main(){
        Child1 obj;
        obj.Print();
        obj.Parent1::Print();
        obj.GrandParent::Print();
        return 0;
}
```

Output

```
Child1::Print
GrandParent::Print
GrandParent::Print
```

Types of Inheritance

There are three types of inheritance

- Public
- Protected
- Private

We can use these keywords (public, private or protected) to specify the type of inheritance

© Virtual University of Pakistan

a. Public Inheritance

class Child: public Parent {...};

Member access in	
Base Class	**Derived Class**
Public	Public
Protected	Protected
Private	Hidden

b. Protected Inheritance

class Child: protected Parent {...};

Member access in	
Base Class	**Derived Class**
Public	Protected
Protected	Protected
Private	Hidden

c. Private Inheritance

class Child: private Parent {...};

Member access in	
Base Class	**Derived Class**
Public	Private
Protected	Private
Private	Hidden

If the user does not specifies the type of inheritance then the default type is private inheritance,

```
class Child: private Parent {…}
```

is equivalent to,

```
class Child: Parent {…}
```

We have seen public inheritance in previous lecture, now we see private and protected inheritance in detail,

Private Inheritance

We use private inheritance when we want to reuse code of some class. Private Inheritance is used to model "Implemented in terms of" relationship

Example

Suppose we have a class collection to store element collection as shown below,

```
class Collection {
…
public:
        void AddElement(int);
        bool SearchElement(int);
        bool SearchElementAgain(int);
        bool DeleteElement(int);
};
```

As you can see it supports the following methods,
AddElement: to add elements in collection
SearchElement: search any element in collection it will true as soon as any element will be found
SearchElementAgain: finds second instance of any element in collection it will return true as soon as it will find any duplicate entry of any element.
DeleteElement: to delete any entry from collection

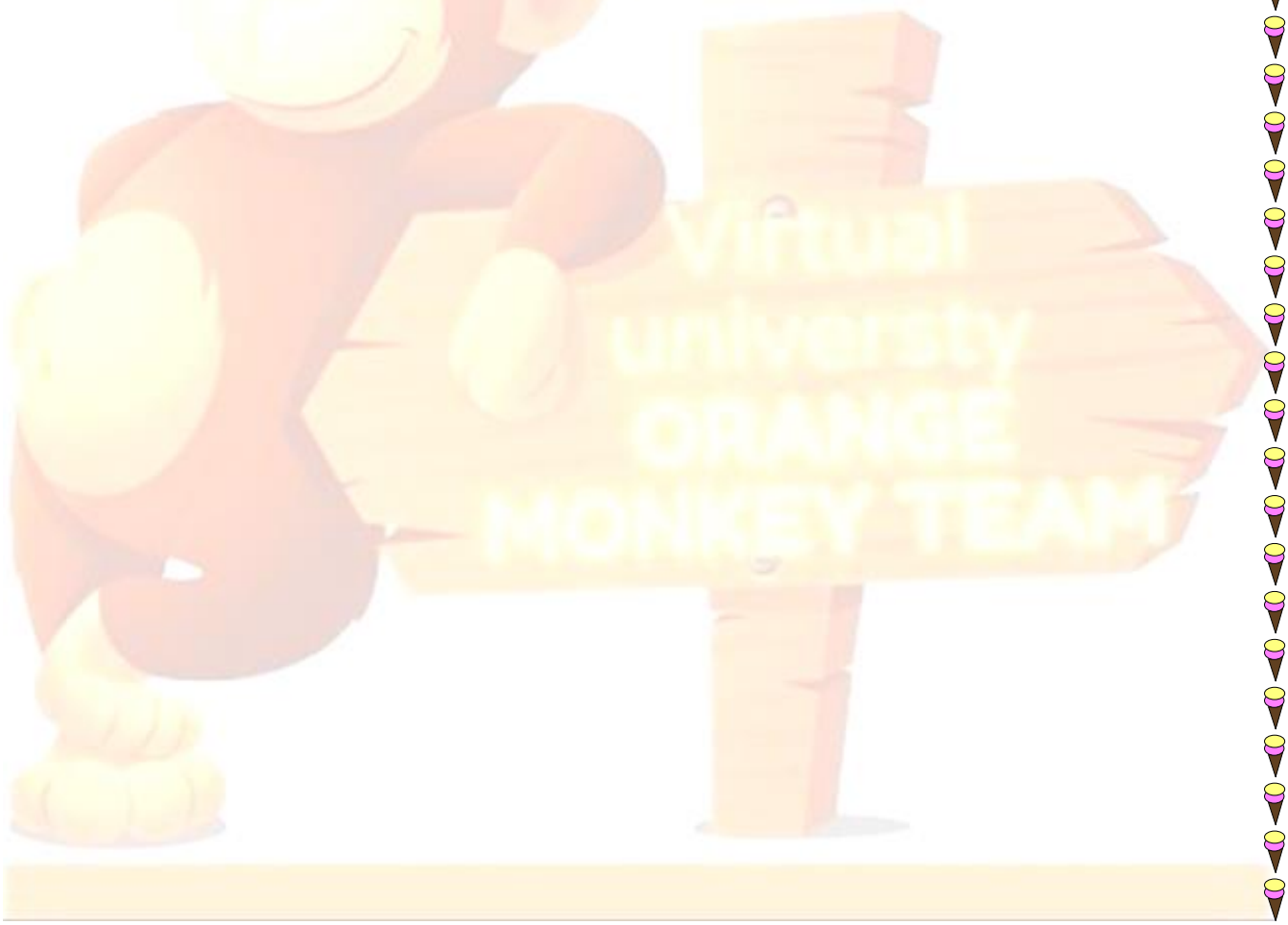
You can see that Class collection allows duplicate elements.
Suppose now we want to implement class Set, class Set has very similar functionality as that of collection class with the difference that Set class can not allow duplicate elements in it, so we can use the concept of inheritance here, we can derive class Set from class Collection.
But we can not use public inheritance here as it allows interface (all functions) of base class to be accessed using derived class object, but we don't want to allow all functions of class Collection to act on Set class object. We only want to use some functions of Collection class (Base class) in derived Set class for to implement Set class functionality so we will use private inheritance here the two main advantages we will achieve through private inheritance in this case are,

1. Specialization of class according to set class (removing extra features)
2. Making interface of collection class inaccessible from outside world using class set reference.

© Virtual University of Pakistan

Class Set

```
class Set: private Collection {
private:
        …
public:
        void AddMember(int);
        bool  IsMember(int);
        bool DeleteMember(int);
};
void Set::AddMember(int i){
                if (! IsMember(i) )
                        AddElement(i);
}
bool Set::IsMember(int i){
        return SearchElement(i);
}
```



Lecture No.27

Previous Lecture Review:

In last lecture we saw the use of private inheritance using the example of classes **Collection and Set,** we saw that class collection was allowing duplicate elements but set class can not allow duplicate elements in it however other functionality was same, so we privately inherited class set from collection and added only those member functions in derived class **set** that were related to set class, the two main advantages we achieved through private inheritance in this case were,

1. Specialization of class collection according to set class (removing extra features)
2. Making interface of collection class inaccessible from outside world using class set reference, to avoid any unintentional use of collection class restricted features in set class.

The code of both classes in shown below,

Class Collection	Class Set

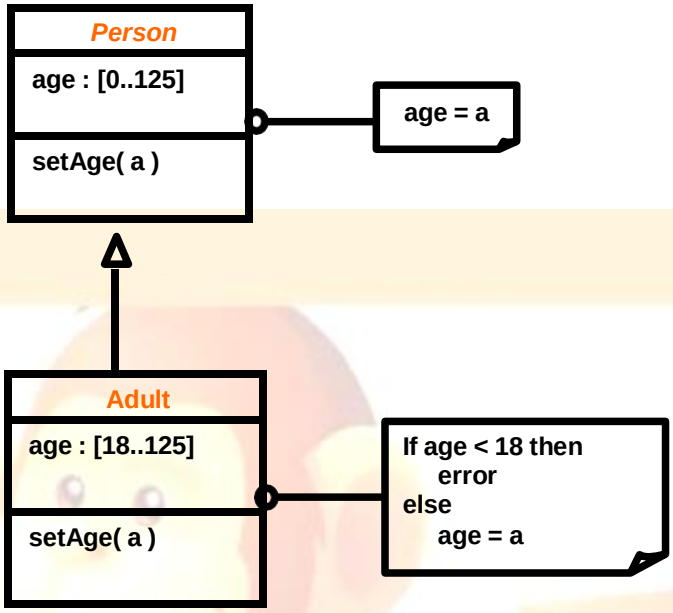
class Collection {
…
public:
void AddElement(int);
bool SearchElement(int);
bool SearchElementAgain(int);
bool DeleteElement(int);
};
	class Set: private Collection {
	private:
	…
	public:
	void AddMember(int);
	bool IsMember(int);
	bool DeleteMember(int);
	};

As we discussed that we achieve specialization using private inheritance so need to see specialization (Specialization concept was discussed in Lecture No.4) again in detail,

Specialization (Restriction)

In specialization derived class is behaviourally incompatible with the base class.
Behaviourally incompatible means that base class can't always be replaced by the derived class.
Specialization is represented by **"Implemented in terms of"** relationship.
Specialization (Restriction) can be implemented using both **private and protected inheritance.**

© Virtual University of Pakistan



Example – Specialization (Restriction)

```
class Person{
        ...
protected:
        int age;
public:
        bool SetAge(int _age){
                if (_age >=0 && _age <= 125) {
                        age = _age;
                        return true;
                }
                return false;
        }
};
class Adult : private Person {
        public:
        bool SetAge(int _age){
                if (_age >=18 && _age <= 125) {
                        age = _age;
                        return true;
                }
                return false;
        }
};
```

Essential properties of Private Inheritance:

1. In Private Inheritance only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object

Example

```
class Parent{
};
class Child : private Parent{
};

int main(){
        Child cobj;
        Parent *pptr = & cobj;      //Error
        return 0;
}
```

```
void DoSomething(const Parent &);

Child::Child(){
        Parent & pPtr = static_cast<Parent &>(*this);  // fine
        DoSomething(pPtr);
        //      DoSomething(*this); // this single line is equal to two lines above.
}
```

2. As was in the case of public inheritance child class object has an anonymous object of parent class.
3. As was in the case of public inheritance the default constructor and copy constructor of parent class are called when constructing object of derived class.

Example

```
class Parent{
public:
        Parent(){
                cout << "Parent Constructor";
        }

        Parent(const Parent & prhs){
          cout << "Parent Copy Constructor";
        }
};
class Child: private Parent{
public:
         Child(){
```


© Virtual University of Pakistan

```
                cout << "Child Constructor";
        }

        Child(const Child & crhs) : Parent(crhs){
                cout << "Child Copy Constructor";
        }
};
int main() {
        Child cobj1; // default constructor will be invoked
        Child cobj2 = cobj1;  // copy constructor will be invoked
        //Child cobj2(cobj1); // another way of calling copy constructor
        return 0;
}
```

Output:

```
Parent Constructor
Child  Constructor
Parent Copy Constructor
Child Copy Constructor
```

4. In private inheritance the derived class that is more than one level down the hierarchy cannot access the member functions of grand parent class as public and protected members functions of derived class become private members of privately derived class for all practical purposes. For example see the code below here Child class is derived class that is more than one level down the hierarchy and hence can not access the member functions of **GrandParent** class.

```
class GrandParent{
public :
        void DoSomething();
};

class Parent: private GrandParent{
        void SomeFunction(){
                DoSomething();
        }
};
class Child: private Parent
{
public:
        Child() {
                DoSomething();          //Error
        }
};
```

5. In private inheritance the derived class that is more than one level down the hierarchy cannot convert the pointer or reference to that of GrandParent , for example in code given below Child class can not convert its reference to its GrandParent reference. The reason is same that in private inheritance we implement specialization and we restrict all features of base class to privately derived class only and its friend classes or functions.

```
void DoSomething(GrandParent&);
class GrandParent{
};
class Parent: private GrandParent{
public:
        Parent()  {DoSomething(*this);}
};
class Child: private Parent {
public:
        Child()
        {
                DoSomething(*this);          //Error
        }
};
```

Protected Inheritance

If a class D has been derived using protected inheritance from class B (If B is a protected base and D is derived class) then public and protected members of B can be accessed by member functions and friends of class D and classes derived from D. Protected inheritance is used to build class hierarchy using **"Implemented in terms of"** relationship.

If B is a protected base and D is derived class then member functions and friends of class D and classes derived from D can access member functions of class B, (note that in private inheritance only derived class can access the member functions of base class). So we can say that protected inheritance lies between public and private inheritance.

© Virtual University of Pakistan

```
class GrandParent{
public :
        void DoSomething();
};

class Parent: protected GrandParent{
        void SomeFunction(){
                DoSomething();
        }
};
class Child: protected Parent
{
public:
        Child()
        {
                DoSomething();
        }
};
```

Properties of Protected Inheritance

If B is a protected base and D is derived class then only friends and members of D and friends and members of class derived from D can convert D* to B* or D& to B& (in private inheritance only derived class or its friends can convert pointer to base class)

```
void DoSomething(GrandParent&);

class GrandParent{
};

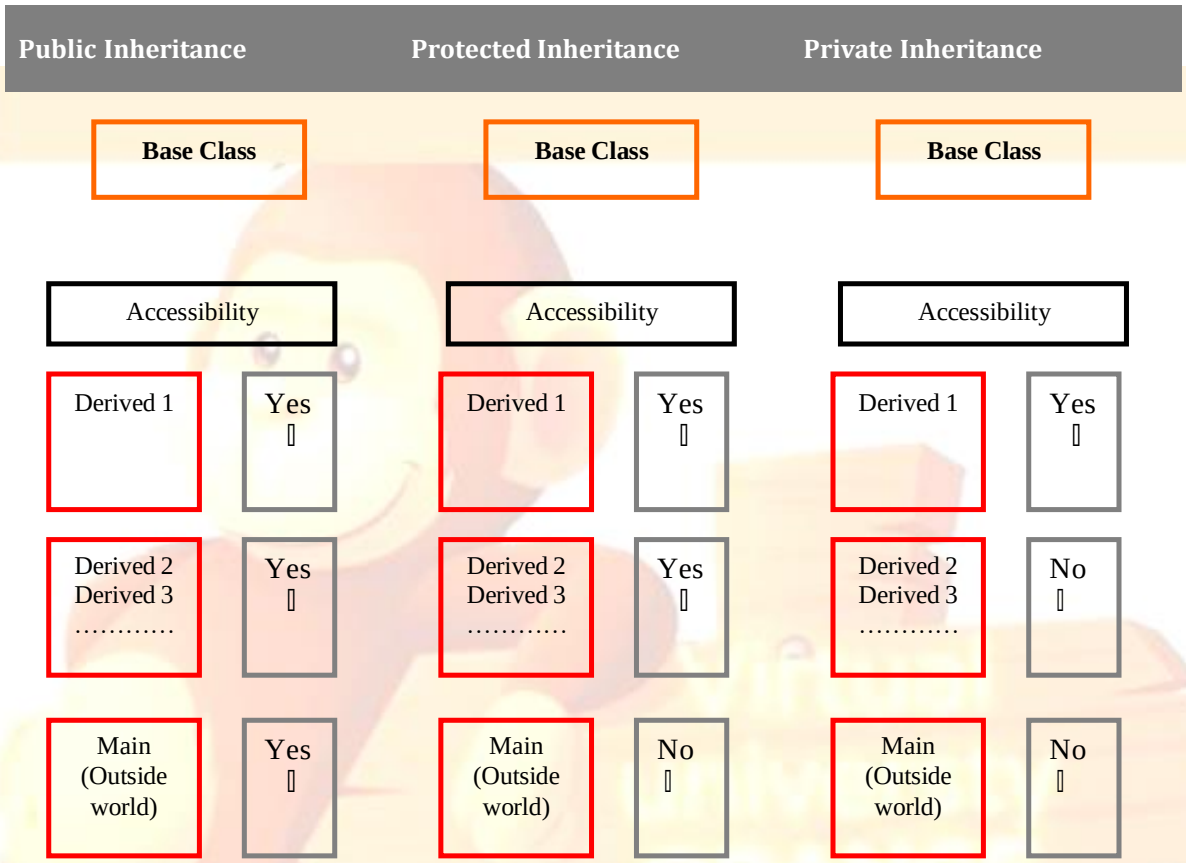
class Parent: protected GrandParent{
};
class Child: protected Parent {
public:
        Child()
        {
                DoSomething(*this);
        }
};
```

Importance of Private and Protected inheritance:

As we have seen that private and protected inheritance is being used for implementing **"Implemented in terms of"** relationship so it is important that we limit different features of base class in child classes that is what we achieve using either private or protected inheritance according to our requirement.

Comparison of public, protected and private inheritance:

We can show accessibility of public member functions of base class in derived classes in these three different types of inheritance as follows,



Private data members will NOT be accessible in any derived class or in main function.

Protected data members will become private data members in case of private inheritance and protected data members of derived class in case of protected inheritance.

A Good Programming Exercise:

A good programming exercise would be that you write a program that shows accessibility of all types of members functions for all types of inheritance in derived class/es.

© Virtual University of Pakistan

Lecture No.28

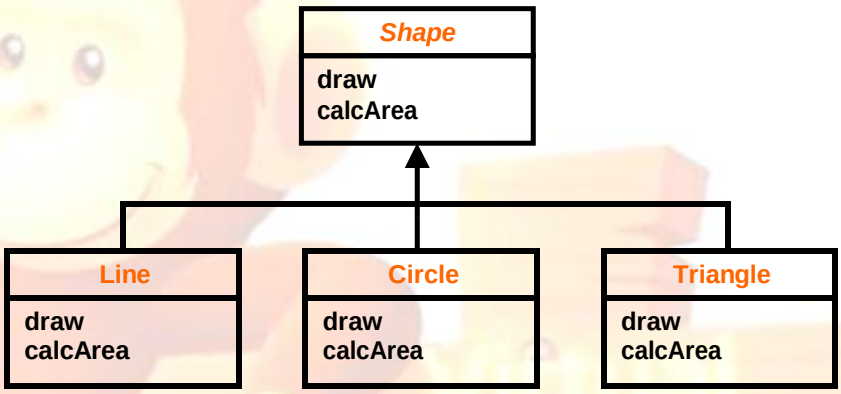
Virtual Functions

Problem Statement:

*Develop a function that can draw different types of **geometric shapes** from an array*

Shape Hierarchy:

We have to implement following shape hierarchy for this,



Problem Description:

a. We want to implement this shape hierarchy in such a manner that our function will take Shape pointers array (Shape * []) and this array size as parameters and then it will draw the appropriate shape.
b. This Shape pointer array (Shape * []) may store pointers to all kinds of Shapes (Line, Circle and Triangle classes) as these classes have IS-A relationship with Shape class (We know base class pointer can store pointer of any of its derived classes in case of public inheritance IS-A relationship).
c. The purpose of implementing this hierarchy in such a way is that we can avoid complex code of calling draw function for each class separately after checking the class type, instead we can use a single function call in a loop to draw all kinds of Shapes as shown below,

```
void drawShapes(Shapes *array[], int size){

        for (int i = 0 ; i < size ; i ++)
            array[i]->draw(); // this function call will work for all types of
                                //Shapes (Line, Circle and Triangle)

}
```

Implementation:

It can be done using concept of inheritance as there exists "IS-A" relationship between general Shape class and derived Line, Circle and Triangle classes.

So in code below, we wrote a general Shape class and then we publicly derived Line, Circle and Triangle classes from it, in main we create objects of those Shapes that we needed like Line, Circle and Triangle and stored them in single shape array, then we called **drawShapes** function by passing this array to draw appropriate shapes.

```
class Shape {
        …
protected:
        char _type;
public:
        Shape() { }
        void draw(){ cout << "Shape\n"; }
        int calcArea() { return 0; }
        char getType() { return _type; }
};
class Line : public Shape {

public:
        Line(Point p1, Point p2) {

        }
        void draw(){ cout << "Line\n"; }
};
class Circle : public Shape {
public:
        Circle(Point center, double radius) {

        }
        void draw(){ cout << "Circle\n"; }
        int calcArea() { … }
};
class Triangle : public Shape {

public:
        Triangle(Line l1, Line l2, double angle)
        {
}
        void draw(){ cout << "Triangle\n"; }
        int calcArea() { … }

};
int main() {
        Shape* _shape[ 10 ];
```

```
        Point p1(0, 0), p2(10, 10);
        shape[1] = new Line(p1, p2);
        shape[2] = new Circle(p1, 15);
        void drawShapes( shape, 10 );
        return 0;
}

/*Function drawShapes()*/

void drawShapes(Shape* _shape[], int size) {
        for (int i = 0; i < size; i++) {
                _shape[i]->draw();
        }
}
```

After compiling this code we will see the following output,

Sample Output

```
Shape
Shape
Shape
Shape
...
```

As you have seen this code is not showing correct output, it is showing text Shape again and again instead of showing appropriate class name that was stored in Shape array like Line, Circle or Triangle so there is some logical error in this code.
Where is problem?
Problem is that as we stored our derived classes (Line, Circle and Triangles) Objects in Shape pointer array and then called draw method; every time draw method of Shape class was called.
Why?
Due to the reason that static type of this array is Shape * so draw method is Shape class will always be called whether pointer stored in it is Line, Circle or Triangle pointer or Shape class pointer.

So we have to think about some modification,

Solution 1:
One solution is that we can modify our **drawShapes** function as follows,

```
void drawShapes( Shape* _shape[], int size) {
        for (int i = 0; i < size; i++) {

                // Determine object type with switch & accordingly call draw()
method
```

```
        }
}
```

The required Switch Logic for this kind of functionality is given below,

```
switch ( _shape[i]->getType() )
{
        case 'L':
                static_cast<Line*>(_shape[i])->draw();
                break;
        case 'C':
                static_cast<Circle*>(_shape[i])->draw();
                break;
        …
}
```

Equivalent If Logic is given below,

```
if ( _shape[i]->getType() == 'L' )
        static_cast<Line*>(_shape[i])->draw();
else if ( _shape[i]->getType() == 'C' )
    static_cast<Circle*>(_shape[i])->draw();
…
```

Sample Output:

```
Line
Circle
Triangle
Circle
…
```

You can see that this code is very complex for both switch or if else conditions.

But there are many Problems with this approach these are given below:

Delocalized Code

Suppose we have to write another function that prints area of each shape from an input array. We have to write same switch or if else logic in that function implementation also to work it correctly.

```
void printArea( Shape* _shape[], int size) {
        for (int i = 0; i < size; i++) {
                // Print shape name.
                // Determine object type with
                // switch & accordingly call
                // calcArea() method.
        }
}
```

Required Switch Logic

```
switch ( _shape[i]->getType() )
{
        case 'L':
                static_cast<Line*>(_shape[i])->calcArea(); break;
        case 'C':
         static_cast<Circle*>(_shape[i])->calcArea(); break;
        …
}
```

The above switch logic is same as was in function drawArray() with difference of name of function being called.

So this approach will result in delocalized code (same code in different places) with following consequences,
Writing same code again and again at different places may produce errors as programmer may forget to write switch cases.
Suppose we have added one more Shape in our program then we will have to add one more switch case or if else condition in all functions where we have used this logic if programmer by mistake forgets to add it in any single function whole program will show incorrect output.
So due to above mentioned reasons this sort of code is very hard to maintain.

Solution?

To avoid switch, we need a mechanism that can select the message target (class) automatically!

Polymorphism Revisited:

In OO model, polymorphism means that different objects can behave in different ways for the same message (stimulus) consequently, sender of a message does not need to know the exact class of receiver.
In other words when we have inheritance relationship and we have written basic structure of our program correctly we need not to worry about the no. of classes in our program. For example in case of shapes hierarchy (Line, Circle, Triangle and so on…) our program will keep working correctly if we add more shapes, draw method of appropriate shape class should automatically be called, that is the benefit of Object Oriented Programming,

```
        void drawShapes(Shapes *array[], int size){

                for (int i = 0 ; i < size ; i ++)
                    array[i]->draw(); // this function call will work for all types of
                                                //Shapes (Line, Circle and Triangle)

        }
```

But this kind of functionality was not being achieved in above approach in which we had to write one separate case for each type of shape class.

To achieve this kind of functionality we have the concept of **virtual functions** we make those functions virtual in base class which will be implemented by derived classes according to their requirements.

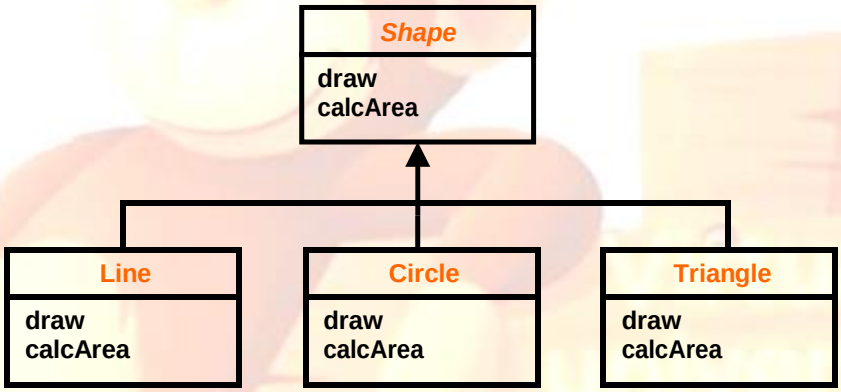
Virtual Functions:

Virtual functions achieve exactly same kind of functionality that was achieved in above code with complex code of switch statement.

- Target class of a virtual function call is determined at run-time **automatically**.
- In C++, we declare a function virtual by preceding the function header with keyword "virtual"

Now we see how we can use virtual functions in case of our shape hierarchy,

Shape Hierarchy



We will make those functions virtual in shape that need to be overridden by derived classes like Draw method,

```
class Shape {
        …
        virtual void draw();
};
```

```
class Shape {
        …
        virtual void draw();
        virtual int calcArea();
};
class Line : public Shape {
        …
        virtual void draw() { cout<<"Line…\n";
                }
};
```

```
class Circle : public Shape {
        ...
        virtual void draw() { cout<<"Circle....\n";
                    }
        virtual int calcArea();
};
class Triangle : public Shape {
        ...
        virtual void draw(){ cout<<"Triangle...\n";
                    }

        virtual int calcArea();
};
void drawShapes(Shape* _shape[], int size) {
                for (int i = 0; i < size; i++) {
                        _shape[i]->draw();
        }
}
```

Sample Output:

```
Line
Circle
Triangle
Circle
...
```

Similarly if we have another function that will calculate and print area of different shapes it will also be written in same way, (Now we have no need to add switch or if else code)

```
void printArea(Shape* _shape[], int size) {
                for (int i = 0; i < size; i++) {
                        // Print shape name
                        cout<< _shape[i]->calcArea();
                        cout << endl;
        }
}
```

Static vs Dynamic Binding

- **Static binding** means that target function for a call is selected at compile time
- **Dynamic binding** means that target function for a call is selected at run time

```
Line _line;
_line.draw();  // Always Line::draw will be called
```

Shape* _shape = new Line();
_shape->draw(); // Shape::draw called if draw() is not virtual because of static type of Shape *

Shape* _shape = new Line();
_shape->draw(); // Line::draw called as draw() is virtual in base Shape class

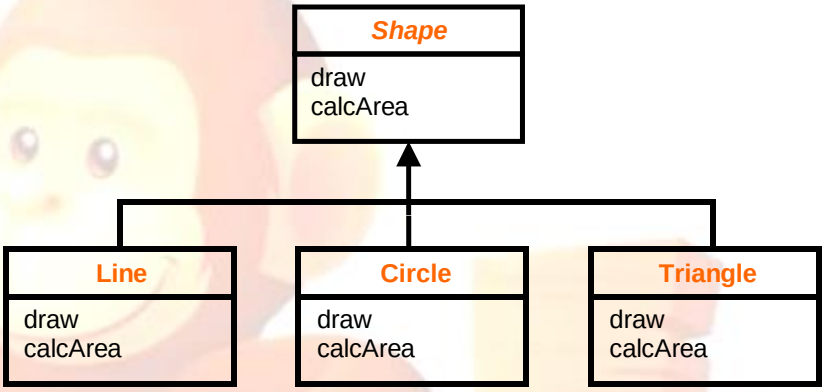


© Virtual University of Pakistan

Lecture No.29

Previous Lecture Review:

The main concept of Polymorphism is that the same method can behave differently according to the object with respect of which it has been called.
We combine all related classes having same kind of functionality in a class hierarchy as shown below,



The draw and calcArea methods here are showing polymorphic behavior as they exist in base as well as in derived classes, the method that will be called is decided at runtime according to the nature of the calling object, for example in the case of above class hierarchy if we call draw method with respect of Line class object, draw of Line class will be called and if we call it with reference of Circle or Triangle class, draw method of Circle or Triangle class will be called.

We achieve this type of polymorphism[13] using virtual functions.
The advantage of this approach is that sender simply pass method call and appropriate method is called automatically, as we saw in previous lecture in which we simply used for loop to draw all kinds of shapes using shape pointers array.
We also saw the drawbacks of achieving same functionality without virtual functions using switch statement that resulted in delocalized and complex code. Then we saw solution of this problem using virtual functions.

Abstract Classes

In our Shape class hierarchy Shape class has no real world existence, there will be no object in real world with name **Shape** there will always be an object of kind of shape like **Line**, **Circle** or **Triangle**, this kind of class with no specific real world existence are called **Abstract Classes**, as these classes are Abstract their behavior becomes also abstract like Draw and calcArea methods in case of Shape class, the actual concepts

[13] Polymorphism may exist in same class (function overloading) or it may exist in different classes (function overriding) here we are referring to function overriding (same functions in base and derived classes).

are realized in derived classes, like all kinds of shapes will have draw and calcArea method, so we add these methods in general Shape class and all kinds of Shapes inheriting for this general shape class given their own implementation of these methods.

Abstract class's objects cannot be instantiated they are used for inheriting interface and/or implementation, so that derived classes can give implementation of these concepts.

Concrete Classes

Concrete classes Implements a concrete concept they can be instantiated they may inherit from an abstract class or another concrete class. So far the classes we studied were concrete classes.

Abstract Classes in C++

In C++, we can make a class abstract by making its function(s) pure virtual. Conversely, a class with no pure virtual function is a concrete class (which object can be instantiated)

Pure Virtual Functions

A pure virtual represents an abstract behavior and may have not implementation for example draw method in Shape class represent abstract behavior as Shape class itself doesn't have its existence in real world so there is no question of drawing it however its derived concrete classes like Line, Circle and Triangle does have physical existence and overridden draw method in these classes will have implementation . A function is declared pure virtual by following its header with "= 0".

```
virtual void draw() = 0;
```

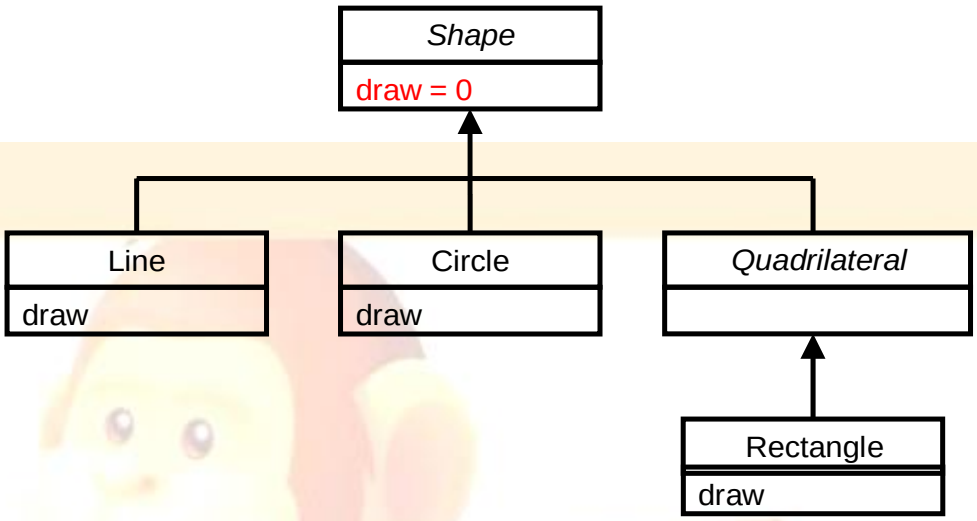
A class having pure virtual function(s) becomes abstract

```
class Shape {
        …
public:
        virtual void draw() = 0;
};
```

Now when we will try to created object of our Shape class in our program compiler will give an error as shown below,

```
Shape s;                    // Error!
```

Shape Hierarchy



A derived class of an abstract class remains abstract until it provides implementation for all pure virtual functions as shown below in other words we can say that at least one class in bottom of class hierarchy should give implementation of pure virtual function (Abstract classes are present at root or near root of the class hierarchy tree, whereas concrete classes are near leaves of class hierarchy tree) see the code below of above class heirarchi here Quadrilateral is also abstract class as it is derived from Shape class but is not giving implementation of draw method we can also not create its object and it is necessary to have one more derived class from quadrilateral that gives implementation of draw method otherwise there will be compiler error as we can not have all abstract classes in heirarchi there should be at least one concrete class at leaf in class heirarchi,

```
class Quadrilateral : public Shape {
        …
        // No overriding draw() method
};
…
Quadrilateral q; // Error!

class Rectangle:public Quadrilateral{
        …
        public:
        // void draw()
        virtual void draw() {       // once a function is declared as virtual it remains
virtual in all hierarchy
                … // function body
        }
};
…
Rectangle r;              // OK
```

Virtual Destructors

The destructor is called according to static type of any class pointer for example if we have saved derived class pointers in shape class pointers array as we did in previous lecture when we will call destructor using delete operator the destructor of base class Shape will be called as static type of array is Shape. This will destroy the base class object only derived class object will not be destroyed this is explained in the example code below,

```
class Shape {
        ...
        public:
        ~Shape() {
                cout << "Shape destructor
        called\n";
        }
};

class Quadrilateral : public Shape {
        ...
        public:
        ~Quadrilateral() {
                cout << "Quadrilateral destructor
        called\n";
        }
};

class Rectangle : public Quadrilateral {
        ...
        public:
        ~Rectangle() {
                cout << "Rectangle destructor
                called\n";
        }
};

int main() {
        Shape* pShape = new Rectangle();
        delete pShape;
        return 0;
}
```

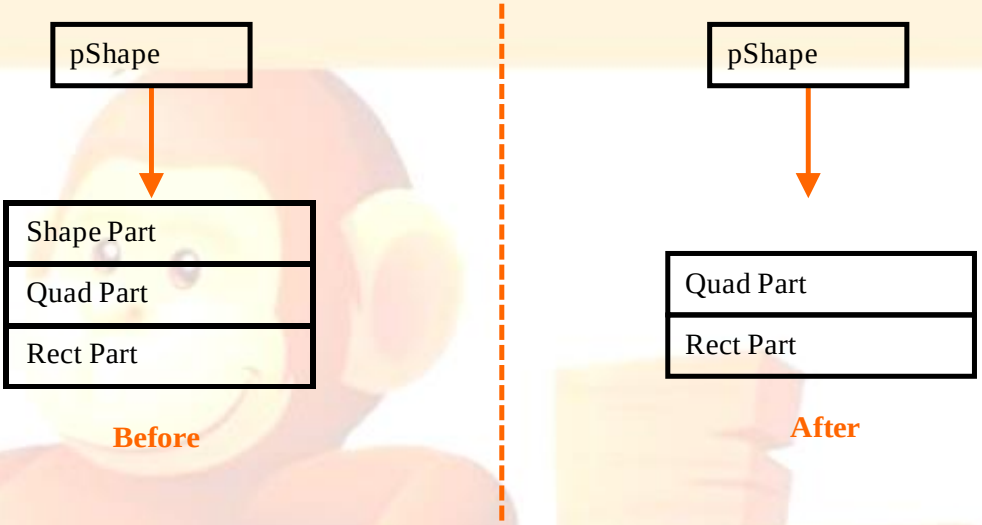
When delete operator is applied to a base class pointer, base class destructor is called regardless of the object type
Output

```
Shape destructor called
```


© Virtual University of Pakistan

Result

You can see by diagram below that only base class part of object will be deleted other parts will remain as it is this result in memory leak (wastage of memory),



This issue will be solved in the same way as we solved the problem of calling derived classes **Draw methods** using base class pointer by the use **Virtual keyword**.

Virtual Destructors

Make the base class destructor virtual as we made Draw method virtual in base class,

```
class Shape {
        ...
        public:
        virtual ~Shape() {
                cout << "Shape destructor called\n"; }
};
class Quadrilateral : public Shape {
        ...
        public:
        virtual ~Quadrilateral() {
                cout << "Quadrilateral destructor called\n";
        }
};
class Rectangle : public Quadrilateral {
        ...
        public:
        virtual ~Rectangle() {
                cout << "Rectangle destructor called\n";
```

```
        }
};
```

Now base class destructor will run after the derived class destructor

```
int main() {
        Shape* pShape = new Recrangle();
        delete pShape;
        return 0;
}
```

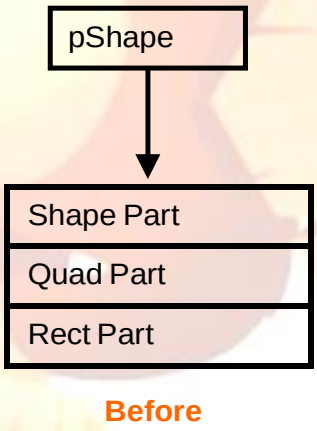
Output

Now you can see that output is correct all destructors are being called in correct order,

```
Rectangle destructor called
Quadrilateral destructor called
Shape destructor called
```

Result

Now you can see result complete object is being deleted so there is no memory leak (waste of memory),





Virtual Functions – Usage

Virtual function are used in two ways,

© Virtual University of Pakistan

- Virtual Functions can be used to when we want to inherit interface and implementation (Simple virtual functions) mean base class as well as derived class will have implementation.

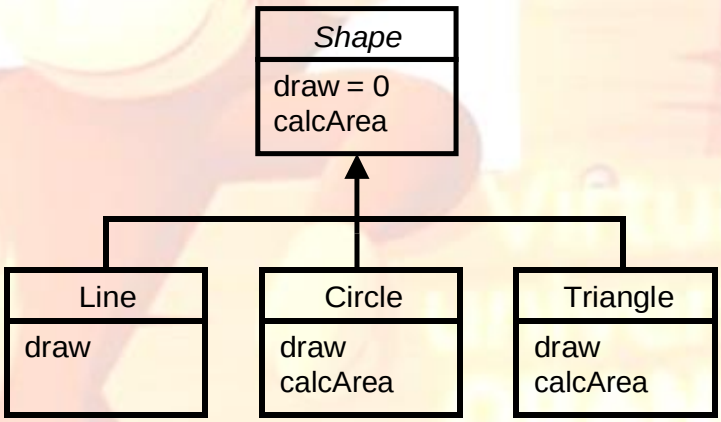
  virtual void draw();

- Just inherit interface (Pure Virtual functions) mean only derived classes will will have implementation base may not have implementation.

  virtual void draw() = 0;

Inherit interface and implementation:

First case of simple virtual functions is useful when we have some derived classes that will also not have implementation of virtual method for example Line is also also Shape but it doesn't have area similarly if we had Point derived class it also doesn't have any area in both cases we will simply not write implementation of calcArea() method and hence calcArea of base Shape class will be called which will simply display area as zero this is shown below,




```
class Shape {
…
        virtual void draw() = 0; // pure virtual functions

        virtual float calcArea() { // simple virtual functions
                return 0;
        }
};
```

So as **calcArea()** is simple virtual function so, each derived class of **Shape** inherits default implementation of **calcArea(), s**ome classes may override this, such as **Circle** and **Triangle** and Others may not, such as **Point** and **Line.**

Inherit interface only:

We have made draw method as pure virtual function because each shape will need to draw whether it is simple point or line or any other shape so its suitable to be decalred as pure virtual,

As **draw()** is Pure Virtual Function so, each derived class of **Shape** inherits interface (prototype) of **draw(), e**ach concrete derived class has to provide body of **draw()** by overriding it.

V Table

Now we see compiler keeps track of virtual functions and call them correctly according to nature of the object with respect to which they are being called,

Compiler builds a virtual function table (vTable) for ***each class having virtual functions***
A vTable contains a pointer for each virtual function,

Pointer to 1st virtual function
Pointer to 2nd virtual function
…….
…….
…….

V Table

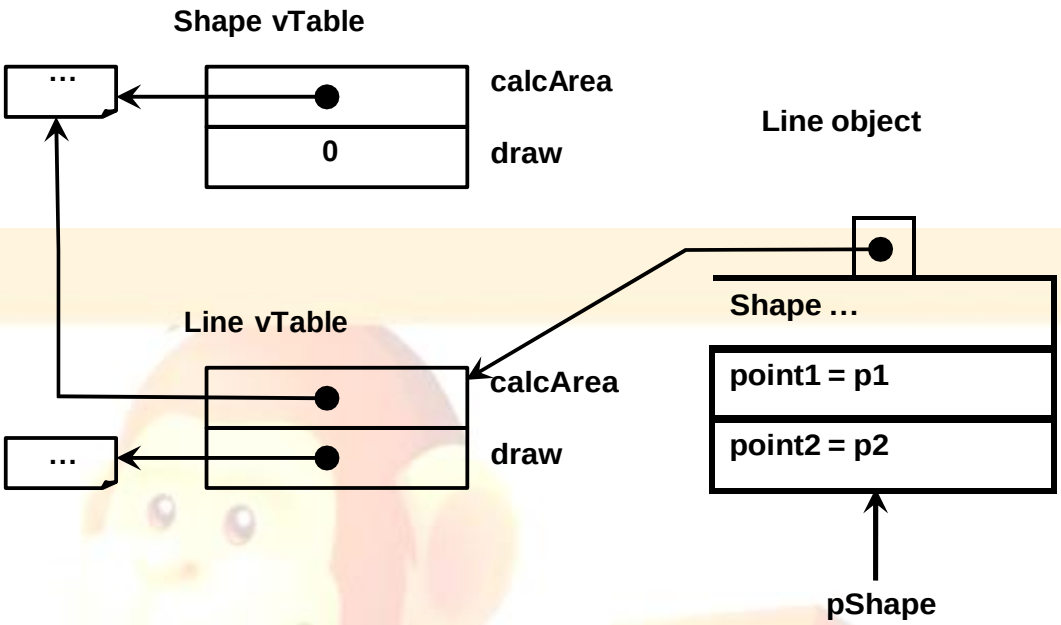
Consider the code below to see how virtual tables are created,

```
int main() {
        Point p1( 10, 10 ), p2( 30, 30 );
        Shape* pShape;

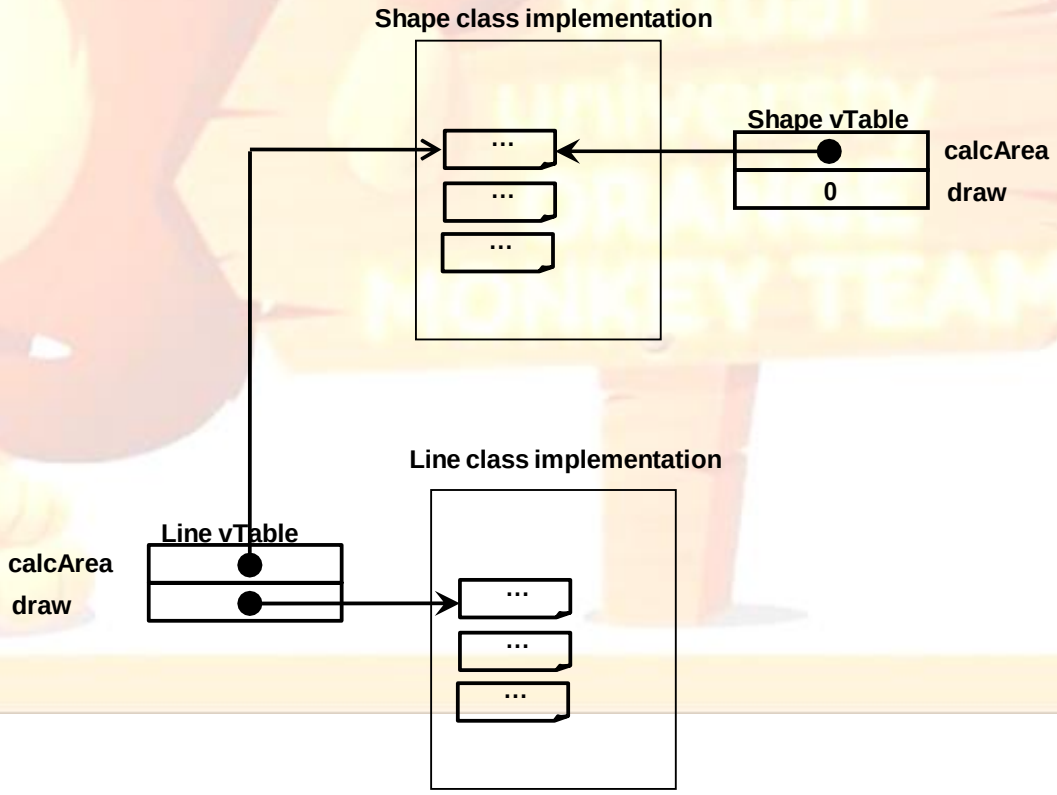
        pShape = new Line( p1, p2 );
        pShape->draw();
        pShape->calcArea();
        return 0;
}
```

We are creating Line class object here and storing its refernence in Shape class pointer and then polymorphically calling methods draw and calcArea as Shape pointer is having Line class object reference so Line class method draw and calcArea will be called we see how this actually happens,

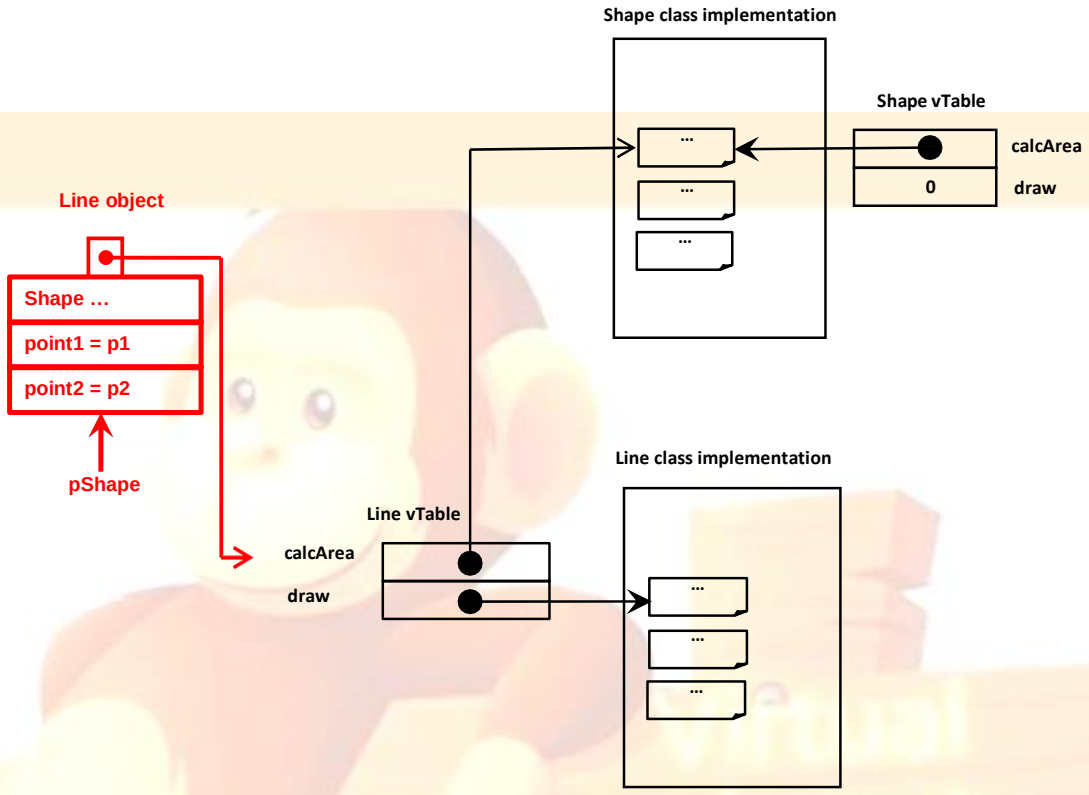
© Virtual University of Pakistan



First of all when this code will be compiled v tables along with implementation code of virtual and non virtual functions for all classes will be generated by compiler as shown below,



When line object will be crated in main it will also have pointer to v table of Line class as shown below,



Dynamic Dispatch (Dynamic Binding)

It happens in case of virtual functions, for non-virtual functions, compiler just generates code to call the function.

In case of virtual functions, compiler generates code to

- access the object
- access the associated vTable
- call the appropriate function

Conclusion

Virtual Functions should be added in code with care because they add,

- Memory overhead due to V-Tables
- Processing overhead due to extra pointer manipulation

However, this overhead is acceptable for many of the applications.

Moral: "Think about performance requirements before making a function virtual".

© Virtual University of Pakistan

Lecture No.30

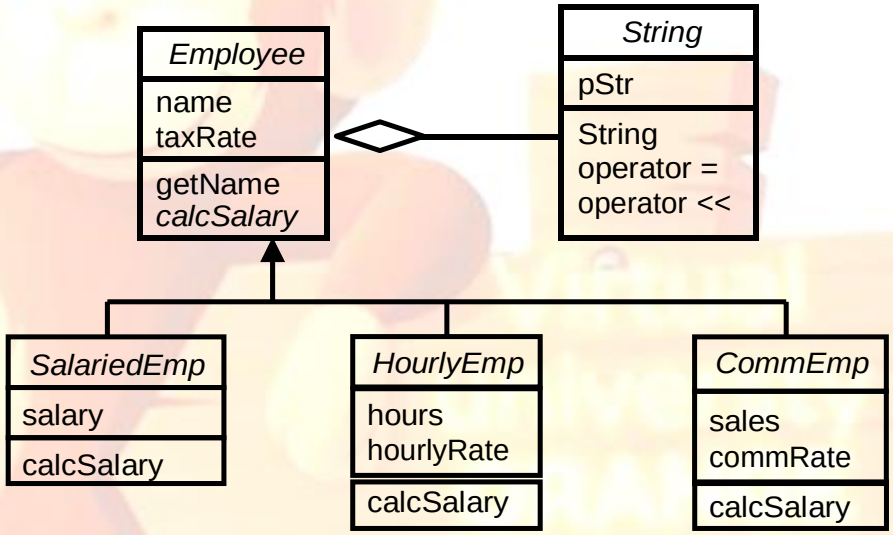
Polymorphism – Case Study: A Simple Payroll Application

We have studied polymorphism implementation in detail using virtual functions now we see an example showing how it may be useful for us,

Problem Statement

Develop a simple payroll application for a company; there are three kinds of employees in the system: salaried employee, hourly employee, and commissioned employee. The system should take input as an array containing employee objects, calculates salary polymorphically (according to employee object), and generates report.

OO Model



You can see that this model is very similar to Shape class hierarchy we saw previously, its implementation in c++ is shown below,

Class *Employee*

```
class Employee {
        private:
                String name;
                double taxRate;
        public:
                Employee( String&, double );
                String getName();
                virtual double calcSalary() = 0;
};
Employee::Employee( String& n, double tr ): name(n){
 taxRate = tr;
}
```

```
String Employee::getName() {
        return name;
}
```

Class SalariedEmp

```
class SalariedEmp : public Employee
{
private:
                double salary;
public:
                SalariedEmp(String&,double,double);
                 virtual double calcSalary();
};
SalariedEmp::SalariedEmp(String& n, double tr, double sal) : Employee( n, tr ) {
        salary = sal;
}
double SalariedEmp::calcSalary() {
        double tax = salary * taxRate;
        return salary – tax;
}
```

Class HourlyEmp

```
class HourlyEmp : public Employee {
private:
        int hours;
        double hourlyRate;
public:
        HourlyEmp(string&,double,int,double);
        virtual double calcSalary();
};
HourlyEmp ::HourlyEmp( String& n, double tr, int h, double hr ) : Employee( n, tr
) {
        hours = h;
        hourlyRate = hr;
}
double  HourlyEmp::calcSalary()
{
        double grossPay, tax;
        grossPay = hours * hourlyRate;
        tax =  grossPay *  taxRate;
        return grossPay – tax;
}
```


© Virtual University of Pakistan

Class CommEmp

```
class CommEmp : public Employee
{
private:
        double sales;
        double  commRate;
public:
        CommEmp( String&, double, double, double );
        virtual double calcSalary();
};
CommEmp::CommEmp( String& n, double tr, double s, double cr ) : Employee( n, tr ) {
        sales = s;
        commRate = cr;
}
double  CommEmp::calcSalary()
{
        double grossPay = sales * commRate;
        double tax = grossPay * taxRate;
        return grossPay – tax;
}
```

A Sample Payroll

```
int main() {
        Employee* emp[10];
        emp[0] = new SalariedEmp( “Aamir”, 0.05, 15000 );
        emp[1] = new HourlyEmp( “Faakhir”, 0.06, 160, 50 );
        emp[2] = new CommEmp( “Fuaad”, 0.04, 150000, 10 );
        …
        generatePayroll( emp, 10 );
        return 0;
}
```

Function that takes Empolyee poinsters array and calls appropriate getName and calcSalary methods

```
void generatePayroll(Employee* emp[], int size) {

        cout << “Name\tNet Salary\n\n”;
        for (int i = 0; i < size; i++) {
        cout << emp[i]->getName() << ‘\t’ << emp[i]->calcSalary() << ‘\n’;

        }
}
```

Sample Output

Name	Net Salary
Aamir	14250
Fakhir	7520
Fuaad	14400
…	

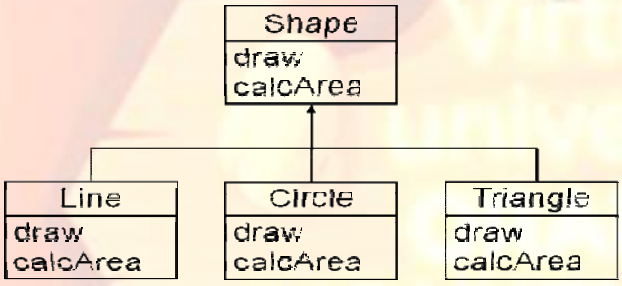
Output is as required displaying employee names and their salary polymorphically.

Important point to note here is that *Polymorphism always works with pointers of class objects not with actual objects*. In above example we used base Employee pointer array to store pointers of derived classes of class Employee and then we polymorphically called generatePayroll method to call getName and calcSal methods in a loop.

Never Treat Arrays Polymorphically:

If we use arrays of Objects Polymorphically then problem occurs as shown below,

Shape Hierarchy Revisited:



We have not made Shape class Abstract so that we can create its objects to show the point that calling array of objects polymorphically results in errors,
See the shape hierarchy again,

```
class Shape {
        …
public:
        Shape();
        virtual void draw(){
                cout << "Shape\n";
        }
        virtual int calcArea() { return 0; }
};

class Line : public Shape {
        …
```

```
public:
        Line(Point p1, Point p2);
        void draw(){ cout << "Line\n"; }
};

void drawShapes( Shape _shape[], int size ) {
        for (int i = 0; i < size; i++) {
                _shape[i].draw();
        }
}

//Polymorphism & Arrays

int main() {
        Shape _shape[ 10 ];
        _shape[ 0 ] = Shape();
        _shape[ 1 ] = Shape();
        ...
        drawShapes( _shape, 10 );
        return 0;
}
```

In above code we created array of ten objects of Shape class and then passed it to drawShapes function, note that in above code we passed Shapes array (Shapes []) instead of Shapes pointers array (Shapes * []) to drawShapes function.
The code works draw method for each Object of Shape class is called and output is displayed below,

Sample Output

```
Shape
Shape
Shape
...
```

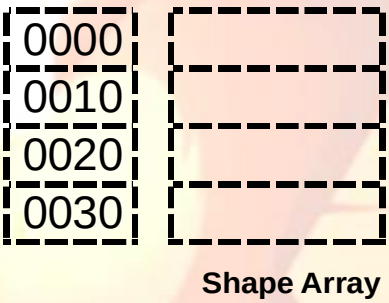
Now we try to do polymorphism here by creating lines objects and passing to this functions as we did previously in case of pointers with the difference that this time we are passing objects array instead of pointers array.

```
int main() {
        Point p1(10, 10), p2(20, 20), ...
        Line _line[ 10 ];
        _line[ 0 ] = Line( p1, p2 );
        _line[ 1 ] = Line( p3, p4 );
        ...
        drawShapes( _line, 10 );
        return 0;
}
```

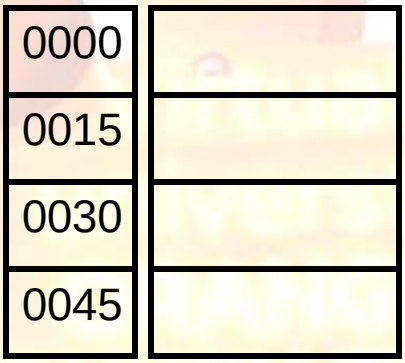
Sample Output:

```
Shape
// Run-time error
```

This time also program is compiled correctly but when we executed it only Shape text is displayed for first object of array and then program terminates abnormally.
The reason behind this output is,
When we passed array of Line objects as parameter this array was converted implicitly to array of Shapes objects by compiler as Line IS A kind of shape, same thing was happened with pointers of array objects but the difference now is that this Line objects array will be treated as Shape objects array, so draw method of first shape object is called and it displays text Shape, for next iteration of loop when compiler calculated address of next object it did it with respect to Shape objects but the actual array consists of Line objects so compiler will incorrectly calculate address of next object as size of Shape class object and Line class objects are different and when it will call draw method with incorrect address of object program will terminate abnormally this is give in diagram below,



Shape Array



Line Array

```
_shape[ i ].draw();
*(_shape + (i * sizeof(Shape))).draw();
```

As shown above compiler will do calculations with respect to Shape array and will calculate next object address as 0010 because it is treating Line array as Shape array but actually next Line object is present at address 0015 so runtime error will be generated as there is no new object at address 0010 and program will terminate abnormally.

Original drawShapes() method that was using pointers is given below

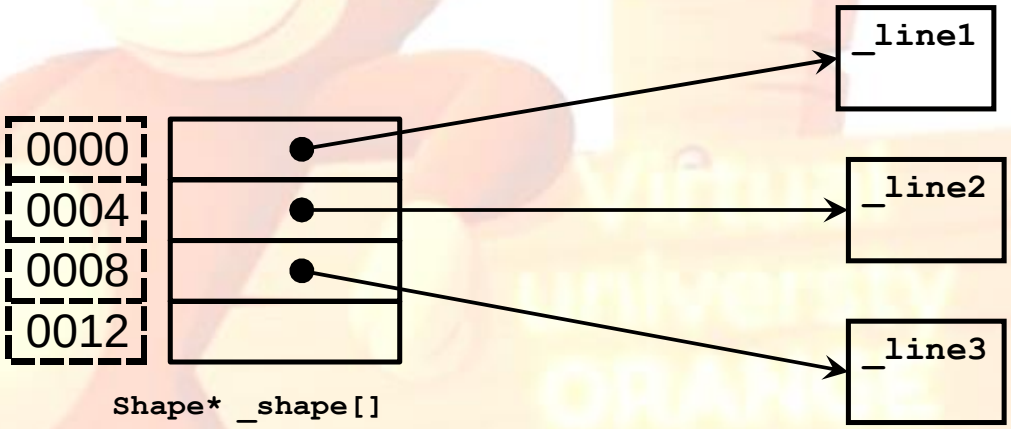
```
void drawShapes(Shape* _shape[], int size) {
        for (int i = 0; i < size; i++) {
```

```
            _shape[i]->draw();
        }
}
```

It shows correct output,
Sample Output:

```
Line
Line
Line
...
```

Because in case of pointers array, the size of each entry is same as each entry contains a pointer and all pointers are of 4 bytes in c++ whatever they are point to Shape class or to Line class or any other built in or user defined type. So in this case whether our array contain Shape * or Line * next object address will be present after 4 bytes, so our drawShapes function will execute correctly,



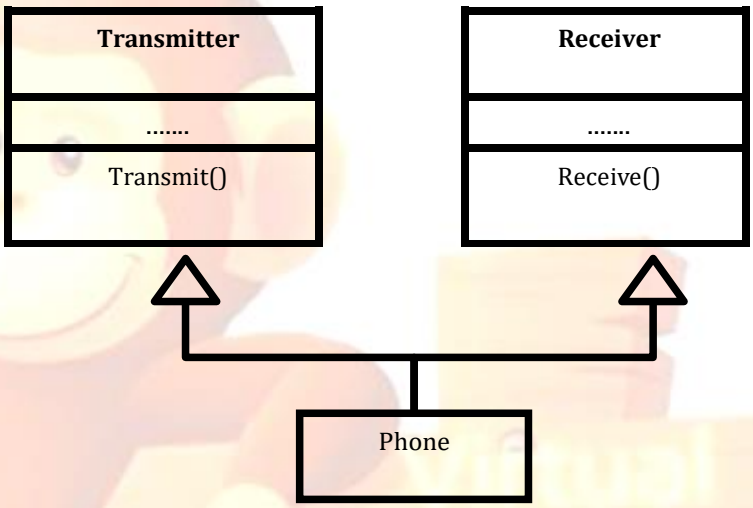

```
_shape[i]->draw();
(_shape + (i * sizeof(Shape*)))->draw();
```

So moral of the story is Never use arrays polymorphically because location of elemetns in any array is calculated using array type and in polymorphism we have the liberty of changing child array to parent array that will result in erraneous calcualation of location of elements in array.

Lecture No.31
Multiple Inheritance

We have seen multiple inheritance in start of OOP now we see its implementation in C++, A class in C++ can inherit from more than one classes like Phone class can inherit from Transmitter or Receiver.

Examples:



class Phone: public Transmitter, public Receiver
// As phone class is publicly inherited so any class derived from Phone class will also have access to public and protected members of Transmitter and Receiver class
{
...
};

Derived class can inherit from public base class as well as private and protected base classes

class Mermaid: private Woman, private Fish
// As Mermaid class is privately inherited from Woman and Fish so any class derived from Mermaid class will Not have access to public and protected members of Woman and Fish classes

{
...
};

Multiple Inheritance
The derived class inherits data members and functions from all the base classes
Object of derived class can perform all the tasks that an object of base class can perform

Example

```
int main(){
        Phone obj;
        obj.Transmit(); // method of Transmitter class
        obj.Receive(); // method of Receiver class
        return 0;
}
```

Multiple Inheritance

When using public multiple inheritance, the object of derived classes can replace the objects of all the base classes
Example

```
int main(){
        Phone obj;
        Transmitter * tPtr = &obj;
        Receiver * rPtr = &obj;
        return 0;
}
```

However note the following,

The pointer of one base class cannot be used to call the function of another base class
The functions are called based on static type

Example:

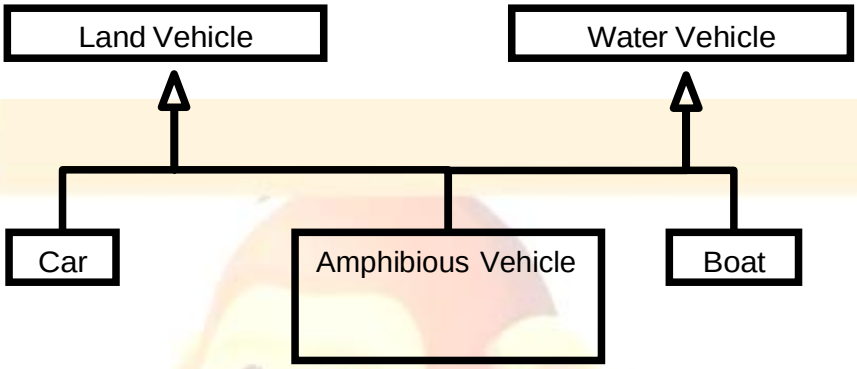
```
int main(){
        Phone obj;
        Transmitter * tPtr = &obj;
        tPtr->Transmit();
        tPtr->Receive();        //Error
        return 0;
}
```

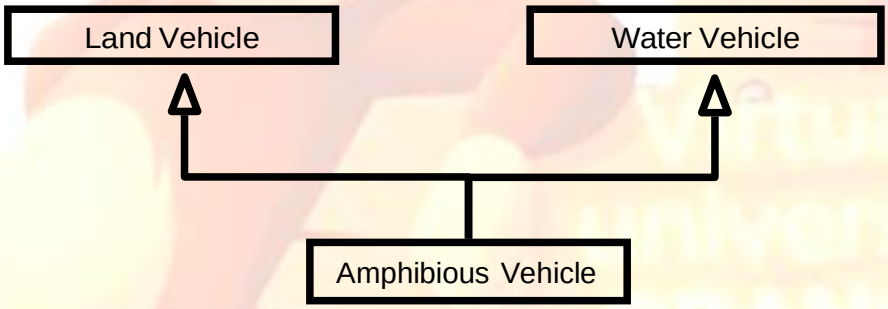
```
int main(){
        Phone obj;
        Receiver * rPtr = &obj;
        rPtr->Receive();
        rPtr->Transmit();       //Error
        return 0;
}
```

Problems in Multiple Inheritance

- If more than one base class have a function with same signature then the child will have two copies of that function.
- Calling such function will result in ambiguity.

Example:






```
class LandVehicle{
public:
        int GetMaxLoad();
};
class WaterVehicle{
public:
        int GetMaxLoad();
};
class AmphibiousVehicle: public LandVehicle, public WaterVehicle {
};
int main(){
        AmphibiousVehicle obj;
        obj.GetMaxLoad();           // Error
        return 0;
}
```

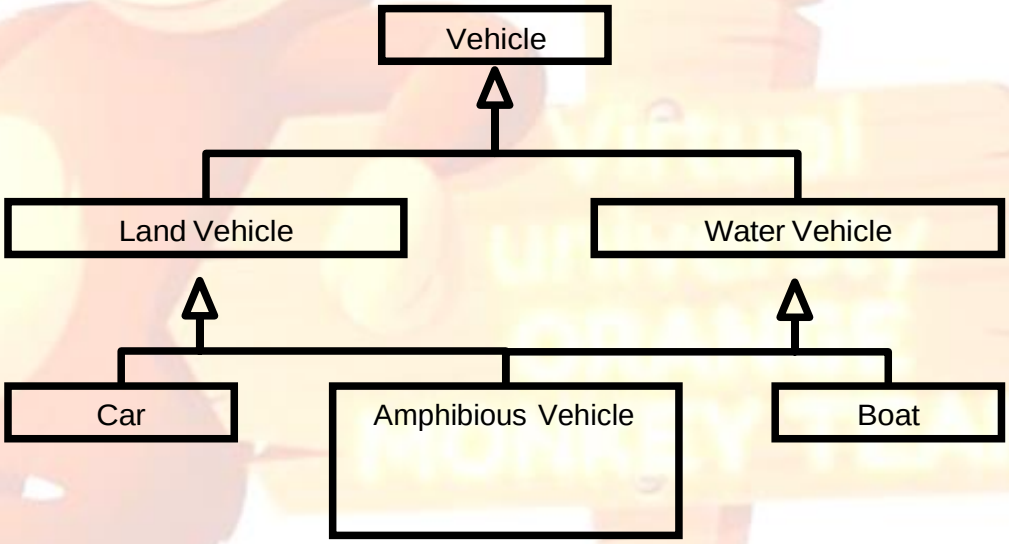
The solution of this problem is that Programmer must explicitly specify the class name when calling ambiguous function

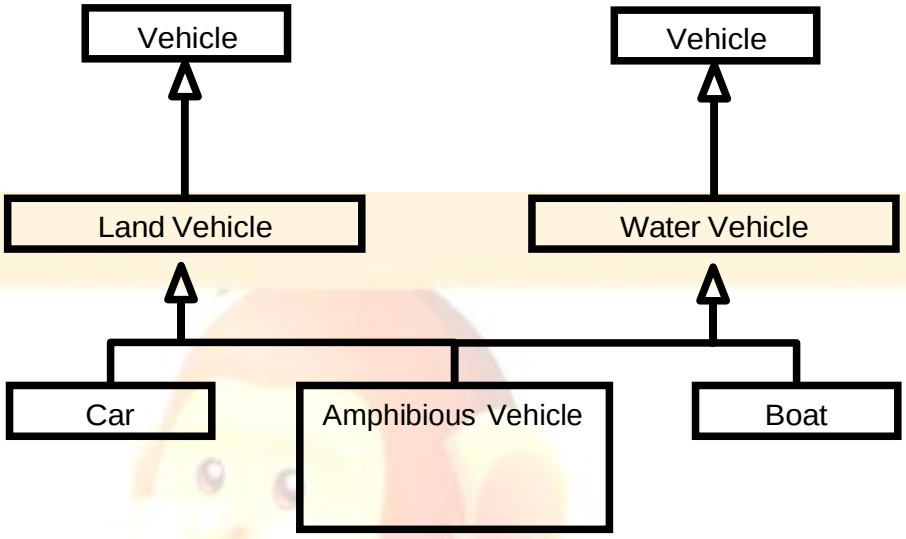
```
int main(){
        AmphibiousVehicle obj;
        obj.LandVehicle::GetMaxLoad();
        obj.WaterVehicle::GetMaxLoad();
        return 0;
}
```

Multiple Inheritance

The ambiguous call problem can arise when dealing with multiple level of multiple inheritance

Example:



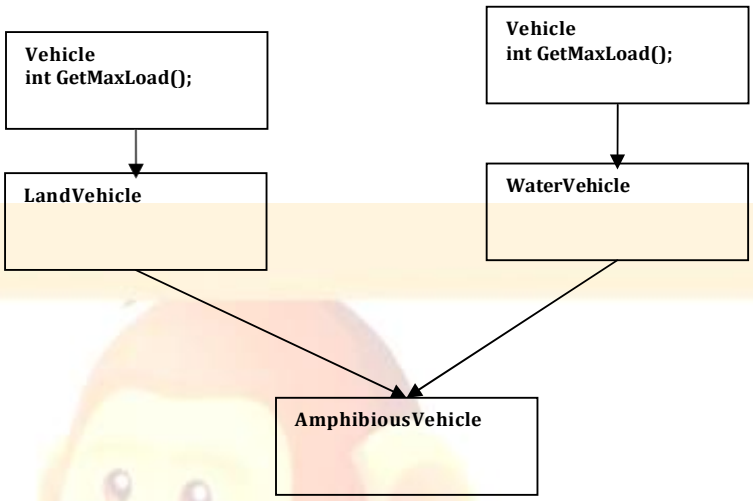



```
class Vehicle{
public:
        int GetMaxLoad();
};
class LandVehicle : public Vehicle{
};
class WaterVehicle : public Vehicle{
};
class AmphibiousVehicle: public LandVehicle, public WaterVehicle {
};

int main(){
        AmphibiousVehicle obj;
        obj.GetMaxLoad();           // Error
        return 0;
}
```

The error is due to the reason that object of AmphibiousVehicle class has two implicit Vehicle class objects one with respect to LandVehicle and one with respect to WaterVehicle class,

© Virtual University of Pakistan



When call to GetMaxLoad method is made, compiler gets confused which copy of this method should be called.
When we try to remove this error by mentioning that this method belongs to Vehicle class as shown below; error remained as it is due to above mentioned fact.

Example

```
int main()
{
        AmphibiousVehicle obj;
        obj.Vehicle::GetMaxLoad(); //Error
        return 0;
}
```

The reason is same that Vehicle is accessible through two paths, we can avoid this error by explicity mentioning the name of intermediate base class LandVehicle or WaterVehicle with respect to which we want to call this GogMaxLoad method as shown below,

Example

```
int main(){
        AmphibiousVehicle obj;
        obj.LandVehicle::GetMaxLoad();
        obj.WaterVehicle::GetMaxLoad();
        return 0;
}
```

Due to the same reason as mentioned above Data member must be used with care when dealing with more than one level on inheritance.

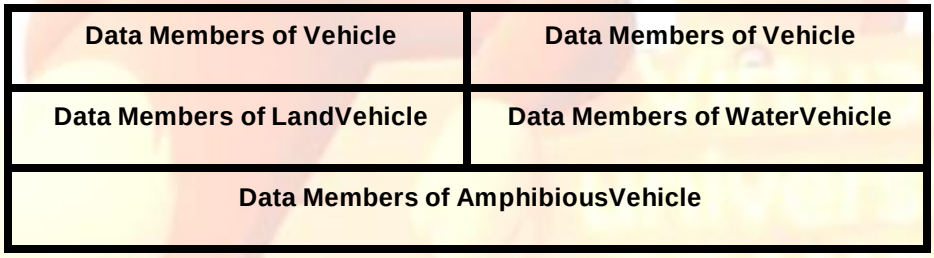
Example

```
class Vehicle{
protected:
```

```
        int weight;
};
class LandVehicle : public Vehicle{
};
class WaterVehicle : public Vehicle{
};
class  AmphibiousVehicle:
                        public LandVehicle,
                        public  WaterVehicle{
public:
         AmphibiousVehicle(){
                LandVehicle::weight = 10;
                WaterVehicle::weight = 10;
         }
};
```

Here AmphibiousVehicle object has multiple copies of data member **weight**.

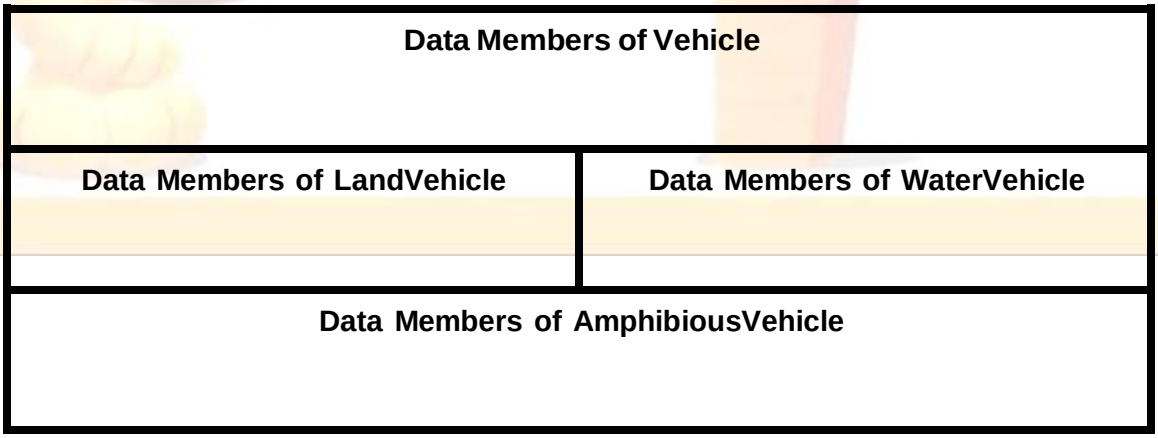
Memory View:



Virtual Inheritance

The solution to avoid this problem is virtual inheritance so that in multiple inheritance only one copy of base class is generated as shown below instead of two separate copies.

Memory View:



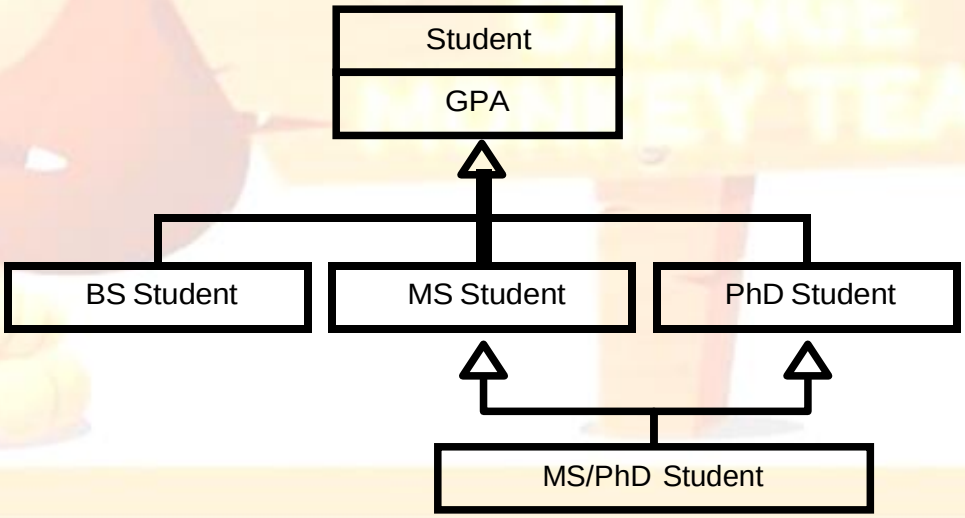
In virtual inheritance there is exactly one copy of the anonymous base class object.
Example:

```
class Vehicle{
protected:
        int weight;
};
class LandVehicle : public virtual Vehicle{
};
class WaterVehicle : public virtual Vehicle{
};
Example
class AmphibiousVehicle: public LandVehicle, public WaterVehicle {
public:
         AmphibiousVehicle(){
                weight = 10;
        }
};
```

When to use Virtual Inheritance?

Virtual inheritance must be used when necessary. It can be used in the situations when programmer wants to use two distinct data members inherited from base class rather than one.

Example



Lecture No.32

Generic Programming

Motivation:

Following function prints an array of integer elements:

```
void printArray(int* array, int size)
{
        for ( int i = 0; i < size; i++ )
                cout << array[ i ] << ", ";
}
```

If we want to print an array of character elements we will write similar function again with different parameters as shown below:

```
void printArray(char* array, int size)
{
        for ( int i = 0; i < size; i++ )
                cout << array[ i ] << ", ";
}
```

similarly for double array we can write,

```
void printArray(double* array, int size)
{
        for ( int i = 0; i < size; i++ )
                cout << array[ i ] << ", ";
}
```

Same will be the case for array of float, short, long and so on….

You can see all these functions are doing same kind of functionality on different data types and it looks repeated task for all data types (basically it is ***function overloading***)

Now if we want to change the way function prints the array. e.g.
from **1, 2, 3, 4, 5**
to **1 2 3 4 5** or **'1' ' 2' '3' '4' '5'** or **"1" "2" "3" "4" "5"**

We will have to change the code in all functions for all data types.

Now consider the **Array** class that is developed to overcome the shortcomings of C++ built in arrays. In C++ arrays, there is no check for array bounds we can insert element at 100th position in an array of size 10 only, our program will compile correctly however this will result in abnormal termination of the program on execution.

```
class Array {
                int* pArray;
                int size;
        public:
```

```
        ...
};
```

The above class has been written for integer arrays. Now, if we want to write the same kind of class for all other data types like double or boolean (true/false), the code will be written as shown below,

```
class Array {
        double* pArray;
        int size;
    public:
        ...
};

class Array {
        bool* pArray;
        int size;
    public:
        ...
};
```

This also looks like repeated code and adds effort to write code. Secondly if we want to add a function sum in **Array** class, we have to change all the three classes.

We want some mechanism that enables us to write **single function or class** that works for all data types this technique of writing programs is called ***Generic Programming***.

Generic Programming

Generic programming refers to programs containing generic abstractions (general code that is same in logic for all data types like printArray function), then we instantiate that generic program abstraction (function, class) for a particular data type, such abstractions can work with many different types of data.

Advantages

Major benefits of this approach are:

a. Reusability: Code can work for all data types.
b. Writability: Code takes lesser time to write.
c. Maintainability: Code is easy to maintain as changes are needed to be made in a single function or class instead of many functions or classes.

Templates

In C++ generic programming is done using templates.

Templates are of two kinds,

a. **Function Templates** (in case we want to write general function like printArray)
b. **Class Templates** (in case we want to write general class like Array class)

Compiler generates different type-specific copies from a single template.
This concept is similar to concept of making prototype in the form of class for all objects of same kind.

Function Templates

A function template can be parameterized to operate on different types of data types.

Declaration:

We write template keyword above any function make any function as template function, they can be declared in any one of the following ways,

```
template< class T >
void funName( T x );

// OR

template< typename T >
void funName( T x );

// OR

template< class T, class U, ... >
void funName( T x, U y, ... );
```

Note here T is typename of class name and we use this T instead of data type(s) for which we want our function to work as template.

For Example – Function Templates
Following function template prints an array having almost any type of elements (note the use of T instead of int or float or char in implementation of function):

```
template< typename T >
void printArray( T* array, int size )
{
        for ( int i = 0; i < size; i++ )
                cout << array[ i ] << ", "; // here data type of array is T
}
```

Similarly we can also write same function as,

```
template< class T >
void printArray( T* array, int size )
{
        for ( int i = 0; i < size; i++ )
                cout << array[ i ] << ", "; // here data type of array is T
}
```

There is no difference in above two implementations.

Template function will be instantiated for a particular data type according to passed argument as shown below,

```
int main() {

        int iArray[5] = { 1, 2, 3, 4, 5 };
        void printArray( iArray, 5 ); // Instantiated for int[] as passed array is of
type int []

        char cArray[3] = { 'a', 'b', 'c' };
        void printArray( cArray, 3 ); // Instantiated for char[] as argument is of type
char []
        return 0;
}
```

Explicit Type Parameterization:

In case a function template does not have any parameter then we have to explicitly mention the data type for which we want to create that function as shown below,

```
        template <typename T>
        T getInput() {
                T x;
                cin >> x;
                return x;
        }
```

Explicit Type Parameterization

```
int main() {
        int x;
        x = getInput();          // Error!

        double y;
        y = getInput();          // Error!
        return 0;
}
```

Explicit Type Parameterization

```
int main() {
        int x;
        x = getInput< int >();

        double y;
        y = getInput< double >();
        return 0;
}
```

User-defined Specializations

A template compiler generated code may not handle all the types successfully; in that case we can give explicit specializations for a particular data type(s).

For example suppose we have written a function isEqual(…, …) that compares two values of data type and return true or false depending upon the values are equal or not,

```
template< typename T >
bool isEqual( T x, T y ) {
        return ( x == y );
}
```

isEqual (6,6) should return true
isEqual (6,7) should return false
isEqual (6.6,6.6) should return true
isEqual (6.5,6.6) should return false
isEqual ('A','A') should return true
isEqual ('A','a') should return false

Until here the function will work correctly but consider the statement below,

isEqual ("abc","xyz")

This is instantiation of isEqual function for built in type char [] or char *[14], this function will fail to give correct result simply because we have given its implementation as

```
return (x == y);
```

So here it will be translated by compiler in,

```
return ( char * == char *);  or  return ( char []  == char []);
```

As arrays consists of many elements, comparison of arrays in this way is not possible it will simply compare first element of both arrays and will return result, this is given in code below,

```
#include <cstdlib>
#include <iostream>

using namespace std;

template< typename T >
bool isEqual( T x, T y ) {
        return ( x == y );
}
```

[14] In C++, both char [] and char * are of same data types.

© Virtual University of Pakistan

```
int main (){
        cout<<isEqual( 5, 6 ); // Ok
        cout<<isEqual( 7.5, 7.5 );      // Ok
        cout<<isEqual( "abc", "bca" ); // Logical Error!

   system("PAUSE");
   return  EXIT_SUCCESS;
}
```

So all statements with starting same characters like below will return **true**,

isEqual("**a**bc", "**a**cc");
isEqual("**b**adac", "**b**acc");
isEqual("**c**afaa", "**c**cda");

This is logical error, solution of this problem is that we give our own correct implementation of **isEqual** template function for char * data type, for this we can write specialization code below general template function as follows,

Example – User Specializations

```
#include <cstdlib>
#include <iostream>

using namespace std;

template< typename T >
bool isEqual( T x, T y ) {
        return ( x == y );
}

template< >
bool isEqual< const char* >(
        const char* x, const char* y ) {
        return ( strcmp( x, y ) == 0 );
}


int main (){
        cout<<isEqual( 5, 6 );  // OK
        cout<<isEqual( 7.5, 7.5 );      // OK
        cout<<isEqual( "abc", "aba" ); //OK will return False


   system("PAUSE");
   return  EXIT_SUCCESS;
}
```

Lecture No.33

Recap

- Templates are generic abstractions
- C++ templates are of two kinds
  - Function Templates
  - Class Templates
- A general template can be **specialized** to specifically handle a particular type like we did for char []

Multiple Type Arguments

Suppose we want to write code to convert different types into one another (like char to int or int to char or float to int or int to float), the problem is same we have to write many functions corresponding to each type, as no of types will increase the required no. of functions will also increase, the concept of templates can be used here as well to write general function to convert one type into another, in this case we will need two type arguments as shown below,

```
template< typename T, typename U >
T my_cast( U u ) {
        return (T)u; // U type will be converted to T type and will be returned
}

int main() {
        double d = 10.5674;
        int j = my_cast( d );     //Error
        int i = my_cast< int >( d );

    // need to explicity mention about type of T (int in this case) as it is used only
for
   // return type not as parameter

        return 0;
}
```

User-Defined Types

Besides primitive types, user-defined types can also be passed as type arguments to templates, compiler performs static type checking to diagnose type errors.
Consider the String class without overloaded operator "=="

```
class String {
        char* pStr;
        …
        // Operator "==" not defined
};

template< typename T >
bool isEqual( T x, T y ) {
```


© Virtual University of Pakistan

```
        return ( x == y );
}
int main() {
        String s1 = “xyz”, s2 = “xyz”;
        isEqual( s1, s2 ); // Error!
        return 0;
}
```

We can use String class objects as arguments to template function **isEqual** also but in this case we should have defined overloaded == operator for our string class as friend function because this operator is being used in **isEqual** function,

```
class String {
        char* pStr;
        …
        friend bool operator ==(    const String&, const String& );
        };

bool operator ==( const String& x, const String& y ) {
        return strcmp(x.pStr, y.pStr) == 0;
}

template< typename T >
bool isEqual( T x, T y ) {
        return ( x == y );
}

int main() {
        String s1 = “xyz”, s2 = “xyz”;
        isEqual( s1, s2 );      // OK
        return 0;
}
```

Overloading vs. Templates

Function templates are used when we want to have exactly identical operations on different data types in case of function templates we can not change implementation from data type to data type however we can specialize implementation for a particular data type.

In case we want to have similar operations on different data types we need function overloading. In case of function overloading, we can give similar but slightly different implementation for each data type.

Example: Overloading vs. Templates

- '+' operator is **overloaded** for different types of opeerands (different implementation in each case).
- A single function template can calculate **sum** of array of many types.

Function Overloading

```
String operator +( const String& x,          const String& y ) {
        String tmp;
        tmp.pStr = new char[strlen(x.pStr) + strlen(y.pStr) + 1 ];
        strcpy( tmp.pStr, x.pStr );
        strcat( tmp.pStr, y.pStr );
        return tmp;
}

String operator +( const char * str1, const String& y ) {
        String tmp;
        tmp.pStr = new char[ strlen(str1) + strlen(y.pStr) + 1 ];
        strcpy( tmp.pStr, str1 );
        strcat( tmp.pStr, y.pStr );
        return tmp;
}
```

Templates

```
template< class T >
T sum( T* array, int size ) {
        T sum = 0;

        for (int i = 0; i < size; i++)
                sum = sum + array[i];

        return sum;
}
```

Template Arguments as Policy:

We can change behaviour of a template using template parameter. We can pass a template argument to enforce some rule (policy). For example see the problem statement below:

"Write a function that compares two given character strings."

This function is similar to built in string comparison function **strcmp** with the difference that it can perform both case sensitive and case insensitive comparisons.

First Solution:

We can write two separate functions for both cases and call any of them as required,

```
int caseSencompare( char* str1, char* str2 )
{
        for (int i = 0; i < strlen( str1 ) && i < strlen( str2 ); ++i)
                if ( str1[i] != str2[i] )
                        return str1[i] - str2[i];

        return strlen(str1) - strlen(str2);
}
```

This function will return 0 in case string length of both strings is same and they have identical elements otherwise it will return 1.

```
int nonCaseSencompare( char* str1,  char* str2 )
{
        for (int i = 0; i < strlen( str1 ) && i < strlen( str2 ); i++)
                if ( toupper( str1[i] ) != toupper( str2[i] ) )
                        return str1[i] - str2[i];

        return strlen(str1) - strlen(str2);
}
```

This function will return 0 in case string length of both strings is same and they have same alphabets (ignoring they are in lower case or in upper case) otherwise it will return 1.

Second Solution:

We write a single compare function and pass a **bool** type parameter to indicate type of comparison and from this function we return result based on passed **bool** parameter used to indicate case sensitive or case insensitive comparison,

```
int compare( char* str1, char* str2, bool caseSen )
{
        for (int i = 0; i < strlen( str1 ) && i < strlen( str2 ); i++)
                if ( ... )
                        return str1[i] - str2[i];

        return strlen(str1) - strlen(str2);
}

// if condition:
```

(caseSen && str1[i] != str2[i]) || (!caseSen && toupper(str1[i]) != toupper(str2[i]))
Here bool variable caseSen will work as flag, and will activate either left sub-expression or right sub-expression in **if-statement**, in case we are performaing case sensitive comparison caseSen will be true (1) and will activate sub-expression,
(caseSen && str1[i] != str2[i])
and in case caseSen is false it will activate second sub-expression involving **!caseSen** as given below,
(!caseSen && toupper(str1[i]) != toupper(str2[i])
other logic is same as in case 1 functions.

Third Solution

Third solution is most elegant solution out of all these, in which we write two classes one for case sensitive and other for case insensitive comparison, and pass one of these classes as argument when instantiating template compare function, in compare function **isEqual** function of passed class is being called to perform either case sensitive or case insensitive comparison,

```
class CaseSenCmp {
public:
        static int isEqual( char x, char y ) {
                return x == y;
        }
};

class NonCaseSenCmp {
public:
        static int isEqual( char x, char y ) {
                return toupper(x) == toupper(y);
        }
};

template< typename C >
int compare( char* str1, char* str2 )
{
        for (int i = 0; i < strlen( str1 ) && i < strlen( str2 ); i++)
                if ( !C::isEqual (str1[i], str2[i]) )
                        return str1[i] - str2[i];

        return strlen(str1) - strlen(str2);
};

int main() {
        int i, j;
        char *x = "hello", *y = "HELLO";
        i = compare< CaseSenCmp >(x, y);
        j = compare< NonCaseSenCmp >(x, y);
        cout << "Case Sensitive: " << i;
        cout << "\nNon-Case Sensitive: “<< j << endl;
        return 0;
```

```
}
```

Sample Output

```
Case Sensitive: 32      // Not Equal
Non-case Sensitive: 0 // Equal
```

Default Policy

We can set default class type as default comparison type as we set default parameters in case of constructors,

```
template< typename C = CaseSenCmp >
int compare( char* str1, char* str2 )
{
        for (int i = 0; i < strlen( str1 ) && i < strlen( str2 ); i++)
                if ( !C::isEqual
                                        (str1[i], str2[i]) )
                        return str1[i] - str2[i];

        return strlen(str1) - strlen(str2);
};

int main() {
        int i, j;
        char *x = "hello", *y = "HELLO";
        i = compare(x, y);
        j = compare< NonCaseSenCmp >(x, y);
        cout << "Case Sensitive: " << i;
        cout << "\nNon-Case Sensitive: "
                        << j << endl;
        return 0;
}
```

Lecture No.34

Generic Algorithms - A Case Study

Consider the template **printArray** function we wrote in previous lecture,

Printing an Array

```
template< typename T >
void printArray( T* array, int size )
{
        for ( int i = 0; i < size; i++ )
                cout << array[ i ] << ", ";
}
```

Although this function will work for all data types but still it is taking array as an argument so this function still depends upon nature of data structure used to pass data, we want this function to be independent from data structure as well means it should print single values as well as arrays of basic data types.

Generic Algorithms

We want to provide such implementation that is independent of data structures also for example in case of printing of values we want printing of both single values and arrays, this can be achieved using *Generic Programming* in which a function work for all types of containers, let us see how we can make a function generic step by step with the help of an example, consider the **find** function below it is similar to **printArray** function with the difference that it tries to find an element in an array and return pointer to that element if it is found in the array otherwise it returns zero (NULL).

Find function that tries to find an integer value in an integer array

```
const int* find( const int* array, int _size, int x ) {
        const int* p = array;
        for (int i = 0; i < _size; i++) {
                if ( *p == x )
                        return p;
                p++;
        }
        return 0;
}
```

First step obviously will be to write it as template so that it may work for all data types instead of only integers,
We write it as template function,

Template function to find a value within an array

```
template< typename T >
T* find( T* array,int _size, const T& x ) {
```


© Virtual University of Pakistan

```
        T* p = array;
        for (int i = 0; i < _size; i++) {
                if ( *p == x )
                        return p;
                p++;
        }
        return 0;
}
```

In next step we simply pass a pointer that is pointing one location next after array instead of passing size of array as shown below, (this will make our code simplified)

```
template< typename T >
T* find( T* array, T* beyond, const T& x ) {
        T* p = array;
        while ( p != beyond ) {
                if ( *p == x )
                        return p;
                p++;
        }
        return 0;
}
```

We also change the return statement of the function by returning the beyond pointer instead of zero in case element is not found, so now we will not check the return value for NULL pointer we will simply check whether it points to any value or it points to beyond pointer, (we are doing it so that we can use a single return statement to return a single pointer as we have done below)

```
template< typename T >
T* find( T* array, T* beyond, const T& x ) {
        T* p = array;
        while ( p != beyond ) {
                if ( *p == x )
                        return p;
                p++;
        }
        return beyond;
}
```

using single return statement,

```
template< typename T >
```

```
T* find( T* array, T* beyond, const T& x ) {
        T* p = array;
        while ( p != beyond && *p != x )
                p++;
        return p;
}
```

Now our code is so generic that it will work for all types of containers (data structures) supporting two operations,

- Increment operator (++) as we are incrementing value in this container
- Dereference operator (*) as we are getting value from container for comparison by dereferencing

Although this template function will now work for all containers in same way but it has one limitation that we will need to pass container pointers in this function this is against the concept of generic programming so we simply remove pointer notation from our code,

```
template< typename P, typename T >
P find( P start, P beyond, const T& x ) {
        while ( start != beyond && *start != x )
                        start++;
        return start;
}
```

Now this implementation will work for all references of containers as shown in code below,

```
int main() {
        int iArray[5];
        iArray[0] = 15;
        iArray[1] = 7;
        iArray[2] = 987;
        …
        int* found;
        found = find(iArray, iArray + 5, 7);
        return 0;
}
```

We can apply the same concept of Generic Algorithms to class templates and develop a generic Vector class that will work for all built-in types as shown below,

Class Templates

© Virtual University of Pakistan

As we saw before a single class template provides functionality to operate on different types of data and in this way facilitates reuse of classes (like we wrote our own array class to overcome the limitations of built-in array class in c++).

We can definition a class template as follows:

- template< class T > class Xyz { … };
- template< typename T > class Xyz { … };

Now we write a template Vector class using the concept of Generic Algorithms such that it can store everything, basically it will act as container to store anything in it, it can store objects that are them self collections like arrays or it can store basic data types. This will be possible because we are going to implement this Vector class using the concept of Generic Algorithms.

Example – Class Template

A **Vector** class template can store data elements of different types, without templates, we need a separate **Vector** class for each data type.

We start with the basic definition of Vector class using templates,

Class Template

```
template< class T >
class Vector {
private:
        int size;
        T* ptr;
public:
        Vector<T>( int =  10  );
        Vector<T>( const Vector< T >& );
        ~Vector<T>();
        int getSize() const;
        const Vector< T >& operator =(
                                const Vector< T >& );
        T& operator []( int );
};
```

Its implementation is,

```
template< class T >
Vector<T>::Vector<T>( int s ) {
        size = s;
        if ( size != 0 )
                ptr = new T[size];
        else
```

```
                ptr = 0;
}
template< class T >
Vector<T>:: Vector<T>(
                        const Vector<T>& copy ) {
        size = copy.getSize();
        if (size != 0) {
                ptr = new T[size];
                for (int i = 0; i < size; i++)
                        ptr[i] = copy.ptr[i];
        }
        else ptr = 0;
}

template< class T >
Vector<T>::~Vector<T>() {
        delete [] ptr;
}

template< class T >
int Vector<T>::getSize() const {
        return size;
}
template< class T >
const Vector<T>& Vector<T>::operator
                        =( const Vector<T>& right) {
        if ( this != &right ) {
                delete [] ptr;
                size = right.size;

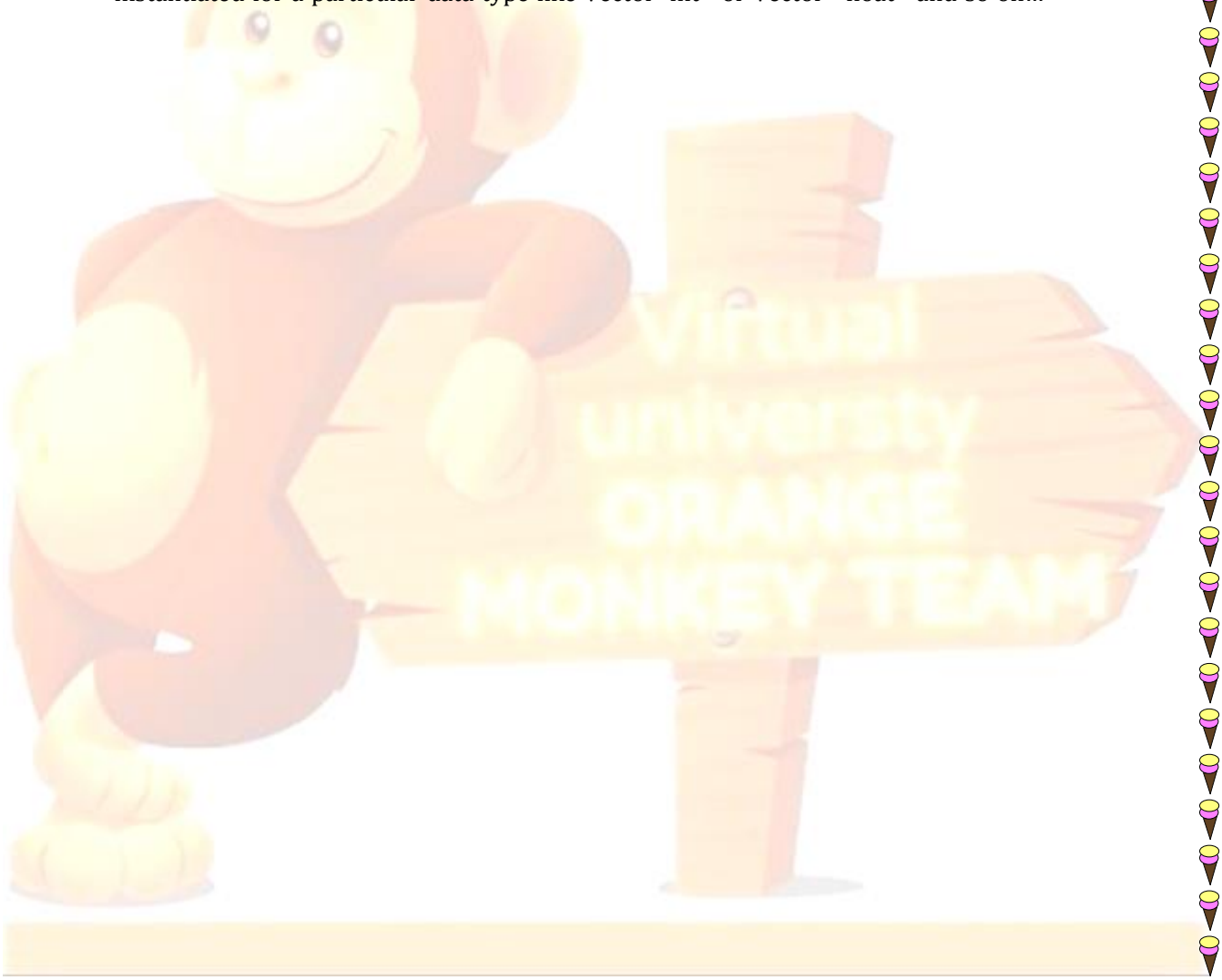
                if ( size != 0 ) {
                        ptr = new T[size];
                        for(int i = 0; i < size;i++)
                                ptr[i] = right.ptr[i];
                }
                else
                        ptr = 0;
        }
        return *this;
}
template< class T >
T& Vector< T >::operator [](        int index ) {
        if ( index < 0 || index >= size ) {
                cout << "Error: index out of
                range\n";
                exit( 1 );
```

```
        }
        return ptr[index];
}
```

We can create this vector class instances for int or char data type as given below,

Vector< int > intVector;
Vector< char > charVector;

This Vector class is parameterized class and will always be instantiated for a particular type only. Now we can not create object of type Vector only it will be instantiated for a particular data type like Vector<int> or Vector <float> and so on…



Lecture No.35

Member Templates:

Member functions of a template class implicitly become functions templates; they work for instantiations (int, char, float, double so on…) of that class, however there are some situations where we need explicit template functions in our class taking more template parameters other than one implicit template parameter (parameter given to this class as parameter while creating its object).

A class or class template can have member functions that are themselves templates

```
template<typename T> class Complex {
        T real, imag;
public:
        // Complex<T>( T r, T im )
        Complex( T r, T im ) :
                real(r), imag(im) {}
        // Complex<T>(const Complex<T>& c)
Complex(const Complex<T>& c) :
                real( c.real ), imag( c.imag ) {}
        …
};
```

Note that in C++ while declaring class there is no need to mention template parameter for class member functions as compiler implicitly understand it, however if we are using some other template parameter like we are doing for copy constructor then we need to give its name as well.

Now see that main function for this class in which we are assigning Complex class float instance to double, it will result in an error,

```
int main() {
        Complex< float > fc(  0,  0  );
        Complex< double > dc = fc; // Error
        return 0;
}
```

Because, our Complex copy constructor is taking argument of same template type T so this copy constructor will work for statements like,

Complex< float > f1c(0, 0);
Complex< float > f2c = f1c;

Complex< double > d1c(0, 0);
Complex< double > d2c = d1c;

But will not work for statements like,

Complex< float > fc(0, 0);

```
Complex< double > dc = fc;

Complex< double > d1c( 0, 0 );
Complex< float > d2c = d1c;
```

When we created Complex object for double compiler generated class for double like shown below,

```
class Complex<double> {
        double real, imag;
public:
        Complex( double r, double im ) :real(r), imag(im)
        {}
        Complex(const Complex<double>& c) :real( c.real ), imag( c.imag )
        {}
        ...
};
```

So, there is need of some sort of overloading of copy constructor such that we can assign two instances of Complex class for two different data types.

Now we change copy constructor as function template explicitly, so that it may work for copy of different types of data types as well,

```
template<typename T> class Complex {
        T real, imag;
public:
        Complex( T r, T im ) :
                real(r), imag(im) {}
        template <typename U>
// this copy constructor is now taking two template parameters one implicit T
// and other explicit U
        Complex(const Complex<U>& c) :
                real( c.real ), imag( c.imag ) {}
        ...
};
```

Now assignment of float Complex instance to double instance will fine,

```
int main() {
        Complex< float > fc( 0, 0 );
        Complex< double > dc = fc; // OK
        return 0;
}
```

Because, here copy constructor

```
Complex(const Complex<U>& c) :
```

Will be instantiated for implicit template type (T) double and explicit template type (U) float because now our Copy constructor is acting as explicity template function written for two different parameters.

Important points to Note:

1. Only that instantiation of copy constructor is generated that is required.
2. Good compilers only generate required template function instances for Complex class instantiation for a particular data type as shown below,

For statement: **Complex< double > dc = fc;**
the functions that will be generated for float and double instance of complex class are shown below,

<double> Instantiation

```
class Complex<double> {
        double real, imag;
public:
        Complex( double r, double im ) :
                real(r), imag(im) {}
        template <typename U>
        Complex(const Complex<U>& c) :
                real( c.real ), imag( c.imag ) {}
        ...
};
```

<float> Instantiation

```
class Complex<float> {
        float real, imag;
public:
        Complex( float r, float im ) :
                real(r), imag(im) {}
// No Copy Constructor code is generated as there is no need for it
        ...
};
```

This approach avoid code bloat (unnecessary code generation).

Class Template Specialization

Like function templates a class template may also not handle all the types successfully, for example for char arrays (char *) the behaviour of template class vector may be not be as desired as shown below,

Vector class to store integers

```
int main() {
        Vector< int > iv1( 2 );
        iv1[0] = 15;
```

```
        iv1[1] = 27;
        Vector< int > iv2( iv1 );
        Vector< int > iv3( 2 );
        iv3 = iv1;
        return 0;
}
```

Vector class to store char arrays (char *)

```
int main() {
        Vector< char* > sv1( 2 );
        sv1[0] = "Aamir";
// compiler will generate a const C String having value Aamir and will assign its
//pointer to sv1[0]
        sv1[1] = "Nasir";
        Vector< char* > sv2( sv1 ); // issue of shallow copy
        Vector< char* > sv3( 2 );
        sv3 = sv1; // issue of shallow copy
        return 0;
}
```

We can write explicit specialization for Vector class for char arrays as we wrote for template function isEqual,

```
template<>
class Vector< char* > {
private:
        int size;
        char** ptr;
public:
        // Vector< char* >( int = 10 );
        Vector( int = 10 );
        Vector( const Vector< char* >& );
        virtual ~Vector();
        int getSize() const;
        const Vector< char* >& operator =( const Vector< char* >& );
        const char*& operator []( int );
        void insert( char*, int );
};
template<>
Vector<char*>::Vector(int s) {
        size = s;
        if ( size != 0 ) {
                ptr = new char*[size];
                for (int i = 0; i < size; i++) ptr[i] = 0;
        }
        else
                ptr = 0;
}
template<>
```

```
Vector< char* >::Vector( const Vector<char*>& copy ) {
        size = copy.getSize();
        if ( size == 0 ) {
                ptr = 0;
                return;
        }
        ptr = new char*[size];
        for (int i = 0; i < size; i++)
                if ( copy.ptr[i] != 0 ) {
                        ptr[i] = new char[ strlen( copy.ptr[i] ) + 1 ];
                        strcpy(ptr[i], copy.ptr[i]);
                }
                else
                        ptr[i] = 0;
}

template<>
Vector<char*>::~Vector() {
        for (int i = 0; i < size; i++)
                delete [] ptr[i];

        delete [] ptr;
}

template<>
int Vector<char*>::getSize() const {
        return size;
}

template<>
const Vector<char*>& Vector<char*>::
operator=(const Vector<char*>& right)
{
        if ( this == &right )
                return *this;
        for (int i = 0; i < size; i++)
                delete [] ptr[i];
        delete [] ptr;
size = right.size;
if ( size == 0 ) {
        ptr = 0;
        return *this;
}
ptr = new char*[size];
for (int i = 0; i < size; i++)
        if ( right.ptr[i] != 0 ) {
                ptr[i] = new char[strlen( right.ptr[i] ) + 1];
                strcpy( ptr[i], right.ptr[i] );
        }
        else
                ptr[i] = 0;
}
```

```
template<>
const char*& Vector<char*>:: operator []( int index ) {
        if ( index < 0 || index >= size ) {
                cout << "Error: index out of range\n";
                exit( 1 );
        }
        return ptr[index];
}

template<>
void Vector< char* >::insert( char* str, int i ) {
        delete [] ptr[i];
        if ( str != 0 ) {
                ptr[i] = new char[strlen(str)+ 1];
                strcpy( ptr[i], str );
        }
        else
                ptr[i] = 0;
}
```

This code is similar to general template Vector class with the difference that now we are allocating and taking care of dynamic memory associated with char arrays.
The main program showing usage of above main program is given below,

```
int main() {
        Vector< char* > sv1( 2 );
        sv1[0] = "Aamir";
   // Error as now we have to changed code of overloaded subscript operator to return //const Vector pointer now and we can not assign new value to constant reference //of Vector now we have to use insert function explicity written for that purpose
        sv1.insert( "Aamir", 0 );
        sv1.insert( "Nasir", 1 );
        Vector< char* > sv2( sv1 );
        Vector< char* > sv3( 2 );
        sv3 = sv1;
        return 0;
}
```

Lecture No.36

Recap

We saw in previous lecture that how we can implement our programming problems easily using Generic Algorithms. Then we saw how we can add explicit template functions to our Class templates to add our desired functionality.
A class template may not handle all the types successfully explicit specializations are required to deal such types we can implement concept of template specialization for such classes as well as we did for function templates.

Now, we move forward to see other cases for member templates,

Member Templates Revisited

We can add member templates for ordinary classes as well, for example following code is adding any instance of Complex class to ComplexSet class, (as we know complex class can be instantiated for int, float or double and ComplexSet class will be collection of Complex class instantiations) as shown below,

Complex Class objects,

real	8
img	3

Complex<int>

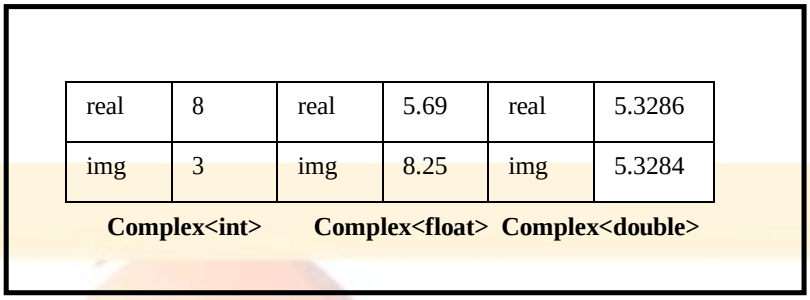
real	5.69
img	8.25

Complex<float>

real	5.3286
img	5.3284

Complex<double>

© Virtual University of Pakistan



ComplexSet Object

```
class ComplexSet {

        template< class T >
        insert( Complex< T > c ) // any instance of complex class
                        {
// Add instance Complex class to Complex set class having many // Complex class
instances
}
        };
int main() {
        Complex< int > ic( 10, 5 );
        Complex< float > fc( 10.5, 5.7 );
        Complex< double > dc( 9.567898, 5 );
        ComplexSet cs;
        cs.insert( ic );
        cs.insert( fc );
        cs.insert( dc );
        return 0;
}
```

Partial Specialization:

We can also perform partial specialization instead of complete specialization, a partial specialization of a template exists between general specialization and complete specialization for example we can specialize a class to behave in a certain manner in case of pointers or in case of parameter of a certain type, see the example below to understand the difference between complete and partial specialization,

```
template< class T, class U, class V > // general template
template< class T, class U, int > // partial specialization
template< class T, float , int > // partial specialization
template< int , float , int > // complete specialization
```

In partial specialization, the number of template parameters remains the same, however, their nature varies (they become more specific).

Example – Partial Specialization

```
template< class T >
class Vector { };
template< class T >
class Vector< T* > { };    // Here T can take any type pointer
```

Example – Partial Specialization

```
template< class T, class U, class V >
class A {};
template< class T, class V >
class A< T, T*, V > {}; // the parameters in header of are two but in class we are using same three parameters
template< class T, class U, int I >
class A< T, U, I > {};   // here we have changed third parameter to non type parameter[15]
template< class T >
class A< int, T*, 5 > {}; // here we have changed first parameter to non type parameter and second is T * any data type pointer and third one is constant expression 5
```

Example – Complete Specialization

```
template< class T >
class Vector { };

template< >
class Vector< char* > { };
```

Example – Complete Specialization

```
template< class T, class U, class V >
class A {};

template< >
class A< int, char*, double > {};
```

Function Templates

Similar to class templates a function template may also have partial specializations,

[15] non type parameters are those parameters which are not template parameters

Example – Partial Specialization

```
template< class T, class U, class V >
void func( T, U, V );

template< class T, class V >
void func( T, T*, V );

template< class T, class U, int I >
void func( T, U );

template< class T >
void func( int, T, 7 );
```

Complete Specialization

We have already used this complete specialization in case of function templates,

```
template< >
        bool isEqual< const char* >(
                const char* x, const char* y ) {
                return ( strcmp( x, y ) == 0 );
        }
```

Example

Consider the following template

```
template< typename T >
        bool isEqual( T x, T y ) {
                return ( x == y );
        }
```

Following partial specialization of this function deals with pointers to objects,

```
template< typename T >
        bool isEqual( T* x, T* y ) {
                return ( *x == *y );
        }
```

So complete having all types of specializations for isEqual function is shown below,

```
template< typename T >
        bool isEqual( T x, T y ) {
                return ( x == y );
        }
template< typename T >
        bool isEqual( T* x, T* y ) {
```

```
                return ( *x == *y );
        }
template< >
        bool isEqual< const char* >(
                const char* x, const char* y ) {
                return ( strcmp( x, y ) == 0 );
        }
```

Using Different Specializations

The code below shows how we can use different type of specializations for isEqual function given above,

```
int main() {
        int i, j;
        char* a, b;
        Shape *s1 = new Line();
        Shape *s2 = new Circle();
        isEqual( i, j ); // Template
        isEqual( a, b );  // Complete Sp.
        isEqual( s1, s2 ); // Partial Sp.
        return 0;
}
```

Non-type Parameters

The parameters given in template definition other than those used for mentioning templates types are called non type parameters, for example,

template <class T, class U, **int I**>

Here int I is not type parameter.

Template parameters may include non-type parameters, the non-type parameters may have default values, for example,

template <class T, class U, **int I = 5**>

They are treated as constants and are commonly used is static memory allocation mean when we want to pass the length of memory we need in template at compile time (statically) the example below shows all this,

Example – template class Array

Consider the Array class again that we discussed in template classes introduction, in this template array class we are creating a c++ array of any built in data type by passing array size in Array object constructor as shown below,

```
template< class T >
class Array {
private:
        T* ptr;
public:
        Array( int size );
        ~Array();
        …
};
template< class T >
Array<T>::Array() {
        if (size > 0)
                ptr = new T[size];
        else
                ptr = NULL;
}
int main() {
        Array< char > cArray( 10 );
        Array< int > iArray( 15 );
        Array< double > dArray( 20 );
        return 0;
}
```

We can do the same by passing the array size as non type parameter while creating an Array class object itself, for this we will need to change to template class definition by adding a non type parameter SIZE as shown below,

```
template< class T, int SIZE >
class Array {
private:
        T ptr[SIZE];
public:
        Array();
        …
};
```

Now we can simply pass array size as argument while creating Array class object,

```
int main() {
        Array< char, 10 > cArray;
        Array< int, 15 > iArray;
        Array< double, 20 > dArray;
```

```
        return 0;
}
```

Default Non-type Parameters

We can set default value for this non type parameters, as we do for parameters passed in ordinary functions,

```
template< class T, int SIZE = 10 >
class Array {
private:
        T ptr[SIZE];
public:
        void doSomething();
        …
};
```

Now default value will be used if we will not pass any value for array size,

```
int main() {
        Array< char > cArray; // here Array of size 10 will be created
        return 0;
}
```

Default Type Parameters

We can also specify default type for type parameters (template parameters like T, U, V), consider the Vector class again we can mention default type of Vector class (that type will be used in case we have not mentioned any type while creating Vector class object)

```
        template< class T = int > // default type for Vector class is now int
        class Vector {
                …
        };

        Vector< > v; // same as Vector< int > v
```

Lecture No.37

© Virtual University of Pakistan

Lecture No. 37

Resolution Order

Consider the following specializations of template **Vector** class,

```
template< typename T >
class Vector { ... };

template< typename T >
class Vector< T* > { ... };

template< >
class Vector< char* > { ... };
```

As these all are template implementations of class **Vector,** when compiler have to use this template class having many specializations from partial to complete, compiler searches all these specializations in a particular order called **resolution order**, so **resolution order** it is the sequence in which compiler searches for required template specialization and is given below,

a. First of all compiler looks for complete specialization
b. If it can not find any required complete specialization then it searches for some partial specialization
c. In the end it searches for some general template

So in other words we can say that compiler searches template specializations from more specific to more general.

Example – Resolution Order

The code below shows which particular instantiation of template **Vector** class will be used,

```
int main() {
        Vector< char* > strVector;
                // Vector< char* > instantiated (complete specialization used)

        Vector< int* > iPtrVector;
                // Vector< T* > instantiated (partial specialization used)

        Vector< int > intVector;
                // Vector< T > instantiated (general specialization used)
        return 0;
}
```

Explanation of code:

In all three cases above, compiler will search for required template specialization in the order given below,

1. Complete specialization (Vector< char* >)
2. Partial specialization (Vector< T* >)
3. General specialization (Vector< T>)

- When compiler will find the following line,

Vector< char* > strVector;

It will find corresponding complete specialization and will stop there.

- When compiler will find the following line,

Vector< int* > iPtrVector;

It will start its searching from complete specializations available but there is only one complete specialization for char * (**Vector< char* >**) and there is no complete specialization for int *, so compiler will move to partial specialization and will try to match some with int *, compiler will find one **(Vector <T** *>) that is working for all pointer data types and therefore it will use that.

- When compiler will find the following line,

Vector< int > intVector;

It will start from complete specializations available but there is only one complete specialization for char * (**Vector< char* >**) and there is no complete specialization for int, so compiler will move to partial specialization and will try to match some with int, compiler will find one partial speciation **(Vector <T *>)** that is working for all pointer data types only hence not applicable here, then compiler will move to general template class Vector <T> and will use it.

Function Template Overloading

We can specialize function templates also, this is called function template overloading,
Consider the specializations of template function sort,

```
template< typename T >
void sort( T ); // general template function
template< typename T >
void sort( Vector< T > & ); // specialization for template Vector class instantiated for any data type
template< >
void sort< Vector<char*> >(Vector< char* > & ); // specialization for template Vector instantiated for char * data type
void sort( char* ); // specialization for char * data type.
```

Resolution Order

Compiler searches target of a function call in the following order in case of function of templates,

- Ordinary Function
- Complete Specialization
- Partial Specialization
- Generic Template

This is the same order as was in case of class template with the addition of ordinary functions (non template functions with the same name).

Example – Resolution Order

```
int main() {

        char* str = "Hello World!";
        sort(str); //  Ordinary function sort( char* )

        Vector<char*> v1 = {"ab", "cd", ... };
        sort(v1); //Complete specialization sort( Vector<char*> & )


        Vector<int> v2 = { 5, 10, 15, 20 };
        sort(v2); // partial specialization sort( Vector<T> &)

        int iArray[] = { 5, 2, 6 , 70 };
        sort(iArray); // general template sort( T )

        return 0;
}
```

Templates and Inheritance

We can use inheritance comfortably with templates or their specializations, but we must follow one rule:

- If we have a template class then all classes derived from it should also be class templates.
- Derived class must take at least as many template parameters as the base class requires for an instantiation.

Derivations in case of a General Template class

- A class template may inherit from another class template. Consider the following code segment in which we are defining a template class A and we are deriving another class B from it which is also a template class.

```
template< class T >
```

```
class A
{ … };

template< class T >
class B : public A< T >
// same template parameter T in both classes declarations ensures that both classes will be instantiated for same types
{ … };

int main() {
    A< int > obj1;
    B< int > obj2;
    return 0;
}
```

- A partial specialization may inherit from a class template

```
template< class T >
class B< T* > : public A< T >// same template parameter T is used here also to //ensure that both classes will be instantiated for same data types
{ … };

int main() {
    A< int > obj1;
    B< int* > obj2;
    return 0;
}
```

- Complete specialization or ordinary class cannot inherit from a class template because as you can see complete specialization is in fact instance of the class for one data type and in this way it is similar to ordinary class as ordinary class also can be instantiated for its members of a particular data type,

```
template< >
class B< char* > : public A< T >
    { … };
// Error: 'T' undefined, derived class is taking less parameters than base class

class B : public A< T >
    { … };
// Error: 'T' undefined, derived class is taking less parameters than base class
```

Derivations in case of a partially specialized class

- A class template may inherit from a partial specialization

```
template< class T >
```


© Virtual University of Pakistan

```
class A
{ … };

template< class T >
class A< T* >
{ … };
template< class T >
class B : public A< T* >
{ … } ;

int main() {
        A< int* > obj1;
        B< int > obj2;
        return 0;
}
```

- A partial specialization may inherit from a partial specialization

```
template< class T >
        class B< T* > : public A< T* >
        { … };

int main() {
        A< int* > obj1;
        B< int* > obj2;
        return 0;
}
```

- Complete specialization or ordinary class cannot inherit from a partial specialization

```
template< >
class B< int* > : public A< T* >
{ … };  // Error: Undefined 'T'

class B : public A< T* >
{ … }; // Error: Undefined 'T'
```

Derivations in case of Completely Specialized class

```
template< class T >
class B : public A< float* >
{ … };
```

```
int main() {
        A< float* > obj1;
        B< int > obj2;
        return 0;
}
```

- A partial specialization may inherit from a complete specialization

```
template< class T >
        class B< T* > : public A< float* >
        { … };

int main() {
        A< float* > obj1;
        B< int* > obj2;
        return 0;
}
```

- A complete specialization may inherit from a complete specialization

```
template< >
        class B< double* > : public A< float* >
        { … };

int main() {
        A< float* > obj1;
        B< double* > obj2;
        return 0;
}
```

- An ordinary class may inherit from a complete specialization

```
class B : public A< float* >
        { … };

int main() {
        A< float* > obj1;
        B obj2;
        return 0;
}
```

Derivations in case of Ordinary Class

- A class template may inherit from an ordinary class

```
class A
```


© Virtual University of Pakistan

```
        { … };

        template< class T >
        class B : public A
        { … };
int main() {
        A obj1;
        B< int > obj2;
        return 0;
}
```

- A partial specialization may inherit from an ordinary class

```
class A{
   };

 template<class T>
 class B {};
template <class T>
class B<T*>: public A{};


int main() {
    A obj1;
  B <int *> obj2;
     return 0;
}
```

- A complete specialization may inherit from an ordinary class

```
        template <class T>
         class B{};
         template< >
        class B< char* > : public A
        { … };

int main() {
        A obj1;
        B< char* > obj2;
        return 0;
}
```

Lecture No.38

Templates and Friends

Templates or their specializations are also compatible with friendship feature of C++ as they are with inheritance.
We consider the following rules when dealing with templates and friend functions and classes.

Templates and Friends – Rule 1

When an ordinary function or class is declared as friend of a class template then it becomes friend of each instantiation of that template class.
Consider the code below in which we are declaring a class A as friend of a class B and in this class A, we are accessing private data of class B without any error,
In case of friend classes,

```
class A {
        …
};

template< class T >
class B {
        int data;
        friend A; // declaring A as friend of B
        …
};

class A {
        void method() {
                B< int > ib;
                B< char > cb
                ib.data = 5; // OK: Accessing private member 'data' for class B instantiation ib
                cb.data = 6; // OK: Accessing private member 'data' for class B instantiation cb
        }
};
```

In case of friend functions,

```
void doSomething( B< char >& );

template< class T >
class B {
        int data;
        friend void doSomething( B<char>& );
   // declaring function doSomething as friend of B
```


© Virtual University of Pakistan

```
        …
};

void doSomething( B< char >& cb ) {
        B< int > ib;
        ib.data = 5;     // OK: Accessing private data of class B
        cb.data = 6;     // OK: Accessing private data of class B
}
```

In combined way both can be written as follows,

```
void doSomething( B< char >& );

class A { … };

template< class T > class B {
        int data;
        friend void doSomething( B<char>& );
        friend A;
        …
};
void doSomething( B< char >& cb ) {
        B< int > ib;
        ib.data = 5;     // OK
        cb.data = 6;     // OK
}
class A {
        void method() {
                B< int > ib;
                B< char > cb
                ib.data = 5;     // OK
                cb.data = 6;     // OK
        }
};
```

The Rule 1 is simple. Now, we go to Rule 2.

Templates and Friends – Rule 2

Rule 2 applies when we make template function or template class friend of another template class.

According to Rule 2 when a friend function template or friend class template is instantiated with the type parameters of class template granting friendship then its instantiation for a specific type is a friend of that class template instantiation for that particular type only. The example below explains this concept,

Consider the code below it has a **template friend function** and a **template friend class** of **template class B**, note that in template class B we have given the **same parameter T** to both template friend function doSomething and template friend class A, according to Rule 2 when we write implementations of friend **template class A**

and friend **template function doSomething** we create instance of class B for same date type for which function **doSomething or class A** will be instantiated as shown below otherwise compiler will generate error,

```
/*************************************************************
One template function and one template class
*************************************************************/
template< class U >
void doSomething( U ); // template function with template parameter U
template< class V >
class A { ... };  // template function with template parameter V

/*************************************************************
Making template function and template class friends of class B
Note that in code of class B below we are using same type parameter T for
instantiating template function doSomething and template class A.
*************************************************************/
template< class T >
class B {
        int data;
        friend  void  doSomething(  T  );     // granting friendship to template
doSomething in // class B
        friend A< T >;  // granting friendship to class A in class B
};

/*************************************************************
Implementation of template function doSometing and template class A
As we have used same type parameter T in class B for declaring doSomething and
class A as friend so now we can only instantiate object of class B according to the
passed parameter
*************************************************************/

template< class U >
void doSomething( U u ) {    // here template T will be passed as U
        B< U > ib;
   // it is OK as we are instantiating class B for same type that is passed from class B
i.e // T
        ib.data = 78;
}
int main() {
        int i = 5;
        char c = 'x';
        doSomething(  i  );  //  OK
        doSomething(  c  );  //  OK
        return 0;
}
```

Now, suppose we change instantiation of doSomething and in it we cerate B class instance for int,

```
template< class U >
void doSomething( U u ) {
        B< int > ib;
        ib.data = 78;
}
```

Now, we can only create doSomething instance for integers as shown below, in case we create that function instance for any other date type error will be generated by compiler ,

```
int main() {
        int i = 5;
        char c = 'x';
        doSomething( i );       // OK
        doSomething( c );       // Error!
        return 0;
}
```

The reason for the error is that we are creating class B instance for int data type in function **doSomething(),** but function **doSomething** itself has been instantiated for char in main.

Same Rule applied to template friend class A is shown below,

```
class B< int > {
        int data;
        friend void doSomething( int );
        friend A< int >;
};
template< class T >
class A {
        void method() {
                B< char > cb;     // Error!
                cb.data = 8;
                B< int > ib; // OK
                ib.data = 9;
        }
};
```

Templates and Friends – Rule 3

When a friend function / class template takes different 'type parameters' from the class template granting friendship, then its each instantiation is a friend of each

instantiation of the class template granting friendship. Basically here we are removing restriction imposed by Rule 2, due to the use of same type parameter in class B while declaring function doSomething and class A as its friends as shown below,

```
template< class U >
void doSomething( U );
template< class V >
class A { ... };


template< class T >
 class B {
        int data;
        template< class W >
        friend void doSomething( W ); // type name is W
        template< class S >
        friend class A; // type name is S
};


template< class U >
void doSomething( U u ) {
        B< int > ib; // Now it is ok to use B for int in function doSomething
instantiated for // char in main
        ib.data = 78;
}
```

```
int main() {
        int i = 5;
        char c = 'x';
        doSomething( i ); // OK
        doSomething( c ); // OK
        return 0;
}
```

For classes same Rule is applicable as follows,

……..

```
/*********************************************************************
class A implementation
*********************************************************************/
template< class T >
class A {
        void method() {
                B< char > cb; // OK!
                cb.data = 8;
                B< int > ib;
                ib.data = 9;
        }
```

```
};
```

Templates and Friends – Rule 4

Rule 4 says that when we declare a template as friend of any class then all kinds specializations of that function – explicit, implicit and partial, also becomes friends of the class granting friendship.

```
template< class T >
class B {
        T data;
        template< class U >
                friend class A;  // granting friendship to class A for all data types (Rule 3)
};

template< class U >
class A {       // general template class A
        A() {
                B< int > ib;
                ib.data = 10;    // OK
        }
};

template< class U >
class A< U* > { // template class A specialized for pointer data types
        A() {
                B< int > ib;
                ib.data = 10;    // OK
        }
};
```

In case of functions demonstration of this Rule is below,

```
template< class T >
class B {
        T data;
        template< class U >
                friend void doSomething( U ); // granting friendship to doSomething for all data types (According to Rule 3)
};

template< class U >
void doSomething( U u ) {
        B< int > ib;
        ib.data = 56;    // OK
}

template< >
void doSomething< char >( char u ) {
```

```
        B< int > ib;
        ib.data = 56;    // OK
}
```

Note:

You can see that there is need to concentrate on Rule 2 as it is somewhat difficult to understand other rules are simple.



Lecture No.39

Templates & Static Members

We have studied in case of general classes that their static members are created and initialized at class scope, meaning that memory is reserved for them when a class is compiled and they are initialized as well at that time, they are not dependent on the creation of any object of a class, all these tings are true in case of static members of template classes as well with different that they are created when a template class is instantiated for each date type, each instantiation of template class has its own copy of static data members, suppose we have template class with static members, compiler will create this template class implementation for different date types as required, and each implementation will have its own copy of static data members.[16]

Consider the class A with a public static data member and a static member function. (It is not good practice to make any data member public but it is declared as public here to demonstrate that separate copy of this data member will be created for each data type).

```
#include <cstdlib>
#include <iostream>

using namespace std;

template< class T >
class A {
public:
        static int data;
        static void doSomething( T & );
};
template<class T>
int A<T>::data = 0;

int main() {
        A< int > ia;
        A< char > ca;
        ia.data = 5;
        ca.data = 7;
        cout << "ia.data = " << ia.data << endl << "ca.data = " << ca.data;
        system("PAUSE");
        return 0;
}
```

Output

ia.data = 5
ca.data = 7

[16] We can take each implementation of template class for a particular data type as a general class written explicitly for that data type.

This output clearly shows that we have two separate copies of data for int and char types.

Also note that each date type instantiation will have single copy of static members as demonstrated below, here objects **ia,ib and ic** will have single copy of **data** being shared by all of them as was the case for static members of a general class,

```
#include <cstdlib>
#include <iostream>

using namespace std;
template< class T >
class A {
public:
        static int data;
        static void doSomething( T & );
};

template<class T>
int A<T>::data = 0;

int main() {
        A< int > ia, ib , ic;

   ia.data = 5;
        ib.data = 7;
   ic.data = 9;
        cout << "ia.data = " << ia.data << endl << "ib.data = " << ib.data << endl <<
"ic.data = " << ic.data << endl;
        system("PAUSE");
        return 0;
}
```

Output

ia.data = 9
ib.data = 9
ic.data = 9

As same data member is being shared so same member value is being changed

Templates – Conclusion
Advantages:
Templates provide

- Reusability
- Writability

© Virtual University of Pakistan

Disadvantages:

- Can consume memory if used without care.
- Templates may affect reliability of a program as they give same type of implementation for all data types with may not be correct for a particular data type like isEqual function given below may give incorrect result for **char ***.

```
template< typename T >
bool isEqual( T x, T y ) {
        return ( x == y );
}
```

- As we are free to give any kind of implementation for a particular data type we may given incorrect implementation, considering the above **isEqual** function, it will produce incorrect result for char * (char arrays) as it will compare first value of both arrays only,

```
int main() {
        char* str1 = "Hello ";
        char* str2 = "World!";
        isEqual( str1, str2 );
                // Compiler accepts!
}
```

Generic Algorithms Revisited

We studied the concept of Generic Algorithms before that made our code type independent as well as independent of underlying data structure,

For this we developed step by step the following find algorithm that is so generic that it works for all containers,

```
template< typename P, typename T >
P find( P start, P beyond, const T& x ) {
        while ( start != beyond && *start != x )
                ++start;
        return start;
}
```

```
int main() {
        int iArray[5];
        iArray[0] = 15;
        iArray[1] = 7;
        iArray[2] = 987;
        …
        int* found;
        found = find(iArray, iArray + 5, 7);
        return 0;
}
```

Generic Algorithms Revisited

We claimed that this algorithm is generic, because it works for any aggregate object (container) that defines following three operations

a. Increment operator (++)
b. Dereferencing operator (*)
c. Inequality operator (!=)

Let us now try to apply this generic algorithms to our template **Vector** class to examine whether our generic algorithm **find** works on this container (our template Vector class can store in it data of all types), for this our Vector class need to support the three operations given above and we will have to add one more integer data member index to track the traversal when we will apply increment operation (++), we will also add its setter and getter members to access its value.
So now our modified Vector class is as follows,

Example – Vector

```
template< class T >
class Vector {
private:
        T* ptr;
        int size;
        int index;        // initialized with zero
public:
        Vector( int = 10 );
    Vector( const Vector< T >& );
        T& operator  [](int);
        int getIndex() const;
        void setIndex( int i );
        T& operator *();
        bool operator !=( const Vector< T >& v );
        Vector< T >& operator ++();
};
template< class T >
int Vector< T >::getIndex() const {
        return index;
}
template< class T >
void Vector< T >::setIndex( int i ) {
        if ( index >= 0 && index < size )
                index = i;
}
template< class T >
Vector<T>& Vector<T>::operator ++() {
        if ( index < size )
                ++index;
        return *this;
}
template< class T >
T& Vector< T >::operator *() {
```


© Virtual University of Pakistan

```
        return ptr[index];
}
template< class T >
bool Vector<T>::operator !=( Vector<T>& v ) {
        if ( size != v.size || index != v.index )
                return true;

        for ( int i = 0; i < size; i++ )
                if ( ptr[i] != v.ptr[i] )
                        return true;

        return false;
}
```

Now we want to apply our generic find method on this Vector container, as our generic **find** algorithm is taking three parameters we will create three instances of our Vector class as follows,

1. First Vector instance to store our data (integers in this case).
2. Second Vector instance to pass address of beyond (One position next to first Vector instance)
3. Third Vector instance to get result

```
int main() {
        Vector<int> iv( 3 );
    // First Vector instance to store integer values from which we need to find
integer
    // value
        iv[0] = 10;
        iv[1] = 20;
        iv[2] = 30;

   Vector<int> beyond( iv ),found( 3 );
     // declaring Second (beyond) and third (found) instance in same
statement
     // beyond is initialized with first Vector instance iv
     // third vector instance of length 3 will be used to store the result
        beyond.setIndex( iv.getSize() ); // We have set index of beyond to 3
where no
     // element exists in it so it is behaving as pointing to one element next to
iv.
        found = find( iv, beyond, 20 );
        cout<<"Index: "<<found.getIndex();
        return 0;
}
```

Generic Algorithm

Now, we want to apply our generic algorithm **find** on this **Vector container**,

```
template< typename P, typename T >
P find( P start, P beyond, const T& x ) {
        while ( start != beyond && *start != x )
                ++start;

        return start;
}
```

This algorithm will run as follows,

First iteration:

Iteration No #	iv size	beyond size	iv index	beyond index	start != beyond	*start != x
1	3	3	0	3	true	true
2	3	3	1	3	true	true
3	3	3	2	3	true	true
4	3	3	3	3	false	undefined

In above scenario this algorithm will run till second iteration only as element 20 will be found iv will be returned with index equal to 1 pointing to second element (20).

As you can see our generic algorithm is working fine with our container **Vector class instance.**

However there are some other things to consider with this generic algorithm implementation.

Problems

a. **Our algorithm doesn't support multiple traversals**

We can move forward in single steps that may be very inefficient approach in case of huge Vector object with large number of values, there is no facility to make multiple movements like one in forward direction and at the same time once in reverse direction,

b. **Our algorithm shows inconsistent behavior**

We generally use pointers to mark a position in a data structure of some primitive type and then return it but here we are using whole container as marker and return it e.g. we returned whole **found** object of Vector class to tell whether we find the value or not we could have done that using single pointer value.

c. **Supports only a single traversal strategy**

Also there is no way to change traversal strategy (moving beyond more than one value in single step).

© Virtual University of Pakistan

Lecture No.40

Recap

In previous lecture we studied that we can apply Generic Algorithms on any container that supports three operations (++, *, !=)
Then we applied Generic Algorithms **find** on our Vector class by giving implementation of these three operations in **Vector** class
At the end we saw that this approach has some drawbacks as given below,

- No support for multiple traversals
- Supports only a single traversal strategy
- Inconsistent behavior

We saw that we have to made distinct three Vector objects for a simple find method this approach may not be efficient when we have a lot of elements in Vector object.
Now, we see how we can use pointers to modify this approach, we will use three cursors (pointers) to vector class to avoid creating three Vector objects, in this approach we will use pointers to store vector elements addresses (basic concept is similar to link lists).

Cursors

A better way is to use *cursors.*
A cursor is a pointer that is declared outside the container / aggregate object
Aggregate object provides methods that help a cursor to traverse the elements

- T* first()
- T* beyond()
- T* next(T*)

In this approach, we will add three methods given above in our Vector class that will return pointer to elements of Vector elements instead of whole vector object.
Now, our vector class is as follows,

Vector

```
template< class T >
class Vector {
private:
        T* ptr;
        int size;
public:
        Vector( int = 10 );
        Vector( const Vector< T >& );
        ~Vector();
        int getSize() const;
        const Vector< T >& operator =( const Vector< T >& );
        T& operator []( int );
        T* first();
        T* beyond();
        T* next( T* );
};
template< class T >
T* Vector< T >::first() {
```

```
        return ptr;
}

template< class T >
T* Vector< T >::beyond() {
        return ( ptr + size );
}

template< class T >
T* Vector< T >::next( T* current )
{
        if ( current < (ptr + size) )
                return ( current + 1 );
        // else
        return current;
}

int main() {
        Vector< int > iv( 3 );
        iv[0] = 10;
        iv[1] = 20;
        iv[2] = 30;
        int* first = iv.first();
        int* beyond = iv.beyond();
        int* found = find(first,beyond,20);
        return 0;
}
```

Note our generic find method remained as it as shown below, simply we used cursors (pointers) in our main method to make our code so efficient.

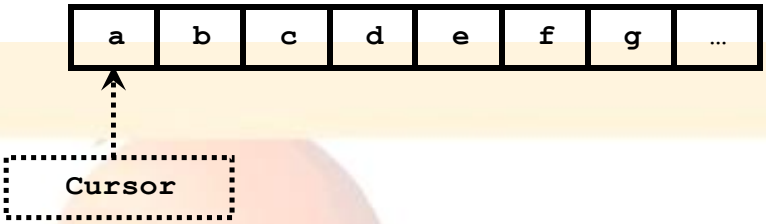
```
template< typename P,  typename  T  >
P find( P start, P beyond, const T& x ) {
        while ( start != beyond && *start != x )
                ++start;
        return start;
}
```

Cursors-Usage

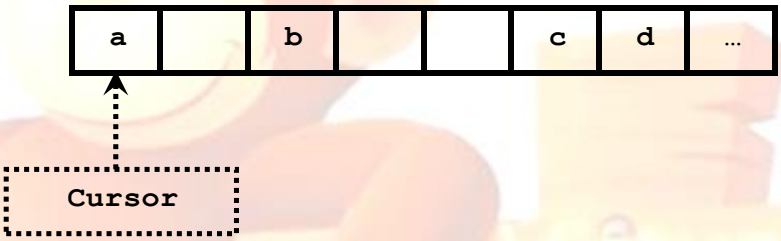
This technique works fine for a contiguous sequence like c++ arrays where elements are placed at consecutive memory locations or Vector whice we have implemented in terms of arrays with extra functionality to overcome shortcomings of c++ arrays, however cursors does now work with containers that use complicated data structures and are non contiguous because in this case we can not simply use ++ operation that we used on our find method, in that case we have to rely on the that container own traversal operations that results in inefficient approach of last lecture,

Example – Works Fine

This cursor works fine when we have



Example – Problem



Example – Problem

In case of non contiguous container we have issues because when we apply find method on non contiguous containers our find method increment operation fails as elements are not placed at next locations in order, in code below we are calling find method for non contiguous container Set that we are going to study in next lectures in this case our find method will give error as indicated,

```
int main() {
        Set< int > is( 3 );
        is.add( 10 );
        is.add( 20 );
        is.add( 30 );
        ET* first = iv.first();
        ET* beyond = iv.beyond();
        ET* found = find(first, beyond, 20);
        return 0;
}
```

```
template< typename P, typename T >
P find( P start, P beyond, const T& x ) {
        while ( start != beyond && *start != x )
                ++start;        // Error
```

```
        return start;
}
```

In this case we need to use that container own increment operation like shown below, then our find algorithm will work fine,

```
template< typename CT, typename ET >
P find( CT& cont, const ET& x ) {
        ET* start = cont.first();
        ET* beyond = cont.beyond();
        while ( start != beyond && *start != x )
                start = cont.next( start );
        return start;
}
```

```
int main() {
        Set< int > is( 3 );
        is.add( 10 );
        is.add( 20 );
        is.add( 30 );
        int* found = find( is, 20 );
        return 0;
}
```

Cursors – Conclusion

The main benefit is that we are using external pointer so we can do now any kind of traversal in any way i.e. as many traversals as we need only we will need one pointer for each traversal, however we can not use cursors in place of pointers for all containers.

Iterators

We studied cursors previously, cursors were external pointer that we accessing internal data of any container like vector, it is against the principle of data hiding as we can access any container data using cursors so it is not good programming practice to given access in container for the use of cursors (first, next, beyond methods) we have alternate to cursors in the form of Iterators which are that traverse a container without exposing its internal representation. Mean they are not external pointers but internal data member of container. Iterators are for containers exactly like pointers are for ordinary data structures (you can see this line as we have made a mechanism to declare pointers to our containers like we declare pointers to ordinary data types, in case of cursors we were using ordinary data type pointer but in case of Iterators we will use container pointers without exposing their internal details)

Generic Iterators

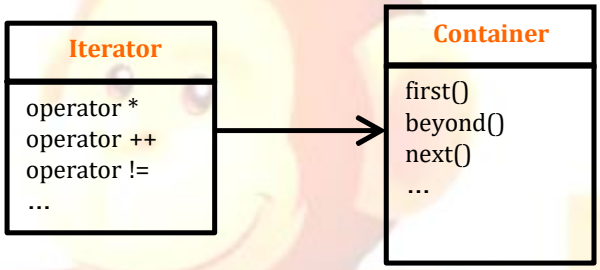
General Iterator class can point to any container because it is implemented using templates, basically it provides us the functionality to create any container object

© Virtual University of Pakistan

using templates and operator overloading this has been explained in code below, now we will not directly use methods first , beyond and next of container but we will use them through Iterator class i.e we will create Iterator class objects and will iterate container through these methods, however these objects will not be able to directly access internal elements of container. A generic Iterator works with any kind of container. We need the same set of operations in container class to use Iterators,

- T* first()
- T* beyond()
- T* next(T*)

Example – Generic Iterator



Generic Iterator

```
template< class CT, class ET >
// Template Iterator class taking two Vector type parameters one for CT (pointer to
// container type) and ET (pointer to individual element of the container)

class Iterator {
        CT* container;
        ET* index;
public:
        Iterator( CT* c, bool pointAtFirst = true );
        Iterator( Iterator< CT, ET >& it );
        Iterator& operator ++();
        ET& operator *();
        bool operator !=( Iterator< CT, ET >& it );
};

template< class CT, class ET >
Iterator< CT, ET >::Iterator( CT* c, bool pointAtFirst ) { // constructor
        container = c;
        if ( pointAtFirst )
                index = container->first();
        else
                index = container->beyond();
```

```
}

template< class CT, class ET >
Iterator< CT, ET >::Iterator(          Iterator< CT, ET >& it ) { // standard copy
constructor
        container = it.container;
        index = it.index;
}

template< class CT, class ET >
Iterator<CT,ET>& Iterator<CT,ET>:: operator ++() { // increment operator
        index = container->next( index );
        return *this;
}

template< class CT, class ET >
ET& Iterator< CT, ET >::operator *() // deference operator
{
        return *index;
}

template< class CT, class ET >
bool Iterator< CT, ET >::operator !=( Iterator< CT, ET >& it ) {
// inequality operator result will be false if containers are pointing
// to same memory location or at same index otherwise true
        if ( container != it.container || index != it.index )
                return true;
// else
        return false;
}
```

```
int main() {
        Vector< int > iv( 2 );
        Iterator < Vector<int>, int > it( &iv ) , beyond( &iv, false );
   // creating two Iterator objects of type <Vector <int >, int >
   // i.e Iterator will have Vector class object for integers and element type
will be
   // integers so index will be integer pointer
   // first object it is taking Vector class object iv as reference and making
iterate
   //index to point to its first element as bool pointAtFirst is true by default
   // second object beyond is also taking same Vector object as reference
   // so that it may also point to it by as poinAtFrist is false now so index will
point
   // to one position beyond the last element in iv
```

```
    iv[0] = 10;
        iv[1] = 20;

   Iterator< Vector<int>, int > found        = find( it, beyond, 20 );

    // creating Iterator object found of type <Vector <int >, int > and saving in
it
    // result of found note that now we have saved index of our vector
    // class element if element is found, without creating extra element of
Vector class.

        return 0;
}
```

Note that in above code we are NOT creating multiple objects of Vector class instead we are creating objects of Iterator class against one Vector class object

```
template< typename P, typename T >
P find( P start, P beyond, const T& x ) {
        while ( start != beyond && *start != x )
                ++start;
        return start;
}
```

Iterators – Advantages

a. With Iterators more than one traversal can be pending on a single container
b. Iterators allow to change the traversal strategy without changing the aggregate object
c. They contribute towards data abstraction by emulating pointers

Lecture No.41

Standard Template Library:

When we started discussing about templates and generic algorithms, our basic aim was to make standard solutions for different problems (like searching or comparison) that should work for all data types and for all containers.
C++ programmers started working on these concepts from very beginning and gave many standard solutions to these problems these standard solutions after the approval of C++ standardization committee were added to a combined library name as Standard Template Library (STL). STL is designed to operate efficiently across many different applications; we can use solutions from this template library in our programs using different header files.

Standard Template Library

STL consists of three key components

- Containers
- Iterators
- Algorithms

STL Promotes Reuse
STL promotes reuse as we don't need to rewrite the already written standard code for different problems, it saves our development time and cost. Secondly these solutions have been thoroughly tested so there is no change of error due to their use.

STL Containers

Container is an object that contains a collection of data elements like we have studied before now we will study them in detail.
STL provides three kinds of containers,

1. Sequence Containers
2. Associative Containers
3. Container Adapters

Sequence Containers

A sequence organizes a finite set of objects, all of the same type, into a strictly linear arrangement

Sequence Containers

a. vector

- Rapid insertions and deletions at back end
- Random access to elements

b. deque[17]

[17] deque behaves like queue (line) such that we can add elements on both side of it.

© Virtual University of Pakistan

- Rapid insertions and deletions at front or back
- Random access to elements

c. list[18]

- Doubly linked list
- Rapid insertions and deletions anywhere

Example – STL Vector

```
#include <vector>
int main() {
        std::vector< int > iv;
        int x, y;
        char ch;
        do {
                cout<<"Enter the first integer:";
                cin >> x;
                cout<<"Enter the second
                integer:";
                cin >> y;
                iv.push_back( x );
                iv.push_back( y );
                cout << “Current capacity of iv = “ << iv.capacity() << endl;
                cout << “Current size of iv =“<< iv.size() << endl;
                cout<<"Do you want to continue?";
                cin >> ch;
        } while ( ch == 'y' );
        return 0;
}
```

Sample Output

```
Enter the first integer: 1

Enter the second integer: 2

Current capacity of iv = 2

Current size of iv = 2

Do you want to continue? y
```

[18] list is linear data structure but can not directly move to any element using index we have to move to a certain by moving sequentially from start element.

Sample Output

```
Enter the first integer: 3

Enter the second integer: 4

Current capacity of iv = 4

Current size of iv = 4

Do you want to continue? y
```

Sample Output

```
Enter the first integer: 5

Enter the second integer: 6

Current capacity of iv = 8

Current size of iv = 6

Do you want to continue? n
```

Example – STL Deque

```
#include <deque>

int main() {
        std::deque< int > dq;
        dq.push_front( 3 );
        dq.push_back( 5 );
        dq.pop_front();
        dq.pop_back()
        return 0;
}
```

Example – STL List

```
#include <list>
int main() {
        std::list< float > _list;
        _list.push_back( 7.8 );
        _list.push_back( 8.9 );
        std::list< float >::iterator it = _list.begin();
```


© Virtual University of Pakistan

```
        _list.insert( ++it, 5.3 );
        return 0;
}
```

Associative Containers

An associative container provide fast retrieval of data based on keys mean we add elements in these containers using some formula and retriever the elements using the same formula again. It ensures that we do not have to traverse the container one by one but we directly move to the required element for example formula may be,

Value % 10 (remainder function),

Using this formula,

6 will be stored at 6th place
11 will be stored at 1st place
13 will be strored at 3rd place

and so on…

Note that this remainder function will always result in a value between 0 and 9 so if we have an array of size 10 we can use this remainder function easily.
When we need to retrieve a value we will use inverse process to find index of value and will then retrieve it.

Associative Containers
set
No duplicates
multiset
Duplicates allowed
map
No duplicate keys
multimap
Duplicate keys allowed

Example – STL Set

```
#include <set>
int main() {
        std::set< char > cs;
        cout << "Size before insertions: " << cs.size() << endl;
        cs.insert( 'a' );
        cs.insert( 'b' );
        cs.insert( 'b' );
        cout << "Size after      insertions: "
                << cs.size();
        return 0;
```

```
}
```

Output

Size before insertions: 0

Size after insertions: 2

Example – STL Multi-Set

```
#include <set>
int main() {
        std::multiset< char > cms;
        cout << "Size before insertions: " << cms.size() << endl;
        cms.insert( 'a' );
        cms.insert( 'b' );
        cms.insert( 'b' );
        cout << "Size after       insertions: "
                << cms.size();
        return 0;
}
```

Output

Size before insertions: 0

Size after insertions: 3

Example – STL Map

```
#include <map>
int main() {
        typedef std::map< int, char > MyMap;
        MyMap m;
        m.insert(MyMap::value_type(1, 'a'));
        m.insert(MyMap::value_type(2,  'b'));
        m.insert(MyMap::value_type(3, 'c'));
        MyMap::iterator it = m.find( 2 );
        cout << "Value @ key " << it->first << " is " << it->second;
        return 0;
}
```

Output
Value @ key 2 is b

© Virtual University of Pakistan

Example – STL Multi-Map

```
#include <map>

int main() {
        typedef std::multimap< int, char > MyMap;
        MyMap m;
        m.insert(MyMap::value_type(1,  'a'));
        m.insert(MyMap::value_type(2,  'b'));
        m.insert(MyMap::value_type(3, 'b'));
        MyMap::iterator it1 = m.find( 2 );
        MyMap::iterator it2 = m.find( 3 );
        cout << "Value @ key " << it1->first << " is " << it1->second << endl;
        cout << "Value @ key " << it2->first << " is " << it2->second << endl;
        return 0;
}
```

Output

```
Value @ key 2 is b

Value @ key 3 is b
```

First-class Containers

Sequence and associative containers are collectively referred to as the first-class containers

Container Adapters

A container adapter is a constrained version of some first-class container

Container Adapters

a. **stack**[19]
   - Last in first out (LIFO)
   - Can adapt **vector**, **deque** or **list**

b. **queue**[20]
- First in first out (FIFO)
- Can adapt **deque** or **list**

c. **priority_queue**[21]

[19] Stack is basically a data structure used to store and retrieve values in speedy way because it grows and shrinks from one end only you can consider it like pile of plates present at somewhere plates can be added or retrieved from front end only. It is very efficient approach however it follows last in first out (LIFO) principle meaning things are retrieved in reverse order from stack.

[20] Queue is also data structure and it is similar to queue of people waiting to submit a form or pay their utility bills queue follows first in first out principle (FIFO).

[21] Priority Queue also follows FIFO principle with difference that we add elements in priority queue according to certain priority its example may be printer queue which has priority option the person send

- Always returns element with highest priority
- Can adapt **vector** or **deque**

Common Functions for All Containers

Common functions that can be applied to all containers are given below (these are basically general functions we can use them with any class),

1. **Default constructor**
2. **Copy Constructor**
3. **Destructor**
4. **empty()**
   - Returns true if container contains no elements
5. **max_size()**
   - Returns the maximum number of elements
6. **size()**
   - Return current number of elements
7. **operator = ()**
   - Assigns one container instance to another
8. **operator < ()**
   - Returns true if the first container is less than the second container
9. **operator <= ()**
   - Returns true if the first container is less than or equal to the second container
10. **operator > ()**
    - Returns true if the first container is greater than the second container
11. **operator >= ()**
    - Returns true if the first container is greater than or equal to the second container
12. **operator == ()**
    - Returns true if the first container is equal to the second container
13. **operator != ()**
    - Returns true if the first container is not equal to the second container
14. **swap ()**
    - swaps the elements of the two containers

Functions for First-class Containers

As first class containers are both sequence and associative containers both allow fast retrieval of data values so they have following additional functions,

1. **begin()**
   - Returns an iterator object that refers to the first element of the container

print having higher priority will find its data at higher level of the queue than other people having low priority

2. **end()**
   - Returns an iterator object that refers to the next position beyond the last element of the container

3. **rbegin()**
   - Returns an iterator object that refers to the last element of the container

4. **rend()**
   - Returns an iterator object that refers to the position before the first element

5. **erase(iterator)**
   - Removes an element pointed to by the iterator

6. **erase(iterator, iterator)**
   - Removes the range of elements specified by the first and the second iterator parameters
7. **clear()**
   - erases all elements from the container

Container Requirements

As each container need to perform certain operations on the elements added to it like their copy while creating another instance of container or their comparison while performaing certain operation on them like their sorting , so the elements that we are going to add in containers should provide this kind of basic functionality.
Examples of these functionalities are given below,

- When an element is inserted into a container, a copy of that element is made using,
  - Copy Constructor
  - Assignment Operator

So, the elements need to be added to any container should provide copy and assignment functionality. Builtin C++ data types already provide these types of functionalities and we have studied that compiler also generates copy constructor and overloaded assignment operator for our user defined data types like structures and classes if we have not done so.

- Associative containers and many algorithms compare elements so the elements that are added to associative containers should have this functionality,
  - Operator ==
  - Operator <

C++ doesnot provide functionality of comparison operator (==) or less than operator (<) by itself so we have to provide this functionality by ourself in element class if we want to use it in associative containers.

[STL components Iterators and Algorithms will be discussed in next lecture.]



Lecture No.42

Iterators

We have studied about Iterators before; they are used to traverse Containers efficiently without accessing internal details, now we see Iterators provided to us in standard template library, Iterators are for containers like pointers are for ordinary data structures STL Iterators provide pointer operations such as * and ++

Iterator Categories

We can divide Iterators given to us in STL in the following categories,

a. Input Iterators
b. Output Iterators
c. Forward Iterators
d. Bidirectional Iterators
e. Random-access Iterators

Input Iterators

Using input iterators we can read an element, and we can only move in forward direction one element at a time, these can be used in implementing those algorithms that can be solved in one pass (moving container once in single direction from start to end like find algorithm we studied in last lecture).

Output Iterators

Using output iterators we can read an element, and we can only move in forward direction one element at a time, these can be used in implementing those algorithms that can be solved in one pass (moving container once in single direction from start to end like find algorithm we studied in last lecture).

Forward Iterators

Forward iterators have the capabilities of both input and output Iterators, in addition they can bookmark a position in the container (we can set one position as bookmark while traversing the container this will be more understandable when we will see example below)

Bidirectional Iterators

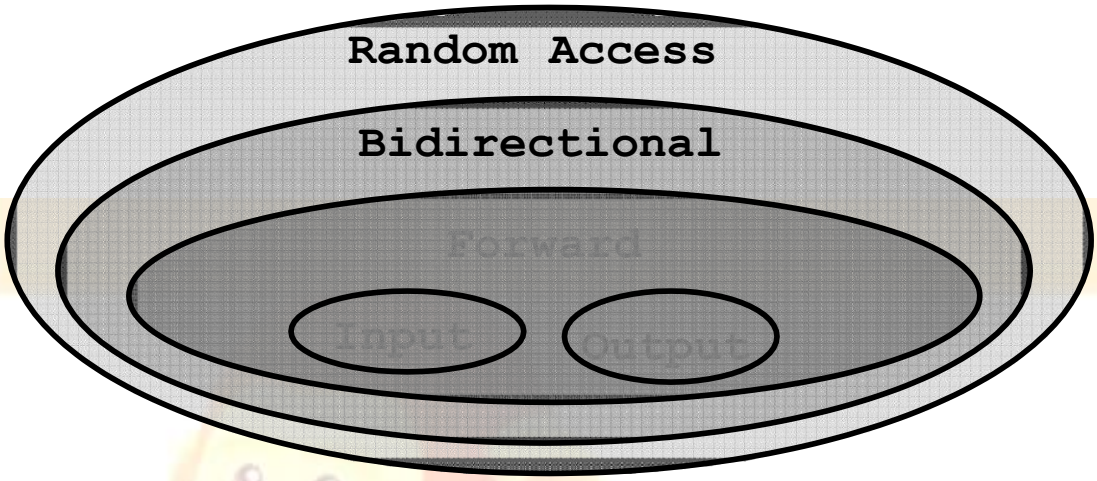
They have all the capabilities of forward Iterators plus they can be moved in backward direction, as a result they support multi-pass algorithms (algorithms that need more that need to traverse container more than once).

Random Access Iterators

They have all the capabilities of bidirectional Iterators plus they can directly access any element of a container.

Iterator Summary:

Following diagram shows the capabilities and scope of different iterators you can see that Random access iterators have all the capabilities and input and output iterators have least capabilities,



Container and Iterator Types:

We can use the following different types of iterators with different types of containers, (it is according to the nature of container)

Sequence Containers

Container Type	Iterator Type	Reason
vector	random access	(as we can access any element of vector using its index so we can use random access iterator)
deque	random access	(in deque we can add elements only in front and back however we can access any element of deque using its index so we can use random access Iterator)
list	bidirectional	(in list we can move in both directions in sequence, however cannot access an element at specific index randomly so we can use bidirectional iterator with list)

Associative Containers

In associative containers we save values based on keys, and we cannot access elements randomly based on indexes as elements are not stored at contiguous memory locations, however, we can traverse them in both directions so, we can use bidirectional iterators with them.

Container Type	Iterator Type	
-- set	--bidirectional	
-- multiset	-- bidirectional	
-- map	-- bidirectional	
-- multimap	-- bidirectional	

Container Adapters

Container adapters are made with special restrictions, most important restriction is that they don't allow free traversal of their elements so we CANNOT use iterators with them as given below,

Container Type	Iterator Type	
-- stack	-- (none)	
-- queue	-- (none)	
-- priority_queue	-- (none)	

Iterator Operations

Iterators support following operations,

All Iterators support,

++p
pre-increment an iterator

p++
post-increment an Iterator

Input Iterators support,

***p**
Dereference operator used as rvalue (right value mean they can be used on right side of expression) for reading only, not for assigning value lvalue (left value, assignment only taken place when we are allowed to use them as left value)
p1 = p2
Assignment (two Iterators)
p1 == p2
Equality operator
p1 != p2
Inequality operator
p->

Access Operator

Output Iterators support,

***p**
Dereference operator (here we can use it for assigning new value lvalue as iterator is output iterator that can be used to set any value)

p1 = p2
Assignment

As Forward Iterators have combined properties of both input and output iterators so they support operations of both input and output Iterators.

Bidirectional Iterators

As bidirectional iterators can move in backward direction also, so they support decrementing operations also (moving pointer one element back),

--p
Pre-decrement operator
p--
Post-decrement operator

Random-access Iterators

Besides the operations of bidirectional Iterators, they also support

p + i
Result is an iterator pointing at **p + i**
p – i
Result is an iterator pointing at **p – i**
p += i
Increment iterator p by i positions
p –= i
Decrement iterator p by i positions
p[i]
Returns a reference of element at p + i
p1 < p2
Returns true if **p1** is before **p2** in the container

p1 <= p2
Returns true if **p1** is before **p2** in the container or **p1** is equal to **p2**
p1 > p2
Returns true if **p1** is after **p2** in the container
p1 >= p2
Returns true if **p1** is after **p2** in the container or **p1** is equal to **p2**

Example – Random Access Iterator

```
typedef std::vector< int > IntVector;
int main() {
        const int SIZE = 3;
        int iArray[ SIZE ] = { 1, 2, 3 };
        IntVector iv(iArray, iArray + SIZE);
        IntVector::iterator it = iv.begin();
        cout << "Vector contents: ";
        for ( int i = 0; i < SIZE; ++i )
                cout << it[i] << ", ";
        return 0;
}
```

Sample Output

```
Vector contents: 1, 2, 3,
```

Example – Bidirectional Iterator

```
typedef std::set< int > IntSet;
int main() {
        const int SIZE = 3;
        int iArray[ SIZE ] = { 1, 2, 3 };
        IntSet is( iArray, iArray + SIZE );
        IntSet::iterator it = is.begin();
        cout << "Set contents: ";
        for (int i = 0; i < SIZE; ++i)
                cout << it[i] << ", "; // Error
        return 0;
}
```

Example – Bidirectional Iterator

```
typedef std::set< int > IntSet;
int main() {
        const int SIZE = 3;
        int iArray[ SIZE ] = { 1, 2, 3 };
        IntSet is( iArray, iArray + SIZE );
        IntSet::iterator it = is.begin();
        cout << "Set contents: ";
        for ( int i = 0; i < SIZE; ++i )
                cout << *it++ << ", "; // OK
        return 0;
}
```

Sample Output

```
Set contents: 1, 2, 3,
```

Example – Bidirectional Iterator

```
typedef std::set< int > IntSet;
int main() {
        const int SIZE = 3;
        int iArray[ SIZE ] = { 1, 2, 3 };
        IntSet is( iArray, iArray + SIZE );
        IntSet::iterator it = is.end();
        cout << "Set contents: ";
        for (int i = 0; i < SIZE; ++i)
                cout << *--it << ", ";
        return 0;
}
```

Sample Output

```
Set contents: 3, 2, 1,
```

Example – Input Iterator

```
#include <iostream>
using std::cin;
using std::cout;
using std::endl;
#include <iterator>

int main() {
        int x, y, z;
        cout << "Enter three integers:\n";
        std::istream_iterator< int > inputIt( cin );
        x = *inputIt++;
        y = *inputIt++;
        z = *inputIt;
        cout << "x = " << x << endl;
        cout << "y = " << y << endl;
        cout << "z = " << z << endl;
        return 0;
}
```

```
int main() {
```


© Virtual University of Pakistan

```
        int x = 5;
        std::istream_iterator< int > inputIt( cin );
        *inputIt = x;     // Error
        return 0;
}
```

Example – Output Iterator

```
int main() {
        int x = 1, y = 2, z = 3;
        std::ostream_iterator< int > outputIt( cout, ", " );
        *outputIt++ = x;
        *outputIt++ = y;
        *outputIt++ = z;
        return 0;
}
```

Example – Output Iterator

```
int main() {
        int x = 1, y = 2, z = 3;
        std::ostream_iterator< int > outputIt( cout, ", " );
        x = *outputIt++;          // Error
        return 0;
}
```

Algorithms

STL includes 70 standard algorithms

These algorithms may use Iterators to manipulate containers

STL algorithms also work for ordinary pointers and data structures

An algorithm works with a particular container only if that container supports a particular Iterator category

A multi-pass algorithm for example, requires bidirectional Iterator(s) at least

Algorithm: Examples

Mutating-Sequence Algorithms (that required changing of elements position)

```
        Copy
```

```
copy_backward
fill
fill_n
generate
generate_n
iter_swap
partition
……..
```

Non-Mutating-Sequence Algorithms (that don't require changing of element position)

```
adjacent_find
count
count_if
equal
find
find_each
find_end
find_first_of
…
```

Numeric Algorithms (involves mathematical calculation)

accumulate

inner_product

partial_sum

adjacent_difference

Example – **copy** Algorithm

```
#include <iostream>
using std::cout;
#include <vector>
#include  <algorithm>
typedef std::vector< int > IntVector;

int main() {
        int iArray[] = {1, 2, 3, 4, 5, 6};
        IntVector iv( iArray, iArray + 6 );
std::ostream_iterator< int >
                                output( cout, ", " );
        std::copy( begin, end, output );

        return 0;
}
```

Output

```
1, 2, 3, 4, 5, 6,
```

Example – **fill** Algorithm

```
#include <iostream>
using  std::cout;
using std::endl;
#include <vector>
#include <algorithm>
typedef std::vector< int > IntVector;
int main() {
        int iArray[] = { 1, 2, 3, 4, 5 };
        IntVector iv( iArray, iArray + 5 );
std::ostream_iterator< int > output( cout, ", " );
        std::copy( iv.begin(), iv.end(), output );
        std::fill(iv.begin(), iv.end(), 0);
        cout << endl;
        std::copy( iv.begin(), iv.end(), output );
        return 0;
}
```

Lecture No.43

Techniques for Error Handling:

Sometimes our program terminates abnormally, sometimes they even crash the system, these errors occur mostly due to incorrect memory access or due to input/output error, sometimes it is our program fault and sometimes it is some external resource error (like network or hard disk).
If we allow these errors to happen we may lose our work for example if a text editor terminates abnormally without allowing us to save our work we will lost our work, so it is important that we add some type of error handling mechanism in our program, we use the following techniques for error handling,

a. Abnormal termination
b. Graceful termination
c. Return the illegal value
d. Return error code from a function
e. Exception handling

Example – Abnormal Termination

In abnormal termination we do nothing and our program is terminated abnormally by operating system if case of any error without saving program data,

```
void GetNumbers( int &a, int &b ) {
        cout << "\nEnter two integers";
        cin >> a >> b;
}
int Quotient( int a, int b ){
        return a / b;
}
void OutputQuotient( int a, int b, int quo ) {
        cout << "Quotient of " << a << " and "
                << b << " is " << quo << endl;
}

int main(){
        int sum = 0, quot;
        int a, b;
        for (int i = 0; i < 10; i++){
                GetNumbers(a,b);
                quot = Quotient(a,b);
                sum += quot;
                OutputQuotient(a,b,quot);
        }
        cout << "\nSum of ten quotients is "<< sum;
        return 0;
}
```


© Virtual University of Pakistan

Output

```
Enter two integers
10
10
Quotient of 10 and 10 is 1
Enter two integers
10
0
Program terminated abnormally
```

Graceful Termination

Program can be designed in such a way that instead of abnormal termination, that causes the wastage of resources, program performs clean up tasks, mean we add check for expected errors (using if conditions),

Example – Graceful Termination

```
int Quotient (int a, int b ) {
        if(b == 0){
                cout << "Denominator can't " << " be zero" << endl;
                // Do local clean up
                exit(1);
        }
        return a / b;
}
```

Output

```
Enter two integers
10
10
Quotient of 10 and 10 is 1
Enter two integers
10
0
Denominator can't be zero
```

Error Handling

a. Return Illegal Value

The clean-up tasks are of local nature only; there remains the possibility of information loss.

Example – Return Illegal Value

```
int Quotient(int a, int b){
        if(b == 0)
                b = 1;
        OutputQuotient(a, b, a/b);
        return a / b ;
}
int main() {
        int a,b,quot;     GetNumbers(a,b);
        quot = Quotient(a,b);
        return 0;
}
```

Output

```
Enter two integers
10
0
Quotient of 10 and 1 is 10
```

b. Return Error Code

Programmer has avoided the system crash but the program is now in an inconsistent state

Example – Return Error Code

```
bool Quotient ( int a, int b, int & retVal ) {
        if(b == 0){
                return false;
        }
        retVal = a / b;
        return true;
}
```

Part of main Function

```
for(int i = 0; i < 10; i++){
        GetNumbers(a,b);
        while ( ! Quotient(a, b, quot) ) {
                cout << "Denominator can't be " << "Zero. Give input again
\n";
                GetNumbers(a,b);
        }
        sum += quot;
        OutputQuotient(a, b, quot);
```


© Virtual University of Pakistan

```
}
```

Output

```
Enter two integers
10
0
Denominator can't be zero. Give input again.
Enter two integers
10
10
Quotient of 10 and 10 is 1
...//there will be exactly ten quotients
```

Error Handling

- Issues in Error Handling:
- Programmer sometimes has to change the design to incorporate error handling
- Programmer has to check the return type of the function to know whether an error has occurred
- Programmer of calling function can ignore the return value
- The result of the function might contain illegal value, this may cause a system crash later
- Program's Complexity Increases
  The error handling code increases the complexity of the code
  - Error handling code is mixed with program logic
  - The code becomes less readable
  - Difficult to modify

The example below shows these concepts,

Example- without error handling

```
int main() {
        function1();
        function2();
        function3();
        return 0;
}
```

Example – with error handling

```
int main(){
        if( function1() ) {
```

```
                    if( function2() ) {
                        if( function3() ) {
                            ...
                        }
                        else    cout << "Error Z has occurred";
                    }
                    else    cout << "Error Y has occurred";
                }
                else    cout << "Error X has occurred";
                return 0;
}
```

Exception Handling

- Exception handling is a much elegant solution as compared to other error handling mechanisms
- It enables separation of main logic and error handling code

Exception Handling Process

- Programmer writes the code that is suspected to cause an exception in **try block**
- Code section that encounters an error **throws an object** that is used to represent exception
- **Catch blocks** follow try block to catch the object thrown

Syntax – Throwing an exception

- The keyword throw is used to throw an exception
- Any expression can be used to represent the exception that has occurred
  - throw X;
  - throw (X);

Examples

```
int a;
Exception obj;
throw 1;                    // throw with literal (const)
throw (a);                  // throw with variable
throw obj;                  // throw with object
throw Exception(); // anonymous object
throw 1+2*9; // mathematical expression
```

- Primitive data types may be avoided as throw expression, as they can cause ambiguity
- Define new classes to represent the exceptions that has occurred
  - This way there are less chances of ambiguity

Syntax – Try and Catch

```
int main () {
        try {
                ... exception with be thrown here in case of error
        }
        catch ( Exception1 ) {    // exception with be caught here if exception was thrown
                ...
        }
        catch ( Exception2 obj ) { // exception with be caught here if exception was
    // thrown
                ...
        }
        return 0;
}
```

Catch Blocks

- Catch handler must be preceded by a try block or an other catch handler
- Catch handlers are only executed when an exception has occurred
- Catch handlers are differentiated on the basis of argument type
- The catch blocks are tried in order they are written
- They can be seen as switch statement that do not need break keyword

Complete Example of try catch and throw:

```
class DivideByZero { // class just use to indicate and throw an exception
public:
        DivideByZero() {
        }
};
int Quotient(int a, int b){
        if(b == 0){
                throw DivideByZero();    // throwing above class object as exception in
            // case of error
        }
        return a / b;
}
```

Related part of main getting user input and calling Quotient method in try catch block this method will throw exception (DividebyZero class object) if user enter no. b (denominator, divider) equal to zero and this exception will be caught in catch block below, (you can write it as complete program to test the actual working of program)

```
for(int i = 0; i < 10; i++) {
        try{
                GetNumbers(a,b);
                quot = Quotient(a,b);
                OutputQuotient(a,b,quot); sum += quot;
        }
        catch(DivideByZero) {
                i--;
                cout << "\nAttempt to divide
    numerator with zero";
        }
}
```

Output

```
Enter two integers
10
10
Quotient of 10 and 10 is 1
Enter two integers
10
0
Attempt to divide numerator (dividend) with zero
...
```

Catch Handler

- The catch handler catches the DivideByZero exception through anonymous object
- Program logic and error handling code are separated
- We can modify this to use the object to carry information about the cause of error

Separation of Program Logic and Error Handling

You can see now that error handling code is separated from man logic of program as shown below,

```
int main() {
        try {
                function1();
                function2();
                function3();
```

```
        }
        catch( ErrorX) { ... }
        catch( ErrorY) { ... }
        catch( ErrorZ) { ... }
        return 0;
}
```



Lecture No.44

Previous Lecture Example of Exception Handling:

```
class DivideByZero {
public:
        DivideByZero() {
        }
};
int Quotient(int a, int b){
        if(b == 0){
                throw DivideByZero();
        }
        return a / b;
}

//main Function

int main() {
        try{    …
                quot = Quotient(a,b);
                …
        }
        catch(DivideByZero) {
                …
        }
        return 0;
}
```

Now, we want to see what happens actually to local variables in try block when an exception is thrown, this concept is called stack unwinding which tells how try catch blocks are un winded (executed) when there are nested function calls involving try catch blocks or nested try catch blocks themselves.

44.1.Stack Unwinding

The flow control (the order in which code statements and function calls are made) as a result of throw statement is referred as "stack unwinding"

Stack Unwinding can take place in the following two ways,

1. When we have nested try catch blocks (one try catch block into other try catch block), for example

```
try {
        try {

        } catch( Exception e) {

        }
```


© Virtual University of Pakistan

```
 } catch(exception e){

}
```

2. When exception is thrown from nested functions having try catch blocks

```
void function1() {

        throw Exception();
}
void function2() {

        function1();

}

int main() {
        try{
                function2();
        } catch( Exception ) { }
        return 0;
}
```

Stack unwinding is more complex than simple nested function calls (or recursive function calls) as in case of nested try catch block, exception can be thrown from any try block, so transfer of control to catch handler is complex, we see this is detail,

First note these points,

- All the local objects of an executing block are destroyed when an exception is thrown
- Dynamically allocated memory is not destroyed automatically
- If no catch handler catches the exception the function terminate is called, which by default calls function abort

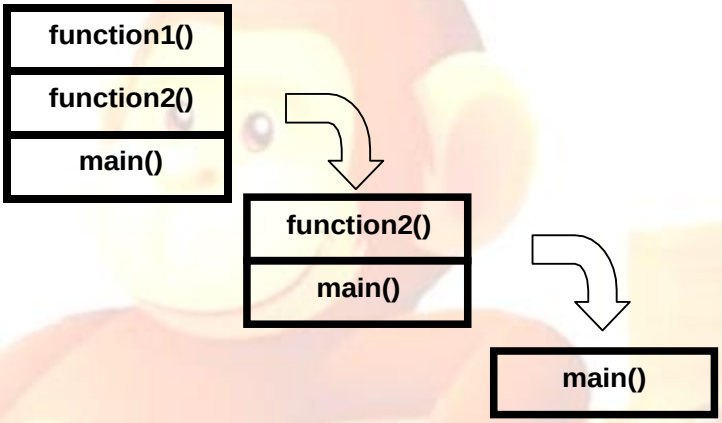
Examples

Nested Functions example:

In example below we have two functions function1 and function2, function2 is calling function1, in function1 we have added exception throwing code so it is necessary now to call function1 in try catch blocks otherwise compiler will generated an error, we are calling function2 in main, now note that function2 ifself is calling function1 that needs try catch block so we need to call function2 in try catch block, (otherwise compiler will generate an error), in case function1 code generates an exception stack unwinding takes place control will be returned to function2 which will return control to main the diagram below code explain this concept,

```
void function1() {
        throw Exception();
}
void function2() {
```

```
                function1();
}
int main() {
                try{
                        function2();
                } catch( Exception ) { }
                return 0;
}
```



Nested Try catch blocks example:

The stack unwinding is also performed if we have nested try catch blocks,

Example

```
int main( ) {
                try {
                        try {
                                throw 1;
                        }
                        catch( float ) { }
                }
                catch( int ) {
                 }
                return 0;
}
```

When an exception will be thrown from a try block control will go to catch block of that try block if it is not appropriate catch block then control will go to ctach blocks of other try blocks above it one by one, Stack Unwinding will occur in following order in above example,

- **If exception is thrown from innermost try block,**

- Firstly the catch handler with float parameter is tried (innermost), this catch handler will not be executed as its parameter is of different type – no coercion (match)
- Secondly the catch handler with int parameter is tried and executed

- **If exception is thrown from outer try block, then as there is no other try block** above it so only this block catch handler will be matched with exception if it matches catch block will be exectuted otherwise default terminate and abort functions (discussed in this lecture start) will be called.

Catch Handler

- We can modify the code in catch handler to use the exception object to carry information about the cause of error
- The exception object thrown is copied to the object given in the handler
- We pass the exception as reference instead of by value in the catch handler to avoid problem caused by shallow copy

Example

We have added a method Print in our exception class that to show the user cause of error,

```
class DivideByZero { // exception class
        int numerator;
public:
        DivideByZero(int i) { // constructor taking one parameter (dividend)
                numerator = i;
        }
        void Print() const{
                cout << endl << numerator
                << " was divided by zero";
        }
};
int Quotient(int a, int b) {
        if(b == 0){
                throw DivideByZero(a);
        }
        return a / b;
}
for ( int i = 0; i < 10; i++ ) {
        try {
                GetNumbers(a, b);
                quot = Quotient(a, b); ...
        } catch(DivideByZero & obj) {
```

```
                obj.Print();
                i--;
        }
}
```

Body of main Function
Output

```
Enter two integers
10
10
Quotient of 10 and 10 is 1
Enter two integers
10
0
10 was divided by zero
...
```

Catch Handler

The object thrown as exception is destroyed when the execution of the catch handler completes

Avoiding too many Catch Handlers

There are two ways to catch more then one object in a single catch handler

- Use inheritance
- Catch every exception

Inheritance of Exceptions

In inheritance we Group all exceptions according to their categories and catch single exception for whole category for example for code below we have divided the exceptions as follows,

- Math exceptions (Divide by Zero and IntegerOutOfRange exception)
- Input Output exceptions(InputStreamError)

```
try{
        ...
}
catch(DivideByZero){
        ...
}
catch(IntergerOutOfRange){
        ...
}
catch (InputStreamError){
```


© Virtual University of Pakistan

```
}
```

Example–With Inheritance

```
try{
                ...
}
catch (MathError){
}
catch (InputStreamError){
}
```

Catch Every Exception

C++ provides a special syntax that allows to catch every object thrown

```
catch ( ... )
{
                //...
}
```

Re-Throw

A function can catch an exception and perform partial handling
Re-throw is a mechanism of throw the exception again after partial handling

```
        throw; /*without any expression*/
```

Example

```
int main ( ) {
                try {
                        Function();
                }
                catch(Exception&) {
                        ...
                }
                return 0;
}
void Function( ) {
                try {
                        /*Code that might throw
                                an Exception*/
                       }

        catch(Exception&) {
                        if( can_handle_completely ) {
                                // handle exception
                        } else {
```

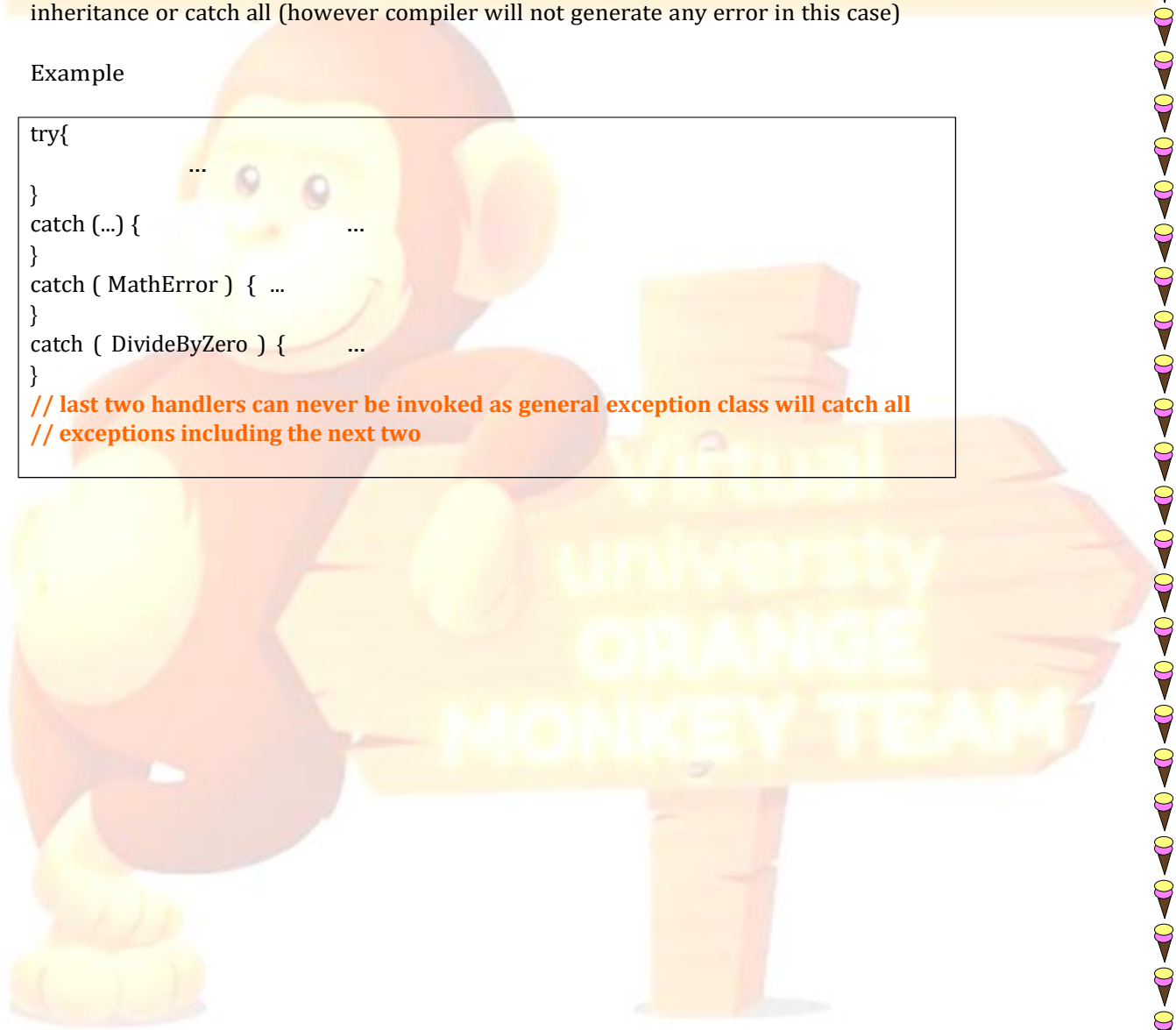
```
                                    // partially handle exception
                                    throw; //re-throw exception
                                }
                            } // end of catch
 } // end of function
```

Order of Handlers

Order of the more than one catch handlers can cause logical errors when using inheritance or catch all (however compiler will not generate any error in this case)

Example

```
try{
                    …
}
catch (...) {                       …
}
catch ( MathError ) { …
}
catch ( DivideByZero ) {            …
}
// last two handlers can never be invoked as general exception class will catch all
// exceptions including the next two
```



Lecture No.45

Resource Management

- Function acquiring a resource must properly release it
- Throwing an exception can cause resource wastage

Example

```
int function1(){
        FILE *fileptr = fopen("filename.txt","w");
        …
        throw exception();
        …
        fclose(fileptr);
        return 0;
}
```

In case of exception the call to fclose will be ignored and file will remain opened.

We can remove this issue in following ways,

First Attempt

```
int function1(){
        try{
                FILE *fileptr = fopen("filename.txt","w");
                fwrite("Hello  World",1,11,fileptr);
                …
                throw exception();
                fclose(fileptr);
        } catch(...) {
                fclose(fileptr);  // adding fclose in catch handler as well
                throw;
        }
        return 0;
}
```

But it is,code duplication.

Second Attempt:

Adding a separate class file handling what will have constructor to open file and destructor to close it, as you know objects and local variables in try block are destroyed automatically when try block complete its execution or in case exception is thrown, so this file object will automatically be destroyed,

```
class FilePtr{
```

```
                FILE * f;
public:
                FilePtr(const char *name, const char * mode) {
                        f = fopen(name, mode);
                    }
                ~FilePtr() {
                        fclose(f);
                    }
                operator FILE * () {
                        return f;
                    }
};

int function1(){
                FilePtr  file("filename.txt","w");
                fwrite("Hello World",1,11,file);
                throw exception();
                …
                return 0;
}
```

- The destructor of the FilePtr class will close the file
- Programmer does not have to close the file explicitly in case of error as well as in normal case

Exception in Constructors

Exception thrown in constructor cause the destructor to be called for any object built as part of object being constructed before exception is thrown
Destructor for partially constructed object is not called

Example

```
class Student{
                String FirstName;
                String SecondName;
                String  EmailAddress;
                …
};
```

If the constructor of the SecondName throws an exception then the destructor for the First Name will be called.

So, generally we can say that in constructor if an exception is thrown than all objects created so far are destroyed, if EmailAddress String object had thrown exception then SecondName and FirstName objects will be destroyed using their destructor. However destructor of Student class itself will not be called in any case as its object was not completely constructed.

Exception in Initialization List

Exception due to constructor of any contained object or the constructor of a parent class can be caught in the member initialization list

Example

```
Student::Student (String aName) : name(aName)
/*The constructor of String can throw a exception*/
{
        ...
}
```

Exception in Initialization List

The programmer may want to catch the exception and perform some action to rectify the problem

Example

```
Student::Student (String aName)
    try     : name(aName) {
        ...
    }
    catch(...) {
    }
```

Exceptions in Destructors

Exception should not leave the destructor, because when a destructor is running it means that we have a stack unwinding going on that has run this destructor to delete this object if this exception will be allowed to run it will run another stack unwinding mechanism and this will be called leaving the exception from destructor and is not allowed. C++ allows running only one stack unwinding process at a time.
If a destructor is called due to stack unwinding, and an exception leaves the destructor then the function std::terminate() is called, which by default calls the std::abort()

Example

```
class Exception;
class Complex{
        ...
public:
        ~Complex(){
            throw Exception();
        }
};
int main(){
        try{
```

```
                    Complex obj;
                    throw Exception();
                    …
            }
            catch(…){
            }
            return 0;
}

// The program will terminate abnormally
```

So add catch exception in destructor itself as shown below, in this case single stack unwinding process may handle the situation.

Example

```
Complex::~Complex()
{
            try{
                    throw Exception();
            }
            catch(…){
            }
}
```

Exception Specification

- Program can specify the list of exceptions a function is allowed to throw
- This list is also called throw list
- If we write empty list then the function wont be able to throw any exception

Syntax

```
void Function1() {…}
void Function2() throw () {…}
void Function3( ) throw (Exception1, …){}
```

Here,

- Function1 can throw any exception
- Function2 cannot throw any Exception
- Function3 can throw any exception of type Exception1 or any class derived from it

Exception Specification

- If a function throws exception other then specified in the throw list then the function unexpected is called

© Virtual University of Pakistan

- The function unexpected calls the function terminate and terminates the program
- If programmer wants to handle such cases also then he must provide a handler function and tell the compiler to call handler using set_unexpected

Course Review

We have studied the following topics in this course,

Object Orientation

- What is an object
- Object-Oriented Model
  - Information Hiding
  - Encapsulation
  - Abstraction
- Classes

Object Orientation

- Inheritance
  - Generalization
  - Sub-Typing
  - Specialization
- "IS-A" relationship
- Abstract classes
- Concrete classes

Object Orientation

- Multiple inheritance
- Types of association
  - Simple association
  - Composition
  - Aggregation
- Polymorphism

Classes – C++ Constructs

- Classes
  - Data members
  - Member functions
- Access specifier
- Constructors
- Copy Constructors
- Destructors

Classes – C++ Constructs

- this pointer

- Constant objects
- Static data member
- Static member function
- Dynamic allocation

Classes – C++ Constructs

- Friend classes
- Friend functions
- Operator overloading
  - Binary operator
  - Unary operator
  - operator []
  - Type conversion

Inheritance – C++ Constructs

- Public inheritance
- Private inheritance
- Protected inheritance
- Overriding
- Class hierarchy

Polymorphism – C++ Constructs

- Static type vs. dynamic type
- Virtual function
- Virtual destructor
- V-tables
- Multiple inheritance
- Virtual inheritance

Templates – C++ Constructs

- Generic programming
- Classes template
- Function templates
- Generic algorithm
- Templates specialization
  - Partial Specialization
  - Complete specialization

Templates – C++ Constructs

- Inheritance and templates
- Friends and templates
- STL
  - Containers
  - Iterators

© Virtual University of Pakistan

- Algorithms

Writing Reliable Programs

- Error handling techniques
  - Abnormal termination
  - Graceful termination
  - Return the illegal value
  - Return error code from a function
  - Exception handling

Note:

Please give your suggestions/comments about this document at the email address cs304@vu.edu.pk, in the following format

Page No.#	Issue/Error

Cs304 Quiz no. 3

All MCqs 100% Confirm

Subscribe My Youtube Channel Sunrise Zeeshan

Contact For Quiz 03438509619

Sunrise Zeeshan

1. **A function call is resolved at run time in**

a) Non virtual members function

b) Virtual member function

c) Both non-virtual and virtual member function

d) None of given

2. **If the user does not specify the type if inheritance, then the default type of inheritance is**

a) Public inheritance

b) Private inheritance

c) Protected inheritance

d) None of given

3. **In case of private inheritance, private members of base class will be in derived class?**

a) Private

b) Public

c) Protected

d) Hidden

4. **Then int member i of base class is accessible in class**,

a) derived 1 only

b) derived 2 only

c) both derived 1 derived

5. **In case of public inheritance, protected members of base class will be in derived class?**

a) Private

b) Public

c) Protected

d) Hidden

6. **By default assignment operator (=) performs**,

a) Shallow copy

b) Deep copy

c) Both of these

d) None of these

7. **In protected inheritance the public members of base class become _ in derived class**.

a) Public

b) Private

c) Protected

d) None of the given

8. **Compiler can automatically generates the following constructor/s for a class,**

a) Default constructor

b) Copy constructor

c) Both of these

d) None of these

9. **Friend's functions of class are _ members of that class.**

Sunrise Zeeshan

a) Public

b) Private

c) Protected

d) None of the given

10. **Methodologies to the development of reusable software relate to.**

a) Structure programming

b) Procedural programming

c) Generic programming

d) None of the given

11. **Function overriding is done in context of**,

a) Single class

b) Single derived class

c) Single base class

d) Derived and base classes

12. **Which of the following is not type of inheritance in c++?**

a) Public

b) Protected

c) Restricted

d) Private

13. **If there is a pointer ,p, to objects of a base class and it contains the address of an object of a derived class and both classes contain a virtual member function, ding(),then the statement p->ding(); will cause the version of ding in the class to be executed**.

a) Base

b) Derived

c) Virtual

Sunrise Zeeshan

d) Implemented

14. **Target class of a function call is determined t run time**.

a) Instance

b) Virtual

c) Operator

d) None of the given

15. **To make a pure virtual, we need to give _after() of this function**.

a) =0;

b) =1;

c) Null;

d) None of the given

16. **If we have not given any constructor for the class, compiler generates which of the following constructors?**

a) Emplicit Default Constructor

b) Implicit Default Constructor

c) Emplicit Parameterized Constructor

17. **In c++ by default access of classes is .**

a) Private

b) Protected

c) Public

d) None of the given

18. **In public inheritance the public members of base class become in derived class**

a) Private

b) Protected

c) Public

Sunrise Zeeshan

d) None of the given

19. **A class hierarchy**

a) Shows the same relationships as a organization chart

b) Describe “has a “relationship.

c) Describe “is a kind of” relationship.

d) Shows the same relationships as a family tree.

20. **In private Inheritance the public members of the base class become_____ in derived class**

a) Public

b) private

c) Protected

d) None of the given option

21. **In the line No.2, which constructor of student class will be**

a) Default constructor of student class

b) Copy constructor of student class

c) Both Default and Copy constructor of student class

d) No constructor will be called

22. **Which of the following is Not casting operator in c++ standard?**

a) Static-cast

b) Var-cast

c) Dynamic-cast

d) Reinterpret-cast

23. **Which of the following type of inheritance is used to model “Implemented in terms of” relationship?**

a) Public

b) Private

Sunrise Zeeshan

c) Protected

d) Confidential

24. **When we want to implement one class in terms of another class then we use,**

a) Public inheritance

b) Protected inheritance

c) Private inheritance

d) None of these options

25. **Public inheritance represents;**

a) "IS A "relationship

b) "Has A" relationship

c) "IS Special kind of" relationship

d) None of these option

26. **Suppose we have defined derived class copy constructor but have not defined base class copy constructor then compiler will,**

a) Use base class default constructor

b) Generate base class copy constructor itself

c) Use base class assignment operator

d) None of these

27. **Child class can call constructor of its**,

a) Direct base class

b) Indirect base class

c) Both direct and indirect base classes

d) None of these

28. **We can have type of member functions in a class**.

a) Public

Sunrise Zeeshan

b) Private

c) Protected

d) All of these options

29. **Compiler generated copy constructor performs**

a) Shallow copy

b) Deep copy

c) Both Shallow and Deep copy

d) None of these options

30. **In c++ generic programming is done using**

a) Procedures

b) Packages

c) Templates

d) None of the given

31. **Target class of a function call is determined at run time**.

a) Instance

b) Virtual

c) Operator

d) None of the given

32. **Suppose person is a user defined class.In statement "Person*pPtr", static type of pPtr is** ----.

a) pPtr

b) pointer

c) person

d) None of the given

33. **Which statement is true for concrete class?**

a) It cannot be inherit from an abstract class

Sunrise Zeeshan

b) It can be instantiated

c) It cannot be instantiated

d) None of given

34. **In c++, compiler can generate which of the following operators 'code**,

a) ==

b) =

c) &

d) &&

35. **In specialization we can,**

a) Replace child class with its base class

b) Replace base class with its child class

c) Replace both child and base classes interchangeably

d) None of the given option

36. **In private inheritance derived class pointer can be assigned to base class pointer in,**

a) Main function

b) In derived class member and friend class function

c) In base class member and friend functions

d) None of the given option

37. **When we access private members of the class from outside the class using operator with its object pointer**

a) ->

b) .

c) &

d) None of above

38. **Which of the following access specifier ensure that base class member is accessible in**

Sunrise Zeeshan

derived class and Not outside of this class

a) Public

b) Private

c) Protected

d) All of above

39. **Static casting is**

a) C++ way of calling base class function from derived class

b) C way of calling base class function from derived class

c) Both of these

d) None of these

40. **Suppose both derive and base classes have compilers generated copy constructors then derived class copy construct is**

a) Compiler generated default constructor

b) Compiler generated copy constructor

c) Both of these

41**. In case of protected inheritance, public members of the base class will be_____ in derived class**

a) Private

b) Public

c) Protected

d) Hidden

42. **Methodologies to the development of reusable software related to**

a) Structured programming

b) Procedural programming

c) Generic programming

Sunrise Zeeshan

d. None of the given

43. **A child class can call constructor of the parent call through**

a. Its constructor initialization list

b) Its constructor body

c) Both from its constructor initialization list or body

d) Cannot call the constructor of its parent class

44. **In case of dynamic memory allocation in or class we should use**

a) Default assignment operator

b) User defined assignment operator

c) Default copy constructor

45. **Which of the following is /are casting operator(s) in c++?**

a) Static_cast

b) Const_cast

c) Dynamic_cast

d) All of the given options

46. A template provides a convenient way to make a family of

a) Variables and data members

b) Functions and classes

c) Classes and exceptions

d) Programs and algorithm

47. **Consider the code below. Class class1{**

Protected:

Int I;

};

Class class2: private calls 1{

Sunrise Zeeshan

};

a) Public

b) Protected

c) Private

d) None of the given options

48. **Consider the following two lines of code written for class student**

Student sobj1;

Student sobj2(sobj1)

In line 2 which constructor of student class will be called?

a) Default constructor of student class

b) Copy constructor of student class

c) Both default and copy constructor of student class

d) No constructor will be called

49. **Consider the code below class class1{**

Private:

Void func1();

};

Class class2: public class1{

};

Function func1 of class 1 is in class2.

a) Public

b) Protected

c) Private

d) None of the given option

50. **We can call base class assignment operator in derived class user defined assignment operator**

Sunrise Zeeshan

a) Implicitly

b) Explicitly

c) Using both of these options

d) Using none of these options

51. **Function func1of class1 is _ in class2.**

a) Public

b) Protected

c) Private

d) None of the given

52. **The default inheritance mode is,**

a) Public inheritance

b) Protected inheritance

c) Private inheritance

d) None of the given

53. **Consider the code below class class1{**

public:

Void func1();

};

Class class2: protected class1{

};

Function func1 of class 1 is in class2.

a) Public

b) Protected

c) Private

54. **Friend Functions of a class are____ members of that Class**

Sunrise Zeeshan

None of the given options.

55. **The C++ code statement class A: public B shows. relationship**.

Inheritance

56. **A parent class can call constructor of its child class through,**

Can not call the constructor of its child class

57. **A class with no pure virtual function is_____ a class**

Concrete

58**. Two functions with same names, parameters and retum type can exist in,**

Function overriding

59. **Sender of the message does not need to know the exact class of receiver in____**

Polymorphism

60. **Which of the following is TRUE,**

Derived class pointer can be used as Base class pointer

Sunrise Zeeshan

CS304-OOP
FINAL TERM MCQS
Prepared by: JUNAID MALIK

AL-JUNAID TECH INSTITUTE



www.vulmshelp.com





Language Courses Training Available

I'm providing paid courses in different languages within 3 Months,Certificate will be awarded after completion.

- HTML
- CSS
- JAVASCRIPT
- BOOTSTRAPS
- JQUERY
- PHP MYSQL
- NODES.JS
- REACT JS

LMS Handling Services

LMS Activities Paid Task

Assignments 95% Results

Quizes 95% Results

GDB 95% Results

For CS619 Project Feel Free To Contact With Me

Ph# 0304-1659294
Email: junaidfazal08@gmail.com

AL-JUNAID INSTITUTE GROUP

ALL answers are verified if found any mistake then Correct ACCORDINGLY

1. Function func1of class1 is_____ in class2.
   - ► Public
   - ► Protected
   - ► Private
   - ► None of the given
2. The default inheritance mode is,
   a) Public inheritance
   b) Protected inheritance
   c) Private inheritance
   d) None of the given
3. A template provides a convenient way to make a family of
   a) Variables and data members
   b) Functions and classes
   c) Classes and exceptions
   d) Programs and algorithms
4. Then int member i of base class is accessible in class,
   a) derived 1 only
   b) derived 2 only
   c) both derived 1 derived 2
   d) none of the given options
5. In case of public inheritance, protected members of base class will be_____in derived class?
   a) Private
   b) Public
   c) Protected
   d) hidden
6. By default assignment operator (=) performs,
   a) Shallow copy
   b) Deep copy
   c) Both of these
   d) None of these
7. In protected inheritance the public members of base class become _____in derived class.
   a) Public
   b) Private
   c) Protected
   d) None of the given
8. We can access public members of the class from outsides the class using_____operator with its object pointer
   a) ->

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

b) .
c) &
d) @

9. Compiler can automatically generates the following constructor/s for a class,
   a) Default constructor
   b) Copy constructor
   c) Both of these
   d) None of these
10. Friend's functions of class are _____members of that class.
   a) Public
   b) Private
   c) Protected
   d) None of the given
11. Methodologies to the development of reusable software relate to_____.
   a) Structure programming
   b) Procedural programming
   c) Generic programming
   d) None of the given
12. Function overriding is done in context of,
   a) Single class
   b) Single derived class
   c) Single base class
   d) Derived and base classes
13. Which of the following is not type of inheritance in c++?
   a) Public
   b) Protected
   c) Restricted
   d) Private
14. If there is a pointer ,p, to objects of a base class and it contains the address of an object of a derived class and both classes contain a virtual member function, ding(),then the statement p->ding(); will cause the version of ding in the_____class to be executed.
   a) Base
   b) Derived
   c) Virtual
   d) Implemented
15. Target class of a____function call is determined t run time.
   a) Instance
   b) Virtual
   c) Operator
   d) None of the given
16. In c++ dynamic binding and polymorphism will be achieved when the member function will be_____

AL-JUNAID TECH INSTITUTE
www.vulmshelp.com
0304-1659294

a) Private
b) Public
c) Virtual
d) Inline

17. To make a pure virtual, we need to give____after() of this function.
a) =0;
b) =1;
c) Null;
d) None of the given

18. If we have not given any constructor for the class,compiler generates which of the following constructors?
a) Emplicit Default Constructor
b) Implicit Default Constructor
c) Emplicit Parameterized Constructor
d) Implicit Parameterized Constructor

19. In c++ by default access of classes is____.
a) Private
b) Protected
c) Public
d) None of the given

20. In public inheritance the public members of base class become_____in derived class
a) Private
b) Protected
c) Public
d) None of the given

21. A class hierarchy
a) Shows the same relationships as a organization chart
b) Describe "has a "relationship.
c) Describe "is a kind of" relationship.
d) Shows the same relationships as a family tree.

22. In private Inheritance the public members of the base class become_____in derived class
a) Public
b) private
c) Protected
d) None of the given option

23. In the line No.2, which constructor of student class will be
a) Default constructor of student class
b) Copy constructor of student class
c) Both Default and Copy constructor of student class
d) No constructor will be called

24. Which of the following is Not casting operator in c++ standard?
a) Static-cast

AL-JUNAID TECH INSTITUTE
@03041659294
www.vulmshelp.com

b) Var-cast
c) Dynamic-cast
d) Reinterpret-cast

25. Which of the following type of inheritance is used to model "Implemented in terms of" relationship?
a) Public
b) Private
c) Protected
d) Confidential

26. When we want to implement one class in terms of another class then we use,a) Public inheritance
b) Protected inheritance
c) Private inheritance
d) None of these options

27. Public inheritance represents;
a) "IS A "relationship
b) "Has A" relationship
c) "IS Special kind of" relationship
d) None of these option

28. Suppose we have defined derived class copy constructor but have not defined base class copy constructor then compiler will,
a) Use base class default constructor
b) Generate base class copy constructor itself
c) Use base class assignment operator
d) Nine of these option is correct

29. Child class can call constructor of its,a) Direct base class
b) Indirect base class
c) Both direct and indirect base classes
d) None of these

30. We can have_____type of member functions in a class.
a) Public
b) Private
c) Protected
d) All of these options

31. Compiler generated copy constructor performs_____
a) Shallow copy
b) Deep copy
c) Both Shallow and Deep copy
d) None of these options

32. In c++ generic programming is done using_____
a) Procedures

AL-JUNAID TECH INSTITUTE 0304 1659294 www.vulmshelp.com

b) Packages
c) Templates
d) None of the given

33. Target class of a____function call is determined at run time.
a) Instance
b) Virtual
c) Operator
d) None of the given

34. Suppose person is a user defined class.In statement "Person*pPtr", static type of pPtr is ----.
a) pPtr
b) pointer
c) person
d) None of the given

35. Which statement is true for concrete class?
a) It cannot be inherit from an abstract class
b) It can be instantiated
c) It cannot be instantiated
d) None of given

36. In c++, compiler can generate which of the following operators 'code,
a) ==
b) =
c) &
d) &&

37. In specialization we can,
a) Replace child class with its base class
b) Replace base class with its child class
c) Replace both child and base classesinterchangeably
d) None of the given option

38. In private inheritance derived class pointer can be assigned to base class pointer in,
a) Main function
b) In derived class member and friend class function
c) In base class member and friend functions
d) None of the given option

39. When we access private members of the class from outside the class using_____operator with its object pointer
a. ->
b. .
c. &
d. None of above

40. Which of the following access specifier ensure that base class member is accessible in derived class and Not outside of this class
a. Public

AL-JUNAID TECH INSTITUTE
0304-1639294
www.vulmshelp.com

b. Private
c. Protected
d. All of above

41. Static casting is
    a. C++ way of calling base class function from derived class
    b. C way of calling base class function from derived class
    c. Both of these
    d. None of these
42. Suppose both derive and base classes have compilers generated copy constructors then derived class copy construct is
    a. Compiler generated default constructor
    b. Compiler generated copy constructor
    c. Both of these options are correct
    d. None of these operators are correct
43. In case of protected inheritance, public members of the base class will be_____in derived class
    a. Private
    b. Public
    c. Protected
    d. Hidden
44. Methodologies to the development of reusable software related to_____
    a. Structured programming
    b. Procedural programming
    c. Generic programming
    d. None of the given
45. A child class can call constructor of the parent call through
    a. Its constructor initialization list
    b. Its constructor body
    c. Both from its constructor initialization list or body
    d. Cannot call the constructor of its parent class
46. In case of dynamic memory allocation in or class we should use
    a. Default assignment operator
    b. User defined assignment operator
    c. Default copy constructor
    d. None of these
47. Consider the code below.

```
Class class1 {
Protected:
Int I;
};
Class class2: private calls 1{
};
```

    a) Public

AL-JUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

b) Protected
c) Private
d) None of the given options

48. Consider the following two lines of code written for class student
    1. Student sobj1;
    2. Student sobj2(sobj1)

In line 2 which constructor of student class will be called?

a) Default constructor of student class
b) Copy constructor of student class
c) Both default and copy constructor of student class
d) No constructor will be called

49. Which of the following is /are casting operator(s) inc++?
    a. Static_cast
    b. Const_cast
    c. Dynamic_cast
    d. All of the given options

50. In case of private inheritance, private members of base class will be_____in derived class?
    a. Private
    b. Public
    c. Protected
    d. Hidden

51. Consider the code below

```
class class1{
Private:
Void func1();
};
Class class2 : public class1{
};
```

Function func1 of class 1 is_____in class2.

    a. Public
    b. Protected
    c. Private
    d. None of the given options

52. We can call base class assignment operator in derived class user defined assignment operator
    a. Implicitly
    b. Explicitly
    c. Using both of these options
    d. Using none of these options

53. A function call is resolved at run time in

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

a. Non virtual members functionb. Virtual member function

c. Both none-virtual and virtual member function

d. None of given

54. If the user does not specify the type if inheritance, then the default type of inheritance is _____

a. Public inheritance b. Private inheritance

c. Protected inheritance

d. None of given

55. Consider the code belowclass class1 {

public:

Void func1();

};

Class class2: protected class1{

};

Function func1 of class 1 is_____in class2.

a. Public

b. Protected

c. Private

d. None of the given options

56. Which of the following causes run time binding?

► Declaring object of abstract class
► Declaring pointer of abstract class
► Declaring overridden methods as non-virtual (Page 226)
► None of the given

57. Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?

► Templates (Page 256)
► Overloading
► Data hiding
► Encapsulation

58. A copy constructor is invoked when

► a function do not returns by value.
► an argument is passed by value. (Page 78)
► a function returns by reference.
► an argument is passed by reference.

59. Like template functions, a class template may not handle all the types successfully.

► True (Page 258)

ALJUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

► False

60. A class template may inherit from another class template.

► True (Page 288)
► False

61. By default the vector data items are initialized to ____
► 0 google
► 0.0
► 1
► null

62. In Private ---------- only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object

►specialization
► inheritance (Page 216)
► abstraction
► composition

63. Which of the following is/are advantage[s] of generic programming?
► Reusability
► Writability
► Maintainability
► All of given (Page 256)

64. Template functions use _____ than ordinary functions.

► Greater Memory
► Lesser Memory
► Equal Memory
► None of the given options

65. Non Template Friend functions of a class are friends of _____instance/s of that class.

► All google
► One specific
► All instances of one date type
► None of the given options

66. A pointer to a base class can point to objects of a derived class.

► True google
► False

67. A template argument is preceded by the keyword _____.

AL JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

► vector
► class
► template
► type*

68. Which one of the following terms must relate to polymorphism?

► Static allocation
► Static typing
► Dynamic binding (Page 239)
► Dynamic allocation

69. Multiple inheritance can be of type

► Public
► Private
► Protected
► All of the given

70. Assume a class Derv that is privately derived from class Base. An object of class Derv located in main() can access

► public members of Derv
► protected members of Derv.
► private members of Derv.
► protected members of Base.

71. A function call is resolved at run-time in_____

► non-virtual member function.
► virtual member function. (Page 239)
► Both non-virtual member and virtual member function.
► None of given

72. Two important STL associative containers are _____ and _____.

► set,map
► sequence,mapping
► setmet,multipule
► sit,mat

73. An abstract class is useful when,

► We do not derive any class from it.
► There are multiple paths from one derived class to another.
► We do not want to instantiate its object
► You want to defer the declaration of the class.

74. Suppose you create an uninitialized vector as follows:

ALJUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

vector<int> evec;
After adding the statment,
evec.push_back(21);
what will happen?

► The following statement will add an element to the start (the back) of evec and will initialize it with the value 21.

► The following statement will add an element to the center of evec and will reinitialize it with the value
21.

► The following statement will delete an element to the end (the back) of evec and will reinitialize it with the value 21.

► The following statement will add an element to the end (the back) of evec and initialize it with the value 21.

75. Default constructor is such constructor which either has no ---------or if it has some parameters these have -----
- values

► Parameter, temporary
► Null, Parameter
► Parameter, default (Page 75)
► non of the given

76. Classes like TwoDimensionalShape and ThreeDimensionalShape would normally be concrete, while classes like Sphere and Cube would normally be abstract.

► True
► False

77. In order to define a class template, the first line of definition must be:

► template <typename T> (Page 257)
► typename <template T>
► Template Class <ClassName>
► Class <Template T>

78. A function template can not be overloaded by another function template.

► True
► False

ALJUNAID TECH INSTITUTE 0304-1659294 www.vulmshelp.com

79. Identify the correct way of declaring an object of user defined template class A for char type members?

► A< char > obj;
► <char>A obj;
► A obj<char>;
► Obj <char> A;

80. The user must define the operation of the copy constructor.

► True
► False

81. The find() algorithm

► finds matching sequences of elements in two containers.
► finds a container that matches a specified container.
► takes iterators as its first two arguments. (Object-Oriented Programming in C++)
► takes container elements as its first two arguments.

82. Compiler performs _____ type checking to diagnose type errors,

► Static (Page 261)
► Dynamic
► Bound
► Unbound

83. Vectors contain contiguous elements stored as a[an] ___.

► variable
► array (Page 306)
► function
► datatype

84. In a de-queue, (chose the best option)

► data can be quickly inserted or deleted at any arbitrary location.
► data can be inserted or deleted at any arbitrary location, but the process is relatively slow.
► data can not be quickly inserted or deleted at either end.
► data can be inserted or deleted at either end, but the process is relatively slow.

85. Algorithms can only be implemented using STL containers.

► True
► False(Object-Oriented Programming in C++)

ARJUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

86. What is a class?

- ► A class is a section of computer memory containing objects.
- ► A class is a section of the hard disk reserved for object oriented programs
- ► A class is the part of an object that contains the variables.
- ► A class is a description of a kind of object. Click here for detail

87. Inheritance is a way to

- ► organize data.
- ► pass arguments to objects of classes.
- ► add features to existing classes without rewriting them. (Page 27)
- ► improve data-hiding and encapsulation.

88. We can use "this" pointer in the constructor in the body and even in the initialization list of any class if we are careful,

- ► True
- ► False

89. _____ and _____ methods may not be declared abstract.

- ► private,staticClick here for detail
- ► private,public
- ► static,public
- ► none of given

90. ---------- members are somewhere between public and private members. They are used in inheritance

- ► protected
- ► public
- ► private
- ► global (Page 187)

91. Which of these are examples of error handling techniques?

- ► Abnormal Termination
- ► Graceful Termination

AJ TECH INSTITUTE
0304-1659294
www.vulmshelp.com

► Return the illegal
► all of the given (Page 329)

92. ---------- follow try block to catch the object thrown

► catch block (Page 333)
► throw block
► main block
► non of the given

93. Graphical representation of the classes and objects is called object model it shows -------

► Class Name only
► Class Name and attributes
► Relationships of the objects and classes
► all of the given

94. Destructor can be overloaded

► True

► False(Page 92)

95. Which of the following is the best approach to implement generic algorithms with minimum number of coding lines?

► Templates (Page 256)
► Overloading
► Overriding
► Friend function/class

ALJUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

96. Each try block can have _____ no. of catch blocks.

- ► 1
- ► 2
- ► 3
- ► As many as necessary.

97. class DocElement

```
{
public:
    virtual void Print() { cout << "Generic element"; }
};
class Heading : public DocElement
{
public:
    void Print() { cout << "Heading element"; }
};
class Paragraph : public DocElement
{
public:
    void Print() { cout << "Paragraph element"; }
};
void main()
{
    DocElement * p = new Paragraph();
```

AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

```
    p->Print();
}
```

98. When you run this program, it will print out a single line to the console output.

What will be in that line?

Select one correct answer from the following list:

► Generic element ►
Heading element
► Paragraph element
► Nothing will be printed.

99. Suppose we have two derived classes from a single class, can we write a method with same name in both these derived classes? Choose the best option.

► No
► Only if the two classes have the same name (Page 204)
► Only if the main program does not declare both kinds
► Yes

100. When a virtual function is called by referencing a specific object by name and using the dot member selection operator (e.g., squareObject.draw()), the reference is resolved at compile time.

► True
► False

101. Considering the resolution order in which Considering the resolution order in which compiler search for functions in a program; the first priority is given to; the first priority is given to,

► general template
► partial specialization
► complete specialization
► ordinary function (Page 287)

102. One purpose of an iterator in the STL is to connect algorithms and containers.

►True
► False

103. In _____, a base class can be replaced by its derived class,

► Sub-typing (Page 31)
► Super-typing
► Multiple-typing
► Restricted-typing

ALJUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

104. one this pointer does not point to current object of any class,

- True
- False

105. Which of the following operator(s) take(s) one or no argument if overloaded?

- ++(Page 162)
- -
- +
- All of the given

106. Which of the following operators can not be overloaded?

- Scope resolution operator (::) (Page 141)
- Insertion operator (<<)
- Extraction operator (>>)
- The relation operator (>)

107. Which of these are examples of error handling techniques ?

- Abnormal Termination
- Graceful Termination
- Return the illegal
- all of the given (Page 329)

108. _____ "is a" relationship

- Inheritance (Page 25)

ARSHAD TECH INSTITUTE
0304-1659294
www.vulmshelp.com

► Polymarphism
► abstraction
► encapsulation

109. Virtual functions allow you to

► create an array of type pointer-to-base class that can hold pointers to derived classes.
► create functions that can never be accessed.
► group objects of different classes so they can all be accessed by the same function code.
► use the same function call to execute member functions of objects from different classes

110. The copy() algorithm returns an iterator to

► the last element copied from.
► the last element copied to.
► the element one past the last element copied from.
► the element one past the last element copied to.

111. If you define a vector v with the default constructor, and define another vector w with a one-argument constructor to a size of 11, and insert 3 elements into each of these vectors with push_back(), then the size() member function will return _____ for v and _____ for w.

► 11 for v and 3 for w.
► 0 for v and 0 for w.
► 0 for v and 3 for w.
► 3 for v and 11 for w.

112. Which is not the Advantage of inheritance?

► providing class growth through natural selection. (Object-Oriented Programming in C++)
► facilitating class libraries.
► avoiding the rewriting of code.
► providing a useful conceptual framework.

113. Which type of inheritance is being represented by the following statement, class X : public A, public B { ... ... };

► Single inheritance
► Multiple inheritance (Page 41)

ALI ISLAM TECH INSTITUTE
0304-1659294
www.vulmshelp.com

► Double inheritance
► None of the given options

114. When we write a class template the first line must be:

► template < class *class_name*>
► template < class *data_type*>
► template < class T > (Page 257)

115. Function templates should be used where code and behavior must be identical.
► True (Page 262)
► False

116. The specialization pattern <T*> after the name says that this specialization is to be used for every,
► data type
► meta type
► virtual type
► pointer type (Page 286)

117. A range is often supplied to an algorithm by two _____ values.
► italic
► iteration (Object-Oriented Programming in C++)
► iterator
► None of given

118. Which of the following is an integral part of an object?

► State
► Behavior
► Unique identity
► All of the given (Page 12)

119. Consider the following statement

Cupboard has books
What is the relationship between Cupboard and books?

► Composition
► Aggregation
► Inheritance
► None of the given options

120. Which sentence clearly defines an object?

► one instance of a class. (Page 23)

ALJUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

► another word for a class.
► a class with static methods.
► a method that accesses class attributes.

121._____, which means if A declares B as its friend it does NOT mean that A can access private
data of B. It only means that B can access all data of A.

► Friendship is one way only
► Friendship is two way only
► NO Friendship between classes
► Any kind of friendship

122.The statement objA=objB; will cause a compiler error if the objects are of different classes.

► True
► False

123.Consider the call given below of an overloaded operator "+", *Rational_number_1 + Rational_number_2*
Where Rational_number_1 and Rational_number_2 are the two objects of Rational_number class (a user defined class). Identify which of the above two objects will be passed as an argument to the overloaded operator function?

► Rational_number_1
► Rational_number_2
► Both Rational_number_1 & Rational_number_2
► any of the two objects, randomly

124.If a class D has been derived using protected inheritance from class B (If B is a protected base and D is derived class) then public and protected members of B -------- accessed by member functions and friends of class D and classes derived from D
► can be
► cannot be
► does restirct to be
► not given

A-JOINED TECH INSTITUTE
www.vulmshelp.com

The type that is used to declare a reference or pointer is called its_____

► abstract type
► reference type
► ► static type
► defaulttype

125.How the information hidden within an object can be accessed?
► Through its interface
► Through its private data members
► Through its private member functions
► Through both public and private members

126.The sub-object's life is not dependant on the life of master class in _____.
► Separation
► Composition

► None of the given

127.Encapsulation means
Select correct option:
►Extending the behaviour of class in another class
►Data and behaviour are tightly coupled within an entity (Page 16)
►One entity takes all the attributes and operations of the other
►Taking out the common features and put those in a separate class

128.An STL container can not be used to,

► hold objects of class employee.
► store elements in a way that makes them quickly accessible.
► compile c++ programs. (Object-Oriented Programming in C++)
► organize the way objects are stored in memory

129.Which of the following may not be an integral part of an object?
► State
► Behavior
► Protected data members (Page 12)
► All of given

130.Public methods of base class can --------- be accessed in its derived class
► directly (Page 179) rep
► inderectly
► simultaniously
► non of the given

ALJUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

131. What is true about function templates?

► The compiler generates only one copy of the function template
► The compiler generates a copy of function respective to each type of data (Page 256)
► The compiler can only generate copy for the int type data
► None of the given.

132. When the base class and the derived class have a member function with the same name, you must be more specific which function you want to call (using _____).

► scope resolution operator
► dot operator
► null operator
► Operator overloading

133. A template provides a convenient way to make a family of

► variables and data members
► functions and classes(Object-Oriented Programming in C++)
► classes and exceptions
► programs and algorithms

134. A non-virtual member function is defined in a base class and overridden in a derived class; if that function is called through a base-class pointer to a derived class object, the derived-class version is used.

► True
► False

135. If there is a pointer p to objects of a base class, and it contains the address of an object of a derived class, and both classes contain a nonvirtual member function, ding(), then the statement p->ding(); will cause the version of ding() in the _____ class to be executed.

► Base (Object-Oriented Programming in C++)
► Derived
► Abstract
► virtual

136. In case of multiple inheritance a derived class inherits,

► Only the public member functions of its base classes
► Only the public data members of its base classes
► Both public data members and member functions of all its base classes
► Data members and member functions of any two base classes

ALJUNAID TECHNOLOGY INSTITUTE
0304-1639294
www.vulmshelp.com

137. Which of the following is incorrect line regarding function template?

► template<class T>
► template <typename U>
► Class<template T> (Page 257)
► template < class T, class U>

138. Consider a class named Vehicle, which of the following can be the instance of class Vehicle?

i. Car
ii. Computer
iii. Desk
iv. Ahmed
v. Bicycle
vi. Truck

- 1,4,5
- 2,5,6
- 1,2,3,6
- 1, 5, 6 Conceptual

139. Consider the code below,

```
class Fred {
public:
Fred();
...
};
int main()
{
Fred a[10];
Fred* p = new Fred[10];
...
}
```

Select the best option,

► Fred a[10]; calls the default constructor 09 times Fred* p = new Fred[10]; calls the default constructor 10 times
► Produce an error

ALJUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

► Fred a[10]; calls the default constructor 11 times
Fred* p = new Fred[10]; calls the default constructor 11 times

► Fred a[10]; calls the default constructor 10 times
Fred* p = new Fred[10]; calls the default constructor 10 times

140. When a variable is define as static in a class then all object of this class,

► Have different copies of this variable
► Have same copy of this variable (Page 110)
► Can not access this variable
► None of given

141. Which of the following operators always takes no argument if overloaded?
► /
► -
► +
► ++ (Page 162)

142. It is sometimes useful to specify a class from which no objects will ever be created.
► True(Object-Oriented Programming in C++)
► False

143. When a subclass specifies an alternative definition for an attribute or method of its superclass, it is _____ the definition in the superclass.

► overload
► overriding (Page 34)
► copy riding
► none of given

144. The mechanism of selecting function at run time according to the nature of calling object is called,
► late binding
► static binding
► virtual binding
► None of the given options (Page 227)

145. Dynamic binding means that target function for a call is selected at run time

Which one of the following functions returns the total number of elements in a vector.
► length();
► size(); (Page 318)
► ele();
► veclen();

JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

146. The main function of scope resolution operator (::) is,
- ► To define an object
- ► To define a data member
- ► To link the definition of an identifier to its declaration Google
- ► To make a class private

147. When is a constructor called?
- ► Each time the constructor identifier is used in a program statement
- ► During the instantiation of a new object (Object-Oriented Programming in C++)
- ► During the construction of a new class
- ► At the beginning of any program execution

148. Associativity can be changed in operator overloading.

- ► True
- ► False (Page 141)

149. A normal C++ operator that acts in special ways on newly defined data types is said to be

- ► glorified.
- ► encapsulated.
- ► classified.
- ► overloaded.

150. Which operator can not be overloaded?

- ► The relation operator (>=)
- ► Assignment operator (=)
- ► Script operator ([])
- ► Conditional operator (? :) (Page 141)

151. Suppose obj1 and obj2 are two objects of a user defined class A. An + operator is overloaded to add obj1 and obj2 using the function call obj1+obj2.

Identify the correct function prototype against the given call?
- ► A operator + (A &obj);
- ► int + operator(); (Page 143)
- ► int operator (plus) ();
- ► A operator(A &obj3);

ALJUNAID TECH INSTITUTE
0304 1659294
www.vulmshelp.com

152. **Suppose that the Test class does not have an overloaded assignment operator. What happens when an assignment a=b; is given for two Test objects a and b?**

- The automatic assignment operator is used
- **The copy constructor is used**
- Compiler error
- Run-time error

153. **A static member function can be called, even when a class is not _____.**

- Declared
- Define
- **Instantiated**
- Called

154. **Identify which of the following overloaded operator function's declaration is appropriate for the given call?**

Rational_number_1 + 2.325

Where Rational_number_1 is an object of user defined class Rational_number.

- Rational_number operator+(Rational_number & obj);
- Rational_number operator+(double& obj);
- Rational_number operator+(Rational_number &obj, double& num);
- operator+(double& obj);

155. **What problem(s) may occur when we copy objects without using deep copy constructor?**

- Dangling pointer
- Memory Leakage
- **All of the given**
- System crash

156. **_____ provide the facility to access the data member.**

- **accesser function**
- private function
- inline function
- None of the given

157. **Constant objects cannot change their state,**

- **True**
- False

`

158. **The _____ relationship indicates that an object contains other objects.**

- None of given
- **'has-a'**
- 'is-a'
- 'be-

159. **Which one of the following features of OOP is used to derive a class from another?**

- Encapsulation
- Polymorphism
- Data hiding
- **Inheritance**

160. **_____satisfy the condition of polymorphism**

- Carbon
- Diamond
- Coal
- **all of the given**

161. **A generalization-specialization relation between classes are implemented using**

- data hiding
- friend classes
- encapsulation
- **inheritance**

162. **The >= operator can be overloaded.**

- **True**
- False

163. **In order to free the memory occupied by the object, we use ----------**

- Constructor
- **Destructor**
- Shallow Copy
- Deep Copy

`

164. **Which of the following is not an example of multiple inheritances?**

- ► Mermaid
- ► **Woman**
- ► None of the given
- ► Amphibious Vehicle

165. **Static variable can be initialized more than once.**

- ► True
- ► **False**

166. **A generic class showing all the common attributes and a behavior of other classes represents a very important feature in oop called --------**

- ► **Inheritance**
- ► Encapsulation
- ► Polymarphism
- ► Abstraction

167. The parameters given in temple definition other than those used for mentioningtemplates types are called ____

a. Type Parameters
b. Non Type parameters
c. Default Type Parameters
d. None of the given option

168. Which of the following is the correct way to define a template class X?

e. Class < typename T>class X{}
f. Template class C{}
g. Template < typename T> classX{}
h. Typename<class T > class X{}

169. In resolution order of function template, compiler searches for_____in the end

a. Complete specialization
b. Partial specialization
c. Ordinary function
d. Generic template

170. __may inherit from a complete specialization

a. Partial specialization
b. Complete specialization
c. Ordinary class
d. All of given options

171. Which of the following may inherit form the ordinary class?

a. Class template

`

b. Partial specialization
c. Complete specialization
d. All of given option

172. Which of the following is a sequence container?
a. Map
b. Vector
c. Set
d. multiset

173. Each_____of a template class by default becomes function template
a. Date member
b. Member function
c. Type parameter
d. Object

174. When we specialize a function template, it is called _____
a. Function overloading
b. Function overriding
c. Function template overloading
d. Function template overriding

175. Which of the following is NOT an associative container?
a. Set
b. Multiset
c. Map
d. Group

176. A_____is a pointer which is declared outside the _____
a. Container, cursor
b. Cursor, container
c. Container, class
d. This, cursor

177. Which of the following are collectively referred to as the first class containers?
a. Associative containers and containers adapters
b. Sequence containers and associative containers
c. Sequence containers and container adapters
d. None of the given options

178. _____class is a single class that provides functionality to operate on different type of data
a. Friend
b. Template
c. Ordinary
d. None of the give options

179. Which of the following can be passed as type argument to template?
a. Primitive type
b. User defined types

`

c. Both primitive type and user defined types
d. None of the given options

180. Which of the following is known as Dereference operator in C++?
a. +b.
*
c. &
d. ::

181. We can change behavior of template using_____
a. Template parameters
b. Function parameters
c. Class Templates
d. none of the given options

182. Which of the following represents partial specialization?
a. Template < int, char >
b. Template < class T, Type t>
c. Template<class T, Class U, int>
d. Template< typename T, class W>

183. When we declare template function as friend of any class, then which of the followingspecialization(s) of template function also become friend of the class granting friendship?
a. Explicit
b. Implicit
c. Partial
d. All of the given options

184. In statement "template <class T, class U, int I = 5>" then non-type parameters is
a. Class T
b. Class U
c. int I
d. All of the given option

185. Which of the following is the correct syntax for passing two type argument to atemplate?
a. template < type T, type U>
b. template < typename T, typename U>
c. template <type T,U>
d. template Typename < T,U >

186. Which of the following represents complete specialization?a. Template < int, char >
b. Template < class T, float>
c. Template<class T, Class U , int>
d. Template< class , class W>

187. Which of the following is/are the key component of standard template library?
a. Containers
b. Iterators

`

c. Algorithm
d. All of the given option

188. Which of the following statement is true about partial specialization?
a. Function templates cannot have partial specialization
b. Class template cannot have partial specialization
c. Both class templates and function template can have partial specialization.
d. None of the given

189. Which of the following is correct code portion to initialize static data member "value"of int type for class X?
a. Template <class T> X::value =0;
b. Template<class T>int X::value =0;
c. Template<class T> int X < T >::value = 0;
d. Template<class T> X < T >::value =0;

190. When we want to have exactly identical operations on different data type, _____are used
a. Function Template
b. Function Overloading
c. Function Overriding
d. None of the given option

191. In order to define a class template the first line of definition must be:a. Template <typename T>
b. typename < template T>
c. Template class <ClassName>
d. Class <Template T>

192. In case of template specialization, if compiler cannot find required completespecialization then it searches for some _
a. General template
b. Complete template
c. Partial specialization
d. None of the given

193. Which of the following is the correct code to instantiate the object of given templatevector class of int type?

```
Template <class T>
class vector
{
};
```

a. Vector obj <int>;
b. Vector obj <> int;
c. Vector <int>obj;
c. Vector int obj;

194. Which of the following types of container is NOT provided by standard TemplateLibrary?
a. Sequence containers

`

b. Structured containers
c. Associative containers
d. Container adapters

195. A class template _____
a. Facilitates rescue of class
b. Dose not facilitates reuse of class
c. Does not support generic methods
d. Does not support static members

196. Suppose a template class "test" has a static data members, how many copies of thisstatic data member will be created when template class is instantiated by writing following line of C++ code?

```
Test <char>x,y,z
```

a. Zero
b. One Correct
c. Two

197. STL stands for:
a. Stack Template Library
b. Standard Template Library
c. Structured Template Library
d. Secure Template Library

198. Which of the following is not a sequence container?
a. Vector
b. Dequec. Map
d. List

199. Suppose a template class "test" has a static data members, how many copies of thisstatic data member will be created when template class is instantiated by writing following line of C++ code?

```
Test <int>x; Test <char>y;
```

a. Zero
b. Onec. Two
d. Three

200. Suppose we have a template class "Test" which have a static data member "data"which is initialized with 0. What will be output when following code is written inmain() function for given class?

```
Test<int> a, b;a.data = 5;
b.data = 7; cout<<a.data;
```

a. 0
b. 1~~c. 5~~
d. 7 Correct

201. In resolution order, highest priority is given to_____in template specialization.
a. General template
b. Complete specialization
c. Partial specialization
d. None of the given options

`

ALHAMD TECH INSTITUTE
0304-1659294
www.vulmshelp.com

202. Which of the following is a constrained version of some first-class container?

a. Sequence containers
b. Associative containers
c. Container Adapter
d. None of the given options

Question No: 1 (Marks: 1) - Please choose one
Which of the following causes run time binding?
► Declaring object of abstract class
► Declaring pointer of abstract class
► Declaring overridden methods as non-virtual (Page 226)
► None of the given
Question No: 2 (Marks: 1) - Please choose one Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?

► Templates (Page 256)
► Overloading
► Data hiding
► Encapsulation
Question No: 3 (Marks: 1) - Please choose one A copy constructor is invoked when

► a function do not returns by value.
► an argument is passed by value. (Page 78)
► a function returns by reference.
► an argument is passed by reference.
Question No: 4 (Marks: 1) - Please choose one Like template functions, a class template may not handle all the types successfully

. ► True (Page 258)
► False 2
Question No: 6 (Marks: 1) - Please choose one A class template may inherit from another class template
.
► True (Page 288)
► False
Question No: 7 (Marks: 1) - Please choose one By default the vector data items are initialized to ____

► 0 Click here for detail
► 0.0
► 1
► null

`

Question No: 8 (Marks: 1) - Please choose one In Private ---------- only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object

► specialization
► inheritance (Page 216)
► abstraction
► composition Question

No: 9 (Marks: 1) - Please choose one Which of the following is/are advantage[s] of generic programming?

► Reusability
► Writability
► Maintainability
► All of given (Page 256)

Question No: 10 (Marks: 1) - Please choose one Template functions use _____ than ordinary functions

. ► Greater Memory
► Lesser Memory
► Equal Memory
► None of the given options

Question No: 11 (Marks: 1) - Please choose one Non Template Friend functions of a class are friends of _____instance/s of that class.
► All Click here for detail
► One specific
► All instances of one date type
► None of the given options 3
Question No: 12 (Marks: 1) - Please choose one A copy constructor is invoked when
► a function do not returns by value.
► an argument is passed by value. (Page 78)
► a function returns by reference.
► an argument is passed by reference
. Question No: 13 (Marks: 1) - Please choose one A pointer to a base class can point to objects of a derived class.
► True Click here for detail
► False
Question No: 14 (Marks: 1) - Please choose one A template argument is preceded by the keyword _____.
► vector
► class Click here for Detail
► template
► type*

`

Question No: 15 (Marks: 1) - Please choose one Which one of the following terms must relate to polymorphism?
► Static allocation
► Static typing
► Dynamic binding (Page 239)
► Dynamic allocation
Question No: 16 (Marks: 1) - Please choose one Multiple inheritance can be of type
► Public
► Private
► Protected
► All of the given Click here for detail
Question No: 17 (Marks: 1) - Please choose one Assume a class Derv that is privately derived from class Base. An object of class Derv located in main() can access
► public members of Derv. (Object-Oriented Programming in C++)
► protected members of Derv.
► private members of Derv.
► protected members of Base. 4
Question No: 18 (Marks: 1) - Please choose one A copy constructor is invoked when
► a function do not returns by value.
► an argument is passed by value. (Page 78) (rep)
► a function returns by reference.
► an argument is passed by reference.
Question No: 19 (Marks: 1) - Please choose one A function call is resolved at run-time in_____
► non-virtual member function.
► virtual member function. (Page 239)
► Both non-virtual member and virtual member function.
► None of given
Question No: 20 (Marks: 1) - Please choose one Two important STL associative containers are _____ and _____.
► set,map (Object-Oriented Programming in C++)
► sequence,mapping
► setmet,multipule
► sit,mat
Question No: 21 (Marks: 1) - Please choose one An abstract class is useful when,
► We do not derive any class from it.
► There are multiple paths from one derived class to another.
► We do not want to instantiate its object. (Object-Oriented Programming in C++)
► You want to defer the declaration of the class.
Question No: 22 (Marks: 1) - Please choose one Which of the following is/are advantage[s] of generic programming?
► Reusability
► Writability
► Maintainability
► All of given (Page 256) rep
Question No: 23 (Marks: 1) - Please choose one By default the vector data items are initialized to ____

`

► 0 Click here for Detail rep
► 0.0
► 1
► null 5
Question No: 24 (Marks: 1) - Please choose one Suppose you create an uninitialized vector as follows: vector evec; After adding the statment, evec.push_back(21); what will happen?
► The following statement will add an element to the start (the back) of evec and will initialize it with the value 21.
► The following statement will add an element to the center of evec and will reinitialize it with the value 21.
► The following statement will delete an element to the end (the back) of evec and will reinitialize it with the value 21.
► The following statement will add an element to the end (the back) of evec and initialize it with the value 21. Click here for detail
Question No: 25 (Marks: 1) - Please choose one Default constructor is such constructor which either has no ---------or if it has some parameters these have ----- --- values
► Parameter, temporary
► Null, Parameter
► Parameter, default (Page 75)
► non of the given
Question No: 26 (Marks: 1) - Please choose one Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?
► Templates (Page 256) rep
► Overloading
► Data hiding
► Encapsulation
Question No: 27 (Marks: 1) Describe the way to declare a template class as a friend of any class.
Question No: 28 (Marks: 1) Classes like TwoDimensionalShape and ThreeDimensionalShape would normally be concrete, while classes like Sphere and Cube would normally be abstract.
► True Click here for detail
► False 6
Question No: 29 (Marks: 1) In order to define a class template, the first line of definition must be:
► template (Page 257)
► typename
► Template Class
► Class
Question No: 30 (Marks: 1) In case of multiple inheritance a derived class inherits,
► Only the public member functions of its base classes
► Only the public data members of its base classes
► Both public data members and member functions of all its base classes Click here for detail
► Data members and member functions of any two base classes
Question No: 31 (Marks: 1) In Private ---------- only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object
► specialization
► inheritance (Page 216) rep

`

► abstraction
► composition
Question No: 32 (Marks: 1) Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?
► Templates (Page 256) rep
► Overloading
► Data hiding
► Encapsulation
FINALTERM EXAMINATION Spring 2010 CS304- Object Oriented Programming
Question No: 1 (Marks: 1) - Please choose one A template argument is preceded by the keyword _____.
► vector
► class (Object-Oriented Programming in C++)
► template
► type*
Question No: 2 (Marks: 1) - Please choose one Which of the following causes run time binding?
► Declaring object of abstract class
► Declaring pointer of abstract class
► Declaring overridden methods as non-virtual (Page 226)
► None of the given
Question No: 3 (Marks: 1) - Please choose one A function template can not be overloaded by another function template.
► True (Object-Oriented Programming in C++)
► False
Question No: 4 (Marks: 1) - Please choose one Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?
► Templates (Page 256) rep
► Overloading
► Data hiding
► Encapsulation Question No: 5 (Marks: 1) - Please choose one Identify the correct way of declaring an object of user defined template class A for char type members?
► A< char > obj; (Object-Oriented Programming in C++)
► A obj;
► A obj;
► Obj A;
Question No: 6 (Marks: 1) - Please choose one The user must define the operation of the copy constructor.
► True Click here for detail
► False
Question No: 7 (Marks: 1) - Please choose one Template functions use _____ than ordinary functions.
► Greater Memory
► Lesser Memory
► Equal Memory
► None of the given options

`

Question No: 8 (Marks: 1) - Please choose one The find() algorithm
► finds matching sequences of elements in two containers.
► finds a container that matches a specified container.
► takes iterators as its first two arguments. (Object-Oriented Programming in C++)
► takes container elements as its first two arguments.
Question No: 9 (Marks: 1) - Please choose one Compiler performs _____ type checking to diagnose type errors,
► Static (Page 261)
► Dynamic
► Bound
► Unbound
Question No: 10 (Marks: 1) - Please choose one Which of the following is/are advantage[s] of generic programming?
► Reusability
► Writability
► Maintainability
► All of given (Page 256) rep
Question No: 11 (Marks: 1) - Please choose one Vectors contain contiguous elements stored as a[an] ___.
► variable
► array (Page 306)
► function
► datatype
Question No: 12 (Marks: 1) - Please choose one Suppose you create an uninitialized vector as follows: vector evec; After adding the statment, evec.push_back(21); what will happen?
► The following statement will add an element to the start (the back) of evec and will initialize it with the value 21.
► The following statement will add an element to the center of evec and will reinitialize it with the value 21.
► The following statement will delete an element to the end (the back) of evec and will reinitialize it with the value 21.
► The following statement will add an element to the end (the back) of evec and initialize it with the value 21. Click here for detail
Question No: 13 (Marks: 1) - Please choose one In a de-queue, (chose the best option)
► data can be quickly inserted or deleted at any arbitrary location.
► data can be inserted or deleted at any arbitrary location, but the process is relatively slow. (Object-Oriented Programming in C++)
► data can not be quickly inserted or deleted at either end.
► data can be inserted or deleted at either end, but the process is relatively slow.
Question No: 14 (Marks: 1) - Please choose one Algorithms can only be implemented using STL containers.
► True
► False (Object-Oriented Programming in C++)
Question No: 15 (Marks: 1) - Please choose one What is a class?
► A class is a section of computer memory containing objects.
► A class is a section of the hard disk reserved for object oriented programs
► A class is the part of an object that contains the variables.

`

► A class is a description of a kind of object. Click here for detail
Question No: 16 (Marks: 1) - Please choose one Inheritance is a way to
► organize data.
► pass arguments to objects of classes.
► add features to existing classes without rewriting them. (Page 27)
► improve data-hiding and encapsulation.
Question No: 17 (Marks: 1) - Please choose one We can use "this" pointer in the constructor in the body and even in the initialization list of any class if we are careful,
► True Click here for detail
► False
Question No: 18 (Marks: 1) - Please choose one _____ and _____ methods may not be declared abstract.
► private,static Click here for detail
► private,public
► static,public
► none of given
Question No: 19 (Marks: 1) - Please choose one Default constructor is such constructor which either has no ---------or if it has some parameters these have -------- values
► Parameter, temporary
► Null, Parameter
► Parameter, default (Page 75) rep
► non of the given
Question No: 20 (Marks: 1) - Please choose one Public methods of base class can --------- be accessed in its derived class
► directly (Page 179)
► inderectly
► simultaniously
► non of the given
Question No: 21 (Marks: 1) - Please choose one The type that is used to declare a reference or pointer is called its ---------
► default type
► static type (Page 185)
► abstract type
► reference type
Question No: 22 (Marks: 1) - Please choose one ---------- members are somewhere between public and private members. They are used in inheritance
► protected (Page 187)
► public
► private
► global
Question No: 23 (Marks: 1) - Please choose one Which of these are examples of error handling techniques?
► Abnormal Termination
► Graceful Termination
► Return the illegal
► all of the given (Page 329)

`

Question No: 24 (Marks: 1) - Please choose one ---------- follow try block to catch the object thrown
► catch block (Page 333)
► throw block
► main block
► non of the given
Question No: 25 (Marks: 1) - Please choose one Graphical representation of the classes and objects is called object model it shows -------
► Class Name only
► Class Name and attributes
► Relationships of the objects and classes
► all of the given Click here for detail
Question No: 26 (Marks: 1) - Please choose one Destructor can be overloaded
► True
► False (Page 92)
FINALTERM EXAMINATION Spring 2010 CS304- Object Oriented Programming (Session - 3) 12
Question No: 1 (Marks: 1) - Please choose one Which one of the following terms must relate to polymorphism?
► Static allocation
► Static typing
► Dynamic binding (Page 239)
► Dynamic allocation
Question No: 2 (Marks: 1) - Please choose one Which of the following causes run time binding?
► Declaring object of abstract class
► Declaring pointer of abstract class
► Declaring overridden methods as non-virtual (Page 226)
► None of the given
Question No: 3 (Marks: 1) - Please choose one Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?
► Templates (Page 256) rep
► Overloading
► Data hiding
► Encapsulation
Question No: 4 (Marks: 1) - Please choose one Which of the following is the best approach to implement generic algorithms with minimum number of coding lines?
► Templates (Page 256)
► Overloading
► Overriding
► Friend function/class
Question No: 5 (Marks: 1) - Please choose one Like template functions, a class template may not handle all the types successfully.
► True (Page 258) rep
► False
Question No: 6 (Marks: 1) - Please choose one A class template may inherit from another class template.
► True (Page 288) rep

`

► False 13
Question No: 7 (Marks: 1) - Please choose one Assume a class Derv that is privately derived from class Base. An object of class Derv located in main() can access
► public members of Derv. (Object-Oriented Programming in C++) rep
► protected members of Derv.
► private members of Derv.
► protected members of Base
. Question No: 8 (Marks: 1) - Please choose one A copy constructor is invoked when
► a function do not returns by value.
► an argument is passed by value. (Page 78) rep
► a function returns by reference.
► an argument is passed by reference.
Question No: 9 (Marks: 1) - Please choose one Each try block can have _____ no. of catch blocks.
► 1
► 2
► 3
► As many as necessary. Click here for detail A single try block can have multiple catch blocks but only one finally block.
Question No: 10 (Marks: 1) - Please choose one class DocElement { public: virtual void Print() { cout << "Generic element"; } }; class Heading : public DocElement { public: void Print() { cout << "Heading element"; } }; class Paragraph : public DocElement { public: void Print() { cout << "Paragraph element"; } }; void main() { DocElement * p = new Paragraph(); 14 p->Print(); } When you run this program, it will print out a single line to the console output. What will be in that line? Select one correct answer from the following list:
► Generic element
► Heading element
► Paragraph element
► Nothing will be printed.
Question No: 11 (Marks: 1) - Please choose one Suppose we have two derived classes from a single class, can we write a method with same name in both these derived classes? Choose the best option.
► No
► Only if the two classes have the same name (Page 204)
► Only if the main program does not declare both kinds
► Yes
Question No: 12 (Marks: 1) - Please choose one When a virtual function is called by referencing a specific object by name and using the dot member selection operator (e.g., squareObject.draw()), the reference is resolved at compile time.
► True
► False (Object-Oriented Programming in C++) (the function invocation is resolved at compile time)
Question No: 13 (Marks: 1) - Please choose one Considering the resolution order in which Considering the resolution order in which compiler search for functions in a program; the first priority is given to; the first priority is given to,
► general template
► partial specialization
► complete specialization

`

► ordinary function (Page 287)
Question No: 14 (Marks: 1) - Please choose one Vectors contain contiguous elements stored as a[an] ___.
► variable
► array (Page 306) rep
► function
► datatype
Question No: 15 (Marks: 1) - Please choose one By default the vector data items are initialized to ____
► 0 Click here for detail rep
► 0.0
► 1
► null
Question No: 16 (Marks: 1) - Please choose one One purpose of an iterator in the STL is to connect algorithms and containers.
► True (Object-Oriented Programming in C++)
► False
Question No: 17 (Marks: 1) - Please choose one Algorithms can only be implemented using STL containers.
► True
► False (Object-Oriented Programming in C++) rep
Question No: 18 (Marks: 1) - Please choose one In _____, a base class can be replaced by its derived class,
► Sub-typing (Page 31)
► Super-typing
► Multiple-typing
► Restricted-typing
Question No: 19 (Marks: 1) - Please choose one this pointer does not point to current object of any class,
► True
► False Click here for detail The this pointer is a hidden pointer inside every class member function that points to the class object the member function is working with.
Question No: 20 (Marks: 1) - Please choose one Which of the following operator(s) take(s) one or no argument if overloaded?
► ++ (Page 162)
► -
► +
► All of the given
16 Question No: 21 (Marks: 1) - Please choose one Which of the following operators can not be overloaded?
► Scope resolution operator (::) (Page 141)
► Insertion operator (<<)
► Extraction operator (>>)
► The relation operator (>)
Question No: 22 (Marks: 1) - Please choose one The type that is used to declare a reference or pointer is called its ---------
► default type

`

► static type (Page 185) rep
► abstract type
► reference type
Question No: 23 (Marks: 1) - Please choose one ---------- members are somewhere between public and private members. They are used in inheritance
► protected (Page 187) rep
► public
► private
► global
Question No: 24 (Marks: 1) - Please choose one Which of these are examples of error handling techniques ?
► Abnormal Termination
► Graceful Termination
► Return the illegal
► all of the given (Page 329)
Question No: 25 (Marks: 1) - Please choose one _____ “is a” relationship

► Inheritance (Page 25)
► Polymarphism
► abstraction ►
encapsulation
Question No: 26 (Marks: 1) - Please choose one Graphical representation of the classes and objects is called object model it shows -------
► Class Name only
► Class Name and attributes
► Relationships of the objects and classes
► all of the given Click here for detail rep
17 FINALTERM EXAMINATION Spring 2010 CS304- Object Oriented Programming (Session - 4)
Question No: 1 (Marks: 1) - Please choose one Classes like TwoDimensionalShape and ThreeDimensionalShape would normally be concrete, while classes like Sphere and Cube would normally be abstract.
► True Click here for detail rep
► False
Question No: 2 (Marks: 1) - Please choose one Virtual functions allow you to
► create an array of type pointer-to-base class that can hold pointers to derived classes.
► create functions that can never be accessed.
► group objects of different classes so they can all be accessed by the same function code.
► use the same function call to execute member functions of objects from different classes (Object-Oriented Programming in C++)
Question No: 3 (Marks: 1) - Please choose one .
► True Click here for detail rep
► False
Question No: 4 (Marks: 1) - Please choose one A copy constructor is invoked when
► a function do not returns by value.
► an argument is passed by value. (Page 78) rep
► a function returns by reference.
► an argument is passed by reference.

`

Question No: 5 (Marks: 1) - Please choose one Each try block can have _____ no. of catch blocks.

► 1
► 2
► 3
► As many as necessary. Click here for detail rep

18 Question No: 6 (Marks: 1) - Please choose one Non Template Friend functions of a class are friends of _____instance/s of

► All Click here for detail rep
► One specific
► All instances of one date type
► None of the given options

Question No: 7 (Marks: 1) - Please choose one Template functions use _____ than ordinary functions.

► Greater Memory
► Lesser Memory
► Equal Memory
► None of the given options

Question No: 8 (Marks: 1) - Please choose one The find() algorithm

► finds matching sequences of elements in two containers.
► finds a container that matches a specified container.
► takes iterators as its first two arguments. (Object-Oriented Programming in C++)
► takes container elements as its first two arguments.

Question No: 9 (Marks: 1) - Please choose one The copy() algorithm returns an iterator to

► the last element copied from.
► the last element copied to.
► the element one past the last element copied from.
► the element one past the last element copied to. (Object-Oriented Programming in C++)

Question No: 10 (Marks: 1) - Please choose one If you define a vector v with the default constructor, and define another vector w with a one-argument constructor to a size of 11, and insert 3 elements into each of these vectors with push_back(), then the size() member function will return _____ for v and _____ for w.

► 11 for v and 3 for w.
► 0 for v and 0 for w.
► 0 for v and 3 for w.
► 3 for v and 11 for w. (Object-Oriented Programming in C++)

19 Question No: 11 (Marks: 1) - Please choose one Which is not the Advantage of inheritance?

► providing class growth through natural selection. (Object-Oriented Programming in C++)
► facilitating class libraries.
► avoiding the rewriting of code.
► providing a useful conceptual framework.

Question No: 12 (Marks: 1) - Please choose one class DocElement { public: virtual void Print() { cout << "Generic element"; } }; class Heading : public DocElement { public: void Print() { cout << "Heading element"; } }; class Paragraph : public DocElement { public: void Print() { cout << "Paragraph element"; } }; void main() { DocElement * p = new Paragraph(); p->Print(); } When you run this program, it will print out a single line to the console output. What will be in that line? Select one correct answer from the following list:

`

► Generic element
► Heading element
► Paragraph element
► Nothing will be printed. 20
Question No: 13 (Marks: 1) - Please choose one Which type of inheritance is being represented by the following statement, class X : public A, public B { ... ... };
► Single inheritance
► Multiple inheritance (Page 41)
► Double inheritance
► None of the given options
Question No: 14 (Marks: 1) - Please choose one When we write a class template the first line must be:
► template < class class_name>
► template < class data_type>
► template < class T > (Page 257) Here T can be replaced with any name but it is preferable.
► class class-name() class template
Question No: 15 (Marks: 1) - Please choose one Function templates should be used where code and behavior must be identical.
► True (Page 262)
► False
Question No: 16 (Marks: 1) - Please choose one Which of the following is/are advantage[s] of generic programming?
► Reusability
► Writability
► Maintainability
► All of given (Page 256) rep
Question No: 17 (Marks: 1) - Please choose one The specialization pattern after the name says that this specialization is to be used for every,
► data type
► meta type
► virtual type
► pointer type (Page 286)
Question No: 18 (Marks: 1) - Please choose one A range is often supplied to an algorithm by two _____ values.
► italic
► iteration (Object-Oriented Programming in C++)
► iterator
► None of given 21
Question No: 19 (Marks: 1) - Please choose one Which of the following is an integral part of an object?
► State
► Behavior
► Unique identity
► All of the given (Page 12)
Question No: 20 (Marks: 1) - Please choose one Consider the following statement Cupboard has books What is the relationship between Cupboard and books?
► Composition

`

► Aggregation
► Inheritance
► None of the given options
Question No: 21 (Marks: 1) - Please choose one Which sentence clearly defines an object?
► one instance of a class. (Page 23)
► another word for a class.
► a class with static methods.
► a method that accesses class attributes.
Question No: 22 (Marks: 1) - Please choose one _____, which means if A declares B as its friend it does NOT mean that A can access private data of B. It only means that B can access all data of A.
► Friendship is one way only Click here for detail
► Friendship is two way only
► NO Friendship between classes
► Any kind of friendship
Question No: 23 (Marks: 1) - Please choose one The statement objA=objB; will cause a compiler error if the objects are of different classes.
► True
► False (Object-Oriented Programming in C++) 22
Question No: 24 (Marks: 1) - Please choose one Consider the call given below of an overloaded operator "+", Rational_number_1 + Rational_number_2 Where Rational_number_1 and Rational_number_2 are the two objects of Rational_number class (a user defined class). Identify which of the above two objects will be passed as an argument to the overloaded operator function?
► Rational_number_1
► Rational_number_2
► Both Rational_number_1 & Rational_number_2
► any of the two objects, randomly
Question No: 25 (Marks: 1) - Please choose one If a class D has been derived using protected inheritance from class B (If B is a protected base and D is derived class) then public and protected members of B -------- accessed by member functions and friends of class D and classes derived from D
► can be Click here for detail
► cannot be
► does restirct to be
► not given
Question No: 26 (Marks: 1) - Please choose one In Private ---------- only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object
► specialization
► inheritance (Page 216) rep
► abstraction
► composition
FINAL TERM EXAMINATION SPRING 2010
Question No: 1 (Marks: 1) - Please choose one Classes like TwoDimensionalShape and ThreeDimensionalShape would normally be concrete, while classes like Sphere and Cube would normally be abstract.
► True Click here for detail rep

`

► False 23
Question No: 2 (Marks: 1) - Please choose one Each try block can have _____ no. of catch blocks.
► 1
► 2
► 3
► As many as necessary. Click here detail
Question No: 3 (Marks: 1) - Please choose one Function templates should be used where code and behavior must be identical.
► True (Page 262)
► False
Question No: 4 (Marks: 1) - Please choose one Consider the following statement Cupboard has books What is the relationship between Cupboard and books?
► Composition
► Aggregation
► Inheritance
► None of the given options
Question No: 5 (Marks: 1) - Please choose one Identify the correct way of declaring an object of user defined template class A for char type members?
► A< char > obj; (Object-Oriented Programming in C++)
► A obj;
► A obj;
► Obj A;
Question No: 6 (Marks: 1) - Please choose one The user must define the operation of the copy constructor.
► True Click here for detail
► False
Question No: 7 (Marks: 1) - Please choose one Default constructor is such constructor which either has no ---------or if it has some parameters these have -------- values
► Parameter, temporary
► Null, Parameter
► Parameter, default (Page 75) rep
► non of the given
Question No: 8 (Marks: 1) - Please choose one The type that is used to declare a reference or pointer is called its ---------
► default type
► static type (Page 185)
► abstract type
► reference type
24 Question No: 9 (Marks: 1) - Please choose one How the information hidden within an object can be accessed?
► Through its interface
► Through its private data members
► Through its private member functions
► Through both public and private members
Question No: 10 (Marks: 1) - Please choose one The sub-object’s life is not dependant on the life of master class in _____.

`

► Separation
► Composition
► Aggregation (Page 134)
► None of the given
Question No: 11 (Marks: 1) - Please choose one Encapsulation means Select correct option:
►Extending the behaviour of class in another class
►Data and behaviour are tightly coupled within an entity (Page 16)
►One entity takes all the attributes and operations of the other
►Taking out the common features and put those in a separate class
Question No: 12 (Marks: 1) - Please choose one Algorithms can only be implemented using STL containers.
► True
► False (Object-Oriented Programming in C++) rep
Question No: 13 (Marks: 1) - Please choose one When we write a class template the first line must be:
► template < class class_name>
► template < class data_type>
► template < class T > (Page 257) rep Here T can be replaced with any name but it is preferable.
► class class-name() class template
Question No: 14 (Marks: 1) - Please choose one An STL container can not be used to,
► hold objects of class employee.
► store elements in a way that makes them quickly accessible.
► compile c++ programs. (Object-Oriented Programming in C++)
► organize the way objects are stored in memory 25
Question No: 15 (Marks: 1) - Please choose one _____, which means if A declares B as its friend it does NOT mean that A can access private data of B. It only means that B can access all data of A.
► Friendship is one way only Click here for detail
► Friendship is two way only
► NO Friendship between classes
► Any kind of friendship
Question No: 16 (Marks: 1) - Please choose one Which of the following may not be an integral part of an object?
► State
► Behavior
► Protected data members (Page 12)
► All of given
Question No: 17 (Marks: 1) - Please choose one Public methods of base class can --------- be accessed in its derived class
► directly (Page 179) rep
► inderectly
► simultaneously
► non of the given
Question No: 18 (Marks: 1) - Please choose one If a class D has been derived using protected inheritance from class B (If B is a protected base and D is derived class) then public and protected members of B -------- accessed by member functions and friends of class D and classes derived from D

`

► can be Click here for detail
► cannot be
► does restirct to be
► not given
Question No: 19 (Marks: 1) - Please choose one What is true about function templates?
► The compiler generates only one copy of the function template
► The compiler generates a copy of function respective to each type of data (Page 256)
► The compiler can only generate copy for the int type data
► None of the given.
Question No: 20 (Marks: 1) - Please choose one Which of the following is an integral part of an object?
► State
► Behavior
► Unique identity
► All of the given (Page 12) rep 26
Question No: 21 (Marks: 1) - Please choose one When the base class and the derived class have a member function with the same name, you must be more specific which function you want to call (using _____).
► scope resolution operator
► dot operator
► null operator
► Operator overloading Click here for detail
FINALTERM EXAMINATION Fall 2009 CS304- Object Oriented Programming (Session - 4)
Question No: 1 (Marks: 1) - Please choose one A template provides a convenient way to make a family of
► variables and data members
► functions and classes (Object-Oriented Programming in C++)
► classes and exceptions
► programs and algorithms
Question No: 2 (Marks: 1) - Please choose one Which one of the following terms must relate to polymorphism?
► Static allocation
► Static typing
► Dynamic binding (Page 239) rep
► Dynamic allocation
Question No: 3 (Marks: 1) - Please choose one What is true about function templates?
► The compiler generates only one copy of the function template
► The compiler generates a copy of function respective to each type of data (Page 256) rep
► The compiler can only generate copy for the int type data
► None of the given.
Question No: 4 (Marks: 1) - Please choose one Which of the following is the best approach if it is required to have more than one functions having exactly same functionality and implemented on different data types?
► Templates (Page 256) rep
► Overloading
► Data hiding
► Encapsulation 27

`

Question No: 5 (Marks: 1) - Please choose one template <> class Vector { } This is an example of partial specialization.
► True (Page 287)
► False
Question No: 6 (Marks: 1) - Please choose one Classes like TwoDimensionalShape and ThreeDimensionalShape would normally be concrete, while classes like Sphere and Cube would normally be abstract.
► True Click here for detail rep
► False
Question No: 7 (Marks: 1) - Please choose one A non-virtual member function is defined in a base class and overridden in a derived class; if that function is called through a base-class pointer to a derived class object, the derived-class version is used.
► True
► False
Question No: 8 (Marks: 1) - Please choose one Assume a class Derv that is privately derived from class Base. An object of class Derv located in main() can access
► public members of Derv. (Object-Oriented Programming in C++) rep
► protected members of Derv.
► private members of Derv.
► protected members of Base.
Question No: 9 (Marks: 1) - Please choose one In order to define a class template, the first line of definition must be:
► template (Page 257) rep
► typename
► Template Class
► Class 28
Question No: 10 (Marks: 1) - Please choose one If there is a pointer p to objects of a base class, and it contains the address of an object of a derived class, and both classes contain a nonvirtual member function, ding(), then the statement p->ding(); will cause the version of ding() in the _____ class to be executed.
► Base (Object-Oriented Programming in C++)
► Derived
► Abstract
► virtual
Question No: 11 (Marks: 1) - Please choose one When the base class and the derived class have a member function with the same name, you must be more specific which function you want to call (using _____).
► scope resolution operator
► dot operator
► null operator
► Operator overloading Click here for detail
Question No: 12 (Marks: 1) - Please choose one Non Template Friend functions of a class are friends of _____instance/s of that class.
► All Click here for detail
► One specific
► All instances of one date type
► None of the given options

`

Question No: 13 (Marks: 1) - Please choose one The find() algorithm
► finds matching sequences of elements in two containers.
► finds a container that matches a specified container.
► takes iterators as its first two arguments. (Object-Oriented Programming in C++) rep
► takes container elements as its first two arguments.
Question No: 14 (Marks: 1) - Please choose one If you define a vector v with the default constructor, and define another vector w with a one-argument constructor to a size of 11, and insert 3 elements into each of these vectors with push_back(), then the size() member function will return _____ for v and _____ for w.
► 11 for v and 3 for w.
► 0 for v and 0 for w.
► 0 for v and 3 for w.
► 3 for v and 11 for w. (Object-Oriented Programming in C++) rep 29
Question No: 15 (Marks: 1) - Please choose one Which of the following may not be an integral part of an object?
► State
► Behavior
► Protected data members (Page 12)
► All of given
Question No: 16 (Marks: 1) - Please choose one Which is not the Advantage of inheritance?
► providing class growth through natural selection. (Object-Oriented Programming in C++) rep
► facilitating class libraries.
► avoiding the rewriting of code
► providing a useful conceptual framework.
Question No: 17 (Marks: 1) - Please choose one 1class DocElement { public: virtual void Print() { cout << "Generic element"; } }; 2class Heading : public DocElement { public: void Print() { cout << "Heading element"; } }; 3class Paragraph : public DocElement { public: void Print() { cout << "Paragraph element"; } }; void main() { DocElement * p = new Paragraph(); p->Print(); } When you run this program, it will print out a single line to the console output. 30 What will be in that line? Select one correct answer from the following list:
► Generic element
► Heading element
► Paragraph element
► Nothing will be printed.
Question No: 18 (Marks: 1) - Please choose one When a virtual function is called by referencing a specific object by name and using the dot member selection operator (e.g., squareObject.draw()), the reference is resolved at compile time
. ► True
► False (Object-Oriented Programming in C++) (the function invocation is resolved at compile time)
Question No: 19 (Marks: 1) - Please choose one In case of multiple inheritance a derived class inherits,
► Only the public member functions of its base classes
► Only the public data members of its base classes
► Both public data members and member functions of all its base classes Click here for detail
► Data members and member functions of any two base classes
Question No: 20 (Marks: 1) - Please choose one When we write a class template the first line must be:

`

► template < class class_name>
► template < class data_type>
► template < class T > (Page 257) rep Here T can be replaced with any name but it is preferable.
► class class-name() class template
Question No: 21 (Marks: 1) - Please choose one Which of the following is incorrect line regarding function template?
► template
► template
► Class (Page 257)
► template < class T, class U> 31
Question No: 22 (Marks: 1) - Please choose one An STL container can not be used to,
► hold objects of class employee.
► store elements in a way that makes them quickly accessible.
► compile c++ programs. (Object-Oriented Programming in C++) rep
► organize the way objects are stored in memory
Question No: 23 (Marks: 1) - Please choose one Algorithms can only be implemented using STL containers.
► True
► False (Object-Oriented Programming in C++) rep
Question No: 24 (Marks: 1) - Please choose one Consider a class named Vehicle, which of the following can be the instance of class Vehicle? 1. Car 2. Computer 3. Desk 4. Ahmed 5. Bicycle 6. Truck
► 1, 4, 5
► 2, 5, 6
► 1, 2, 3, 6
► 1, 5, 6 (correct)
Question No: 25 (Marks: 1) - Please choose one Consider the code below, class Fred { public: Fred(); ... }; int main() { Fred a[10]; Fred* p = new Fred[10]; ... } Select the best option,
► Fred a[10]; calls the default constructor 09 times Fred* p = new Fred[10]; calls the default constructor 10 times
► Produce an error 32
► Fred a[10]; calls the default constructor 11 times Fred* p = new Fred[10]; calls the default constructor 11 times
► Fred a[10]; calls the default constructor 10 times Fred* p = new Fred[10]; calls the default constructor 10 times Click here for detail
Question No: 26 (Marks: 1) - Please choose one When a variable is define as static in a class then all object of this class,
► Have different copies of this variable
► Have same copy of this variable (Page 110)
► Can not access this variable
► None of given
Question No: 27 (Marks: 1) - Please choose one The life of sub object is dependant on the life of master class in _____.
► Separation
► Composition (Page 53)
► Aggregation
► None of the given

`

Question No: 28 (Marks: 1) - Please choose one _____, which means if A declares B as its friend it does NOT mean that A can access private data of B. It only means that B can access all data of A.

► Friendship is one way only Click here for detail
► Friendship is two way only
► NO Friendship between classes
► Any kind of friendship

Question No: 29 (Marks: 1) - Please choose one Which of the following operators always takes no argument if overloaded?

► /
► -
► +
► ++ (Page 162)

Question No: 30 (Marks: 1) - Please choose one In Private ---------- only member functions and friend classes or functions of a derived class can convert pointer or reference of derived object to that of parent object

► specialization
► inheritance (Page 216) rep
► abstraction
► composition

33 FINALTERM EXAMINATION Fall 2009 CS304- Object Oriented Programming (Session - 1)

Question No: 1 (Marks: 1) - Please choose one Which one of the following terms must relate to polymorphism?

► Static allocation
► Static typing
► Dynamic binding (Page 239) rep
► Dynamic allocation

Question No: 2 (Marks: 1) - Please choose one Multiple inheritance can be of type

► Public
► Private
► Protected
► All of the given Click here for detail

Question No: 3 (Marks: 1) - Please choose one When a subclass specifies an alternative definition for an attribute or method of its superclass, it is _____ the definition in the superclass.

► overload
► overriding (Page 34)
► copy riding
► none of given

Question No: 4 (Marks: 1) - Please choose one Like template functions, a class template may not handle all the types successfully.

► True (Page 258) rep
► False

Question No: 5 (Marks: 1) - Please choose one It is sometimes useful to specify a class from which no objects will ever be created.

► True (Object-Oriented Programming in C++)
► False

`

Question No: 6 (Marks: 1) - Please choose one Assume a class Derv that is privately derived from class Base. An object of class Derv located in main() can access
► public members of Derv. (Object-Oriented Programming in C++) rep
► protected members of Derv.
► private members of Derv.
► protected members of Base. 34
Question No: 7 (Marks: 1) - Please choose one A pointer to a base class can point to objects of a derived class. \
► True Click here for detail
► False
Question No: 8 (Marks: 1) - Please choose one A copy constructor is invoked when
► a function do not returns by value.
► an argument is passed by value. (Page 78) rep
► a function returns by reference.
► an argument is passed by reference.
Question No: 9 (Marks: 1) - Please choose one A function call is resolved at run-time in_____
► non-virtual member function.
► virtual member function. (Page 239) rep
► Both non-virtual member and virtual member function.
► None of given
Question No: 10 (Marks: 1) - Please choose one When the base class and the derived class have a member function with the same name, you must be more specific which function you want to call (using _____).
► scope resolution operator
► dot operator
► null operator
► Operator overloading Click here for detail
Question No: 11 (Marks: 1) - Please choose one Each try block can have _____ no. of catch blocks.
► 1
► 2
► 3
► As many as necessary. Click here for detail rep
Question No: 12 (Marks: 1) - Please choose one Two important STL associative containers are _____ and _____.
► set,map (Object-Oriented Programming in C++) rep
► sequence,mapping
► setmet,multipule
► sit,mat 35
Question No: 13 (Marks: 1) - Please choose one The mechanism of selecting function at run time according to the nature of calling object is called,
► late binding
► static binding
► virtual binding
► None of the given options (Page 227) Dynamic binding means that target function for a call is selected at run time

`

Question No: 14 (Marks: 1) - Please choose one An abstract class is useful when
► We do not derive any class from it.
► There are multiple paths from one derived class to another.
► We do not want to instantiate its object. (Object-Oriented Programming in C++) rep
► You want to defer the declaration of the class.
Question No: 15 (Marks: 1) - Please choose one Which of the following is incorrect line regarding function template?
► template
► template
► Class (Page 257) rep
► template < class T, class U>
Question No: 16 (Marks: 1) - Please choose one Which of the following is/are advantage[s] of generic programming?
► Reusability
► Writability
► Maintainability
► All of given (Page 256) rep
Question No: 17 (Marks: 1) - Please choose one By default the vector data items are initialized to ____
► 0 Click here for detail rep
► 0.0
► 1
► null
Question No: 18 (Marks: 1) - Please choose one Which one of the following functions returns the total number of elements in a vector.
► length();
► size(); (Page 318)
► ele();
► veclen();
36 Question No: 19 (Marks: 1) - Please choose one Suppose you create an uninitialized vector as follows: vector evec; After adding the statment, evec.push_back(21); what will happen?
► The following statement will add an element to the start (the back) of evec and will initialize it with the value 21.
► The following statement will add an element to the center of evec and will reinitialize it with the value 21.
► The following statement will delete an element to the end (the back) of evec and will reinitialize it with the value 21.
► The following statement will add an element to the end (the back) of evec and initialize it with the value 21. Click here for detail
Question No: 20 (Marks: 1) - Please choose one An STL container can not be used to,
► hold objects of class employee.
► store elements in a way that makes them quickly accessible.
► compile c++ programs. (Object-Oriented Programming in C++) rep
► organize the way objects are stored in memory
Question No: 21 (Marks: 1) - Please choose one Algorithms can only be implemented using STL containers.
► True

`

► False (Object-Oriented Programming in C++) rep
Question No: 22 (Marks: 1) - Please choose one The main function of scope resolution operator (::) is,
► To define an object
► To define a data member
► To link the definition of an identifier to its declaration Click here for detail
► To make a class private
Question No: 23 (Marks: 1) - Please choose one When is a constructor called?
► Each time the constructor identifier is used in a program statement
► During the instantiation of a new object (Object-Oriented Programming in C++)
► During the construction of a new class
► At the beginning of any program execution 37
Question No: 24 (Marks: 1) - Please choose one Consider the code below, class Fred { public: Fred(); ... }; int main() { Fred a[10]; Fred* p = new Fred[10]; ... } Select the best option,
► Fred a[10]; calls the default constructor 09 times Fred* p = new Fred[10];
► Produce an error
► Fred a[10]; calls the default constructor 11 times Fred* p = new Fred[10]; calls the default constructor 11 times
► Fred a[10]; calls the default constructor 10 times Fred* p = new Fred[10]; calls the default constructor 10 times Click here for detail
Question No: 25 (Marks: 1) - Please choose one Associativity can be changed in operator overloading.
► True
► False (Page 141)
Question No: 26 (Marks: 1) - Please choose one A normal C++ operator that acts in special ways on newly defined data types is said to be
► glorified.
► encapsulated.
► classified.
► overloaded. (Object-Oriented Programming in C++)
Question No: 27 (Marks: 1) - Please choose one Which operator can not be overloaded?
► The relation operator (>=)
► Assignment operator (=)
► Script operator ([])
► Conditional operator (? :) (Page 141) 38
Question No: 28 (Marks: 1) - Please choose one Suppose obj1 and obj2 are two objects of a user defined class A. An + operator is overloaded to add obj1 and obj2 using the function call obj1+obj2. Identify the correct function prototype against the given call?
► A operator + (A &obj);
► int + operator(); (Page 143)
► int operator (plus) ();
► A operator(A &obj3);
Question No: 29 (Marks: 1) - Please choose one Default constructor is such constructor which either has no ---------or if it has some parameters these have ----- --- values
► Parameter, temporary
► Null, Parameter
► Parameter, default (Page 75) rep

`

► non of the given

Question No: 30 (Marks: 1) - Please choose one Public methods of base class can --------- be accessed in its derived class

► directly (Page 179) rep

► inderectly

► simultaneously

► non of the given

0304-1659294
www.vulmshelp.com


AL-JUNAID TECH INSTITUTE
AJ
0304-1659294
www.vulmshelp.com


AL-JUNAID TECH INSTITUTE
0304-1659294
www.vulmshelp.com

CS304-Object Oriented Programming-Finalterm-Solved-MCQs

Shared & Solved by Umair Saulat

A template provides a convenient way to make a family of.
variables and data members
functions and classes
classes and exceptions
programs and algorithms

A class template may inherit from another class template.
True
False

Target of a _____ function call is determined at run time.
instance
virtual
operator
none of given

A class hierarchy .
shows the same relationships as an organization chart.
describes "has a" relationships.
describes "is a kind of" relationships.
shows the same relationships as a family tree.

Sender of the message does not need to know the exact class of receiver in_____.
Abstraction
Polymorphism
Inheritance
none of the given

A function call is resolved at run-time in_____ .
non-virtual member function
virtual member function
Both non-virtual member and virtual member function.
None of given

Adding a derived class to a base class requires fundamental changes to the base class.
True
False

User can make virtual table explicitly.
True
False

Binding means that target function for a call is selected at compile time.
Static
Dynamic
Automatic
None of given

Target of a _____ function call is determined at run time.
instance
virtual
operator
none of given

Which line will produce error. Class phone: private Transmit, private Receiver { } 1. int main() 2. { 3. phone obj; 4. Tranmit* obj1 = &obj; 5. Received obj2 = &obj; 6. }.
3rd line will produce error
4th line will produce error
3rd and 4th line will produce error.
5th line will produce error

Function overriding is done in context of,
Single class
Single derived class
Single base class
Derived and base classes

Consider the code below, class class1{ public: void func1(); }; class class2 : protected class1 { }; Function func1 of class1 is ____ in class2,
public
protected
private
none of the given options

The following statements: 1) int iArray[5]; 2) int *pArr = iArray;
These statements will compile successfully
Error in first statement
Error in second statement
None of given options

Methodologies to the development of reusable software relate to_____.
Structure programming
procedural programming
generic programming
None of the given

Function template must have a parameter.
True
False

The default inheritance mode is,
Public inheritance
Protected inheritance
Private inheritance
None of these options

Two functions with same names, parameters and return type can exist in,
Function overloading
Function overriding
Operator overloading
None of these options

Consider the code below, class c1{ }; class c2 : public c1 { }; class c3 : public c2 { }; Then c2 is,
Direct base class of c3
Direct child class of c3
Direct base class of c1
None of these

Virtual functions allow you to
create an array of type pointer-to-base class that can hold pointers to derived classes.
create functions that can never be accessed.
group objects of different classes so they can all be accessed by the same function code.
use the same function call to execute member functions of objects from different classes.

User can make virtual table explicitly.
True
False

In order to define a class template the first line of definition must be:
template <typename T>
typename <template T>
Template Class <ClassName>
Class <Template T>

Consider the following statements: 1) int iArray[5]; 2) int *pArr = iArray;
These statements will compile successfully
Error in first statement
Error in second statement
None of given options

In c++ dynamic binding and polymorphism will be achieved when member function will be __.
private
public

virtual
inline

In type in depended function template should be use where code and behavior must be identical.
True
False

Consider the code below, class class1{ protected: int i; }; class class2 : private class1 { }; Then int member i of class1 is ____ in class2,
public
protected
private
none of the given options

In specialization we can,
Replace child class with its base class
Replace base class with its child class (Not Sure)
Replace both child and base classes interchangeably
None of the given options

Consider the code below, class class1{ public: void func1(); }; class class2 : public class1 { }; Function func1 of class1 is ____ in class2,
public
protected
private
none of the given options

It is illegal to make objects of one class members of another class.
True
False

An abstract class is useful when
no classes should be derived from it.
there are multiple paths from one derived class to another.
no objects should be instantiated from its.
you want to defer the declaration of the class.

In resolution order compiler search firstly _____.
Generic Template
Partial Specialization
Complete Specialization
Ordinary function

template<> class Vector{ void p; //.... void*& operator[] ((int i); };**
This specialization can then be used as the common implementation for all Vectors of pointers.
This specialization can then be used as the all type implementation for one type classes.
This specialization can then be used double type pointers.
This specialization should be used for Vectors of all type int types.

In private inheritance derived class pointer can be assigned to base class pointer in.
Main function
In derived class member and friend functions
In base class member and friend functions
None of the given options

Which statement will be true for concrete class?
it implements an virtual concept.
it can be instantiated
it cannot be instantiated
none of given

Target of a _____ function call is determined at run time.
instance
virtual
operator
none of given

The Specialization pattern after the name says that this specialization is to be used for every___.
data types
meta types
virtual types
pointers type

c++ dynamic binding and polymorphism will be achieved when member function will be __.
private
public
virtual
inline

Consider the code below, class class1{ protected: void func1(); }; class class2 : public class1 { }; Function func1 of class1 is ____ in class2,
public
protected
private
none of the given options

Consider the code below, class class1{ protected: int i; }; class class2 : protected class1 { }; Then int member i of class1 is ____ in class2,
public
protected
private
none of the given options

Consider the code below, class class1{ private: void func1(); }; class class2 : private class1 { }; Function func1 of class1 is ____ in class2,
public

protected
private
none of the given options

Target of a _____ function call is determined at run time.
instance
virtual
operator
none of given

Consider the following statements: 1) int iArray[5]; 2) int *pArr = iArray;
These statements will compile successfully
Error in first statement
Error in second statement
None of given options

Consider the code below, class class1{ private: int i; }; class class2 : private class1 { }; Then int member i of class1 is ____ in class2,
public
protected
private
none of the given options

If there is a pointer, p, to objects of a base class, and it contains the address of an object of a derived class, and both classes contain a virtual member function, ding(), then the statement p->ding(); will cause the version of ding() in the ___ class to be executed.
base
derived
virtual
implemented

A class template may inherit from another class template.
True
False

Derived class can inherit from public base class as well as private and protected base classes
True
False

Two functions with same names, parameters and return type can exist in,
Function overloading
Function overriding
Operator overloading
None of these options

Consider the code below, class class1{ private: int i; }; class class2 : public class1 { }; Then int member i of class1 is ____ in class2,
public

protected
private
none of the given options

Target of a _____ function call is determined at run time.
instance
virtual
operator
none of given

A class template may inherit from another class template.
True
False

A function call is resolved at run-time in_____
non-virtual member function
virtual member function
Both non-virtual member and virtual member function.
None of given

A class hierarchy
shows the same relationships as an organization chart.
describes “has a” relationships.
describes “is a kind of” relationships.
shows the same relationships as a family tree.

Consider the code below, class class1{ public: int i; }; class class2 : public class1 { }; Then int member i of class1 is ____ in class2,
public
protected
private
none of the given options

Consider the code below, class c1{ }; class c2 : public c1 { }; class c3 : public c2 { }; Then c1 is,
Direct base class of c3
Direct child class of c3
Direct base class of c2
Direct child class of c2

A class can inherit from more then one class is called.
Simple inheritance
Multiple inheritances
Single inheritance
Double inheritance

template<> class Vector{ void p; //.... void*& operator[] ((int i); };**
This specialization can then be used as the common implimentation for all Vectors of pointers.
This spcialization can then be used as the all type implimentation for one type classes.

This specialization can then be used double type pointers.
This specialization should be used for Vectors of all type int types.

Consider the code below, class class1{ public: int i; }; class class2 : protected class1 { }; Then int member i of class1 is ____ in class2,
public
protected
private
none of the given options

Consider the code below, class class1{ private: void func1(); }; class class2 : public class1 { }; Function func1 of class1 is ____ in class2,
public
protected
private
none of the given options

Templates automatically create different versions of a function, depending on user input.
True
False

_____ Binding means that target function for a call is selected at run time
Automatic
Dynamic
Static
Dramatic

When we create objects, then space is allocated to:
Member function
Access specifier
Data member
None of given

There is only one form of copy constructor.
True
False

Which of the following features of OOP is used to deal with only relevant details?
Abstraction
Information hiding
Object

_____ Binding means that targets function for a call is selected at compile time.
Static
Dynamic
Automatic
None of given

A Class hierarchy
Shows the same relationships as an organization chart
Describes "has a" relationships.
Describes "is a kind of" relationships.
Shows the same relationships as a family tree

In C++, we declare a function virtual by preceding the function header with keyword "Inline".
True
False

It is illegal to make objects of one class members of another class.
True
False

In Resolution order compiler search firstly_____.
Generic Template
Partial Specification
Complete Specification
Ordinary function

Derived class can inherit from public base class as well as private and protected base classes
True
False

Which line will produce error. Class phone: Private Transmit, private Receiver { } 1.int main () 2. { 3.phone obj; 4.Tranmit*obj1 = &obj; 5.Received obj2 = &obj; 6.}

3rd line will produce error
4th line will produce error
3rd and 4th line will produce error.
5th line will produce error.

Methodologies to the development of reusable software relate to _____.
Structure programming
Procedural programming
Generic programming
None of the given

A template argument is preceded by the keyword_____.
Vector
Class
Template
Type*

Friends are used exactly the same for template and non-template classes.
True
False

A function template must have a parameter
True
False

Child class can call constructor of its,
Direct base class
Indirect base class
Both direct and indirect base classes
None of these.

Which statement will be true for concrete class?
It implements an virtual concept.
It can be instantiated
It cannot be instantiated
None of given

A class D can be derived from a class C, which is derived froma class B, which is derived from a class A
True
False

Adding a derived class to a base class requires fundamental changes to the base class.
True
False

A Class or class template can have member _____ that are themselves templates.
Variable
Function
Objects
None of given

Which will be the Primary task or tasks of generic programming?
Categorize the abstractions in a domain into concepts
Implement generic algorithms based on the concepts
Build concrete models of the concepts
All of given

The default inheritance mode is,

Public inheritance
Protected Inheritance
Private Inheritance
None of these options

If there is a pointer, p, to objects of a base class, and it contains the address of an object of a derived class, and both classes contain a virtual member function, ding(), then the statement p->ding(); will cause the version of ding() in the _____class to be executed.
Base
Derived
Virtual
Implemented

CS304-Object Oriented Programming-Finalterm-Solved-MCQs

Shared & Solved by Umair Saulat

A template provides a convenient way to make a family of.
variables and data members
functions and classes
classes and exceptions
programs and algorithms

A class template may inherit from another class template.
True
False

Target of a _____ function call is determined at run time.
instance
virtual

operator
none of given

A class hierarchy .
shows the same relationships as an organization chart.
describes “has a” relationships.
describes “is a kind of” relationships.
shows the same relationships as a family tree.

Sender of the message does not need to know the exact class of receiver in_____.
Abstraction
Polymorphism
Inheritance
none of the given

A function call is resolved at run-time in_____ .
non-virtual member function
virtual member function
Both non-virtual member and virtual member function.
None of given

Adding a derived class to a base class requires fundamental changes to the base class.
True
False

User can make virtual table explicitly.
True
False

Binding means that target function for a call is selected at compile time.
Static
Dynamic
Automatic
None of given

Target of a _____ function call is determined at run time.
instance
virtual
operator
none of given

Which line will produce error. Class phone: private Transmit, private Receiver { } 1. int main() 2. { 3. phone obj; 4. Tranmit* obj1 = &obj; 5. Received obj2 = &obj; 6. }.
3rd line will produce error
4th line will produce error
3rd and 4th line will produce error.
5th line will produce error

Function overriding is done in context of,
Single class
Single derived class
Single base class
Derived and base classes

Consider the code below, class class1{ public: void func1(); }; class class2 : protected class1 { }; Function func1 of class1 is ____ in class2,
public
protected
private
none of the given options

The following statements: 1) int iArray[5]; 2) int *pArr = iArray;
These statements will compile successfully
Error in first statement
Error in second statement
None of given options

Methodologies to the development of reusable software relate to_____.
Structure programming
procedural programming
generic programming
None of the given

Function template must have a parameter.
True
False

The default inheritance mode is,
Public inheritance
Protected inheritance
Private inheritance
None of these options

Two functions with same names, parameters and return type can exist in,
Function overloading
Function overriding
Operator overloading
None of these options

Consider the code below, class c1{ }; class c2 : public c1 { }; class c3 : public c2 { }; Then c2 is,
Direct base class of c3
Direct child class of c3
Direct base class of c1
None of these

Virtual functions allow you to
create an array of type pointer-to-base class that can hold pointers to derived classes.
create functions that can never be accessed.
group objects of different classes so they can all be accessed by the same function code.
use the same function call to execute member functions of objects from different classes.

User can make virtual table explicitly.
True
False

In order to define a class template the first line of definition must be:
template <typename T>
typename <template T>
Template Class <ClassName>
Class <Template T>

Consider the following statements: 1) int iArray[5]; 2) int *pArr = iArray;
These statements will compile successfully
Error in first statement
Error in second statement
None of given options

In c++ dynamic binding and polymorphism will be achieved when member function will be __.
private
public
virtual
inline

In type in depended function template should be use where code and behavior must be identical.
True
False

Consider the code below, class class1{ protected: int i; }; class class2 : private class1 { };
Then int member i of class1 is ____ in class2,
public
protected
private
none of the given options

In specialization we can,
Replace child class with its base class
Replace base class with its child class (Not Sure)
Replace both child and base classes interchangeably
None of the given options

Consider the code below, class class1{ public: void func1(); }; class class2 : public class1 { };
Function func1 of class1 is ____ in class2,

public
protected
private
none of the given options

It is illegal to make objects of one class members of another class.
True
False

An abstract class is useful when
no classes should be derived from it.
there are multiple paths from one derived class to another.
no objects should be instantiated from its.
you want to defer the declaration of the class.

In resolution order compiler search firstly _____.
Generic Template
Partial Specialization
Complete Specialization
Ordinary function

template<> class Vector{ void p; //.... void*& operator[] ((int i); };**
This specialization can then be used as the common implementation for all Vectors of pointers.
This specialization can then be used as the all type implementation for one type classes.
This specialization can then be used double type pointers.
This specialization should be used for Vectors of all type int types.

In private inheritance derived class pointer can be assigned to base class pointer in.
Main function
In derived class member and friend functions
In base class member and friend functions
None of the given options

Which statement will be true for concrete class?
it implements an virtual concept.
it can be instantiated
it cannot be instantiated
none of given

Target of a _____ function call is determined at run time.
instance
virtual
operator
none of given

The Specialization pattern after the name says that this specialization is to be used for every___.
data types
meta types

virtual types
pointers type

c++ dynamic binding and polymorphism will be achieved when member function will be __.
private
public
virtual
inline

Consider the code below, class class1{ protected: void func1(); }; class class2 : public class1 { }; Function func1 of class1 is ____ in class2,
public
protected
private
none of the given options

Consider the code below, class class1{ protected: int i; }; class class2 : protected class1 { }; Then int member i of class1 is ____ in class2,
public
protected
private
none of the given options

Consider the code below, class class1{ private: void func1(); }; class class2 : private class1 { }; Function func1 of class1 is ____ in class2,
public
protected
private
none of the given options

Target of a _____ function call is determined at run time.
instance
virtual
operator
none of given

Consider the following statements: 1) int iArray[5]; 2) int *pArr = iArray;
These statements will compile successfully
Error in first statement
Error in second statement
None of given options

Consider the code below, class class1{ private: int i; }; class class2 : private class1 { }; Then int member i of class1 is ____ in class2,
public
protected
private
none of the given options

If there is a pointer, p, to objects of a base class, and it contains the address of an object of a derived class, and both classes contain a virtual member function, ding(), then the statement p->ding(); will cause the version of ding() in the ___ class to be executed.
base
derived
virtual
implemented

A class template may inherit from another class template.
True
False

Derived class can inherit from public base class as well as private and protected base classes
True
False

Two functions with same names, parameters and return type can exist in,
Function overloading
Function overriding
Operator overloading
None of these options

Consider the code below, class class1{ private: int i; }; class class2 : public class1 { }; Then int member i of class1 is ____ in class2,
public
protected
private
none of the given options

Target of a _____ function call is determined at run time.
instance
virtual
operator
none of given

A class template may inherit from another class template.
True
False

A function call is resolved at run-time in_____
non-virtual member function
virtual member function
Both non-virtual member and virtual member function.
None of given

A class hierarchy
shows the same relationships as an organization chart.
describes “has a” relationships.

describes “is a kind of” relationships.
shows the same relationships as a family tree.

Consider the code below, class class1{ public: int i; }; class class2 : public class1 { }; Then int member i of class1 is ____ in class2,
public
protected
private
none of the given options

Consider the code below, class c1{ }; class c2 : public c1 { }; class c3 : public c2 { }; Then c1 is,
Direct base class of c3
Direct child class of c3
Direct base class of c2
Direct child class of c2

A class can inherit from more then one class is called.
Simple inheritance
Multiple inheritances
Single inheritance
Double inheritance

template<> class Vector{ void p; //.... void*& operator[] ((int i); };**
This specialization can then be used as the common implimentation for all Vectors of pointers.
This spcialization can then be used as the all type implimentation for one type classes.
This specialization can then be used double type pointers.
This specialization should be used for Vectors of all type int types.

Consider the code below, class class1{ public: int i; }; class class2 : protected class1 { }; Then int member i of class1 is ____ in class2,
public
protected
private
none of the given options

Consider the code below, class class1{ private: void func1(); }; class class2 : public class1 { }; Function func1 of class1 is ____ in class2,
public
protected
private
none of the given options

Templates automatically create different versions of a function, depending on user input.
True
False

_____ Binding means that target function for a call is selected at run time
Automatic

Dynamic
Static
Dramatic

When we create objects, then space is allocated to:
Member function
Access specifier
Data member
None of given

There is only one form of copy constructor.
True
False

Which of the following features of OOP is used to deal with only relevant details?
Abstraction
Information hiding
Object

_____ Binding means that targets function for a call is selected at compile time.
Static
Dynamic
Automatic
None of given

A Class hierarchy
Shows the same relationships as an organization chart
Describes "has a" relationships.
Describes "is a kind of" relationships.
Shows the same relationships as a family tree

In C++, we declare a function virtual by preceding the function header with keyword "Inline".
True
False

It is illegal to make objects of one class members of another class.
True
False

In Resolution order compiler search firstly_____.
Generic Template
Partial Specification
Complete Specification
Ordinary function

Derived class can inherit from public base class as well as private and protected base classes

True
False

Which line will produce error. Class phone: Private Transmit, private Receiver { } 1.int main () 2. { 3.phone obj; 4.Tranmit*obj1 = &obj; 5.Received obj2 = &obj; 6.}

3rd line will produce error
4th line will produce error
3rd and 4th line will produce error.
5th line will produce error.

Methodologies to the development of reusable software relate to _____.
Structure programming
Procedural programming
Generic programming
None of the given

A template argument is preceded by the keyword_____.
Vector
Class
Template
Type*

Friends are used exactly the same for template and non-template classes.
True
False

A function template must have a parameter
True
False

Child class can call constructor of its,
Direct base class
Indirect base class
Both direct and indirect base classes
None of these.

Which statement will be true for concrete class?
It implements an virtual concept.
It can be instantiated
It cannot be instantiated
None of given

A class D can be derived from a class C, which is derived froma class B, which is derived from a class A
True
False

Adding a derived class to a base class requires fundamental changes to the base class.
True
False

A Class or class template can have member _____ that are themselves templates.
Variable
Function
Objects
None of given

Which will be the Primary task or tasks of generic programming?
Categorize the abstractions in a domain into concepts
Implement generic algorithms based on the concepts
Build concrete models of the concepts
All of given

The default inheritance mode is,

Public inheritance
Protected Inheritance
Private Inheritance
None of these options

If there is a pointer, p, to objects of a base class, and it contains the address of an object of a derived class, and both classes contain a virtual member function, ding(), then the statement p->ding(); will cause the version of ding() in the _____class to be executed.
Base
Derived
Virtual
Implemented

CS304 Object Oriented Programming Solved MCQs from Quiz # 2

CS304 Object Oriented Programming Solved MCQs from Quiz # 2

Child class can call constructor of its,
Direct base class
Indirect base class

Both direct and indirect base classes
None of these

A pure virtual function is a virtual function that
causes its class to be abstract.
returns nothing.
is used in a derived class.
takes no arguments.

Two functions with same names, parameters and return type can exist in,
Function overloading
Function overriding
Operator overloading
None of these options
Adding a derived class to a base class requires fundamental changes to the base class.
True CORRECT
FALSE
User can make virtual table explicitly.
TRUE
False CORRECT
Consider the following statements: 1) int iArray[5]; 2) int *pArr = iArray;
These statements will compile successfully
Error in first statement
Error in second statement CORRECT
None of given options
Consider the code below, class class1{ public: void func1(); }; class class2 : protected class1 { };
Function func1 of class1 is _____ in class2,
public
protected
private CORRECT
none of the given options
Consider the code below, class c1{ }; class c2 : public c1 { }; class c3 : public c2 { }; Then c2 is,
Direct base class of c3
Direct child class of c3 CORRECT
Direct base class of c1
None of these
None of these